

Intel® Integrated Performance Primitives for Intel® Architecture

Reference Manual

Volume 4: Cryptography

January 2007

Document Number: 303881-07US

World Wide Web: <http://developer.intel.com>



Version	Version Information	Date
-01	Documents Intel® Integrated Performance Primitives (Intel® IPP) 4.0 beta release.	05/ 2003
-02	Documents Intel® Integrated Performance Primitives (Intel® IPP) 4.0 gold release.	10/2003
-03	Documents Intel® Integrated Performance Primitives (Intel® IPP) 5.0 release with ECC and DL-based functionalities.	03/2005
-04	Documents Intel® Integrated Performance Primitives (Intel® IPP) 5.0 gold release.	08/2005
-05	Documents Intel® Integrated Performance Primitives (Intel® IPP) 5.1 gold release.	02/2006
-06	Documents Intel® Integrated Performance Primitives (Intel® IPP) 5.2 beta release.	09/2006
-07	Documents Intel® Integrated Performance Primitives (Intel® IPP) 5.2 gold release.	01/2007

INFORMATION IN THIS DOCUMENT IS PROVIDED IN CONNECTION WITH INTEL® PRODUCTS. NO LICENSE, EXPRESS OR IMPLIED, BY ESTOPPEL OR OTHERWISE, TO ANY INTELLECTUAL PROPERTY RIGHTS IS GRANTED BY THIS DOCUMENT. EXCEPT AS PROVIDED IN INTEL'S TERMS AND CONDITIONS OF SALE FOR SUCH PRODUCTS, INTEL ASSUMES NO LIABILITY WHATSOEVER, AND INTEL DISCLAIMS ANY EXPRESS OR IMPLIED WARRANTY, RELATING TO SALE AND/OR USE OF INTEL PRODUCTS INCLUDING LIABILITY OR WARRANTIES RELATING TO FITNESS FOR A PARTICULAR PURPOSE, MERCHANTABILITY, OR INFRINGEMENT OF ANY PATENT, COPYRIGHT OR OTHER INTELLECTUAL PROPERTY RIGHT. Intel products are not intended for use in medical, life saving, life sustaining, critical control or safety systems, or in nuclear facility applications.

Intel may make changes to specifications and product descriptions at any time, without notice.

The software described in this document may contain software defects which may cause the product to deviate from published specifications. Current characterized software defects are available on request.

This manual as well as the software described in it is furnished under license and may only be used or copied in accordance with the terms of the license. The information in this manual is furnished for informational use only, is subject to change without notice, and should not be construed as a commitment by Intel Corporation. Intel Corporation assumes no responsibility or liability for any errors or inaccuracies that may appear in this document or any software that may be provided in association with this document.

Except as permitted by such license, no part of this document may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without the express written consent of Intel Corporation.

Developers must not rely on the absence or characteristics of any features or instructions marked "reserved" or "undefined." Improper use of reserved or undefined features or instructions may cause unpredictable behavior or failure in developer's software code when running on an Intel processor. Intel reserves these features or instructions for future definition and shall have no responsibility whatsoever for conflicts or incompatibilities arising from their unauthorized use.

BunnyPeople, Celeron, Celeron Inside, Centrino, Centrino logo, Core Inside, FlashFile, i960, InstantIP, Intel, Intel logo, Intel386, Intel486, Intel740, IntelDX2, IntelDX4, IntelSX2, Intel Core, Intel Inside, Intel Inside logo, Intel. Leap ahead., Intel. Leap ahead. logo, Intel NetBurst, Intel NetMerge, Intel NetStructure, Intel SingleDriver, Intel SpeedStep, Intel StrataFlash, Intel Viiv, Intel vPro, Intel XScale, IPLink, Itanium, Itanium Inside, MCS, MMX, Oplus, OverDrive, PDCharm, Pentium, Pentium Inside, skool, Sound Mark, The Journey Inside, VTune, Xeon, and Xeon Inside are trademarks or registered trademarks of Intel Corporation or its subsidiaries in the United States and other countries.

* Other names and brands may be claimed as the property of others.

Copyright © 2001 - 2007, Intel Corporation. All rights reserved

Contents

Chapter 1 Overview

Basic Features	1-1
About This Software	1-2
Hardware and Software Requirements	1-2
Platforms Supported	1-3
Cross-Architecture Alignment	1-3
API Changes in Version 5.0	1-4
Technical Support	1-5
About This Manual	1-5
Manual Organization	1-5
Function Descriptions	1-6
Audience for This Manual	1-6
Notational Conventions	1-7
Online Version	1-7

Chapter 2 Symmetric Cryptography Primitive Functions

Block Cipher Modes of Operation	2-1
DES/TDES Functions	2-3
DESGetSize	2-5
DESInit	2-6
DESEncryptECB	2-6
DESDecryptECB	2-7
DESEncryptCBC	2-8
DESDecryptCBC	2-9
DESEncryptCFB	2-10

DESDecryptCFB.....	2-11
DESEncryptOFB.....	2-12
DESDecryptOFB	2-13
DESEncryptCTR.....	2-14
DESDecryptCTR	2-15
TDESEncryptECB	2-16
TDESDecryptECB.....	2-17
TDESEncryptCBC.....	2-18
TDESDecryptCBC.....	2-19
TDESEncryptCFB	2-20
TDESDecryptCFB.....	2-21
TDESEncryptOFB	2-23
TDESDecryptOFB.....	2-24
TDESEncryptCTR	2-25
TDESDecryptCTR.....	2-26
Rijndael Functions	2-29
Rijndael128GetSize.....	2-32
Rijndael128Init	2-33
Rijndael128EncryptECB	2-33
Rijndael128DecryptECB	2-34
Rijndael128EncryptCBC	2-35
Rijndael128DecryptCBC.....	2-36
Rijndael128EncryptCFB	2-37
Rijndael128DecryptCFB	2-38
Rijndael128EncryptOFB	2-39
Rijndael128DecryptOFB.....	2-40
Rijndael128EncryptCTR	2-41
Rijndael128DecryptCTR.....	2-42
Rijndael128EncryptCCM	2-43
Rijndael128DecryptCCM	2-44
Rijndael128EncryptCCM_u8	2-45
Rijndael128DecryptCCM_u8	2-47
Rijndael192GetSize.....	2-48
Rijndael192Init	2-49

Rijndael192EncryptECB	2-49
Rijndael192DecryptECB	2-50
Rijndael192EncryptCBC	2-51
Rijndael192DecryptCBC	2-52
Rijndael192EncryptCFB	2-53
Rijndael192DecryptCFB	2-54
Rijndael192EncryptOFB	2-55
Rijndael192DecryptOFB	2-56
Rijndael192EncryptCTR	2-57
Rijndael192DecryptCTR	2-58
Rijndael256GetSize	2-59
Rijndael256Init	2-60
Rijndael256EncryptECB	2-60
Rijndael256DecryptECB	2-61
Rijndael256EncryptCBC	2-62
Rijndael256DecryptCBC	2-63
Rijndael256EncryptCFB	2-64
Rijndael256DecryptCFB	2-65
Rijndael256EncryptOFB	2-66
Rijndael256DecryptOFB	2-67
Rijndael256EncryptCTR	2-68
Rijndael256DecryptCTR	2-69
Blowfish Functions	2-73
BlowfishGetSize	2-74
BlowfishInit	2-75
BlowfishEncryptECB	2-75
BlowfishDecryptECB	2-76
BlowfishEncryptCBC	2-77
BlowfishDecryptCBC	2-78
BlowfishEncryptCFB	2-79
BlowfishDecryptCFB	2-80
BlowfishEncryptOFB	2-81
BlowfishDecryptOFB	2-82
BlowfishEncryptCTR	2-83

BlowfishDecryptCTR.....	2-84
Twofish Functions	2-87
TwofishGetSize.....	2-88
TwofishInit.....	2-89
TwofishEncryptECB	2-89
TwofishDecryptECB	2-90
TwofishEncryptCBC	2-91
TwofishDecryptCBC.....	2-92
TwofishEncryptCFB	2-93
TwofishDecryptCFB	2-94
TwofishEncryptOFB	2-95
TwofishDecryptOFB.....	2-96
TwofishEncryptCTR	2-97
TwofishDecryptCTR.....	2-98
RC5* Functions	2-102
RC5* Algorithm Functions for 64-bit Block Size.....	2-104
ARCFive64GetSize	2-104
ARCFive64Init	2-105
ARCFive64EncryptECB.....	2-106
ARCFive64DecryptECB	2-107
ARCFive64EncryptCBC	2-108
ARCFive64DecryptCBC	2-109
ARCFive64EncryptCFB.....	2-110
ARCFive64DecryptCFB.....	2-111
ARCFive64EncryptOFB.....	2-112
ARCFive64DecryptOFB	2-113
ARCFive64EncryptCTR.....	2-114
ARCFive64DecryptCTR	2-115
RC5* Algorithm Functions for 128-bit Block Size.....	2-116
ARCFive128GetSize	2-116
ARCFive128Init	2-116
ARCFive128EncryptECB	2-117
ARCFive128DecryptECB.....	2-118
ARCFive128EncryptCBC.....	2-119

ARCFive128DecryptCBC	2-120
ARCFive128EncryptCFB	2-121
ARCFive128DecryptCFB	2-122
ARCFive128EncryptOFB	2-123
ARCFive128DecryptOFB	2-124
ARCFive128EncryptCTR	2-125
ARCFive128DecryptCTR	2-126
ARCFour Functions	2-127
ARCFourGetSize	2-128
ARCFourCheckKey	2-128
ARCFourInit	2-129
ARCFourEncrypt	2-130
ARCFourDecrypt	2-131
ARCFourReset	2-132

Chapter 3 One-Way Hash Primitives

Hash Functions	3-4
MD5GetSize	3-4
MD5Init	3-5
MD5Duplicate	3-6
MD5Update	3-6
MD5Final	3-7
SHA1GetSize	3-8
SHA1Init	3-8
SHA1Duplicate	3-9
SHA1Update	3-10
SHA1Final	3-11
SHA224GetSize	3-11
SHA224Init	3-12
SHA224Duplicate	3-12
SHA224Update	3-13
SHA224Final	3-14
SHA256GetSize	3-15
SHA256Init	3-15

SHA256Duplicate.....	3-16
SHA256Update.....	3-17
SHA256Final	3-18
SHA384GetSize	3-18
SHA384Init.....	3-19
SHA384Duplicate.....	3-19
SHA384Update.....	3-20
SHA384Final	3-21
SHA512GetSize	3-22
SHA512Init.....	3-22
SHA512Duplicate.....	3-23
SHA512Update.....	3-24
SHA512Final	3-25
Generalized Hash Functions for Non-Streaming Messages	3-25
General Definition of a Hash Function	3-26
MD5MessageDigest.....	3-26
SHA1MessageDigest	3-27
SHA224MessageDigest.....	3-29
SHA256MessageDigest.....	3-30
SHA384MessageDigest.....	3-31
SHA512MessageDigest.....	3-32
Mask Generation Functions	3-32
User's Implementation of a Mask Generation Function	3-33
MGF_MD5.....	3-34
MGF_SHA1	3-34
MGF_SHA224.....	3-35
MGF_SHA256.....	3-36
MGF_SHA384.....	3-37
MGF_SHA512.....	3-37

Chapter 4 Data Authentication Primitive Functions

Message Authentication Functions.....	4-1
HMACSHA1GetSize	4-4
HMACSHA1Init	4-5

HMACSHA1Duplicate	4-5
HMACSHA1Update	4-6
HMACSHA1Final	4-7
HMACSHA1MessageDigest	4-8
HMACSHA224GetSize	4-9
HMACSHA224Init	4-9
HMACSHA224Duplicate	4-10
HMACSHA224Update	4-11
HMACSHA224Final	4-12
HMACSHA224MessageDigest	4-12
HMACSHA256GetSize	4-13
HMACSHA256Init	4-14
HMACSHA256Duplicate	4-15
HMACSHA256Update	4-15
HMACSHA256Final	4-16
HMACSHA256MessageDigest	4-17
HMACSHA384GetSize	4-20
HMACSHA384Init	4-20
HMACSHA384Duplicate	4-21
HMACSHA384Update	4-22
HMACSHA384Final	4-23
HMACSHA384MessageDigest	4-23
HMACSHA512GetSize	4-24
HMACSHA512Init	4-25
HMACSHA512Duplicate	4-26
HMACSHA512Update	4-26
HMACSHA512Final	4-27
HMACSHA512MessageDigest	4-28
HMACMD5GetSize	4-29
HMACMD5Init	4-29
HMACMD5Duplicate	4-30
HMACMD5Update	4-31
HMACMD5Final	4-32
HMACMD5MessageDigest	4-32

CMACRijndael128GetSize.....	4-34
CMACRijndael128Init	4-35
CMACRijndael128Update	4-36
CMACRijndael128Final.....	4-37
CMACRijndael128MessageDigest.....	4-37
Data Authentication Functions.....	4-39
DAADESGetSize	4-41
DAADESInit	4-41
DAADESUpdate	4-42
DAADESFinal	4-43
DAADESMessageDigest	4-44
DAATDESGetSize.....	4-44
DAATDESInit.....	4-45
DAATDESUpdate.....	4-46
DAATDESFinal	4-47
DAATDESMessageDigest.....	4-47
DAARijndael128GetSize.....	4-48
DAARijndael128Init.....	4-49
DAARijndael128Update	4-50
DAARijndael128Final.....	4-51
DAARijndael128MessageDigest.....	4-51
DAARijndael192GetSize.....	4-52
DAARijndael192Init.....	4-53
DAARijndael192Update	4-54
DAARijndael192Final.....	4-55
DAARijndael192MessageDigest.....	4-55
DAARijndael256GetSize.....	4-56
DAARijndael256Init.....	4-57
DAARijndael256Update	4-58
DAARijndael256Final.....	4-59
DAARijndael256MessageDigest.....	4-59
DAABlowfishGetSize.....	4-60
DAABlowfishInit	4-61
DAABlowfishUpdate	4-61

DAABlowfishFinal.....	4-62
DAABlowfishMessageDigest.....	4-63
DAATwofishGetSize	4-64
DAATwofishInit.....	4-64
DAATwofishUpdate.....	4-65
DAATwofishFinal	4-66
DAATwofishMessageDigest	4-67

Chapter 5 **Public Key Cryptography Functions**

Big Number Arithmetic.....	5-1
Add_BNU	5-3
Sub_BNU	5-4
MulOne_BNU	5-5
MACOne_BNU_I	5-6
Mul_BNU4	5-7
Mul_BNU8	5-7
Div_64u32u.....	5-8
Sqr_32u64u	5-9
Sqr_BNU4	5-10
Sqr_BNU8	5-10
SetOctString_BNU	5-11
GetOctString_BNU	5-12
BigNumGetSize	5-13
BigNumInit.....	5-13
Set_BN	5-14
SetOctString_BN	5-15
GetSize_BN	5-17
Get_BN	5-18
GetOctString_BN.....	5-19
Cmp_BN	5-20
CmpZero_BN	5-21
Add_BN	5-22
Sub_BN	5-24
Mul_BN	5-25

MAC_BN_I	5-26
Div_BN	5-27
Mod_BN	5-28
Gcd_BN.....	5-29
ModInv_BN.....	5-30
Montgomery Reduction Scheme Functions	5-31
MontGetSize	5-33
MontInit	5-34
MontSet	5-35
MontGet.....	5-36
MontForm.....	5-36
MontMul	5-37
MontExp.....	5-40
Pseudorandom Number Generation Functions.....	5-41
User's Implementation of a Pseudorandom Number Generator	5-42
PRNGGetSize	5-43
PRNGInit	5-43
PRNGSetSeed	5-44
PRNGSetAugment.....	5-45
PRNGSetModulus.....	5-46
PRNGSetH0	5-47
PRNGen	5-47
PRNGen_BN.....	5-48
Prime Number Generation Functions	5-51
PrimeGetSize	5-52
PrimeInit.....	5-53
PrimeGen	5-54
PrimeTest	5-55
PrimeSet	5-56
PrimeSet_BN	5-56
PrimeGet.....	5-57
PrimeGet_BN	5-58
RSA Algorithm Functions	5-61
Functions for Building RSA System.....	5-61

RSAGetSize	5-62
RSASInit	5-63
RSASetKey	5-64
RSAGetKey	5-65
RSAGenerate	5-66
RSASValidate	5-67
RSA Primitives	5-68
RSAEncrypt	5-69
RSADecrypt	5-70
RSA-OAEP Scheme Functions	5-74
RSASOAEP Encrypt	5-75
RSASOAEP Encrypt_MD5	5-76
RSASOAEP Encrypt_SHA1	5-77
RSASOAEP Encrypt_SHA224	5-78
RSASOAEP Encrypt_SHA256	5-79
RSASOAEP Encrypt_SHA384	5-80
RSASOAEP Encrypt_SHA512	5-81
RSASOAEP Decrypt	5-82
RSASOAEP Decrypt_MD5	5-83
RSASOAEP Decrypt_SHA1	5-84
RSASOAEP Decrypt_SHA224	5-84
RSASOAEP Decrypt_SHA256	5-85
RSASOAEP Decrypt_SHA384	5-86
RSASOAEP Decrypt_SHA512	5-87
RSA-SSA Scheme Functions	5-87
RSASSASign	5-88
RSASSASign_MD5	5-90
RSASSASign_SHA1	5-91
RSASSASign_SHA224	5-92
RSASSASign_SHA256	5-93
RSASSASign_SHA384	5-94
RSASSASign_SHA512	5-95
RSASSAVerify	5-96
RSASSAVerify_MD5	5-97

RSASSAVerify_SHA1	5-98
RSASSAVerify_SHA224.....	5-98
RSASSAVerify_SHA256.....	5-99
RSASSAVerify_SHA384.....	5-100
RSASSAVerify_SHA512.....	5-100
Discrete-Logarithm-Based Cryptography Functions.....	5-101
DLPGetSize.....	5-103
DLPInit	5-103
DLPSet.....	5-104
DLPGet	5-105
DLPSetDP.....	5-106
DLPGetDP.....	5-107
DLPGenKeyPair	5-108
DLPPublicKey	5-109
DLPValidateKeyPair	5-110
DLPSetKeyPair	5-111
DLPGenerateDSA.....	5-112
DLPValidateDSA	5-113
DLPSignDSA	5-114
DLPVerifyDSA	5-115
DLPGenerateDH	5-120
DLPValidateDH	5-121
DLPSharedSecretDH	5-122
Elliptic Curve Cryptography Functions.....	5-123
Functions Based on GF(p).....	5-126
ECCPGetSize.....	5-127
ECCPInit.....	5-128
ECCPSet.....	5-128
ECCPSetStd	5-130
ECCPGet	5-131
ECCPGetOrderBitSize	5-132
ECCPValidate	5-133
ECCPPointGetSize	5-135
ECCPPointInit.....	5-135

ECCPSetPoint	5-136
ECCPSetPointAtInfinity	5-137
ECCPGetPoint	5-138
ECCPCheckPoint	5-139
ECCPComparePoint	5-140
ECCPNegativePoint	5-141
ECCPAddPoint	5-142
ECCPMulPointScalar	5-143
ECCPGenKeyPair	5-144
ECCPPublicKey	5-145
ECCPValidateKeyPair	5-146
ECCPSetKeyPair	5-147
ECCPSharedSecretDH	5-148
ECCPSharedSecretDHC	5-150
ECCPSignDSA	5-151
ECCPVerifyDSA	5-153
ECCPSignNR	5-154
ECCPVerifyNR	5-156
Functions Based on GF(2 ^m)	5-157
ECCBGetSize	5-158
ECCBInit	5-158
ECCBSet	5-159
ECCBSetStd	5-161
ECCBGet	5-162
ECCBGetOrderBitSize	5-163
ECCBValidate	5-164
ECCBPointGetSize	5-166
ECCBPointInit	5-166
ECCBSetPoint	5-167
ECCBSetPointAtInfinity	5-168
ECCBGetPoint	5-169
ECCBCheckPoint	5-170
ECCBComparePoint	5-171
ECCBNegativePoint	5-172

ECCBAddPoint	5-173
ECCBMulPointScalar	5-174
ECCBGenKeyPair	5-175
ECCBPublicKey	5-176
ECCBValidateKeyPair	5-177
ECCBSetKeyPair	5-178
ECCBSharedSecretDH	5-179
ECCBSharedSecretDHC	5-181
ECCBSignDSA	5-182
ECCBVerifyDSA	5-184
ECCBSignNR	5-185
ECCBVerifyNR	5-186

Appendix A Functions Removed from Intel® Integrated Performance Primitives 5.0

Appendix B Support Functions and Classes

Version Information Function	B-1
GetLibVersion	B-1
Classes and Functions Used in Examples.....	B-3
BigNumber Class	B-3
Functions for Creation of Cryptographic Contexts.....	B-15

Bibliography

Index

Examples

Example 2-1 DES/TDES Encryption and Decryption	2-27
Example 2-2 AES Encryption and Decryption	2-71
Example 2-3 Blowfish Encryption and Decryption	2-85
Example 2-4 Twofish Encryption and Decryption	2-100
Example 3-1 MD5 Digest of a Message	3-27
Example 3-2 SHA1 Digest of a Message.....	3-28

Example 4-1 SHA256 HMACs of a Message	4-18
Example 4-2 MD5 HMAC of a Message.....	4-34
Example 5-1 Create a Big Number.....	5-15
Example 5-2 Create a Big Number from a String	5-17
Example 5-3 Type a Big Number.....	5-20
Example 5-4 Add Big Numbers.....	5-23
Example 5-5 Montgomery Multiplication	5-38
Example 5-6 Find Pseudorandom Co-primes.....	5-49
Example 5-7 Check Primality	5-59
Example 5-8 The Use of RSA Primitives	5-71
Example 5-9 The Use of DLPSignDSA and DLPVerifyDSA.....	5-117
Example B-1 Using the ippcpGetLibVersion Function.....	B-2

Overview

1

This manual describes the structure, operation, and functions of Intel® Integrated Performance Primitives (Intel® IPP) for cryptography. This is the fourth volume of the Intel IPP Reference Manual, which also comprises descriptions of Intel IPP for signal processing (volume 1), Intel IPP for image and video processing (volume 2), and Intel IPP for small matrices and realistic rendering (volume 3).

The Intel IPP software package supports many functions whose performance can be significantly enhanced on the Intel® Architecture (IA), particularly using the MMX™ technology and Streaming SIMD Extensions.

Intel IPP for cryptography is a cross-platform software layer optimized for IA-32, Intel® 64, and IA-64 (Itanium®) architectures as well as for Intel® IXP4XX product line of network processors and running on Microsoft* Windows*, Linux*, and Mac OS* operating systems.

This manual provides detailed descriptions of Intel IPP functions developed for cryptographic operations.

This chapter introduces the Intel IPP cryptography software and explains the organization of this manual.

Basic Features

Like other members of Intel® Performance Libraries, Intel Integrated Performance Primitives is a collection of high-performance code that performs domain-specific operations. It is distinguished by providing a low-level, stateless interface.

Based on experience in developing and using Intel Performance Libraries, Intel IPP has the following major distinctive features:

- Intel IPP provides basic low-level functions for creating applications in several different domains, such as signal processing, image and video processing, operations on small matrices, and cryptography applications.

- Intel IPP functions follow the same interface conventions, including uniform naming conventions and similar composition of prototypes for primitives that refer to different application domains.
- Intel IPP functions use an abstraction level which is best suited to achieve superior performance figures by the application programs.

To speed up the performance, Intel IPP functions are optimized to use all benefits of Intel® architecture processors. Besides this, most of Intel IPP functions do not use complicated data structures, which helps reduce overall execution overhead.

Intel IPP is well-suited for cross-platform applications. For example, the functions developed for IA-32 architecture-based platform can be readily ported to Intel® 64, IA-64 architecture-based platforms and systems with Intel® XScale™ technology. For more information on platform compatibility, see [Cross-Architecture Alignment](#). In addition, each Intel IPP function has its reference code written in ANSI C, which clearly presents the algorithm used and provides for compatibility with different operating systems.

About This Software

The Intel IPP software enables to take advantage of the single-instruction, multiple data (SIMD) instructions that comprise the core of the MMX technology and Streaming SIMD Extensions.

The Intel Integrated Performance Primitives for cryptography provide a broad set of cryptographic primitive functions optimized for maximum performance on Intel® platforms, including IA-32, Intel® 64, IA-64 architectures and Intel® IXP4XX product line. The set of cryptography primitive functions can be thought of as a set of “building blocks” that enable independent software vendors to build their own FIPS-conformant security solutions.

The package of Intel IPP cryptography functions offers developers cross-platform and cross operating system API for routines commonly used for cryptographic operations. Use of Intel IPP primitive functions can help to drastically reduce development costs and accelerate time-to market by eliminating the need of writing processor-specific code for computation intensive routines.

Hardware and Software Requirements

Intel IPP for Intel architecture software runs on personal computers that are based on processors using IA-32, Intel® 64 or IA-64 architecture and running Microsoft Windows*, Linux*, or Mac OS*. Intel IPP integrates into the customer's application or library written in C or C++.

Platforms Supported

Intel IPP for Intel architecture software runs on Windows*, Linux*, and Mac OS* platforms. The code and syntax used in this manual for function and variable declarations are written in the ANSI C style. However, versions of Intel IPP for different processors or operating systems may, of necessity, vary slightly.

Cross-Architecture Alignment

Cross Architecture Overview

Intel IPP has been designed to support application development on various Intel® architectures.

By providing a single cross-architecture API, Intel IPP allows software application repurposing and enables developers to port to unique features across Intel® processor-based desktop, server, mobile, and handheld platforms. Developers can write their code once in order to realize the application performance over many processor generations.

The [Table 1-1](#) summarizes the functionality covered in each Intel IPP implementation. Only integer flavors of the most functions described in this manual are supported for Intel® IXP4XX product line of network processors. Please refer to the corresponding header files.

For additional information on API additions and changes from previous versions, please refer to the product release notes and the document Migration Guide for Intel® IPP Cross Architecture API Alignment available at <http://www.intel.com/software/products/ipp>.

The following table summarizes the functionality covered in each Intel IPP implementation.

Table 1-1 Function Coverage in Intel IPP

Function Group	Intel® Pentium® 4 processors	Intel® Xeon™ processors using Intel® 64 architecture	Intel® Itanium® 2 processors	Intel® IXP4XX Product Line
Signal Processing	available	available	available	available ¹
Image Processing	available	available	available	available ¹
JPEG	available	available	available	available ¹
Speech Recognition	available	available	available	available ¹
Speech Coding	available	available	available	available ¹
Audio Codecs	available	available	available	available ¹
Video Codecs	available	available	available	available ¹

Table 1-1 Function Coverage in Intel IPP (continued)

Function Group	Intel® Pentium® 4 processors	Intel® Xeon™ processors using Intel® 64 architecture	Intel® Itanium® 2 processors	Intel® IXP4XX Product Line
Matrix	available	available	available	n/a
Vector Math	available	available	available	n/a
Computer Vision	available	available	available	available ¹
Cryptography	available	available	available	available ¹
Data Compression	available	available	available	available ¹
Color Conversion	available	available	available	available ¹
String Processing	available	available	available	available ¹
Realistic Rendering	available	available	available	n/a

1. Only a subset of these functions is supported, please refer to the corresponding header file.

API Changes in Version 5.0

As Intel IPP evolved and the functions got aligned for the Intel Pentium, Xeon and Itanium processors and the Intel PCA processors, the libraries accumulated numerous changes, which, in particular, caused quite a few functions grow obsolete. So, in Intel IPP 5.0, compatibility in API with previous versions had to be compromised to achieve the following library improvements:

- A number of functions were replaced by newer functions with extended functionality.
- Some other functions were changed to make the API more consistent.
- Odd and unusable functions were removed.

Several modifications of Intel IPP, and especially those that improve directory structure, also break *binary* backward compatibility of version 5.0. So, to migrate to Intel IPP 5.0, you have at least to rebuild your applications. If an application calls a function missing from Intel IPP 5.0, you have also to modify the source code. To help you migrate to Intel IPP 5.0, Appendix A lists functions removed from Intel IPP 5.0 for cryptography and specifies Intel IPP 5.0 function(s) to substitute for the missing ones. Higher versions of Intel IPP, starting from 5.1, will be fully backward compatible with version 5.0.

Technical Support

Intel IPP provides a product web site that offers timely and comprehensive product information, including product features, white papers, and technical articles. For the latest information, see <http://developer.intel.com/software/products/>.

Intel also provides a support web site that contains a rich repository of self-help information, including getting started tips, known product issues, product errata, license information, and more (visit <http://support.intel.com/support/>).

Registering your product entitles you to one-year technical support and product updates through Intel® Premier Support. Intel Premier Support is an interactive issue management and communication web site providing the following services:

- Submit issues and review their status.
- Download product updates anytime of the day.

To register your product, or contact Intel, or seek product support, please visit <http://www.intel.com/software/products/support>.

About This Manual

This manual provides a background for cryptography concepts used in the Intel IPP software as well as detailed description of the respective Intel IPP functions. The Intel IPP functions are combined in groups by their functionality. Each group of functions is described in a separate chapter.

Manual Organization

This manual contains the following chapters and appendices:

- Chapter 1 [Overview](#). Introduces Intel IPP for cryptography operations, provides information on manual organization, and explains notational conventions.
- Chapter 2 [Symmetric Cryptography Primitive Functions](#). Explains basic concepts underlying the Intel IPP functions used for symmetric cryptography algorithm operations and describes the supported data layout and operation modes.
- Chapter 3 [One-Way Hash Primitives](#). Describes functions used for hash cryptography operations and data authentication.
- Chapter 4 [Data Authentication Primitive Functions](#). Describes functions for generating message authentication code using the Keyed Hash Functions (HMAC) and the Data authentication Functions (DAA) schemes.
- Chapter 5 [Public Key Cryptography Functions](#). Explains basic concepts underlying the Intel IPP functions used for asymmetric cryptography algorithm operations and describes the supported data layout and operation modes. In addition, the chapter covers the Diffie-Hellman functionality functions and the elliptic cryptography primitives operated on finite fields.

Appendix A [Functions Removed from Intel® Integrated Performance Primitives 5.0](#). Lists cryptography functions removed from Intel IPP 5.0 along with their version 5.0 substitutes.

Appendix B [Support Functions and Classes](#). Presents miscellaneous information on support functions and classes that are used in Intel IPP cryptography software or in examples given in the manual chapters.

The manual also includes a [Bibliography](#) and [Index](#) of major terms and definitions used in this volume.

Function Descriptions

In Chapters 2 through 4, each function is introduced by its short name (without the `ipps` prefix and descriptors) and a brief description of its purpose. This is followed by an example of the function call sequence, definition of its parameters, and a more detailed explanation of the function purpose. The following sections are included in the function description:

<i>Syntax</i>	Lists function prototypes.
<i>Parameters</i>	Describes all function parameters.
<i>Description</i>	Defines the function and details the operation performed by the function. Code examples and equations that the function implements may be included in the description.
<i>Return Value</i>	Describes values indicating status codes set as the result of the function execution.

Audience for This Manual

This manual is intended for cryptography system developers who may benefit from avoiding hand optimization of cryptography operations. In the Intel IPP primitive functions for cryptography, developers may find a convenient and easy-to-use building block resource that covers cryptography algorithms widely used for creating products that require data security and integrity.

The audience must have experience using C and a working knowledge of the vocabulary and principles of basic crypto algorithms.

Notational Conventions

In this manual, notational conventions include:

- Fonts used for distinction between the text and the code

- Naming conventions for different items.

Font Conventions

The following font conventions are used throughout this manual:

<i>This type style</i>	Mixed with the uppercase in function names, code examples, and call statements, for example, <code>ippsAdd_BNU</code> .
<i>This type style</i>	Parameters in function prototype parameters and parameters description, for example, <code>pCtx</code> , <code>pSrcMsg</code> .

Naming Conventions

The naming conventions for different items are the same as used by the Intel IPP software.

- All names of the functions used for cryptographic operations have the `ipps` prefix. In code examples, you can distinguish the Intel IPP interface functions from the application functions by this prefix.



NOTE. In this manual, the `ipps` prefix in function names is always used in code examples and function prototypes. In the text, this prefix is omitted when referring to the function group.

- Each new part of a function name starts with an uppercase character, without underscore, for example, `ippsDESInit`.

Online Version

This manual is available in an electronic format (Portable Document Format, or PDF). To obtain a hard copy of the manual, print the file using the printing capability of Adobe Acrobat*, the tool used for the online presentation of the document.

Symmetric Cryptography

Primitive Functions



In the context of secure data communication, symmetric cryptography primitive functions protect messages transferred over open communication media by offering adequate security strength to meet application security requirement, as well as algorithmic efficiency to enable secure communication in real time.

Intel® Integrated Performance Primitives (Intel® IPP) for cryptography offer operations using the following symmetric cryptography algorithms:

- DES, Triple DES (TDES) [[FIPS PUB 46-3](#)], Rijndael [[AES](#)], Twofish [[TF](#)], Blowfish [[BF](#)] and RC5* [[RC5](#)] block ciphers
- ARC4 stream cipher, described, for example, in [[AC](#)] and producing the same encryption/decryption as the RC4* proprietary cipher of RSA Security Inc.

Symmetric cryptography primitive functions implemented in Intel IPP are accordingly divided into the following groups: [DES/TDES Functions](#), [Rijndael Functions](#), [Blowfish Functions](#), [Twofish Functions](#), [RC5* Functions](#), and [ARC4 Functions](#).

Block Cipher Modes of Operation

Most of Symmetric Cryptography Algorithms implemented in Intel IPP are Block Ciphers, which operate on data blocks of the fixed size. Block Ciphers encrypt a plaintext block into a ciphertext block or decrypts a ciphertext block into a plaintext block. The size of the data blocks depends on the specific algorithm. [Table 2-1](#) shows the correspondence between Block Ciphers applied and their data block size.

Table 2-1 Block Sizes in Symmetric Algorithms

Block Cipher Name	Data Block Size (bits)
DES	64
TDES	64
Rijndael128	128
Rijndael192	192

Table 2-1 **Block Sizes in Symmetric Algorithms** (continued)

Block Cipher Name	Data Block Size (bits)
Rijndael256	256
Twofish	128
Blowfish	64
RC5	64 or 128

Block Cipher modes of executing the operation of encryption/decryption are applied in practice more frequently than “pure” Block Ciphers. On one hand, the modes enable you to process arbitrary length data stream. On the other hand, they provide additional security strength.

Intel IPP for cryptography supports five widely used modes, as specified in [\[NIST SP 800-38A\]](#):

- Electronic Code Book (ECB) mode
- Cipher Block Chain (CBC) mode
- Cipher Feedback (CFB) mode
- Output Feedback (OFB) mode
- Counter (CTR) mode.



NOTE. For simplicity and consistency, the mathematical expression and pseudo code in this chapter describes the behaviour of each function.

The cryptographic functions described in this chapter require the application to specify both the plaintext message and the ciphertext message lengths as multiples of block size of the respective algorithm (see [Table 2-1](#)). To meet this requirement in ciphering the message, the application may use any padding scheme, for example, the scheme defined in [\[PKCS7\]](#). In case padding is used, the application is responsible for correct interpretation and processing of the last deciphered message block. So of the three padding schemes available for earlier releases,

```
typedef enum {
    NONE = 0, IppsCPPaddingNONE = 0,
    PKCS7 = 1, IppsCPPaddingPKCS7 = 1,
    ZEROS = 2, IppsCPPaddingZEROS = 2
} IppsCPPadding;
```

only IppsCPPaddingNONE remains acceptable.

DES/TDES Functions

Data Encryption Standard (DES) is a well-known symmetric cipher and also the first modern commercial-grade algorithm with open and fully specified implementation details. DES consists of a Feistel network iterated 16 times with the block size of 64 bits and the effective key size of 56 bits.

Triple Data Encryption Standard (TDES) is a revised symmetric algorithm scheme built on the DES system. TDES encryption process includes three consecutive DES operations in the encryption, decryption, and encryption (E-D-E) sequence again in accordance with the American standard FIPS 46-3.

Although the functions that support TDES operations require three sets of round keys, the functions can operate under TDES cipher system with a two-set round keys by simply setting the third set of round keys to be the same as the first set.

You can use the functions described in this section for performing various operational modes under the DES/TDES cipher systems.

[Table 2-2](#) lists Intel IPP DES/TDES functions:

Table 2-2 Intel IPP DES/TDES Functions

Function Base Name	Operation
DES Functions	
DESGetSize	Gets the size of the IppsDESSpec context.
DESInit	Initializes user-supplied memory as IppsDESSpec context for future use.
DESEncryptECB	Encrypts a variable length data stream in the ECB mode.
DESDecryptECB	Decrypts a variable length data stream in the ECB mode.
DESEncryptCBC	Encrypts a variable length data stream in the CBC mode.
DESDecryptCBC	Decrypts a variable length data stream in the CBC mode.
DESEncryptCFB	Encrypts a variable length data stream in the CFB mode.
DESDecryptCFB	Decrypts a variable length data stream in the CFB mode.
DESEncryptOFB	Encrypts a variable length data stream in the OFB mode.
DESDecryptOFB	Decrypts a variable length data stream in the OFB mode.
DESEncryptCTR	Encrypts a variable length data stream in the CTR mode.
DESDecryptCTR	Decrypts a variable length data stream in the CTR mode.
TDES Functions	
TDESEncryptECB	Encrypts variable length data stream in the ECB mode.
TDESDecryptECB	Decrypts variable length data stream in the ECB mode.

Table 2-2 Intel IPP DES/TDES Functions (continued)

Function Base Name	Operation
TDESEncryptCBC	Encrypts variable length data stream in the CBC mode.
TDESDecryptCBC	Decrypts variable length data stream in the CBC mode.
TDESEncryptCFB	Encrypts variable length data stream in the CFB mode.
TDESDecryptCFB	Decrypts variable length data stream in the CFB mode.
TDESEncryptOFB	Encrypts variable length data stream in the OFB mode.
TDESDecryptOFB	Decrypts variable length data stream in the OFB mode.
TDESEncryptCTR	Encrypts a variable length data stream in the CTR mode.
TDESDecryptCTR	Decrypts a variable length data stream in the CTR mode.
ARCFourDecrypt	Decrypts a variable length data stream according to ARCFour.
ARCFourReset	Resets the <code>IppsARCFourState</code> context to the initial state.



NOTE. Intel IPP functions for cryptography operations do not allocate memory internally. The function `GetSize` does not require allocated memory. You need to call the function `GetSize` to find out how much available memory you need to have to work with the selected algorithm and after that you call the initialization function to create a memory buffer and initialize it.

Intel IPP for cryptography supports ECB, CBC, CFB, and CTR modes. You can tell which algorithm a given function supports from the function base name, for example, the function `DESEncryptECB` operates under the ECB mode for DES encryption and the function `TDESEncryptECB` operates under the ECB mode under the TDES scheme.

The encryption functions [DESEncryptCBC](#) and [TDESEncryptCBC](#) operate under the CBC mode using their respective cipher scheme and require to have an initialization vector *iv*. Since there exists a number of ways to initialize the initialization vector *iv*, you should remember which of these ways you used to be able to decrypt the message when needed.

Functions [DESEncryptCFB](#) and [TDESEncryptCFB](#) operate under CFB mode for encryption using their respective cipher scheme, both require having the initialization vector *pIV*, and CFB block size *cfbBlkSize*.

All functions described in this section use the context `IppsDESSpec` to serve as an operational vehicle that carries a set of round keys.

The application code for conducting a typical encryption under CBC mode using the TDES scheme must perform the following sequence of operations:

1. Get the size required to configure the context `IppsDESSpec` by calling the function [DESGetSize](#).
2. Call operating system memory allocation service function to allocate three buffers whose sizes are not less than the one specified by the function `DESGetSize`. Initialize pointers to contexts `pCtx1`, `pCtx2`, and `pCtx3` by calling the function [DESInit](#) three times, each with the allocated buffer and the respective DES key.
3. Specify the initialization vector and then call the function [TDESEncryptCBC](#) to encrypt the input data stream under CBC mode using TDES scheme.
4. Free the memory allocated to the buffer once TDES encryption under the CBC mode has been completed and the data structures allocated for set of round keys are no longer required.



NOTE. Similar procedure can be applied for ECB, CFB, and CTR mode operation.

DESGetSize

Gets the size of the `IppsDESSpec` context.

Syntax

```
IppStatus ippsDESGetSize(int* pSize)
```

Parameters

`pSize` Pointer to the `IppsDESSpec` context size value.

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsDESSpec` context size in bytes and stores it in `*pSize`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

DESInit

Initializes user-supplied memory as the IppsDESSpec context for future use.

Syntax

```
IppStatus ippsDESInit(const Ipp8u* pKey, IppsDESSpec* pCtx);
```

Parameters

<i>pKey</i>	Pointer to the DES key.
<i>pCtx</i>	Pointer to the IppsDESSpec context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as IppsDESSpec context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

DESEncryptECB

Encrypts a variable length data stream in the ECB mode.

Syntax

```
IppStatus ippsDESEncryptECB(const Ipp8u* pSrc, Ipp8u* pDst, int srcLen,  
                             const IppsDESSpec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.

<i>pCtx</i>	Pointer to the <code>DECSpec</code> context.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippscp.h` file. The function encrypts the input data stream of a variable length according to the cipher scheme specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by cipher block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DESDecryptECB

Decrypts a variable length data stream in the ECB mode.

Syntax

```
IppStatus ippsDESDecryptECB(const Ipp8u* pSrc, Ipp8u* pDst, int srclen,  
    const IppsDESSpec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream of variable length.
<i>srclen</i>	Length of the ciphertext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsDESSpec</code> context.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the ECB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <code>srclen</code> is not divisible by cipher block size.

DESEncryptCBC

Encrypts a variable length data stream in the CBC mode.

Syntax

```
IppStatus ippDESEncryptCBC(const Ipp8u* pSrc, Ipp8u* pDst, int srclen,
                           const IppsDESSpec* pCtx, const Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<code>pSrc</code>	Pointer to the input plaintext data stream of variable length.
<code>pDst</code>	Pointer to the resulting ciphertext data stream.
<code>srclen</code>	Length of the plaintext data stream length in bytes.
<code>pCtx</code>	Pointer to the <code>IppsDESSpec</code> context.
<code>pIV</code>	Pointer to the initialization vector for the CBC mode operation.
<code>padding</code>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the CBC mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <code>srclen</code> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DESDecryptCBC

Decrypts a variable length data stream in the CBC mode.

Syntax

```
IppStatus ippDESDecryptCBC(const Ipp8u* pSrc, Ipp8u* pDst, int srclen,  
                           const IppsDESSpec* pCtx, const Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<code>pSrc</code>	Pointer to the input ciphertext data stream.
<code>pDst</code>	Pointer to the resulting plaintext data stream of the variable length.
<code>srclen</code>	Length of the ciphertext data stream length in bytes.
<code>pCtx</code>	Pointer to the <code>IppsDESSpec</code> context.
<code>pIV</code>	Pointer to the initialization vector for CBC mode operation.
<code>padding</code>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the CBC mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <code>srcLen</code> is not divisible by cipher block size.

DESEncryptCFB

Encrypts a variable length data stream in the CFB mode.

Syntax

```
IppStatus ippSDESEncryptCFB(const Ipp8u* pSrc, Ipp8u* pDst, int srcLen, int
    cfbBlkSize, const IppsDESSpec* pCtx, const Ipp8u* pIV, IppsCPPadding
    padding);
```

Parameters

<code>pSrc</code>	Pointer to the input plaintext data stream of variable length.
<code>pDst</code>	Pointer to the resulting ciphertext data stream.
<code>srcLen</code>	Length of the plaintext data stream in bytes.
<code>cfbBlkSize</code>	Size of the CFB block in bytes.
<code>pCtx</code>	Pointer to the <code>IppsDESSpec</code> context.
<code>pIV</code>	Pointer to the initialization vector for the CFB mode operation.
<code>padding</code>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of variable length according to the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <code>srcLen</code> is not divisible by <code>cfbBlkSize</code> parameter value.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <code>cfbBlkSize</code> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DESDecryptCFB

Decrypts a variable length data stream in the CFB mode.

Syntax

```
IppStatus ippDESDecryptCFB(const Ipp8u* pSrc, Ipp8u* pDst, int srclen,  
    int cfbBlkSize, const IppsDESSpec* pCtx, const Ipp8u* pIV,  
    IppsCPPadding padding);
```

Parameters

<code>pSrc</code>	Pointer to the input ciphertext data stream.
<code>pDst</code>	Pointer to the resulting plaintext data stream of variable length.
<code>srclen</code>	Length of the ciphertext data stream in bytes.
<code>cfbBlkSize</code>	Size of the CFB block in bytes.
<code>pCtx</code>	Pointer to the <code>IppsDESSpec</code> context.
<code>pIV</code>	Pointer to the initialization vector for the CFB mode operation.

padding IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of variable length according to the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <i>cfbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by cipher block size.

DESEncryptOFB

Encrypts a variable length data stream according to the DES algorithm in the OFB mode.

Syntax

```
IppStatus ippDESEncryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int srclen, int
    ofbBlkSize, const IppsDESSpec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>len</i>	Length of the plaintext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the IppsDESSpec context.

pIV Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DESDecryptOFB

Decrypts a variable length data stream according to the DES algorithm in the OFB mode.

Syntax

```
IppStatus ippDESDecryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int srclen, int  
    ofbBlkSize, const IppsDESSpec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>srclen</i>	Length of the ciphertext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsDESSpec</code> context.

pIV Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DESEncryptCTR

Encrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippSDESEncryptCTR(const Ipp8u* pSrc, Ipp8u* pDst, int srcLen,
                             const IppsDESSpec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of a variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsDESSpec</code> context.
<i>pCtrValue</i>	Pointer to the counter data block.

ctrNumBitSize Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the CTR mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <i>ctrNumBitSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DESDecryptCTR

Decrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippDESDecryptCTR(const Ipp8u* pSrc, Ipp8u* pDst, int srcLen,  
                           const IppsDESSpec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream.
<i>pDst</i>	Pointer to the resulting plaintext data stream of a variable length.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsDESSpec</code> context.
<i>pCtrValue</i>	Pointer to the counter data block.
<i>ctrNumBitSize</i>	Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the CTR mode as specified in the [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsCTRSIZEErr</code>	Indicates an error condition if the value of the <code>ctrNumBitSize</code> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

TDESEncryptECB

Encrypts variable length data stream in ECB mode.

Syntax

```
ippStatus ippSTDESEncryptECB(const Ipp8u *pSrc, Ipp8u *pDst, int srclen,
    const IppsDESSpec *pCtx1, const IppsDESSpec *pCtx2, const IppsDESSpec
    *pCtx3, IppsCPPadding padding);
```

Parameters

<code>pSrc</code>	Input plaintext data stream of a variable length.
<code>pDst</code>	Resulting ciphertext data stream.
<code>srclen</code>	Input data stream length in bytes.
<code>pCtx1</code>	First set of round keys scheduled for TDES internal operations.
<code>pCtx2</code>	Second set of round keys scheduled for TDES internal operations.
<code>pCtx3</code>	Third set of round keys scheduled for TDES internal operations.
<code>padding</code>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the cipher scheme specified in [\[NIST SP 800-38A\]](#). The function uses three sets of supplied round keys in the ECB mode. The function returns the ciphertext result.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if the input data stream length is not divisible by cipher block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

TDESDecryptECB

Decrypts variable length data stream in the ECB mode.

Syntax

```
IppStatus ippSTDESDecryptECB(const Ipp8u *pSrc, Ipp8u *pDst, int srclen,  
    const IppsDESSpec *pCtx1, const IppsDESSpec *pCtx2, const IppsDESSpec  
    *pCtx3, IppsCPPadding padding);
```

Parameters

<code>pSrc</code>	Input ciphertext data stream of variable length.
<code>pDst</code>	Resulting plaintext data stream.
<code>srclen</code>	Input data stream length in bytes.
<code>pCtx1</code>	First set of round keys scheduled for TDES internal operations.
<code>pCtx2</code>	Second set of round keys scheduled for TDES internal operations.
<code>pCtx3</code>	Third set of round keys scheduled for TDES internal operations.

padding IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the cipher scheme specified in [\[NIST SP 800-38A\]](#). The function uses three sets of supplied round keys in the ECB mode. The function returns the ciphertext result and validates the final plaintext block.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the decrypted plaintext data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by cipher block size.

TDESEncryptCBC

Encrypts variable length data stream in the CBC mode.

Syntax

```
IppStatus ippdTDESEncryptCBC(const Ipp8u *pSrc, Ipp8u *pDst, int srclen,
                             const IppsDESSpec *pCtx1, const IppsDESSpec *pCtx2, const IppsDESSpec
                             *pCtx3, const Ipp8u *pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Input plaintext data stream of a variable length.
<i>pDst</i>	Resulting ciphertext data stream.
<i>pIV</i>	Initialization vector for TDES CBC mode operation.
<i>srclen</i>	Input data stream length in bytes.
<i>pCtx1</i>	First set of round keys scheduled for TDES internal operations.
<i>pCtx2</i>	Second set of round keys scheduled for TDES internal operations.
<i>pCtx3</i>	Third set of round keys scheduled for TDES internal operations.
<i>padding</i>	IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the cipher scheme specified in [\[NIST SP 800-38A\]](#). The function uses three sets of the supplied round keys in the Cipher Block Chaining (CBC) mode with the initialization vector. The function returns the ciphertext result.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if the input data stream length is not divisible by cipher block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

TDESDecryptCBC

Decrypts variable length data stream in the CBC mode.

Syntax

```
IppStatus ippstDESDecryptCBC(const Ipp8u *pSrc, Ipp8u *pDst, int srclen,  
    const IppsDESSpec *pCtx1, const IppsDESSpec *pCtx2, const IppsDESSpec  
    *pCtx3, const Ipp8u *pIV, IppsCPPadding padding);
```

Parameters

<code>pSrc</code>	Input ciphertext data stream of a variable length.
<code>pDst</code>	Resulting plaintext data stream.
<code>pIV</code>	Initialization vector for TDES CBC mode operation.
<code>srclen</code>	Input data stream length in bytes.
<code>pCtx1</code>	First set of round keys scheduled for TDES internal operations.
<code>pCtx2</code>	Second set of round keys scheduled for TDES internal operations.

<i>pCtx3</i>	Third set of round keys scheduled for TDES internal operations.
<i>padding</i>	IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the cipher scheme specified in [\[NIST SP 800-38A\]](#). The function uses three sets of the supplied round keys in the Cipher Block Chaining (CBC) mode with the initialization vector. The function returns the ciphertext result and validates the final plaintext block.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the decrypted plaintext data stream length is less than or equal to zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by cipher block size.

TDESEncryptCFB

Encrypts variable length data stream in the CFB mode.

Syntax

```
ippStatus ippstTDESEncryptCFB(const Ipp8u *pSrc, Ipp8u *pDst, int srclen,
    int cfbBlkSize, const IppsDESSpec *pCtx1, const IppsDESSpec *pCtx2,
    const IppsDESSpec *pCtx3, const Ipp8u *pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Input plaintext data stream of variable length.
<i>pDst</i>	Resulting ciphertext data stream.
<i>pIV</i>	Initialization vector for TDES CFB mode operation.

<code>srclen</code>	Input data stream length in bytes.
<code>pCtx1</code>	First set of round keys scheduled for TDES internal operations.
<code>pCtx2</code>	Second set of round keys scheduled for TDES internal operations.
<code>pCtx3</code>	Third set of round keys scheduled for TDES internal operations.
<code>cfbBlkSize</code>	CFB block size in bytes.
<code>padding</code>	IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the cipher scheme specified in [\[NIST SP 800-38A\]](#). The function uses three sets of the supplied round keys in the Cipher Feedback (CFB) mode with the initialization vector. The function returns the ciphertext result.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <code>srclen</code> is not divisible by <code>cfbBlkSize</code> parameter value.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <code>cfbBlkSize</code> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

TDESDecryptCFB

Decrypts variable length data stream in the CFB mode.

Syntax

```
IppStatus ippstDESDecryptCFB(const Ipp8u *pSrc, Ipp8u *pDst, int srclen,
                             int cfbBlkSize, const IppsDESSpec *pCtx1, const IppsDESSpec *pCtx2,
                             const IppsDESSpec *pCtx3, const Ipp8u *pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Input ciphertext data stream of variable length.
<i>pDst</i>	Resulting plaintext data stream.
<i>pIV</i>	Initialization vector for TDES CFB mode operation.
<i>srcLen</i>	Ciphertext data stream length in bytes.
<i>pCtx1</i>	First set of round keys scheduled for TDES internal operations.
<i>pCtx2</i>	Second set of round keys scheduled for TDES internal operations.
<i>pCtx3</i>	Third set of round keys scheduled for TDES internal operations.
<i>cfbBlkSize</i>	CFB block size in bytes.
<i>padding</i>	IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the cipher scheme specified in [\[NIST SP 800-38A\]](#). The function uses three sets of the supplied round keys in the Cipher Feedback (CFB) mode with the initialization vector. The function returns the ciphertext result and validates the final plaintext block.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the decrypted plaintext data stream length is less than or equal to zero.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <i>cfbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by cipher block size.

TDESEncryptOFB

Encrypts a variable length data stream according to the TDES algorithm in the OFB mode.

Syntax

```
IppStatus ippSTDESEncryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int srclen, int  
    ofbBlkSize, const IppsDESSpec *pCtx1, const IppsDESSpec *pCtx2, const  
    IppsDESSpec *pCtx3, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the plaintext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx1</i>	First set of round keys scheduled for TDES internal operations.
<i>pCtx2</i>	Second set of round keys scheduled for TDES internal operations.
<i>pCtx3</i>	Third set of round keys scheduled for TDES internal operations.
<i>pIV</i>	Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.

<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
------------------------------------	---

TDSEDecryptOFB

Decrypts a variable length data stream according to the TDES algorithm in the OFB mode.

Syntax

```
IppStatus ippSTDESDecryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int srclen, int
    ofbBlkSize, const IppsDESSpec *pCtx1, const IppsDESSpec *pCtx2, const
    IppsDESSpec *pCtx3, Ipp8u* pIV);
```

Parameters

<code>pSrc</code>	Pointer to the input ciphertext data stream of variable length.
<code>pDst</code>	Pointer to the resulting plaintext data stream.
<code>srclen</code>	Length of the ciphertext data stream in bytes.
<code>ofbBlkSize</code>	Size of the OFB block in bytes.
<code>pCtx1</code>	First set of round keys scheduled for TDES internal operations.
<code>pCtx2</code>	Second set of round keys scheduled for TDES internal operations.
<code>pCtx3</code>	Third set of round keys scheduled for TDES internal operations.
<code>pIV</code>	Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.

<code>ippStsUnderRunErr</code>	Indicates an error condition if <code>srclen</code> is not divisible by the <code>ofbBlkSize</code> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <code>ofbBlkSize</code> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

TDESEncryptCTR

Encrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippdTDESEncryptCTR(const Ipp8u *pSrc, Ipp8u *pDst, int srclen,  
    const IppsDESSpec *pCtx1, const IppsDESSpec *pCtx2, const IppsDESSpec  
    *pCtx3, Ipp8u *pCtrValue, int ctrNumBitSize);
```

Parameters

<code>pSrc</code>	Input plaintext data stream of a variable length.
<code>pDst</code>	Resulting ciphertext data stream.
<code>srclen</code>	Input data stream length in bytes.
<code>pCtx1</code>	First set of round keys scheduled for TDES internal operations.
<code>pCtx2</code>	Second set of round keys scheduled for TDES internal operations.
<code>pCtx3</code>	Third set of round keys scheduled for TDES internal operations.
<code>pCtrValue</code>	Counter.
<code>ctrNumBitSize</code>	Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function `TDESEncryptCTR` encrypts the input data stream of a variable length according to the cipher scheme specified in the [\[NIST SP 800-38A\]](#) recommendation. The function uses three sets of the supplied round keys. The standard incrementing function is applied to increment counter value. The function returns the ciphertext result.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <i>ctrNumBitSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

TDESDecryptCTR

Decrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippSTDESDecryptCTR(const Ipp8u *pSrc, Ipp8u *pDst, int srcLen,
    const IppsDESSpec *pCtx1, const IppsDESSpec *pCtx2, const IppsDESSpec
    *pCtx3, Ipp8u *pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Input ciphertext data stream of a variable length.
<i>pDst</i>	Resulting plaintext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>pCtx1</i>	First set of round keys scheduled for TDES internal operations.
<i>pCtx2</i>	Second set of round keys scheduled for TDES internal operations.
<i>pCtx3</i>	Third set of round keys scheduled for TDES internal operations.
<i>pCtrValue</i>	Counter.
<i>ctrNumBitSize</i>	Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function `TDESDecryptCTR` decrypts the input data stream of a variable length according to the cipher scheme specified in the [\[NIST SP 800-38A\]](#) recommendation. The function uses three sets of the supplied round keys. The standard incrementing function is applied to increment value of counter. The function returns the ciphertext result.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the descrypted plaintext data stream length is less that or equal to zero.
<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <i>ctrNumBitSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Example 2-1 DES/TDES Encryption and Decryption

```
// use of the ECB mode
void DES_sample(void){
    // size of the DES algorithm block is equal to 8
    const int desBlkSize = 8;

    // get size of the context needed for the encryption/decryption operation
    int ctxSize;
    ippDESGetSize(&ctxSize);
    // and allocate one
    IppsDESSpec* pCtx = (IppsDESSpec*)( new Ipp8u [ctxSize] );
```

Example 2-1 DES/TDES Encryption and Decryption (continued)

```
// define the key
Ipp8u key[] = {0x01,0x2,0x3,0x4,0x5,0x6,0x7,0x8};
// and prepare the context for the DES usage
ippsDESInit(key, pCtx);
// define the message to be encrypted
Ipp8u ptext[] = {"quick brown fox jum over lazy dog"};

// allocate enough memory for the ciphertext
// note that
// the size of ciphertext is always multiple of cipher block size
Ipp8u ctext[(sizeof(ptext)+desBlkSize-1) & ~(desBlkSize-1)];
// encrypt (ECB mode) ptext message
// pay attention to the 'length' parameter
// it defines the number of bytes to be encrypted
ippsDESEncryptECB(ptext, ctext, sizeof(ctext),
                  pCtx,
                  IppsCPPaddingNONE);

// allocate memory for the decrypted message
Ipp8u rtext[sizeof(ctext)];
// decrypt (ECB mode) ctext message
// pay attention to the 'length' parameter
// it defines the number of bytes to be decrypted
ippsDESDecryptECB(ctext, rtext, sizeof(ctext),
                  pCtx,
                  IppsCPPaddingNONE);

delete (Ipp8u*)pCtx;
}
```

Rijndael Functions

Rijndael cipher scheme is an iterated block cipher with a variable block size and a variable key length. You can independently specify the lengths of the data block and the key as 128, 192, or 256 bits.

This section describes the functions operating in various operational modes under the various Rijndael cipher systems. The functions in this section are categorized by their data block sizes of the baseline Rijndael cipher functions:

- `Rijndael128` refers to the Rijndael cipher scheme with 128-bit data block size
- `Rijndael192` refers to the Rijndael cipher scheme with 192-bit data block size
- `Rijndael256` refers to the Rijndael cipher scheme with 256-bit data block size.

To specify the key length for these baseline Rijndael cipher schemes, all the functions in this section use the following enumeration

```
typedef enum {  
    IppsRijndaelKey128 = 128, // 128-bit key  
    IppsRijndaelKey192 = 192, // 192-bit key  
    IppsRijndaelKey256 = 256, // 256-bit key  
} IppsRijndaelKeyLength;
```

The functions for Rijndael128 with the 128-bit key length described in this section are, in fact, American Encryption Standard (AES) cipher functions implemented in the way to comply with the American Standard FIPS 197. All other functions for various other Rijndael block cipher schemes fully comply to the respective cipher schemes documented by Joan Daeman and Vincent Rijmen.

[Table 2-3](#) lists Intel IPP Rijndael functions:

Table 2-3 Intel IPP Rijndael Algorithm Functions

Function Base Name	Operation
Rijndael128GetSize	Gets the size of the <code>IppsRijndael128Spec</code> context.
Rijndael128Init	Initializes user-supplied memory as <code>IppsRijndael128Spec</code> context for future use.
Rijndael128EncryptECB	Encrypts plaintext message using Rijndael128 algorithm in the ECB encryption mode.
Rijndael128DecryptECB	Decrypts byte data stream using Rijndael128 algorithm in the ECB mode.
Rijndael128EncryptCBC	Encrypts byte data stream according to Rijndael128 in the CBC mode.

Table 2-3 Intel IPP Rijndael Algorithm Functions (continued)

Function Base Name	Operation
<u>Rijndael128DecryptCBC</u>	Decrypts byte data stream according to Rijndael128 in the CBC mode.
<u>Rijndael128EncryptCFB</u>	Encrypts byte data stream according to Rijndael128 in the CFB mode.
<u>Rijndael128DecryptCFB</u>	Decrypts byte data stream according to Rijndael128 in the CFB mode.
<u>Rijndael128EncryptOFB</u>	Encrypts byte data stream according to Rijndael128 in the OFB mode.
<u>Rijndael128DecryptOFB</u>	Decrypts byte data stream according to Rijndael128 in the OFB mode.
<u>Rijndael128EncryptCTR</u>	Encrypts a variable length data stream according to Rijndael128 in the CTR mode.
<u>Rijndael128DecryptCTR</u>	Decrypts a variable length data stream according to Rijndael128 in the CTR mode.
<u>Rijndael128EncryptCCM</u>	Encrypts a variable length data stream and generates its authentication tag in the CCM mode.
<u>Rijndael128DecryptCCM</u>	Decrypts and verifies a variable length data stream in the CCM mode.
<u>Rijndael128EncryptCCM_u8</u>	Encrypts a variable length data stream and generates its authentication tag in the CCM mode using enhanced interface.
<u>Rijndael128DecryptCCM_u8</u>	Decrypts and verifies a variable length data stream in the CCM mode using enhanced interface.
<u>Rijndael192GetSize</u>	Gets the size of the IppsRijndael192Spec context.
<u>Rijndael192Init</u>	Initializes user-supplied memory as IppsRijndael192Spec context for future use.
<u>Rijndael192EncryptECB</u>	Encrypts a byte data stream according to Rijndael192 in the ECB mode.
<u>Rijndael192DecryptECB</u>	Decrypts byte data stream according to Rijndael192 in the ECB mode.
<u>Rijndael192EncryptCBC</u>	Encrypts a byte data stream according to Rijndael192 in the CBC mode.
<u>Rijndael192DecryptCBC</u>	Decrypts a byte data stream according to Rijndael192 in the CBC mode.
<u>Rijndael192EncryptCFB</u>	Encrypts a byte data stream according to Rijndael192 in the CFB mode.
<u>Rijndael192DecryptCFB</u>	Decrypts a byte data stream according to Rijndael192 in the CFB mode.
<u>Rijndael192EncryptOFB</u>	Encrypts a byte data stream according to Rijndael192 in the OFB mode.

Table 2-3 Intel IPP Rijndael Algorithm Functions (continued)

Function Base Name	Operation
<u>Rijndael192DecryptOFB</u>	Decrypts a byte data stream according to Rijndael192 in the OFB mode.
<u>Rijndael192EncryptCTR</u>	Encrypts a variable length data stream according to Rijndael192 in the CTR mode.
<u>Rijndael192DecryptCTR</u>	Decrypts a variable length data stream according to Rijndael192 in the CTR mode.
<u>Rijndael256GetSize</u>	Gets the size of the <code>Rijndael256Spec</code> context.
<u>Rijndael256Init</u>	Initializes user-supplied memory as <code>IppsRijndael256Spec</code> context for future use.
<u>Rijndael256EncryptECB</u>	Encrypts a byte data stream according to Rijndael256 in the ECB mode.
<u>Rijndael256DecryptECB</u>	Decrypts a byte data stream according to Rijndael256 in the ECB mode.
<u>Rijndael256EncryptCBC</u>	Encrypts byte data stream according to Rijndael256 in the CBC mode.
<u>Rijndael256DecryptCBC</u>	Decrypts byte data stream according to Rijndael256 in the CBC mode.
<u>Rijndael256EncryptCFB</u>	Encrypts byte data stream according to Rijndael256 in the CFB mode.
<u>Rijndael256DecryptCFB</u>	Decrypts byte data stream according to Rijndael256 in the CFB mode.
<u>Rijndael256EncryptOFB</u>	Encrypts byte data stream according to Rijndael256 in the OFB mode.
<u>Rijndael256DecryptOFB</u>	Decrypts byte data stream according to Rijndael256 in the OFB mode.
<u>Rijndael256EncryptCTR</u>	Encrypts a variable length data stream according to Rijndael256 in the CTR mode.
<u>Rijndael256DecryptCTR</u>	Decrypts a variable length data stream according to Rijndael256 in the CTR mode.

Throughout this section, the functions for Rijndael128 baseline cipher schemes employ the context `IppsRijndael128Spec`, the functions for Rijndael192 baseline cipher schemes employ the context `IppsRijndael192Spec`, and the functions for Rijndael256 baseline cipher schemes employ the context `IppsRijndael256Spec`. They serve as operational vehicles not only to carry a set of round keys and a set of round inverse keys at the same time, but also the key management information.

Once the respective initialization function generates the round keys, the functions for ECB, CBC, CFB, and other modes are ready for the execution of either encrypting or decrypting the streaming data with the specified padding scheme.

The application code for conducting a typical encryption under CBC mode using the AES scheme, that is, the Rijndael128 with a 128-bit key, should follow the sequence of operations as outlined below:

1. Get the size required to configure the context `IppsRijndael128Spec` by calling the function [Rijndael128GetSize](#).
2. Call the operating system memory-allocation service function to allocate a buffer whose size is no less than the one specified by the function `Rijndael128GetSize`.
3. Initialize the context `IppsRijndael128Spec *pCtx` by calling the function [Rijndael128Init](#) with the allocated buffer and the respective 128-bit AES key.
4. Specify the initialization vector and the padding scheme, then call the function [Rijndael128EncryptCBC](#) to encrypt the input data stream using the AES encryption function with CBC mode.
5. Call the operating system memory free service function to release the buffer allocated for the context `IppsRijndael128Spec`, if needed.

Rijndael128GetSize

Gets the size of the `IppsRijndael128Spec` context.

Syntax

```
IppStatus ippsRijndael128GetSize(int* pSize)
```

Parameters

pSize Pointer to the `IppsRijndael128Spec` context size value.

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsRijndael128Spec` context size in bytes and stores it in **pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

Rijndael128Init

*Initializes user-supplied memory as
IppsRijndael128Spec context for future use.*

Syntax

```
IppStatus ippsRijndael128Init(const Ipp8u *pKey, IppsRijndaelKeyLength  
    keylen, IppsRijndael128Spec* pCtx);
```

Parameters

<i>pKey</i>	Pointer to the Rijndael128 key.
<i>keylen</i>	Key byte stream length in bytes defined by the IppsRijndaelKeyLength enumerator.
<i>pCtx</i>	Pointer to the IppsRijndael128Spec context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as `IppsRijndael128Spec`. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Returns an error condition if <i>keyLen</i> is not <code>IppsRijndaelKey128</code> , <code>IppsRijndaelKey192</code> or <code>IppsRijndaelKey256</code> .

Rijndael128EncryptECB

*Encrypts plaintext message by using ECB
encryption mode.*

Syntax

```
IppStatus ippsRijndael128EncryptECB(const Ipp8u *pSrc, Ipp8u *pDst, int  
    srclen, const IppsRijndael128Spec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the input plaintext data in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael128Spec</code> context.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the cipher scheme specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by cipher block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael128DecryptECB

Decrypts byte data stream by using Rijndael algorithm in the ECB mode.

Syntax

```
IppStatus ippRijndael128DecryptECB(const Ipp8u* pSrc, Ipp8u* pDst, int
    srclen, const IppsRijndael128Spec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream of variable length.

<code>srclen</code>	Length of the ciphertext data stream in bytes.
<code>pCtx</code>	Pointer to the <code>IppsRijndael128Spec</code> context.
<code>padding</code>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the ECB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <code>srclen</code> is not divisible by cipher block size.

Rijndael128EncryptCBC

Encrypts byte data stream according to Rijndael in the CBC mode.

Syntax

```
IppStatus ippRijndael128EncryptCBC(const Ipp8u* pSrc, Ipp8u* pDst, int srclen,  
    const IppsRijndael128Spec* pCtx, const Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<code>pSrc</code>	Pointer to the input plaintext data stream of variable length.
<code>pDst</code>	Pointer to the resulting ciphertext data stream.
<code>srclen</code>	Length of the plaintext data stream length in bytes.
<code>pCtx</code>	Pointer to the <code>IppsRijndael128Spec</code> context.
<code>pIV</code>	Pointer to the initialization vector for the CBC mode operation.

padding IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the CBC mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael128DecryptCBC

Decrypts byte data stream according to Rijndael in the CBC mode.

Syntax

```
ippStatus ippRijndael128DecryptCBC(const Ipp8u* pSrc, Ipp8u* pDst, int
    srclen, const IppsRijndael128Spec* pCtx, const Ipp8u* pIV,
    IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream.
<i>pDst</i>	Pointer to the resulting plaintext data stream of the variable length.
<i>srclen</i>	Length of the ciphertext data stream length in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael128Spec</code> context.
<i>pIV</i>	Pointer to the initialization vector for CBC mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the CBC mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by cipher block size.

Rijndael128EncryptCFB

Encrypts byte data stream according to Rijndael in the CFB mode.

```
IppStatus ippRijndael128EncryptCFB(const Ipp8u* pSrc, Ipp8u* pDst, int
    srcLen, int cfbBlkSize, const IppsRijndael128Spec* pCtx, const Ipp8u
    *pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>cfbBlkSize</i>	Size of the CFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael128Spec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the CFB mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of variable length according to the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <code>srcLen</code> is not divisible by <code>cfbBlkSize</code> parameter value.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <code>cfbBlkSize</code> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael128DecryptCFB

Decrypts byte data stream according to Rijndael in CFB mode.

Syntax

```
IppStatus ippRijndael128DecryptCFB(const Ipp8u* pSrc, Ipp8u* pDst, int
    srclen, int cfbBlkSize, const IppsRijndael128Spec* pCtx, const
    Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<code>pSrc</code>	Pointer to the input ciphertext data stream.
<code>pDst</code>	Pointer to the resulting plaintext data stream of variable length.
<code>srclen</code>	Length of the ciphertext data stream in bytes.
<code>cfbBlkSize</code>	Size of the CFB block in bytes.
<code>pCtx</code>	Pointer to the <code>IppsRijndael128Spec</code> context.
<code>pIV</code>	Pointer to the initialization vector for the CFB mode operation.

padding IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of variable length according to the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <i>cfbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by cipher block size.

Rijndael128EncryptOFB

Encrypts a variable length data stream according to Rijndael128 in the OFB mode.

Syntax

```
IppStatus ippRijndael128EncryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int  
    srcLen, int ofbBlkSize, const IppsRijndael128Spec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the IppsRijndael128Spec context.

pIV Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael128DecryptOFB

Decrypts a variable length data stream according to Rijndael128 in the OFB mode.

Syntax

```
IppStatus ippRijndael128DecryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int
    srclen, int ofbBlkSize, const IppsRijndael128Spec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>srclen</i>	Length of the ciphertext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael128Spec</code> context.

pIV Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael128EncryptCTR

Encrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippRijndael128EncryptCTR(const Ipp8u* pSrc, Ipp8u* pDst, int srcLen,  
    const IppsRijndael128Spec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of a variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael128Spec</code> context.
<i>pCtrValue</i>	Pointer to the counter data block.

ctrNumBitSize Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the CTR mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <i>ctrNumBitSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael128DecryptCTR

Decrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippRijndael128DecryptCTR(const Ipp8u* pSrc, Ipp8u* pDst, int srcLen,
    const IppsRijndael128Spec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream.
<i>pDst</i>	Pointer to the resulting plaintext data stream of a variable length.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael128Spec</code> context.
<i>pCtrValue</i>	Pointer to the counter data block.
<i>ctrNumBitSize</i>	Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the CTR mode as specified in the [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <code>ctrNumBitSize</code> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael128EncryptCCM

Encrypts a variable length data stream and generates its authentication tag in the CCM mode.

Syntax

```
IppStatus ippRijndael128EncryptCCM(const Ipp8u* pNonce, Ipp32u nonceLen, const Ipp8u* pAssc, Ipp64u asscLen, const Ipp8u* pSrc, Ipp64u srcLen, int macLen, Ipp8u* pDst, const IppsRijndael128Spec* pCtx);
```

Parameters

<code>pNonce</code>	Pointer to the nonce.
<code>nonceLen</code>	Length of the nonce <code>*pNonce</code> (in octets).
<code>pAssc</code>	Pointer to the associated data.
<code>asscLen</code>	Length of the associated data <code>*pAssc</code> (in octets).
<code>pSrc</code>	Pointer to the input plaintext data.
<code>srcLen</code>	Length of the input plaintext <code>*pSrc</code> (in octets).
<code>macLen</code>	Length of the authentication tag in octets.
<code>pDst</code>	Pointer to the resulting ciphertext data stream.

pCtx Pointer to the `IppsRijndael128Spec` context.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length and computes the authentication tag according to the Counter with Cipher Block Chaining-Message Authentication Code (CCM) mode, as specified in [\[NIST SP 800-38C\]](#). Unless you've successfully implemented the encryption with this function, you are encouraged to use the newer function [Rijndael128EncryptCCM_u8](#) having the same functionality.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the length input data does not meet any of the following conditions: $7 \leq \text{nonceLen} \leq 13$ $\text{asscLen} \geq 0$ $\text{srcLen} \geq 1$ $\text{macLen} = 2n, 1 \leq n \leq 8$.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael128DecryptCCM

Decrypts and verifies a variable length data stream in the CCM mode.

Syntax

```
IppStatus ippRijndael128DecryptCCM(const Ipp8u* pNonce, Ipp32u nonceLen, const
Ipp8u* pAssc, Ipp64u asscLen, const Ipp8u* pSrc, Ipp64u srcLen, int macLen,
Ipp8u* pDst, IppBool* pResult, const IppsRijndael128Spec* pCtx);
```

Parameters

<i>pNonce</i>	Pointer to the nonce of length.
<i>nonceLen</i>	Length of the nonce <i>pNonce</i> (in octets).
<i>pAssc</i>	Pointer to the associated data.

<i>asscLen</i>	Length of the associated data <i>*pAssc</i> (in octets).
<i>pSrc</i>	Pointer to the input ciphertext data.
<i>srcLen</i>	Length of the input ciphertext <i>*pSrc</i> (in octets).
<i>macLen</i>	Length of the authentication tag in octets.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>pResult</i>	Pointer to the result of verification.
<i>pCtx</i>	Pointer to the <code>IppsRijndael128Spec</code> context.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length and verifies the authentication tag according to the CCM mode, as specified in [\[NIST SP 800-38C\]](#). Unless you've successfully implemented the decryption with this function, you are encouraged to use the newer function [Rijndael128DecryptCCM_u8](#) having the same functionality.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the length input data does not meet any of the following conditions: $7 \leq \text{nonceLen} \leq 13$ $\text{srcLen} > \text{macLen}$ $\text{macLen} = 2n, 1 \leq n \leq 8$.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael128EncryptCCM_u8

Encrypts a variable length data stream and generates its authentication tag in the CCM mode.

Syntax

```
IppStatus ippRijndael128EncryptCCM_u8(const Ipp8u* pNonce, int nonceLen, const
    Ipp8u* pAssc, int asscLen, const Ipp8u* pSrc, int srcLen, int macLen, Ipp8u*
    pDst, const IppsRijndael128Spec* pCtx);
```

<i>pNonce</i>	Pointer to the nonce.
<i>nonceLen</i>	Length of the nonce <i>*pNonce</i> (in octets).
<i>pAssc</i>	Pointer to the associated data.
<i>asscLen</i>	Length of the associated data <i>*pAssc</i> (in octets).
<i>pSrc</i>	Pointer to the input plaintext data.
<i>srcLen</i>	Length of the input plaintext <i>*pSrc</i> (in octets).
<i>macLen</i>	Length of the authentication tag in octets.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>pCtx</i>	Pointer to the IppsRijndael128Spec context.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length and computes the authentication tag according to the CCM mode, as specified in [\[NIST SP 800-38C\]](#). Unlike [Rijndael128EncryptCCM](#), [Rijndael128EncryptCCM_u8](#) employs the `int` data type for *all* length parameters. Use this function rather than [Rijndael128EncryptCCM](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the length input data does not meet any of the following conditions: $7 \leq \text{nonceLen} \leq 13$ $\text{asscLen} \geq 0$ $\text{srcLen} \geq 1$ $\text{macLen} = 2n, 1 \leq n \leq 8$.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael128DecryptCCM_u8

Decrypts and verifies a variable length data stream in the CCM mode.

Syntax

```
IppStatus ippsRijndael128DecryptCCM_u8(const Ipp8u* pNonce, int nonceLen, const Ipp8u* pAssc, int asscLen, const Ipp8u* pSrc, int srcLen, int macLen, Ipp8u* pDst, IppBool* pResult, const IppsRijndael128Spec* pCtx);
```

Parameters

<i>pNonce</i>	Pointer to the nonce of length.
<i>nonceLen</i>	Length of the nonce <i>*pNonce</i> (in octets).
<i>pAssc</i>	Pointer to the associated data.
<i>asscLen</i>	Length of the associated data <i>*pAssc</i> (in octets).
<i>pSrc</i>	Pointer to the input ciphertext data.
<i>srcLen</i>	Length of the input ciphertext <i>*pSrc</i> (in octets).
<i>macLen</i>	Length of the authentication tag in octets.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>pResult</i>	Pointer to the result of verification.
<i>pCtx</i>	Pointer to the IppsRijndael128Spec context.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length and verifies the authentication tag according to the CCM mode, as specified in [\[NIST SP 800-38C\]](#). Unlike [Rijndael128DecryptCCM](#), `Rijndael128DecryptCCM_u8` employs the `int` data type for *all* length parameters. Use this function rather than `Rijndael128DecryptCCM`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsLengthErr</code>	Indicates an error condition if the length input data does not meet any of the following conditions: $7 \leq \text{nonceLen} \leq 13$ $\text{asscLen} \geq 0$ $\text{srcLen} > \text{macLen}$ $\text{macLen} = 2n, 1 \leq n \leq 8$.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael192GetSize

Gets the size of the IppsRijndael192Spec context.

Syntax

```
IppStatus ippRijndael192GetSize(int* pSize)
```

Parameters

pSize Pointer to the IppsRijndael192Spec context size value.

Description

This function is declared in the `ippcp.h` file. The function gets the IppsRijndael192Spec context size in bytes and stores it in **pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

Rijndael192Init

Initializes user-supplied memory as the IppsRijndael192Spec context for future use.

Syntax

```
IppStatus ippsRijndael192Init(const Ipp8u *pKey, IppsRijndaelKeyLength  
    keylen, IppsRijndael192Spec* pCtx);
```

Parameters

<i>pKey</i>	Pointer to the Rijndael192 key.
<i>keylen</i>	Key byte stream length in bytes as defined by the IppsRijndaelKeyLength enumerator.
<i>pCtx</i>	Pointer to the IppsRijndael192Spec context.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed to by *pCtx* as the IppsRijndael192Spec context. In addition, the function uses the key to provide all the necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>keylen</i> is not set to IppsRijndaelKey128, IppsRijndaelKey192 or IppsRijndaelKey256.

Rijndael192EncryptECB

Encrypts a byte data stream according to Rijndael in the ECB mode.

Syntax

```
IppStatus ippsRijndael192EncryptECB(const Ipp8u *pSrc, Ipp8u *pDst, int  
    srclen, const IppsRijndael192Spec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the input data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael192Spec</code> context.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the cipher scheme specified in [\[NIST SP 800-38A\]](#).

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to 0.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael192DecryptECB

Decrypts byte data stream according to Rijndael in the ECB mode.

Syntax

```
IppStatus ippRijndael192DecryptECB(const Ipp8u* pSrc, Ipp8u* pDst, int
    srclen, const IppsRijndael192Spec *pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream of variable length.
<i>srclen</i>	Length of the ciphertext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael192Spec</code> context.

padding IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the ECB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by cipher block size.

Rijndael192EncryptCBC

Encrypts a byte data stream according to Rijndael in the CBC mode.

Syntax

```
IppStatus ippRijndael192EncryptCBC(const Ipp8u* pSrc, Ipp8u* pDst, int srclen,  
    const IppsRijndael192Spec* pCtx, const Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the plaintext data stream length in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael192Spec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the CBC mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the CBC mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael192DecryptCBC

Decrypts a byte data stream according to Rijndael in the CBC mode.

Syntax

```
IppStatus ippRijndael192DecryptCBC(const Ipp8u* pSrc, Ipp8u* pDst, int
    srclen, const IppsRijndael192Spec* pCtx, const Ipp8u* pIV,
    IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream.
<i>pDst</i>	Pointer to the resulting plaintext data stream of the variable length.
<i>srclen</i>	Length of the ciphertext data stream length in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael192Spec</code> context.
<i>pIV</i>	Pointer to the initialization vector for CBC mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the CBC mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by cipher block size.

Rijndael192EncryptCFB

Encrypts a byte data stream according to Rijndael in the CFB mode.

Syntax

```
IppStatus ippRijndael192EncryptCFB(const Ipp8u* pSrc, Ipp8u* pDst, int
    srcLen, int cfbBlkSize, const IppsRijndael192Spec* pCtx, const Ipp8u*
    pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>cfbBlkSize</i>	Size of the CFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael192Spec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the CFB mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of variable length according to the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <code>srcLen</code> is not divisible by <code>cfbBlkSize</code> parameter value.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <code>cfbBlkSize</code> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael192DecryptCFB

Decrypts a byte data stream according to Rijndael in the CFB mode.

Syntax

```
IppStatus ippRijndael192DecryptCFB(const Ipp8u* pSrc, Ipp8u* pDst, int
    srclen, int cfbBlkSize, const IppsRijndael192Spec* pCtx, const Ipp8u*
    pIV, IppsCPPadding padding);
```

Parameters

<code>pSrc</code>	Pointer to the input ciphertext data stream.
<code>pDst</code>	Pointer to the resulting plaintext data stream of variable length.
<code>srclen</code>	Length of the ciphertext data stream in bytes.
<code>cfbBlkSize</code>	Size of the CFB block in bytes.
<code>pCtx</code>	Pointer to the <code>IppsRijndael192Spec</code> context.
<code>pIV</code>	Pointer to the initialization vector for the CFB mode operation.

padding IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of variable length according to the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <i>cfbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by cipher block size.

Rijndael192EncryptOFB

Encrypts a variable length data stream according to Rijndael192 in the OFB mode.

Syntax

```
IppStatus ippRijndael192EncryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int
    srcLen, int ofbBlkSize, const IppsRijndael192Spec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the IppsRijndael192Spec context.

pIV Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael192DecryptOFB

Decrypts a variable length data stream according to Rijndael192 in the OFB mode.

Syntax

```
IppStatus ippRijndael192DecryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int
    srclen, int ofbBlkSize, const IppsRijndael192Spec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>srclen</i>	Length of the ciphertext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael192Spec</code> context.

pIV Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael192EncryptCTR

Encrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippRijndael192EncryptCTR(const Ipp8u* pSrc, Ipp8u* pDst, int  
    srcLen, const IppsRijndael192Spec* pCtx, Ipp8u* pCtrValue, int  
    ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of a variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael192Spec</code> context.
<i>pCtrValue</i>	Pointer to the counter data block.

ctrNumBitSize Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the CTR mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <i>ctrNumBitSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael192DecryptCTR

Decrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippRijndael192DecryptCTR(const Ipp8u* pSrc, Ipp8u* pDst, int srcLen,
    const IppsRijndael192Spec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream.
<i>pDst</i>	Pointer to the resulting plaintext data stream of a variable length.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael192Spec</code> context.
<i>pCtrValue</i>	Pointer to the counter data block.
<i>ctrNumBitSize</i>	Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the CTR mode as specified in the [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <code>ctrNumBitSize</code> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael256GetSize

Gets the size of the `IppsRijndael256Spec` context.

Syntax

```
IppStatus ippRijndael256GetSize(int* pSize)
```

Parameters

pSize Pointer to the `IppsRijndael256Spec` context size value.

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsRijndael256Spec` context size in bytes and stores it in **pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

Rijndael256Init

Initializes user-supplied memory as IppsRijndael256Spec context for future use.

Syntax

```
IppStatus ippsRijndael256Init(const Ipp8u *pKey, IppsRijndaelKeyLength
    keylen, IppsRijndael256Spec* pCtx);
```

Parameters

<i>pKey</i>	Pointer to the Rijndael256 key.
<i>keylen</i>	Key byte stream length in bytes defined by the IppsRijndaelKeyLength enumerator.
<i>pCtx</i>	Pointer to the IppsRijndael256Spec context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as the IppsRijndael256Spec context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>keylen</i> is not set to IppsRijndaelKey128, IppsRijndaelKey192 or IppsRijndaelKey256.

Rijndael256EncryptECB

Encrypts a byte data stream according to Rijndael in the ECB mode.

Syntax

```
IppStatus ippsRijndael256EncryptECB(const Ipp8u *pSrc, Ipp8u *pDst, int
    srclen, const IppsRijndael256Spec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the input data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael256Spec</code> context.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the cipher scheme specified in [\[NIST SP 800-38A\]](#).

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to 0.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael256DecryptECB

Decrypts a byte data stream according to Rijndael in the ECB mode.

Syntax

```
IppStatus ippRijndael256DecryptECB(const Ipp8u* pSrc, Ipp8u* pDst, int  
    srclen, const IppsRijndael256Spec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream of variable length.
<i>srclen</i>	Length of the ciphertext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael256Spec</code> context.

padding

IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the ECB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by cipher block size.

Rijndael256EncryptCBC

Encrypts byte data stream according to Rijndael in the CBC mode.

Syntax

```
IppStatus ippRijndael256EncryptCBC(const Ipp8u* pSrc, Ipp8u* pDst, int srclen,
    const IppsRijndael256Spec* pCtx, const Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the plaintext data stream length in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael256Spec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the CBC mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the CBC mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <code>srcLen</code> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael256DecryptCBC

Decrypts byte data stream according to Rijndael in the CBC mode.

Syntax

```
IppStatus ippRijndael256DecryptCBC(const Ipp8u* pSrc, Ipp8u* pDst, int
    srcLen, const IppsRijndael256Spec* pCtx, const Ipp8u* pIV,
    IppsCPPadding padding);
```

Parameters

<code>pSrc</code>	Pointer to the input ciphertext data stream.
<code>pDst</code>	Pointer to the resulting plaintext data stream of the variable length.
<code>srcLen</code>	Length of the ciphertext data stream length in bytes.
<code>pCtx</code>	Pointer to the <code>IppsRijndael256Spec</code> context.
<code>pIV</code>	Pointer to the initialization vector for CBC mode operation.
<code>padding</code>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the CBC mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by cipher block size.

Rijndael256EncryptCFB

Encrypts byte data stream according to Rijndael in the CFB mode.

Syntax

```
IppStatus ippRijndael256EncryptCFB(const Ipp8u* pSrc, Ipp8u* pDst, int
    srcLen, int cfbBlkSize, const IppsRijndael256Spec* pCtx, const Ipp8u*
    pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>cfbBlkSize</i>	Size of the CFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael256Spec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the CFB mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of variable length according to the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <code>srcLen</code> is not divisible by <code>cfbBlkSize</code> parameter value.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <code>cfbBlkSize</code> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael256DecryptCFB

Decrypts byte data stream according to Rijndael in the CFB mode.

Syntax

```
IppStatus ippRijndael256DecryptCFB(const Ipp8u* pSrc, Ipp8u* pDst, int  
    srclen, int cfbBlkSize, const IppsRijndael256Spec* pCtx, const Ipp8u*  
    pIV, IppsCPPadding padding);
```

Parameters

<code>pSrc</code>	Pointer to the input ciphertext data stream.
<code>pDst</code>	Pointer to the resulting plaintext data stream of variable length.
<code>srclen</code>	Length of the ciphertext data stream in bytes.
<code>cfbBlkSize</code>	Size of the CFB block in bytes.
<code>pCtx</code>	Pointer to the <code>IppsRijndael256Spec</code> context.
<code>pIV</code>	Pointer to the initialization vector for the CFB mode operation.

padding

IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of variable length according to the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <i>cfbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by cipher block size.

Rijndael256EncryptOFB

Encrypts a variable length data stream according to Rijndael256 in the OFB mode.

Syntax

```
IppStatus ippRsRijndael256EncryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int
    srclen, int ofbBlkSize, const IppsRijndael256Spec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the plaintext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the IppsRijndael256Spec context.

pIV Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael256DecryptOFB

Decrypts a variable length data stream according to Rijndael256 in the OFB mode.

Syntax

```
IppStatus ippRijndael256DecryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int  
    srclen, int ofbBlkSize, const IppsRijndael256Spec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>srclen</i>	Length of the ciphertext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael256Spec</code> context.

pIV Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael256EncryptCTR

Encrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippRijndael256EncryptCTR(const Ipp8u* pSrc, Ipp8u* pDst, int srcLen,
    const IppsRijndael256Spec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of a variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsRijndael256Spec</code> context.
<i>pCtrValue</i>	Pointer to the counter data block.

ctrNumBitSize Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the CTR mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsCTRSIZEErr</code>	Indicates an error condition if the value of the <i>ctrNumBitSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Rijndael256DecryptCTR

Decrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippRijndael256DecryptCTR(const Ipp8u* pSrc, Ipp8u* pDst, int srcLen,  
    const IppRijndael256Spec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream.
<i>pDst</i>	Pointer to the resulting plaintext data stream of a variable length.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppRijndael256Spec</code> context.
<i>pCtrValue</i>	Pointer to the counter data block.
<i>ctrNumBitSize</i>	Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the CTR mode as specified in the [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <i>ctrNumBitSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Example 2-2 AES Encryption and Decryption

```
// use of the CTR mode
void AES_sample(void){
    // size of Rijndael-128 algorithm block is equal to 16
    const int aesBlkSize = 16;

    // get the size of the context needed for the encryption/decryption operation
    int ctxSize;
    ippsRijndael128GetSize(&ctxSize);

    // and allocate one
    IppsRijndael128Spec* pCtx = (IppsRijndael128Spec*)( new Ipp8u [ctxSize] );
    // define the key
    Ipp8u key[16] = {0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,
                     0x08,0x09,0x10,0x11,0x12,0x13,0x14,0x15};

    // and prepare the context for Rijndael128 usage
    ippsRijndael128Init(key,IppsRijndaelKey128, pCtx);

    // define the message to be encrypted
    Ipp8u ptext[] = {"quick brown fox jum over lazy dog"};

    // define an initial vector
    Ipp8u crt0[aesBlkSize] = {0xff,0xee,0xdd,0xcc,0xbb,0xaa,0x99,0x88,
                              0x77,0x66,0x55,0x44,0x33,0x22,0x11,0x00};
    Ipp8u crt[aesBlkSize];

    // counter the variable number of bits
    int ctrNumBitSize = 64;
```

Example 2-2 AES Encryption and Decryption (continued)

```
// allocate enough memory for the ciphertext
// note that
// the size of the ciphertext is always equal to that of the plaintext
Ipp8u ctext[sizeof(ptext)];

// init the counter
memcpy(crt, crt0, sizeof(crt0));
// encrypt (CTR mode) ptext message
// pay attention to the 'length' parameter
// it defines the number of bytes to be encrypted
memcpy(crt, crt0, sizeof(crt0));
ippsRijndael128EncryptCTR(ptext, ctext, sizeof(ptext),
                           pCtx,
                           crt, ctrNumBitSize);

// allocate memory for the decrypted message
Ipp8u rtext[sizeof(ptext)];

// init the counter
memcpy(crt, crt0, sizeof(crt0));

// decrypt (ECTR mode) ctext message
// pay attention to the 'length' parameter
// it defines the number of bytes to be decrypted
ippsRijndael128DecryptCTR(ctext, rtext, sizeof(ptext),
                           pCtx,
                           crt, ctrNumBitSize);

delete (Ipp8u*)pCtx;
}
```

Blowfish Functions

Blowfish is a 16-round Feistel block cipher. Under this algorithm, the block size is 64 bits and the key can be of any size up to 448 bits. Although the algorithm requires a computationally intensive key expansion process that creates a set of eighteen 32-bit subkeys plus four 8x32-bit S-boxes derived from the input key, for a total of 4168 bytes, the actual encryption of streaming data is very efficient for software implementation.

This section describes the functions performing various operational modes under the Blowfish cipher systems. The implementation of the functions described in this section complies with the Blowfish cipher schemes.

[Table 2-4](#) lists Intel IPP Blowfish algorithm functions.

Table 2-4 Intel IPP Blowfish Algorithm Functions

Function Base Name	Operation
BlowfishGetSize	Gets the size of the <code>IppsBlowfishSpec</code> context.
BlowfishInit	Initializes user-supplied memory as <code>IppsBlowfishSpec</code> context for future use.
BlowfishEncryptECB	Encrypts input plaintext according to Blowfish scheme in the ECB mode.
BlowfishDecryptECB	Decrypts byte data stream according to Blowfish scheme in the EBC mode.
BlowfishEncryptCBC	Encrypts byte data stream according to Blowfish scheme in the CBC mode.
BlowfishDecryptCBC	Decrypts byte data stream according to Blowfish scheme in the CBC mode.
BlowfishEncryptCFB	Encrypts byte data stream according to Blowfish scheme in the CFB mode.
BlowfishDecryptCFB	Decrypts byte data stream according to Blowfish scheme in the CFB mode.
BlowfishEncryptOFB	Encrypts byte data stream according to Blowfish scheme in the OFB mode.
BlowfishDecryptOFB	Decrypts byte data stream according to Blowfish scheme in the OFB mode.
BlowfishEncryptCTR	Encrypts a variable length data stream in the CTR mode.
BlowfishDecryptCTR	Decrypts a variable length data stream in the CTR mode.

Throughout this section, the functions for Blowfish baseline cipher scheme employ the context `IppsBlowfishSpec`. It is so defined as to serve as the operational vehicles to not only carry both a set of subkeys and a set of S-Boxes, but also the key management information.

Once the respective initialization function has generated a set of subkeys and S-Boxes, the functions for ECB, CBC, CFB, and CTR modes are ready for the execution of either encrypting or decrypting the streaming data with the selected padding scheme.

The application code for conducting a typical encryption under CBC mode using Blowfish scheme should follow the sequence of operations as outlined below:

1. Get the buffer size required to configure the context `IppsBlowfishSpec` by calling the function [BlowfishGetSize](#).
2. Call the operating system memory allocation service function to allocate a buffer whose size is no less than the one specified by the function [BlowfishGetSize](#). Initialize the context `IppsBlowfishSpec *pCtx` by calling the function [BlowfishInit](#) with the allocated buffer and the respective Blowfish cipher key of the specified size.
3. Specify the initialization vector and the padding scheme, then call the function [BlowfishEncryptCBC](#) to encrypt the input data stream using the Blowfish encryption function with CBC mode.
4. Call the operating system memory free service function to release the buffer allocated for the context `IppsBlowfishSpec`, if needed.

BlowfishGetSize

Gets the size of the `IppsBlowfishSpec` context.

Syntax

```
IppStatus ippsBlowfishGetSize(int* pSize)
```

Parameters

`pSize` Pointer to the `IppsBlowfishSpec` context size value.

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsBlowfishSpec` context size in bytes and stores it in `*pSize`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

BlowfishInit

Initializes user-supplied memory as the IppsBlowfishSpec context for future use.

Syntax

```
IppStatus ippsBlowfishInit(const Ipp8u *pKey, int keylen,  
    IppsBlowfishSpec* pCtx);
```

Parameters

<i>pKey</i>	Pointer to the Blowfish key.
<i>keylen</i>	Key byte stream length in bytes.
<i>pCtx</i>	Pointer to the IppsBlowfishSpec context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as the `IppsBlowfishSpec` context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if <i>keylen</i> is less than 1 or greater than 56.

BlowfishEncryptECB

Encrypts input plaintext in the ECB mode.

Syntax

```
IppStatus ippsBlowfishEncryptECB(const Ipp8u *pSrc, Ipp8u *pDst, int  
    scrlen, const IppsBlowfishSpec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the input data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsBlowfishSpec</code> context.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the cipher scheme specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to 0.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

BlowfishDecryptECB

Decrypts byte data stream according to Blowfish scheme in the ECB mode.

Syntax

```
IppStatus ippBlowfishDecryptECB(const Ipp8u* pSrc, Ipp8u* pDst, int
    srclen, const IppsBlowfishSpec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream of variable length.

<i>srclen</i>	Length of the ciphertext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsBlowfishSpec</code> context.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the ECB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by cipher block size.

BlowfishEncryptCBC

Encrypts byte data stream according to Blowfish scheme in the CBC mode.

Syntax

```
IppStatus ippBlowfishEncryptCBC(const Ipp8u* pSrc, Ipp8u* pDst, int srclen,  
                                const IppsBlowfishSpec* pCtx, const Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsBlowfishSpec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the CBC mode operation.

padding

IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the CBC mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

BlowfishDecryptCBC

Decrypts byte data stream according to Blowfish scheme in the CBC mode.

Syntax

```
IppStatus ippSBlowfishDecryptCBC(const Ipp8u* pSrc, Ipp8u* pDst, int srclen,
    const IppsBlowfishSpec* pCtx, const Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream.
<i>pDst</i>	Pointer to the resulting plaintext data stream of the variable length.
<i>srclen</i>	Length of the ciphertext data stream length in bytes.
<i>pCtx</i>	Pointer to the <code>IppsBlowfishSpec</code> context.
<i>pIV</i>	Pointer to the initialization vector for CBC mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the CBC mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by cipher block size.

BlowfishEncryptCFB

Encrypts byte data stream according to Blowfish scheme in the CFB mode.

Syntax

```
IppStatus ippBlowfishEncryptCFB(const Ipp8u* pSrc, Ipp8u* pDst, int  
    srcLen, int cfbBlkSize, const IppsBlowfishSpec* pCtx, const Ipp8u*  
    pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>cfbBlkSize</i>	Size of the CFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsBlowfishSpec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the CFB mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of variable length according to the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <code>srcLen</code> is not divisible by <code>cfbBlkSize</code> parameter value.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <code>cfbBlkSize</code> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

BlowfishDecryptCFB

Decrypts byte data stream according to Blowfish scheme in the CFB mode.

Syntax

```
IppStatus ippBlowfishDecryptCFB(const Ipp8u* pSrc, Ipp8u* pDst, int
    srclen, int cfbBlkSize, const IppsBlowfishSpec* pCtx, const Ipp8u*
    pIV, IppsCPPadding padding);
```

Parameters

<code>pSrc</code>	Pointer to the input ciphertext data stream.
<code>pDst</code>	Pointer to the resulting plaintext data stream of variable length.
<code>srclen</code>	Length of the ciphertext data stream in bytes.
<code>cfbBlkSize</code>	Size of the CFB block in bytes.
<code>pCtx</code>	Pointer to the <code>IppsBlowfishSpec</code> context.
<code>pIV</code>	Pointer to the initialization vector for the CFB mode operation.

padding IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of variable length according to the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <i>cfbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by cipher block size.

BlowfishEncryptOFB

Encrypts a variable length data stream according to Blowfish scheme in the OFB mode.

Syntax

```
IppStatus ippBlowfishEncryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int srclen,
    int ofbBlkSize, const IppsBlowfishSpec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the plaintext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the IppsBlowfishSpec context.

pIV Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

BlowfishDecryptOFB

Decrypts a variable length data stream according to Blowfish scheme in the OFB mode.

Syntax

```
IppStatus ippBlowfishDecryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int srclen,
    int ofbBlkSize, const IppsBlowfishSpec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>srclen</i>	Length of the ciphertext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsBlowfishSpec</code> context.

pIV Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

BlowfishEncryptCTR

Encrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippBlowfishEncryptCTR(const Ipp8u* pSrc, Ipp8u* pDst, int srcLen,  
                                const IppsBlowfishSpec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of a variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsBlowfishSpec</code> context.
<i>pCtrValue</i>	Pointer to the counter data block.

ctrNumBitSize Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the CTR mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <i>ctrNumBitSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

BlowfishDecryptCTR

Decrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippSBlowfishDecryptCTR(const Ipp8u* pSrc, Ipp8u* pDst, int srcLen,
    const IppsBlowfishSpec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream.
<i>pDst</i>	Pointer to the resulting plaintext data stream of a variable length.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsBlowfishSpec</code> context.
<i>pCtrValue</i>	Pointer to the counter data block.
<i>ctrNumBitSize</i>	Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the CTR mode as specified in the [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <i>ctrNumBitSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Example 2-3 Blowfish Encryption and Decryption

```
// use of the CBC mode
void BF_sample(void){
    // size of Blowfish algorithm block is equal to 8
    const int bfBlkSize = 8;

    // get the size of the context needed for the encryption/decryption operation
    int ctxSize;
    ippBlowfishGetSize(&ctxSize);

    // and allocate one
    IppsBlowfishSpec* pCtx = (IppsBlowfishSpec*)( new Ipp8u [ctxSize] );

    // define the key
    // note that the key length may vary from 1 to 56 bytes
    Ipp8u key[] = {0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,
                  0x08,0x09,0x10,0x11,0x12,0x13,0x14,0x15,
                  0x16,0x17};
```

Example 2-3 Blowfish Encryption and Decryption (continued)

```
// and prepare the context for Blowfish usage
ippsBlowfishInit(key, sizeof(key), pCtx);

// define the message to be encrypted
Ipp8u ptext[] = {"quick brown fox jum over lazy dog"};

// define an initial vector
Ipp8u iv[bfBlkSize] = {0x77, 0x66, 0x55, 0x44, 0x33, 0x22, 0x11, 0x00};

// allocate enough memory for the ciphertext
// note that
// the size of the ciphertext is always multiple of the cipher block size
Ipp8u ctext[(sizeof(ptext)+bfBlkSize-1) &~(bfBlkSize-1)];

// encrypt (CBC mode) ptext message
// pay attention to the 'length' parameter
// it defines the number of bytes to be encrypted
ippsBlowfishEncryptCBC(ptext, ctext, sizeof(ptext),
                        pCtx,
                        iv,
                        IppsCPPaddingPKCS7);

// allocate memory for the decrypted message
Ipp8u rtext[sizeof(ptext)];
// decrypt (CBC mode) ctext message
// pay attention to the 'length' parameter
// it defines the number of bytes to be decrypted
ippsBlowfishDecryptCBC(ctext, rtext, sizeof(ptext),
                       pCtx,
                       iv,
                       IppsCPPaddingPKCS7);

delete (Ipp8u*)pCtx;
}
```

Twofish Functions

Twofish is a 16-round Feistel block cipher. The block size is 128 bits and the key can be any size up to 256 bits. The algorithm is one of the five Advanced Encryption Standard (AES) finalists. The cipher design for both the round function and key schedule enables an efficient implementation in software. Even though Twofish is free and not patented, it is nevertheless efficient and highly secure block cipher.

This section describes the functions for various operational modes under the Twofish cipher systems. The functions in this section are implemented to comply to the Twofish cipher schemes documented and submitted to NIST for candidacy of AES by B. Schneier.

[Table 2-5](#) lists Intel IPP Twofish algorithm functions:

Table 2-5 Intel IPP Twofish Algorithm Functions

Function Base Name	Operation
TwofishGetSize	Gets the size of the <code>IppsTwofishSpec</code> context.
TwofishInit	Initializes user-supplied memory as <code>IppsTwofishSpec</code> context for future use.
TwofishEncryptECB	Encrypts input plaintext according to Twofish scheme in the ECB mode.
TwofishDecryptECB	Decrypts byte data stream according to Twofish scheme in the EBC mode.
TwofishEncryptCBC	Encrypts byte data stream according to Twofish scheme in the CBC mode.
TwofishDecryptCBC	Decrypts byte data stream according to Twofish scheme in the CBC mode.
TwofishEncryptCFB	Encrypts byte data stream according to Twofish scheme in the CFB mode.
TwofishDecryptCFB	Decrypts byte data stream according to Twofish scheme in the CFB mode.
TwofishEncryptOFB	Encrypts byte data stream according to Twofish scheme in the OFB mode.
TwofishDecryptOFB	Decrypts byte data stream according to Twofish scheme in the OFB mode.
TwofishEncryptCTR	Encrypts a variable length data stream in the CTR mode.
TwofishDecryptCTR	Decrypts a variable length data stream in the CTR mode.

Throughout this section, the functions for Twofish baseline cipher scheme employ the context `IppsTwofishSpec`. This structure is defined to serve as an operational vehicle to not only carry both a set of subkeys and a set of S-Boxes, but also the key management information.

Once the respective initialization function has generated a set of subkeys and S-Boxes, the functions for ECB, CBC, CFB, and CTR modes are ready to the execution of either encrypting or decrypting the streaming data with the selected padding scheme.

The application code for conducting typical encryption under the CBC mode using the Twofish scheme should follow the sequence of operations as outlined below:

1. Get the buffer size required to configure the context `IppsTwofishSpec` by calling the function [TwofishGetSize](#).
2. Call operating system memory allocation service function to allocate a buffer whose size is no less than the one specified by the function [TwofishGetSize](#). Initialize the context `IppsTwofishSpec *pCtx` by calling the function [TwofishInit](#) with the allocated buffer and the respective Twofish cipher key of the specified size.
3. Specify the initialization vector and the padding scheme, then call the function [TwofishEncryptCBC](#) to encrypt the input data stream using the Twofish encryption function with CBC mode.
4. Call the operating system memory free service function to release the buffer allocated for the context [Rijndael128Init](#), if needed.

TwofishGetSize

Gets the size of the `IppsTwofishSpec` context.

Syntax

```
IppStatus ippsTwofishGetSize(int* pSize)
```

Parameters

pSize Pointer to the `IppsTwofishSpec` context size value.

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsTwofishSpec` context size in bytes and stores it in **pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

TwofishInit

Initializes user-supplied memory as the IppsTwofishSpec context for future use.

Syntax

```
IppStatus ippsTwofishInit(const Ipp8u *pKey, int keylen,  
                          IppsTwofishSpec* pCtx);
```

Parameters

<i>pKey</i>	Pointer to the Twofish key.
<i>keylen</i>	Key byte stream length in bytes.
<i>pCtx</i>	Pointer to the IppsTwofishSpec context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as the `IppsTwofishSpec` context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if <i>keylen</i> is less than 1 or greater than 32.

TwofishEncryptECB

Encrypts input plaintext in the ECB mode.

Syntax

```
IppStatus ippsTwofishEncryptECB(const Ipp8u *pSrc, Ipp8u *pDst, int  
                                scrlen, const IppsTwofishSpec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the input data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsTwofishSpec</code> context.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the cipher scheme specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to 0.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

TwofishDecryptECB

Decrypts byte data stream according to Twofish scheme in the ECB mode.

Syntax

```
IppStatus ippstTwofishDecryptECB(const Ipp8u* pSrc, Ipp8u* pDst, int
    srclen, const IppsTwofishSpec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream of variable length.

<i>srclen</i>	Length of the ciphertext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsTwofishSpec</code> context.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the ECB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by cipher block size.

TwofishEncryptCBC

Encrypts byte data stream according to Twofish scheme in the CBC mode.

Syntax

```
IppStatus ippstWofishEncryptCBC(const Ipp8u* pSrc, Ipp8u* pDst, int srclen,  
    const IppsTwofishSpec* pCtx, const Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the plaintext data stream length in bytes.
<i>pCtx</i>	Pointer to the <code>IppsTwofishSpec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the CBC mode operation.

padding IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the CBC mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

TwofishDecryptCBC

Decrypts byte data stream according to Twofish scheme in the CBC mode.

Syntax

```
IppStatus ippStwofishDecryptCBC(const Ipp8u* pSrc, Ipp8u *pDst, int srclen,
    const IppsTwofishSpec* pCtx, const Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream.
<i>pDst</i>	Pointer to the resulting plaintext data stream of the variable length.
<i>srclen</i>	Length of the ciphertext data stream length in bytes.
<i>pCtx</i>	Pointer to the <code>IppsDESSpec</code> context.
<i>pIV</i>	Pointer to the initialization vector for CBC mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the CBC mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by cipher block size.

TwofishEncryptCFB

Encrypts byte data stream according to Twofish scheme in the CFB mode.

Syntax

```
IppStatus ippStwofishEncryptCFB(const Ipp8u* pSrc, Ipp8u* pDst, int  
    srcLen, int cfbBlkSize, const IppsTwofishSpec* pCtx, const Ipp8u*  
    pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>cfbBlkSize</i>	Size of the CFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsTwofishSpec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the CFB mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of variable length according to the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <code>srcLen</code> is not divisible by <code>cfbBlkSize</code> parameter value.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <code>cfbBlkSize</code> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

TwofishDecryptCFB

Decrypts byte data stream according to Twofish scheme in the CFB mode.

Syntax

```
IppStatus ippStwofishDecryptCFB(const Ipp8u* pSrc, Ipp8u* pDst, int
    srclen, int cfbBlkSize, const IppsTwofishSpec* pCtx, const Ipp8u*
    pIV, IppsCPPadding padding);
```

Parameters

<code>pSrc</code>	Pointer to the input ciphertext data stream.
<code>pDst</code>	Pointer to the resulting plaintext data stream of variable length.
<code>srclen</code>	Length of the ciphertext data stream in bytes.
<code>cfbBlkSize</code>	Size of the CFB block in bytes.
<code>pCtx</code>	Pointer to the <code>IppsTwofishSpec</code> context.
<code>pIV</code>	Pointer to the initialization vector for the CFB mode operation.

padding IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of variable length according to the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value for <i>cfbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by cipher block size.

TwofishEncryptOFB

Encrypts a variable length data stream according to Twofish scheme in the OFB mode.

Syntax

```
IppStatus ippStsTwofishEncryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int srclen,
int ofbBlkSize, const IppsTwofishSpec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the plaintext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the IppsTwofishSpec context.

pIV Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srclen</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

TwofishDecryptOFB

Decrypts a variable length data stream according to Twofish scheme in the OFB mode.

Syntax

```
IppStatus ippstTwofishDecryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int srclen,
    int ofbBlkSize, const IppsTwofishSpec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>srclen</i>	Length of the ciphertext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsTwofishSpec</code> context.

pIV Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>srcLen</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

TwofishEncryptCTR

Encrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippStwofishEncryptCTR(const Ipp8u* pSrc, Ipp8u* pDst, int srcLen,  
                                const IppsTwofishSpec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of a variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsTwofishSpec</code> context.
<i>pCtrValue</i>	Pointer to the counter data block.

ctrNumBitSize Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length according to the CTR mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <i>ctrNumBitSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

TwofishDecryptCTR

Decrypts a variable length data stream in the CTR mode.

Syntax

```
IppStatus ippstTwofishDecryptCTR(const Ipp8u* pSrc, Ipp8u* pDst, int srcLen,
    const IppsTwofishSpec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream.
<i>pDst</i>	Pointer to the resulting plaintext data stream of a variable length.
<i>srcLen</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsTwofishSpec</code> context.
<i>pCtrValue</i>	Pointer to the counter data block.
<i>ctrNumBitSize</i>	Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the CTR mode as specified in the [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the output data stream length is less than or equal to zero.
<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <i>ctrNumBitSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Example 2-4 Twofish Encryption and Decryption

```
// use of the FCB mode
void TF_sample(void){
    // size of the Twofish algorithm block is equal to 16
    const int tfBlkSize = 16;

    // get the size of the context needed for the encryption/decryption operation
    int ctxSize;
    ippsTwofishGetSize(&ctxSize);

    // and allocate one
    IppsTwofishSpec* pCtx = (IppsTwofishSpec*)( new Ipp8u [ctxSize] );

    // define the key
    // note that the key length may vary from 16 to 32 bytes
    Ipp8u key[] = {0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,
                   0x08,0x09,0x10,0x11,0x12,0x13,0x14,0x15,
                   0x16,0x17};

    // and prepare the context for Twofish usage
    ippsTwofishInit(key,sizeof(key), pCtx);

    // define the message to be encrypted
    Ipp8u ptext[] = {"quick brown fox jum over lazy dog"};

    // fcb value (bytes)
    const int cfbBlkSize = 2;
```

Example 2-4 Twofish Encryption and Decryption (continued)

```
// define an initial vector
Ipp8u iv[tfBlkSize] = {0xff,0xee,0xdd,0xcc,0xbb,0xaa,0x99,0x88,
                      0x77,0x66,0x55,0x44,0x33,0x22,0x11,0x00};

// allocate enough memory for the ciphertext
// note that
// the size of the ciphertext is always multiple of the cfbBlkSize size
Ipp8u ctext[(sizeof(ptext)+cfbBlkSize-1) &~(cfbBlkSize-1)];

// encrypt (CFB mode) ptext message
// pay attention to the 'length' parameter
// it defines the number of bytes to be encrypted
ippsTwofishEncryptCFB(ptext, ctext, sizeof(ptext), cfbBlkSize,
                      pCtx,
                      iv,
                      IppsCPPaddingPKCS7);

// allocate memory for the decrypted message
Ipp8u rtext[sizeof(ptext)];
// decrypt (CFB mode) ctext message

// pay attention to the 'length' parameter
// it defines the number of bytes to be decrypted
ippsTwofishDecryptCFB(ctext, rtext, sizeof(ptext), cfbBlkSize,
                      pCtx,
                      iv,
                      IppsCPPaddingPKCS7);

delete (Ipp8u*)pCtx;
}
```

RC5* Functions

RC5, introduced by R.Rivest, is a fast symmetric block cipher that has a variable block size, variable key length, variable number of rounds and heavily uses data-independent rotations. Intel IPP for Cryptography implements functions for RC5 algorithm with 64- and 128-bit block sizes.

This section describes functions for various operational modes of RC5 cipher systems. The functions are implemented to comply with the RC5 cipher schemes documented in [RC5].

[Table 2-6](#) lists Intel IPP RC5 functions.

Table 2-6 Intel IPP RC5 Functions

Function Base Name	Operation
Functions for 64-bit Block Size	
ARCFive64GetSize	Gets the size of the IppsARCFive64Spec context.
ARCFive64Init	Initializes user-supplied memory as the IppsARCFive64Spec context for future use.
ARCFive64EncryptECB	Encrypts a variable length data stream using the RC5 cipher with 64-bit block size in the ECB mode.
ARCFive64DecryptECB	Decrypts a variable length data stream using the RC5 cipher with 64-bit block size in the ECB mode.
ARCFive64EncryptCBC	Encrypts a variable length data stream using the RC5 cipher with 64-bit block size in the CBC mode.
ARCFive64DecryptCBC	Decrypts a variable length data stream using the RC5 cipher with 64-bit block size in the CBC mode.
ARCFive64EncryptCFB	Encrypts a variable length data stream using the RC5 cipher with 64-bit block size in the CFB mode.
ARCFive64DecryptCFB	Decrypts a variable length data stream using the RC5 cipher with 64-bit block size in the CFB mode.
ARCFive64EncryptOFB	Encrypts a variable length data stream using the RC5 cipher with 64-bit block size in the OFB mode.
ARCFive64DecryptOFB	Decrypts a variable length data stream using the RC5 cipher with 64-bit block size in the OFB mode.
ARCFive64EncryptCTR	Encrypts a variable length data stream using the RC5 cipher with 64-bit block size in the CTR mode.
ARCFive64DecryptCTR	Decrypts a variable length data stream using the RC5 cipher with 64-bit block size in the CTR mode.
Functions for 128-bit Block Size	
ARCFive128GetSize	Gets the size of the IppsARCFive128Spec context.

Table 2-6 Intel IPP RC5 Functions (continued)

Function Base Name	Operation
<u>ARCFive128Init</u>	Initializes user-supplied memory as the <code>IppsARCFive128Spec</code> context for future use.
<u>ARCFive128EncryptECB</u>	Encrypts a variable length data stream using the RC5 cipher with 128-bit block size in the ECB mode.
<u>ARCFive128DecryptECB</u>	Decrypts a variable length data stream using the RC5 cipher with 128-bit block size in the ECB mode.
<u>ARCFive128EncryptCBC</u>	Encrypts a variable length data stream using the RC5 cipher with 128-bit block size in the CBC mode.
<u>ARCFive128DecryptCBC</u>	Decrypts a variable length data stream using the RC5 cipher with 128-bit block size in the CBC mode.
<u>ARCFive128EncryptCFB</u>	Encrypts a variable length data stream using the RC5 cipher with 128-bit block size in the CFB mode.
<u>ARCFive128DecryptCFB</u>	Decrypts a variable length data stream using the RC5 cipher with 128-bit block size in the CFB mode.
<u>ARCFive128EncryptOFB</u>	Encrypts a variable length data stream using the RC5 cipher with 128-bit block size in the OFB mode.
<u>ARCFive128DecryptOFB</u>	Decrypts a variable length data stream using the RC5 cipher with 128-bit block size in the OFB mode.
<u>ARCFive128EncryptCTR</u>	Encrypts a variable length data stream using the RC5 cipher with 128-bit block size in the CTR mode.
<u>ARCFive128DecryptCTR</u>	Decrypts a variable length data stream using the RC5 cipher with 128-bit block size in the CTR mode.

Throughout this section, the functions for the RC5 cipher with 64-bit block size employ the context `IppsARCFive64Spec` and the functions for the RC5 cipher with 128-bit block size employ the context `IppsARCFive128Spec`. These contexts serve as operational vehicles to carry variables needed to accomplish RC5 encryption/decryption, namely the number of rounds and expanded key table `S`.

Once the respective initialization function generates these variables, the functions for ECB, CBC, CFB, and other modes are ready for either encrypting or decrypting the streaming data with the specified padding scheme.

The application code for conducting a typical encryption in the CBC mode using the RC5 cipher with 64-bit block size should follow the sequence of operations outlined below:

1. Get the size required to configure the context `IppsARCFive64Spec` by calling the function [ARCFive64GetSize](#).
2. Call operating system memory allocation service function to allocate a buffer whose size is not less than the one specified by the function `ARCFive64GetSize`. Initialize the context `IppsARCFive64Spec *pCtx` by calling the function [ARCFive64Init](#) with the allocated buffer and the respective RC5 cipher key of the specified size.
3. Specify the initialization vector and the padding scheme and then call the function [ARCFive64EncryptCBC](#) to encrypt the input data stream using the RC5 cipher in the CBC mode.
4. Call the operating system memory free service function to release the buffer allocated for the context `IppsARCFive64Spec`, if needed.



NOTE. Similar procedure can be applied for ECB, CFB, OFB, and CTR modes of operation as well as the RC5 cipher with 128-bit block size.

RC5* Algorithm Functions for 64-bit Block Size

ARCFive64GetSize

Gets the size of the `IppsARCFive64Spec` context.

Syntax

```
IppStatus ippsARCFive64GetSize (int rounds, int* pSize);
```

Parameters

<i>rounds</i>	The number of rounds for the cipher.
<i>pSize</i>	Pointer to the size value of the <code>IppsARCFive64Spec</code> context.

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsARCFive64Spec` context size in bytes and stores it in *pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

ARCFive64Init

Initializes user-supplied memory as the IppsARCFive64Spec context for future use.

Syntax

```
IppStatus ipp5ARCFive64Init (const Ipp8u *pKey, int keylen, int rounds,  
    IppsARCFive64Spec* pCtx);
```

Parameters

<code>pKey</code>	Pointer to the RC5 key.
<code>keylen</code>	Key byte stream length in bytes.
<code>rounds</code>	The number of rounds for the cipher.
<code>pCtx</code>	Pointer to the IppsARCFive64Spec context.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by `pCtx` as the `IppsARCFive64Spec` context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <code>keylen</code> is less than 1 or greater than 256.

ARCFive64EncryptECB

Encrypts a variable length data stream using the RC5 cipher with 64-bit block size in the ECB mode.

Syntax

```
IppStatus ippsARCFive64EncryptECB (const Ipp8u* pSrc, Ipp8u* pDst, int length,
    const IppsARCFive64Spec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>length</i>	Length of the input data stream in bytes.
<i>pCtx</i>	Pointer to the IppsARCFive64Spec context.
<i>padding</i>	IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the ECB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to 0.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive64DecryptECB

Decrypts a variable length data stream using the RC5 cipher with 64-bit block size in the ECB mode.

Syntax

```
IppStatus ippsARCFive64DecryptECB (const Ipp8u* pSrc, Ipp8u* pDst, int length,
    const IppsARCFive64Spec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>length</i>	Length of the ciphertext data stream in bytes.
<i>pCtx</i>	Pointer to the IppsARCFive64Spec context.
<i>padding</i>	IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the ECB mode as specified in the [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to 0.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive64EncryptCBC

Encrypts a variable length data stream using the RC5 cipher with 64-bit block size in the CBC mode.

Syntax

```
IppStatus ippsARCFive64EncryptCBC (const Ipp8u* pSrc, Ipp8u* pDst, int length,
    const IppsARCFive64Spec* pCtx, const Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>length</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the IppsARCFive64Spec context.
<i>pIV</i>	Pointer to the initialization vector for the CBC mode operation.
<i>padding</i>	IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the CBC mode as specified in the [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to 0.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive64DecryptCBC

Decrypts a variable length data stream using the RC5 cipher with 64-bit block size in the CBC mode.

Syntax

```
IppStatus ippsARCFive64DecryptCBC (const Ipp8u* pSrc, Ipp8u* pDst, int length,  
    const IppsARCFive64Spec* pCtx, const Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>length</i>	Length of the ciphertext data stream in bytes.
<i>pCtx</i>	Pointer to the IppsARCFive64Spec context.
<i>pIV</i>	Pointer to the initialization vector for the CBC mode operation.
<i>padding</i>	IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the CBC mode as specified in the [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to 0.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive64EncryptCFB

Encrypts a variable length data stream using the RC5 cipher with 64-bit block size in the CFB mode.

Syntax

```
IppStatus ippsARCFive64EncryptCFB (const Ipp8u* pSrc, Ipp8u* pDst, int length,
    int cfbBlkSize, const IppsARCFive64Spec* pCtx, const Ipp8u* pIV,
    IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>length</i>	Length of the plaintext data stream in bytes.
<i>cfbBlkSize</i>	Size of the CFB block in bytes.
<i>pCtx</i>	Pointer to the IppsARCFive64Spec context.
<i>pIV</i>	Pointer to the initialization vector for the CFB mode operation.
<i>padding</i>	IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by the <i>cfbBlkSize</i> parameter value.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value of <i>cfbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive64DecryptCFB

Decrypts a variable length data stream using the RC5 cipher with 64-bit block size in the CFB mode.

Syntax

```
IppStatus ippSARCFive64DecryptCFB (const Ipp8u* pSrc, Ipp8u* pDst, int length,
int cfbBlkSize, const IppsARCFive64Spec* pCtx, const Ipp8u* pIV,
IppsCFFPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>length</i>	Length of the ciphertext data stream in bytes.
<i>cfbBlkSize</i>	Size of the CFB block in bytes.
<i>pCtx</i>	Pointer to the IppsARCFive64Spec context.
<i>pIV</i>	Pointer to the initialization vector for the CFB mode operation.
<i>padding</i>	IppsCFFPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by the <i>cfbBlkSize</i> parameter value.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value of <i>cfbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive64EncryptOFB

Encrypts a variable length data stream using the RC5 cipher with 64-bit block size in the OFB mode.

Syntax

```
IppStatus ippsARCFive64EncryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int length,
    int ofbBlkSize, const IppsARCFive64Spec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>length</i>	Length of the plaintext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the IppsARCFive64Spec context.
<i>pIV</i>	Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive64DecryptOFB

Decrypts a variable length data stream using the RC5 cipher with 64-bit block size in the OFB mode.

Syntax

```
IppStatus ippSARCFive64DecryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int length,
    int ofbBlkSize, const IppsARCFive64Spec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>length</i>	Length of the ciphertext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the IppsARCFive64Spec context.
<i>pIV</i>	Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive64EncryptCTR

Encrypts a variable length data stream using the RC5 cipher with 64-bit block size in the CTR mode.

Syntax

```
IppStatus ippSARCFive64EncryptCTR (const Ipp8u* pSrc, Ipp8u* pDst, int length,
    const IppsARCFive64Spec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of a variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>length</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the IppsARCFive64Spec context.
<i>pCtrValue</i>	Pointer to the counter data block.
<i>ctrNumBitSize</i>	Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the CTR mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsCTRSIZEErr</code>	Indicates an error condition if the value of the <i>ctrNumBitSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive64DecryptCTR

Decrypts a variable length data stream using the RC5 cipher with 64-bit block size in the CTR mode.

Syntax

```
IppStatus ippsARCFive64DecryptCTR (const Ipp8u* pSrc, Ipp8u* pDst, int length,  
    const IppsARCFive64Spec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>length</i>	Length of the ciphertext data stream in bytes.
<i>pCtx</i>	Pointer to the IppsARCFive64Spec context.
<i>pCtrValue</i>	Pointer to the counter data block.
<i>ctrNumBitSize</i>	Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the CTR mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsCTRSIZEErr</code>	Indicates an error condition if the value of the <i>ctrNumBitSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

RC5* Algorithm Functions for 128-bit Block Size

ARCFive128GetSize

Gets the size of the IppsARCFive128Spec context.

Syntax

```
IppStatus ippsARCFive128GetSize (int rounds, int* pSize);
```

Parameters

<i>rounds</i>	The number of rounds for the cipher.
<i>pSize</i>	Pointer to the size of the IppsARCFive128Spec context.

Description

This function is declared in the `ippcp.h` file. The function gets the IppsARCFive128Spec context size in bytes and stores it in **pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

ARCFive128Init

Initializes user-supplied memory as the IppsARCFive128Spec context for future use.

Syntax

```
IppStatus ippsARCFive128Init (const Ipp8u *pKey, int keylen, int rounds,  
                             IppsARCFive128Spec* pCtx);
```

Parameters

<i>pKey</i>	Pointer to the RC5 key.
<i>keylen</i>	Key byte stream length in bytes.

<i>rounds</i>	The number of rounds for the cipher.
<i>pCtx</i>	Pointer to the <code>IppsARCFive128Spec</code> context.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as the `IppsARCFive128Spec` context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if <i>keylen</i> is less than 1 or greater than 256.

ARCFive128EncryptECB

Encrypts a variable length data stream using the RC5 cipher with 128-bit block size in the ECB mode.

Syntax

```
IppStatus ippARCFive128EncryptECB (const Ipp8u* pSrc, Ipp8u* pDst, int length,  
    const IppsARCFive128Spec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>length</i>	Length of the input data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsARCFive128Spec</code> context.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the ECB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to 0.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive128DecryptECB

Decrypts a variable length data stream using the RC5 cipher with 128-bit block size in the ECB mode.

Syntax

```
IppStatus ippArcFive128DecryptECB (const Ipp8u* pSrc, Ipp8u* pDst, int length,
    const IppsARCFive128Spec* pCtx, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>length</i>	Length of the ciphertext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsARCFive128Spec</code> context.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the ECB mode as specified in the [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
--------------------------	--

<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to 0.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive128EncryptCBC

Encrypts a variable length data stream using the RC5 cipher with 128-bit block size in the CBC mode.

Syntax

```
IppStatus ippSARCFive128EncryptCBC (const Ipp8u* pSrc, Ipp8u* pDst, int length,  
    const IppsARCFive128Spec* pCtx, const Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>length</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsARCFive128Spec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the CBC mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the CBC mode as specified in the [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to 0.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive128DecryptCBC

Decrypts a variable length data stream using the RC5 cipher with 128-bit block size in the CBC mode.

Syntax

```
IppStatus ipp5ARCFive128DecryptCBC (const Ipp8u* pSrc, Ipp8u* pDst, int length,
    const IppsARCFive128Spec* pCtx, const Ipp8u* pIV, IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>length</i>	Length of the ciphertext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsARCFive128Spec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the CBC mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the CBC mode as specified in the [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to 0.

<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by data block size.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive128EncryptCFB

Encrypts a variable length data stream using the RC5 cipher with 128-bit block size in the CFB mode.

Syntax

```
IppStatus ipp5ARCFive128EncryptCFB (const Ipp8u* pSrc, Ipp8u* pDst, int length,
int cfbBlkSize, const IppsARCFive128Spec* pCtx, const Ipp8u* pIV,
IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>length</i>	Length of the plaintext data stream in bytes.
<i>cfbBlkSize</i>	Size of the CFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsARCFive128Spec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the CFB mode operation.
<i>padding</i>	<code>IppsCPPaddingNONE</code> padding scheme.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.

<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by the <i>cfbBlkSize</i> parameter value.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value of <i>cfbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive128DecryptCFB

Decrypts a variable length data stream using the RC5 cipher with 128-bit block size in the CFB mode.

Syntax

```
IppStatus ipp5ARCFive128DecryptCFB (const Ipp8u* pSrc, Ipp8u* pDst, int length,
                                     int cfbBlkSize, const IppsARCFive128Spec* pCtx, const Ipp8u* pIV,
                                     IppsCPPadding padding);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>length</i>	Length of the ciphertext data stream in bytes.
<i>cfbBlkSize</i>	Size of the CFB block in bytes.
<i>pCtx</i>	Pointer to the IppsARCFive128Spec context.
<i>pIV</i>	Pointer to the initialization vector for the CFB mode operation.
<i>padding</i>	IppsCPPaddingNONE padding scheme.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the CFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by the <i>cfbBlkSize</i> parameter value.
<code>ippStsCFBSizeErr</code>	Indicates an error condition if the value of <i>cfbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive128EncryptOFB

Encrypts a variable length data stream using the RC5 cipher with 128-bit block size in the OFB mode.

Syntax

```
IppStatus ippSARCFive128EncryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int length,
                                     int ofbBlkSize, const IppsARCFive128Spec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>length</i>	Length of the plaintext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsARCFive128Spec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive128DecryptOFB

Decrypts a variable length data stream using the RC5 cipher with 128-bit block size in the OFB mode.

Syntax

```
IppStatus ipp5ARCFive128DecryptOFB (const Ipp8u* pSrc, Ipp8u* pDst, int length,
    int ofbBlkSize, const IppsARCFive128Spec* pCtx, Ipp8u* pIV);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>length</i>	Length of the ciphertext data stream in bytes.
<i>ofbBlkSize</i>	Size of the OFB block in bytes.
<i>pCtx</i>	Pointer to the <code>IppsARCFive128Spec</code> context.
<i>pIV</i>	Pointer to the initialization vector for the OFB mode operation.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the OFB mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsUnderRunErr</code>	Indicates an error condition if <i>length</i> is not divisible by the <i>ofbBlkSize</i> parameter value.
<code>ippStsOFBSizeErr</code>	Indicates an error condition if the value of <i>ofbBlkSize</i> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive128EncryptCTR

Encrypts a variable length data stream using the RC5 cipher with 128-bit block size in the CTR mode.

Syntax

```
IppStatus ippSARCFive128EncryptCTR (const Ipp8u* pSrc, Ipp8u* pDst, int length,
    const IppsARCFive128Spec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of a variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>length</i>	Length of the plaintext data stream in bytes.
<i>pCtx</i>	Pointer to the <code>IppsARCFive128Spec</code> context.
<i>pCtrValue</i>	Pointer to the counter data block.
<i>ctrNumBitSize</i>	Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length in the CTR mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.
<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <code>ctrNumBitSize</code> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFive128DecryptCTR

Decrypts a variable length data stream using the RC5 cipher with 128-bit block size in the CTR mode.

Syntax

```
IppStatus ipp5ARCFive128DecryptCTR (const Ipp8u* pSrc, Ipp8u* pDst, int length,
    const IppsARCFive128Spec* pCtx, Ipp8u* pCtrValue, int ctrNumBitSize);
```

Parameters

<code>pSrc</code>	Pointer to the input ciphertext data stream of variable length.
<code>pDst</code>	Pointer to the resulting plaintext data stream.
<code>length</code>	Length of the ciphertext data stream in bytes.
<code>pCtx</code>	Pointer to the <code>IppsARCFive128Spec</code> context.
<code>pCtrValue</code>	Pointer to the counter data block.
<code>ctrNumBitSize</code>	Number of bits in the specific part of the counter to be incremented.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length in the CTR mode as specified in [\[NIST SP 800-38A\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than or equal to zero.

<code>ippStsCTRSizeErr</code>	Indicates an error condition if the value of the <code>ctrNumBitSize</code> is illegal.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFour Functions

As the RC4* stream cipher, widely used for file encryption and secure communications, is the property of RSA Security Inc., a cipher discussed in this section and resulting in the same encryption/decryption as RC4* is called ARCFour.

The ARCFour stream cipher ([AC]) uses a variable length key of up to 256 octets (bytes). ARCFour operates in the Output Feedback mode (OFB), defined in [NIST SP 800-38A], which creates the keystream independently of both the plaintext and the ciphertext.

[Table 2-7](#) lists Intel IPP ARCFour algorithm functions:

Table 2-7 Intel IPP ARCFour Algorithm Functions

Function Base Name	Operation
ARCFourGetSize	Gets the size of the <code>IppsARCFourState</code> context.
ARCFourCheckKey	Checks weakness of a user defined key.
ARCFourInit	Initializes user-supplied memory as the <code>IppsARCFourState</code> context for future use.
ARCFourEncrypt	Encrypts a variable length data stream according to ARCFour.
ARCFourDecrypt	Decrypts a variable length data stream according to ARCFour.
ARCFourReset	Resets the <code>IppsARCFourState</code> context to the initial state.

The ARCFour algorithm functions, described in this section, use the context `IppsARCFourState` as an operational vehicle to carry variables needed to execute the algorithm: S-Boxes and a current pair of indices.

The typical application code for conducting an encryption or decryption using ARCFour should follow the sequence of operations listed below:

1. Get the buffer size required to configure the context `IppsARCFourState` by calling the function [ARCFourGetSize](#).
2. Call the operating system memory allocation service function to allocate a buffer whose size is not less than the one specified by the function [ARCFourGetSize](#). Initialize the pointer `pCtx` to the `IppsARCFourState` context by calling the function [ARCFourInit](#) with the allocated buffer and the respective ARCFour cipher key of the specified size.

3. Call the [ARCFourEncrypt](#) or [ARCFourDecrypt](#) function to encrypt or decrypt the input data stream, respectively.
4. Call the operating system memory free service function to release the buffer allocated for the `IppsARCFourState` context, if needed.

ARCFourGetSize

Gets the size of the `IppsARCFourState` context.

Syntax

```
IppStatus ippsARCFourGetSize(int* pSize);
```

Parameters

pSize Pointer to the size value of the `IppsARCFourState` context.

Description

This function is declared in the `ippcp.h` file. The function gets the size of the `IppsARCFourState` context in bytes and stores it in **pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if the specified pointer is NULL.

ARCFourCheckKey

Checks weakness of a user-defined key.

Syntax

```
IppStatus ippsARCFourCheckKey(const Ipp8u* pKey, int keyLen, IppsBool* pIsWeak);
```

Parameters

<i>pKey</i>	Pointer to the user-defined key.
<i>keyLen</i>	Length of the user-defined key in octets.

pIsWeak Pointer to the result of checking.

Description

This function is declared in the `ippcp.h` file. The function checks weakness of user-defined key. The function allows to make sure that the supplied key provides sufficient security.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>keyLen</i> < 1 or <i>keyLen</i> > 256.

ARCFourInit

*Initializes user-supplied memory as the
IppsARCFourState context for future use.*

Syntax

```
IppStatus ippARCFourInit(const Ipp8u* pKey, int keyLen,  
                        IppsARCFourState* pCtx);
```

Parameters

<i>pKey</i>	Pointer to the user-defined key.
<i>keyLen</i>	Length of the user-defined key in octets.
<i>pCtx</i>	Pointer to the <code>IppsARCFourState</code> context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as `IppsARCFourState` context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>keyLen</i> < 1 or <i>keyLen</i> > 256.

ARCFourEncrypt

Encrypts a variable length data stream according to ARCFour.

Syntax

```
IppStatus ippsARCFourEncrypt(const Ipp8u* pSrc, Ipp8u* pDst, int srclen,
                             IppsARCFourState* pCtx);
```

Parameters

<i>pSrc</i>	Pointer to the input plaintext data stream of variable length.
<i>pDst</i>	Pointer to the resulting ciphertext data stream.
<i>srclen</i>	Length of the plaintext data stream in octets.
<i>pCtx</i>	Pointer to the <code>ARCFourState</code> context.

Description

This function is declared in the `ippcp.h` file. The function encrypts the input data stream of a variable length using the ARCFour algorithm.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if length of the input data stream is less than one octet.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFourDecrypt

Decrypts a variable length data stream according to ARCFour.

Syntax

```
IppStatus ippSARCFourDecrypt(const Ipp8u* pSrc, Ipp8u* pDst, int srclen,  
                             IppsARCFourState* pCtx);
```

Parameters

<i>pSrc</i>	Pointer to the input ciphertext data stream of variable length.
<i>pDst</i>	Pointer to the resulting plaintext data stream.
<i>srclen</i>	Length of the ciphertext data stream in octets.
<i>pCtx</i>	Pointer to the <code>ARCFourState</code> context.

Description

This function is declared in the `ippcp.h` file. The function decrypts the input data stream of a variable length according to the ARCFour algorithm.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if length of the input data stream is less than one octet.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

ARCFourReset

Resets the IppsARCFourState context to the initial state.

Syntax

```
IppStatus ippsARCFourReset(IppsARCFourState* pCtx);
```

Parameters

pCtx Pointer to the IppsARCFourState context being reset.

Description

This function is declared in the `ippcp.h` file. The function resets the `IppsARCFourState` context to the state it had immediately after the [ARCFourInit](#) function call. Contrary to `ARCFourInit`, `ARCFourReset` requires no secret key to initialize the S-Box.

Return Values

<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

One-Way Hash Primitives

3

Hash functions are used in cryptography with digital signatures and for ensuring data integrity.

When used with digital signatures, a publicly available hash function hashes the message and signs the resulting hash value. The party who receives the message can then hash the message and check if the block size is authentic for the given hash value.

Hash functions are also referred to as “message digests” and “one-way encryption functions”. Both terms are appropriate since hash algorithms do not have a key like symmetric and asymmetric algorithms and you can recover neither the length nor the contents of the plaintext message from the ciphertext.

To ensure data integrity, hash functions are used to compute the hash value that corresponds to a particular input. Then, if necessary, you can check if the input data has remained unmodified; you can re-compute the hash value again using the available input and compare it to the original hash value.

The [Hash Functions](#) section of this chapter describes functions that implement the following hash algorithms for streaming messages: MD5 [[RFC 1321](#)], SHA-1, SHA-224, SHA-256, SHA-384, and SHA-512 [[FIPS PUB 180-2](#)]. These algorithms are widely used in enterprise applications nowadays.

Subsequent sections of the chapter describe [Generalized Hash Functions for Non-Streaming Messages](#), which apply hash algorithms to entire (non-streaming) messages, and [Mask Generation Functions](#), whose algorithms are often based on hash computations.

Each of the above algorithms is implemented as a set of primitive functions.

The full list of Intel® Integrated Performance Primitives (Intel® IPP) Hash Primitive Functions is given in [Table 3-1](#).

Table 3-1 One-way Hash Primitive Functions

Function Base Name	Operation
Hash Functions	
<u>MD5GetSize</u>	Gets the size of the IppsMD5State context.
<u>MD5Init</u>	Initializes user-supplied memory as IppsMD5State context for future use.
<u>MD5Duplicate</u>	Copies one IppsMD5State context to another.
<u>MD5Update</u>	Digests the current input message stream of the specified length.
<u>MD5Final</u>	Completes computation of MD5 digest value.
<u>SHA1GetSize</u>	Gets the size of the SHA1 context.
<u>SHA1Init</u>	Initializes user-supplied memory as IppsSHA1State context for future use.
<u>SHA1Duplicate</u>	Copies one IppsSHA1State context to another.
<u>SHA1Update</u>	Digests the current input message stream of the specified length.
<u>SHA1Final</u>	Completes computation of SHA-1 digest value.
<u>SHA224GetSize</u>	Gets the size of the IppsSHA224State context.
<u>SHA224Init</u>	Initializes user-supplied memory as IppsSHA224State context for future use.
<u>SHA224Duplicate</u>	Copies one IppsSHA224State context to another.
<u>SHA224Update</u>	Digests the current input message stream of the specified length.
<u>SHA224Final</u>	Completes computation of SHA-224 digest value.
<u>SHA256GetSize</u>	Gets the size of the IppsSHA256State context.
<u>SHA256Init</u>	Initializes user-supplied memory as IppsSHA256State context for future use.
<u>SHA256Duplicate</u>	Copies one IppsSHA256State context to another.
<u>SHA256Update</u>	Digests the current input message stream of the specified length.
<u>SHA256Final</u>	Completes computation of SHA-256 digest value.
<u>SHA384GetSize</u>	Gets the size of the IppsSHA384State context.
<u>SHA384Init</u>	Initializes user-supplied memory as IppsSHA384State context for future use.
<u>SHA384Duplicate</u>	Copies one IppsSHA384State context to another.

Table 3-1 One-way Hash Primitive Functions (continued)

Function Base Name	Operation
<u>SHA384Update</u>	Digests the current input message stream of the specified length.
<u>SHA384Final</u>	Completes computation of SHA-384 digest value.
<u>SHA512GetSize</u>	Gets the size of the <code>IppsSHA512State</code> context.
<u>SHA512Init</u>	Initializes user-supplied memory as <code>IppsSHA512State</code> context for future use.
<u>SHA512Duplicate</u>	Copies one <code>IppsSHA512State</code> context to another.
<u>SHA512Update</u>	Digests the current input message stream of the specified length.
<u>SHA512Final</u>	Completes computation of SHA-512 digest value.
Generalized Hash Functions for Non-Streaming Messages	
<u>MD5MessageDigest</u>	Computes MD5 digest value of the input message.
<u>SHA1MessageDigest</u>	Computes SHA-1 digest value of the input message.
<u>SHA256MessageDigest</u>	Computes SHA-224 digest value of the input message.
<u>SHA256MessageDigest</u>	Computes SHA-256 digest value of the input message.
<u>SHA384MessageDigest</u>	Computes SHA-384 digest value of the input message.
<u>SHA512MessageDigest</u>	Computes SHA-512 digest value of the input message.
Mask Generation Functions	
<u>MGF_MD5</u>	Generates a pseudorandom mask of the specified length using MD5 hash function.
<u>MGF_SHA1</u>	Generates a pseudorandom mask of the specified length using SHA-1 hash function.
<u>MGF_SHA224</u>	Generates a pseudorandom mask of the specified length using SHA-224 hash function.
<u>MGF_SHA256</u>	Generates a pseudorandom mask of the specified length using SHA-256 hash function.
<u>MGF_SHA384</u>	Generates a pseudorandom mask of the specified length using SHA-384 hash function.
<u>MGF_SHA512</u>	Generates a pseudorandom mask of the specified length using SHA-512 hash function.

Hash Functions

Functions featured in this section apply hash algorithms to digesting streaming messages. A primitive implementing a hash algorithm uses the `State` context as an operational vehicle to carry all necessary variables to manage the computation of the chaining digest value. For example, the primitive implementing the SHA-1 hash algorithm must use the `ippsSHA1State` context.

The function `Init` initializes (`MD5Init`, `SHA1Init`, `SHA224Init`, `SHA256Init`, `SHA384Init`, and `SHA512Init`) the context and sets up specified initialization vectors. Once initialized, the function `Update` (`MD5Update`, `SHA1Update`, `SHA224Update`, `SHA256Update`, `SHA384Update`, and `SHA512Update`) digests the input message stream with the selected hash algorithm till it exhausts all message blocks. The function `Final` (`MD5Final`, `SHA1Final`, `SHA224Final`, `SHA256Final`, `SHA384Final`, and `SHA512Final`) is designed to pad the partial message block into a final message block with the specified padding scheme, and then uses the hash algorithm to transform the final block into a message digest value.

The following example illustrates how the application code can apply the implemented SHA-1 hash standard to digest the input message stream.

1. Call the function [SHA1GetSize](#) to get the size required to configure the `ippsSHA1State` context.
2. Ensure that the required memory space is properly allocated. With the allocated memory, call the [SHA1Init](#) function to set up the initial context state with the SHA-1 specified initialization vectors.
3. Keep calling the function [SHA1Update](#) to digest incoming message stream in the queue till its completion.
4. Call the function [SHA1Final](#) for padding the partial block into a final SHA-1 message block and transforming it into a 160-bit message digest value.
5. Call the operating system memory free service function to release the `ippsSHA1State` context.

MD5GetSize

Gets the size of the `IppsMD5State` context in bytes.

Syntax

```
IppsStatus ippsMD5GetSize(int *pSize);
```

Parameters

pSize Pointer to the IppsMD5State context size value.

Description

This function is declared in the `ippcp.h` file. The function gets the IppsMD5State context size in bytes and stores it in **pSize*.

Return Values

`ippStsNoErr` Indicates no error. Any other value indicates an error or warning.

`ippStsNullPtrErr` Indicates an error condition if any of the specified pointers is NULL.

MD5Init

Initializes user-supplied memory as IppsMD5State context for future use.

Syntax

```
IppStatus ippMD5Init(IppsMD5State* pCtx);
```

Parameters

pCtx Pointer to the IppsMD5State context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as IppsMD5State context.

Return Values

`ippStsNoErr` Indicates no error. Any other value indicates an error or warning.

`ippStsNullPtrErr` Indicates an error condition if any of the specified pointers is NULL.

MD5Duplicate

Copies one IppsMD5State context to another.

Syntax

```
IppStatus ippsMD5Duplicate(const IppsMD5State* pSrcCtx, IppsMD5State* pDstCtx)
```

Parameters

<i>pSrcCtx</i>	Pointer to the source IppsMD5State context.
<i>pDstCtx</i>	Pointer to the IppsMD5State context to be cloned.

Description

The function is declared in the `ippcp.h` file. The function copies one IppsMD5State context to another.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

MD5Update

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippsMD5Update(const Ipp8u *pSrcMesg, int mesglen, IppsMD5State  
    *pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part of or the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.

pCtx Pointer to the `IppsMD5State` context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length.

The function first integrates the previous partial block with the input message stream and then partitions them into multiple message blocks (as specified by the applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected hash algorithm to transform the block into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

MD5Final

Completes computation of the MD5 digest value.

Syntax

```
IppStatus ippMD5Final(Ipp8u *pMD, IppsMD5State *pCtx);
```

Parameters

<i>pMD</i>	Pointer to the resultant digest value.
<i>pCtx</i>	Pointer to the <code>IppsMD5State</code> context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stores the result into the specified memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

SHA1GetSize

Gets the size of the `IppsSHA1State` context in bytes.

Syntax

```
IppStatus ippSHA1GetSize(int *pSize);
```

Parameters

pSize Pointer to the `IppsSHA1State` context size value.

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsSHA1State` context size in bytes and stores it in **pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

SHA1Init

Initializes user-supplied memory as `IppsSHA1State` context for future use.

Syntax

```
IppStatus ippSHA1Init(IppsSHA1State* pCtx);
```

Parameters

pCtx Pointer to the `IppsSHA1State` context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as `IppsSHA1State` context.

Return Values

`ippStsNoErr` Indicates no error. Any other value indicates an error or warning.
`ippStsNullPtrErr` Indicates an error condition if any of the specified pointers is NULL.

SHA1Duplicate

Copies one `IppsSHA1State` context to another.

Syntax

```
IppStatus ippSHA1Duplicate(const IppsSHA1State* pSrcCtx, IppsSHA1State*  
                        pDstCtx)
```

Parameters

pSrcCtx Pointer to the source `IppsSHA1State` context.
pDstCtx Pointer to the `IppsSHA1State` context to be cloned.

Description

The function is declared in the `ippcp.h` file. The function copies one `IppsSHA1State` context to another.

Return Values

`ippStsNoErr` Indicates no error. Any other value indicates an error or warning.
`ippStsNullPtrErr` Indicates an error condition if any of the specified pointers is NULL.
`ippStsContextMatchErr` Indicates an error condition if the context parameter does not match the operation.

SHA1Update

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippSHA1Update(const Ipp8u *pSrcMesg, int mesglen,  
                        IppsSHA1State *pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part of or the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the IppsSHA1State context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length.

The function first integrates the previous partial block with the input message stream and then partitions them into multiple message blocks (as specified by the applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected hash algorithm to transform the block into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

SHA1Final

Completes computation of the SHA1 digest value.

Syntax

```
IppStatus ippSHA1Final(Ipp8u *pMD, IppsSHA1State *pCtx);
```

Parameters

<i>pMD</i>	Pointer to the resultant digest value.
<i>pCtx</i>	Pointer to the IppsSHA1State context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stores the result into the specified memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

SHA224GetSize

Gets the size of the IppsSHA224State context in bytes.

Syntax

```
IppStatus ippSHA224GetSize(int *pSize);
```

Parameters

<i>pSize</i>	Pointer to the IppsSHA224State context size value.
--------------	--

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsSHA224State` context size in bytes and stores it in `*pSize`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

SHA224Init

*Initializes user-supplied memory as
`IppsSHA224State` context for future use.*

Syntax

```
IppStatus ippSHA224Init(IppsSHA224State* pCtx);
```

Parameters

<code>pCtx</code>	Pointer to the <code>IppsSHA224State</code> context being intialized.
-------------------	---

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by `pCtx` as `IppsSHA224State` context.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

SHA224Duplicate

Copies one `IppsSHA224State` context to another.

Syntax

```
IppStatus ippSHA224Duplicate(const IppsSHA224State* pSrcCtx,  
                             IppsSHA224State* pDstCtx)
```

Parameters

<i>pSrcCtx</i>	Pointer to the source SHA224State context.
<i>pDstCtx</i>	Pointer to the IppsSHA224State context to be cloned.

Description

The function is declared in the `ippcp.h` file. The function copies one `IppsSHA224State` context to another.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

SHA224Update

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippsha224Update(const Ipp8u *pSrcMesg, int mesglen,  
                          IppsSHA224State *pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part of or the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the IppsSHA224State context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length.

The function first integrates the previous partial block with the input message stream and then partitions them into multiple message blocks (as specified by the applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected hash algorithm to transform the block into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

SHA224Final

Completes computation of the SHA224 digest value.

Syntax

```
IppStatus ippSHA224Final(Ipp8u *pMD, IppsSHA224State *pCtx);
```

Parameters

<i>pMD</i>	Pointer to the resultant digest value.
<i>pCtx</i>	Pointer to the <code>IppsSHA224State</code> context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stores the result into the specified memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
------------------------------------	---

SHA256GetSize

Gets the size of the `IppsSHA256State` context in bytes.

Syntax

```
IppStatus ippSHA256GetSize(int *pSize);
```

Parameters

`pSize` Pointer to the `IppsSHA256State` context size value.

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsSHA256State` context size in bytes and stores it in `*pSize`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

SHA256Init

Initializes user-supplied memory as `IppsSHA256State` context for future use.

Syntax

```
IppStatus ippSHA256Init(IppsSHA256State *pCtx);
```

Parameters

`pCtx` Pointer to the `IppsSHA256State` context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by `pCtx` as `IppsSHA256State` context.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

SHA256Duplicate

Copies one `IppsSHA256State` context to another.

Syntax

```
IppStatus ippSHA256Duplicate(const IppsSHA256State* pSrcCtx,
                             IppsSHA256* pDstCtx)
```

Parameters

<code>pSrcCtx</code>	Pointer to the source <code>IppsSHA256State</code> context.
<code>pDstCtx</code>	Pointer to the <code>IppsSHA256State</code> context to be cloned.

Description

The function is declared in the `ippcp.h` file. The function copies one `IppsSHA256State` context to another.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

SHA256Update

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippSHA256Update(const Ipp8u *pSrcMesg, int mesglen,  
                          IppsSHA256State *pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part of or the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the IppsSHA256State context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length.

The function first integrates the previous partial block with the input message stream and then partitions them into multiple message blocks (as specified by the applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected hash algorithm to transform the block into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

SHA256Final

Completes computation of the SHA256 digest value.

Syntax

```
IppStatus ippSHA256Final(Ipp8u *pMD, IppsSHA256State *pCtx);
```

Parameters

<i>pMD</i>	Pointer to the resultant digest value.
<i>pCtx</i>	Pointer to the IppsSHA256State context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stores the result into the specified memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

SHA384GetSize

Gets the size of the IppsSHA384State context in bytes.

Syntax

```
IppStatus ippSHA384GetSize(int *pSize);
```

Parameters

<i>pSize</i>	Pointer to the IppsSHA384State context size value.
--------------	--

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsSHA384State` context size in bytes and stores it in `*pSize`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

SHA384Init

*Initializes user-supplied memory as
IppsSHA384State context for future use.*

Syntax

```
IppStatus ippSHA384Init(IppsSHA384State* pCtx);
```

Parameters

<code>pCtx</code>	Pointer to the <code>IppsSHA384State</code> context being initialized.
-------------------	--

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by `pCtx` as `IppsSHA384State` context.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

SHA384Duplicate

Copies one IppsSHA384State context to another.

Syntax

```
IppStatus ippSHA384Duplicate(const IppsSHA384State* pSrcCtx,  
                             IppsSHA384State* pDstCtx)
```


Parameters

<i>pSrcCtx</i>	Pointer to the source <code>IppsSHA384State</code> context.
<i>pDstCtx</i>	Pointer to the <code>IppsSHA384State</code> context to be cloned.

Description

The function is declared in the `ippcp.h` file. The function copies one `IppsSHA384State` context to another.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

SHA384Update

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippsha384Update(const Ipp8u *pSrcMesg, int mesglen,
                          IppsSHA384State *pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part of or the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the <code>IppsSHA384State</code> context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length.

The function first integrates the previous partial block with the input message stream and then partitions them into multiple message blocks (as specified by the applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected hash algorithm to transform the block into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

SHA384Final

Completes computing of the SHA384 digest value.

Syntax

```
IppStatus ippSHA384Final(Ipp8u *pMD, IppsSHA384State *pCtx);
```

Parameters

<code>pMD</code>	Pointer to the resultant digest value.
<code>pCtx</code>	Pointer to the <code>IppsSHA384State</code> context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stores the result into the specified memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

SHA512GetSize

Gets the size of the IppsSHA512State context in bytes.

Syntax

```
IppStatus ippsSHA512GetSize(int *pSize);
```

Parameters

pSize Pointer to the IppsSHA512State context size value.

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsSHA512State` context size in bytes and stores it in **pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

SHA512Init

Initializes user-supplied memory as IppsSHA512State context for future use.

Syntax

```
IppStatus ippsSHA512Init(IppsSHA512State* pState);
```

Parameters

pCtx Pointer to the IppsSHA512State context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as `IppsSHA512State` context.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

SHA512Duplicate

Copies one `IppsSHA512State` context to another.

Syntax

```
IppStatus ippSHA512Duplicate(const IppsSHA512State* pSrcCtx,  
                             IppsSHA512* pDstCtx)
```

Parameters

<code>pSrcCtx</code>	Pointer to the source <code>IppsSHA512State</code> context.
<code>pDstCtx</code>	Pointer to the <code>IppsSHA512State</code> context to be cloned.

Description

The function is declared in the `ippcp.h` file. The function copies one `IppsSHA512State` context to another.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

SHA512Update

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippsSHA512Update(const Ipp8u *pSrcMesg, int mesglen,  
                           IppsSHA512State *pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part of or the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the IppsSHA512State context.

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length.

The function first integrates the previous partial block with the input message stream and then partitions them into multiple message blocks (as specified by the applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected hash algorithm to transform the block into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

SHA512Final

Completes computation of the SHA512 digest value.

Syntax

```
IppStatus ippSHA512Final(Ipp8u *pMD, IppsSHA512State *pCtx);
```

Parameters

<i>pMD</i>	Pointer to the resultant digest value.
<i>pCtx</i>	Pointer to the <code>IppsSHA512State</code> context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stores the result into the specified memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

Generalized Hash Functions for Non-Streaming Messages

Intel IPP hash functions [MD5MessageDigest](#), [SHA1MessageDigest](#), [SHA224MessageDigest](#), [SHA256MessageDigest](#), [SHA384MessageDigest](#), and [SHA512MessageDigest](#) are used to calculate a digest of an entire (non-streaming) input message by applying a selected hash algorithm.

Having the six hash algorithms currently available in the Intel IPP, you may still prefer to use a different implementation of a hash algorithm, based on some other function. In this case you can also use the IPPCP library. To do this, use the definition of a hash function introduced in IPPCP and given in the subsection below. This definition is also applied to a hash function when it is passed as a parameter that some Public Key Cryptography operations optionally use.

General Definition of a Hash Function

Syntax

```
typedef IppStatus(_STDCALL *IppHASH)(const Ipp8u* pMsg, int msgLen,
                                     Ipp8u* pMD);
```

Parameters

<i>pMsg</i>	Pointer to the input octet string.
<i>msgLen</i>	Length of the input string in octets.
<i>pMD</i>	Pointer to the output message digest.

Description

This declaration is included in the `ippcp.h` file. The function calculates the digest of a non-streaming message using the implemented hash algorithm.



NOTE. Definition of a hash function used in Intel IPP limits length (in octets) of an input message for any specific hash function by the range of the `int` data type, with the upper bound of $2^{32}-1$.

MD5MessageDigest

Computes MD5 digest value of the input message.

Syntax

```
IppStatus ippMD5MessageDigest(const Ipp8u *pSrcMsg, int msgLen, Ipp8u *pMD);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>msglen</i>	Message length in octets.
<i>pMD</i>	Pointer to the resultant digest.

Description

This function is declared in the `ippcp.h` file. The function uses the selected hash algorithm to compute digest value of the entire (non-streaming) input message.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

Example 3-1 MD5 Digest of a Message

```
void MD5_sample(void) {
    // define message
    Ipp8u msg[] = "abcdefghijklmnopqrstuvwxyz";

    // once the whole message is placed into memory,
    // one can use the integrated primitive
    Ipp8u digest[16];
    ippMD5MessageDigest(msg, strlen((char*)msg), digest);
}
```

SHA1MessageDigest

Computes SHA-1 digest value of the input message.

Syntax

```
IppStatus ippSHA1MessageDigest(const Ipp8u *pSrcMesg, int mesglen,
                                Ipp8u *pMD);
```

Parameters

<code>pSrcMesg</code>	Pointer to the input message.
<code>mesglen</code>	Message length in octets.

pMD Pointer to the resultant digest.

Description

This function is declared in the `ippcp.h` file. The function uses the selected hash algorithm to compute the digest value of the entire (non-streaming) input message.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

Example 3-2 SHA1 Digest of a Message

```
//      Compute two SHA1 digests of a message:
//      1-st will correspond of 1/2 message
//      2-nd will correspond of whole message
void SHA1_sample(void){
    // get size of the SHA1 context
    int ctxSize;
    ippSHA1GetSize(&ctxSize);

    // allocate the SHA1 context
    IppsSHA1State* pCtx = (IppsSHA1State*)( new Ipp8u [ctxSize] );

    // and initialize the context
    ippSHA1Init(pCtx);

    // define a message
    Ipp8u msg[] = "abcdbcdecdefdefgefghfghighijhijkijkljklmklmnlmnomnopnopq";

    int n;
```

Example 3-2 SHA1 Digest of a Message (continued)

```
// update digest using a piece of message
for(n=0; n<(sizeof(msg)-1)/2; n++)
    ippsSHA1Update(msg+n, 1, pCtx);
// clone the SHA1 context
IppsSHA1State* pCtx2 = (IppsSHA1State*)( new Ipp8u [ctxSize] );
ippsSHA1Init(pCtx2);
ippsSHA1Duplicate(pCtx, pCtx2);
// finalize and extract digest of a half message
Ipp8u digest[20];
ippsSHA1Final(digest, pCtx);

// update digest using the SHA1 clone context
ippsSHA1Update(msg+n, sizeof(msg)-1-n, pCtx2);

// finalize and extract digest of a whole message
Ipp8u digest2[20];
ippsSHA1Final(digest2, pCtx2);

delete [] (Ipp8u*)pCtx;
delete [] (Ipp8u*)pCtx2;
}
```

SHA224MessageDigest

Computes SHA-224 digest value of the input message.

Syntax

```
IppStatus ippsSHA224MessageDigest(const Ipp8u *pSrcMsg, int msglen,
    Ipp8u *pMD);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>mesglen</i>	Message length in octets.
<i>pMD</i>	Pointer to the resultant digest.

Description

This function is declared in the `ippcp.h` file. The function uses the selected hash algorithm to compute the digest value of the entire (non-streaming) input message.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

SHA256MessageDigest

Computes SHA-256 digest value of the input message.

Syntax

```
IppStatus ippSHA256MessageDigest(const Ipp8u *pSrcMsg, int mesglen,
                                   Ipp8u *pMD);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>mesglen</i>	Message length in octets.
<i>pMD</i>	Pointer to the resultant digest.

Description

This function is declared in the `ippcp.h` file. The function uses the selected hash algorithm to compute the digest value of the entire (non-streaming) input message.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
--------------------------	--

<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

SHA384MessageDigest

Computes SHA-384 digest value of the input message.

Syntax

```
IppStatus ippSHA384MessageDigest(const Ipp8u *pSrcMsg, int mesglen,  
                                Ipp8u *pMD);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>mesglen</i>	Message length in octets.
<i>pMD</i>	Pointer to the resultant digest.

Description

This function is declared in the `ippcp.h` file. The function uses the selected hash algorithm to compute the digest value of the entire (non-streaming) input message.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

SHA512MessageDigest

Computes SHA-512 digest value of the input message.

Syntax

```
IppStatus ippsSHA512MessageDigest(const Ipp8u *pSrcMsg, int msglen,
                                   Ipp8u *pMD);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>msglen</i>	Message length in octets.
<i>pMD</i>	Pointer to the resultant digest.

Description

This function is declared in the `ippcp.h` file. The function uses the selected hash algorithm to compute the digest value of the entire (non-streaming) input message.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

Mask Generation Functions

Public Key Cryptography frequently uses mask generation functions (MGFs) to achieve a particular security goal. For example, MGFs are used both in RSA-OAEP encryption and RSA-SSA signature schemes.

MGF function takes an octet string of a variable length and generates an octet string of a desired length. MGFs are deterministic, which means that the input octet string completely determines the output one. The output of an MGF should be pseudorandom, that is, infeasible to predict. The provable security of such cryptography schemes as RSA-OAEP or RSA-SSA, relies on the random nature of the MGF output. That is why one-way hash functions is one of the well-known ways to implement an MGF. The exact definition of an MGF based on a one-way hash function may be found in [\[PKCS 1.2.1\]](#).

This section describes MGFs based on widely-used MD5, SHA-1, SHA-224, SHA-256, SHA-384 and SHA-512 hash algorithms as well as the possibility to use a different implementation of MGF.



NOTE. Intel IPP implementation of MGFs limits length (in octets) of an input message for any specific MGF by the range of the `int` data type, with the upper bound of $2^{32}-1$.

User's Implementation of a Mask Generation Function

In case you prefer or have to use a different implementation of an MGF you can still use IPPCP. To do this, use the definition of MGF introduced in the IPPCP library and described in this section. The declaration provided below also defines an MGF when it is used as a parameter in some Public Key Cryptography operations.

Syntax

```
typedef IppStatus(_STD_CALL *IppMGF)(const Ipp8u* pSeed, int seedLen,  
    Ipp8u* pMask, int maskLen);
```

Parameters

<i>pSeed</i>	Pointer to the input octet string.
<i>seedLen</i>	Length of the input string.
<i>pMask</i>	Pointer to the output pseudorandom mask.
<i>maskLen</i>	Desired length of the output.

Description

This declaration is included in the `ippcp.h` file. The function generates an octet string of length *maskLen* according to the implemented algorithm, providing pseudorandom output.

MGF_MD5

Generates a pseudorandom mask of the specified length using MD5 hash function.

Syntax

```
IppStatus ippsMGF_MD5(const Ipp8u *pSeed, int seedLen, Ipp8u* pMask, int maskLen);
```

Parameters

<i>pSeed</i>	Pointer to the input octet string.
<i>seedLen</i>	Length of the input string.
<i>pMask</i>	Pointer to the output pseudorandom mask.
<i>maskLen</i>	Desired length of the output.

Description

This function is declared in the `ippcp.h` file. The function generates a pseudorandom mask of the specified length using MD5 hash algorithm.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if <i>pMask</i> pointer is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if any of the specified lengths is negative or zero.

MGF_SHA1

Generates a pseudorandom mask of the specified length using SHA-1 hash function.

Syntax

```
IppStatus ippsMGF_SHA1(const Ipp8u *pSeed, int seedLen, Ipp8u* pMask, int maskLen);
```

Parameters

<i>pSeed</i>	Pointer to the input octet string.
<i>seedLen</i>	Length of the input string.
<i>pMask</i>	Pointer to the output pseudorandom mask.
<i>maskLen</i>	Desired length of the output.

Description

This function is declared in the `ippcp.h` file. The function generates a pseudorandom mask of the specified length using SHA-1 hash algorithm.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if <i>pMask</i> pointer is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if any of the specified lengths is negative or zero.

MGF_SHA224

Generates a pseudorandom mask of the specified length using SHA-224 hash function.

Syntax

```
IppStatus ippmGF_SHA224(const Ipp8u *pSeed, int seedLen, Ipp8u* pMask,  
                        int maskLen);
```

Parameters

<i>pSeed</i>	Pointer to the input octet string.
<i>seedLen</i>	Length of the input string.
<i>pMask</i>	Pointer to the output pseudorandom mask.
<i>maskLen</i>	Desired length of the output.

Description

This function is declared in the `ippcp.h` file. The function generates a pseudorandom mask of the specified length using SHA-224 hash algorithm.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if <i>pMask</i> pointer is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if any of the specified lengths is negative or zero.

MGF_SHA256

Generates a pseudorandom mask of the specified length using SHA-256 hash function.

Syntax

```
IppStatus ippmMGF_SHA256(const Ipp8u *pSeed, int seedLen, Ipp8u* pMask,
                        int maskLen);
```

Parameters

<i>pSeed</i>	Pointer to the input octet string.
<i>seedLen</i>	Length of the input string.
<i>pMask</i>	Pointer to the output pseudorandom mask.
<i>maskLen</i>	Desired length of the output.

Description

This function is declared in the `ippcp.h` file. The function generates a pseudorandom mask of the specified length using SHA-256 hash algorithm.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if <i>pMask</i> pointer is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if any of the specified lengths is negative or zero.

MGF_SHA384

Generates a pseudorandom mask of the specified length using SHA-384 hash function.

Syntax

```
IppStatus ippsMGF_SHA384(const Ipp8u *pSeed, int seedLen, Ipp8u* pMask,  
    int maskLen);
```

Parameters

<i>pSeed</i>	Pointer to the input octet string.
<i>seedLen</i>	Length of the input string.
<i>pMask</i>	Pointer to the output pseudorandom mask.
<i>maskLen</i>	Desired length of the output.

Description

This function is declared in the `ippcp.h` file. The function generates a pseudorandom mask of the specified length using SHA-384 hash algorithm.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if <i>pMask</i> pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if any of the specified lengths is negative or zero.

MGF_SHA512

Generates a pseudorandom mask of the specified length using SHA-512 hash function.

Syntax

```
IppStatus ippsMGF_SHA512(const Ipp8u *pSeed, int seedLen, Ipp8u* pMask,  
    int maskLen);
```

Parameters

<i>pSeed</i>	Pointer to the input octet string.
<i>seedLen</i>	Length of the input string.
<i>pMask</i>	Pointer to the output pseudorandom mask.
<i>maskLen</i>	Desired length of the output.

Description

This function is declared in the `ippcp.h` file. The function generates a pseudorandom mask of the specified length using SHA-512 hash algorithm.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if <i>pMask</i> pointer is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if any of the specified lengths is negative or zero.

Data Authentication Primitive Functions



This chapter describes functions for generating message authentication code (MAC), that is, [Message Authentication Functions](#), and the [Data Authentication Functions](#) (DAA) schemes.

Hash function-based MAC (HMAC) is widely used in the applications requiring message authentication and data integrity check. HMAC was initially put forward in [RFC 2401] and adopted by ANSI X9.71 and [FIPS PUB 198]. A MAC algorithm based on a symmetric key block cipher, in other words, a cipher-based MAC (CMAC), is standardized in [NIST SP 800-38B]. CMAC may be appropriate for information systems where an approved block cipher is available rather than an approved hash function.

DAA is widely used in the applications to detect unauthorized modifications, both intentional and accidental, to data. DAA specification can be found in [FIPS PUB 113].

Message Authentication Functions

Intel® Integrated Performance Primitives (Intel® IPP) HMAC primitive functions, described in this chapter, use various HMAC schemes based on one-way hash functions described in the [One-Way Hash Primitives](#) chapter.

The Intel IPP CMAC primitive functions use CMAC schemes based on block ciphers described in the [Symmetric Cryptography Primitive Functions](#) chapter.

The full list of Intel IPP message authentication functions is given in [Table 4-1](#).

Table 4-1 Intel IPP HMAC Functions

Function Base Name	Operation
Keyed Hash Primitive Functions	
HMACSHA1GetSize	Gets the size of the <code>IppsHMACSHA1State</code> context.

Table 4-1 Intel IPP HMAC Functions (continued)

Function Base Name	Operation
<u>HMACSHA1Init</u>	Initializes user-supplied memory as IppsHMACSHA1State context for future use.
<u>HMACSHA1Duplicate</u>	Copies one IppsHMACSHA1State context to another.
<u>HMACSHA1Update</u>	Digests the current input message stream of the specified length.
<u>HMACSHA1Final</u>	Completes computation of the HMAC value.
<u>HMACSHA1MessageDigest</u>	Computes the HMAC value of the message.
<u>HMACSHA224GetSize</u>	Gets the size of the IppsHMACSHA224State context.
<u>HMACSHA224Init</u>	Initializes user-supplied memory as IppsHMACSHA224State context for future use.
<u>HMACSHA224Duplicate</u>	Copies one IppsHMACSHA224State context to another.
<u>HMACSHA224Update</u>	Digests the current input message stream of the specified length.
<u>HMACSHA224Final</u>	Completes computation of the HMAC value.
<u>HMACSHA224MessageDigest</u>	Computes the HMAC value of the message.
<u>HMACSHA256GetSize</u>	Gets the size of the IppsHMACSHA256State context.
<u>HMACSHA256Init</u>	Initializes user-supplied memory as IppsHMACSHA256State context for future use.
<u>HMACSHA256Duplicate</u>	Copies one IppsHMACSHA256State context to another.
<u>HMACSHA256Update</u>	Digests the current input message stream of the specified length.
<u>HMACSHA256Final</u>	Completes computation of the HMAC value.
<u>HMACSHA256MessageDigest</u>	Computes the HMAC value of the message.
<u>HMACSHA384GetSize</u>	Gets the size of the IppsHMACSHA384State context.
<u>HMACSHA384Init</u>	Initializes user-supplied memory as IppsHMACSHA384State context for future use.
<u>HMACSHA384Duplicate</u>	Copies one IppsHMACSHA384State context to another.
<u>HMACSHA384Update</u>	Digests the current input message stream of the specified length.
<u>HMACSHA384Final</u>	Completes computation of the HMAC value.
<u>HMACSHA384MessageDigest</u>	Computes the HMAC value of the message.
<u>HMACSHA512GetSize</u>	Gets the size of the IppsHMACSHA512State context.
<u>HMACSHA512Init</u>	Initializes user-supplied memory as IppsHMACSHA512State context for future use.
<u>HMACSHA512Duplicate</u>	Copies one IppsHMACSHA512State context to another.

Table 4-1 Intel IPP HMAC Functions (continued)

Function Base Name	Operation
<u>HMACSHA512Update</u>	Digests the current input message stream of the specified length.
<u>HMACSHA512Final</u>	Completes computation of the HMAC value.
<u>HMACSHA512MessageDigest</u>	Computes the HMAC value of the message.
<u>HMACCMD5GetSize</u>	Gets the size of the <code>IppsHMACCMD5State</code> context.
<u>HMACCMD5Init</u>	Initializes user-supplied memory as <code>IppsHMACCMD5State</code> context for future use.
<u>HMACCMD5Duplicate</u>	Copies one <code>IppsHMACCMD5State</code> context to another.
<u>HMACCMD5Update</u>	Digests the current input message stream of the specified length.
<u>HMACCMD5Final</u>	Completes computation of the HMAC value.
<u>HMACCMD5MessageDigest</u>	Computes the HMAC value of the message.
CMAC Primitive Functions	
<u>CMACRijndael128GetSize</u>	Gets the size of the <code>IppsCMACRijndael128State</code> context.
<u>CMACRijndael128Init</u>	Initializes user-supplied memory as <code>IppsCMACRijndael128State</code> context for future use.
<u>CMACRijndael128Update</u>	Updates the MAC value depending on the current input message stream of the specified length.
<u>CMACRijndael128Final</u>	Completes computation of the MAC value.
<u>CMACRijndael128MessageDigest</u>	Computes the MAC value of the entire message.

Each MAC scheme is implemented as a set of the primitive functions tabled above.

For example, the primitive implementing HMAC that is based on SHA-1 hash algorithm uses the `ippsHMACSHA1State` context as an operational vehicle to carry all necessary variables to manage computation of chaining digest value.

The function `HMACSHA1Init` initializes the context and sets up the specified initialization vectors. After the initialization, the function `HMACSHA1Update` digests the input message stream with the selected hash algorithm till it exhausts all message blocks.

The function `HMACSHA1Final` is designed to pad the partial message block into a final message block with the specified padding scheme, and then use the hash algorithm to transform the final block into a message digest value.

The following example illustrates how the application code can apply the implemented HMAC-SHA1 hash standard to digest the input message stream:

1. Call the function [HMACSHA1GetSize](#) to get the size required to configure the `IppsHMACSHA1State` context.
2. Ensure that the required memory space is properly allocated. With the allocated memory, call the function [HMACSHA1Init](#) to set up key material and the initial context state with the SHA-1 specified initialization vectors.
3. Keep calling the function [HMACSHA1Update](#) to digest incoming message stream in the queue till its completion.
4. Call the function [HMACSHA1Final](#) for padding the partial block into a final SHA-1 message block and transforming it into a resulting HMAC value.
5. Call the operating system memory free service function to release the `IppsHMACSHA1State` context.

HMACSHA1GetSize

Gets the size of the `IppsHMACSHA1State` context.

Syntax

```
IppStatus ippsHMACSHA1GetSize(int *pSize);
```

Parameters

pSize Pointer to the `IppsHMACSHA1State` context size value.

Description

The function is declared in the `ippcp.h` file. The function gets the `IppsHMACSHA1State` context size in bytes and stores it in *pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

HMACSHA1Init

*Initializes user-supplied memory as
IppsHMACSHA1State context for future use.*

Syntax

```
IppStatus ippsHMACSHA1Init(const Ipp8u *pKey, int keyLen,  
                           IppsHMACSHA1State *pCtx;
```

Parameters

<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length in bytes.
<i>pCtx</i>	Pointer to the IppsHMACSHA1State context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed to by *pCtx* as the IppsHMACSHA1State context. The function also sets up the initial chaining digest value according to the Hash algorithm specified by the function base name and computes necessary key material from the supplied key *pKey*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>keyLen</i> is less than one.

HMACSHA1Duplicate

*Copies one IppsHMACSHA1State context to
another.*

Syntax

```
IppStatus ippsHMACSHA1Duplicate(const IppsHMACSHA1State* pSrcCtx,  
                                IppsHMACSHA1State* pDstCtx);
```


Parameters

<i>pSrcCtx</i>	Pointer to the source <code>IppsHMACSHA1State</code> context.
<i>pDstCtx</i>	Pointer to the source <code>IppsHMACSHA1State</code> context to be cloned.

Description

The function is declared in the `ippcp.h` file. The function copies one `IppsHMACSHA1State` context to another.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

HMACSHA1Update

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippsha1Update(const Ipp8u *pSrcMesg, int mesglen,
                        IppsHMACSHA1State *pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part of the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the <code>IppsHMACSHA1State</code> context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length.

The function first integrates the previous partial block with the input message stream and then partitions them into multiple message blocks (as specified by the applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected hash algorithm to transform the block into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

HMACSHA1Final

Completes computation of the HMAC value.

Syntax

```
IppStatus ippHMACSHA1Final(Ipp8u *pMAC, int macLen, IppsHMACSHA1State *pCtx);
```

Parameters

<code>pMAC</code>	Pointer to the resultant HMAC value.
<code>macLen</code>	Specified HMAC length.
<code>pCtx</code>	Pointer to the <code>IppsHMACSHA1State</code> context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stores the result into the specified *pMD* memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>macLen</i> is less than one or greater than the length of the hash value.

HMACSHA1MessageDigest

Computes the HMAC value of the message.

Syntax

```
IppStatus ippSHA1MessageDigest(const Ipp8u *pSrcMesg, int msgLen,
                               const Ipp8u *pKey, int keyLen, Ipp8u *pMAC, int macLen);
```

Parameters

<i>pSrcMesg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length in bytes.
<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey* of the specified key length *keyLen* and applies the keyed hash-based message authentication code scheme to transform the input message into the respective message authentication code *pMAC* of the specified length *macLen*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero and <i>macLen</i> is less than one or greater than the length of the hash value.

HMACSHA224GetSize

Gets the size of the IppsHMACSHA224State context.

Syntax

```
IppStatus ippsHMACSHA224GetSize(int *pSize);
```

Parameters

pSize Pointer to the IppsHMACSHA224State context size value.

Description

The function is declared in the `ippcp.h` file. The function gets the IppsHMACSHA224State context size in bytes and stores it in *pSize*.

Return Values

`ippStsNoErr` Indicates no error. Any other value indicates an error or warning.
`ippStsNullPtrErr` Indicates an error condition if any of the specified pointers is NULL.

HMACSHA224Init

Initializes user-supplied memory as IppsHMACSHA224State context for future use.

Syntax

```
IppStatus ippsHMACSHA224Init(const Ipp8u *pKey, int keyLen,  
                              IppsHMACSHA224State *pCtx);
```

Parameters

pKey Pointer to the user-supplied key.
keyLen Key length in bytes.
pCtx Pointer to the IppsHMACSHA224State context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by `pCtx` as the `IppsHMACSHA224State` context. The function also sets up the initial chaining digest value according to the Hash algorithm specified by the function base name and computes necessary key material from the supplied key `pKey`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <code>keyLen</code> is less than one.

HMACSHA224Duplicate

Copies one IppsHMACSHA224State context to another.

Syntax

```
IppStatus ippHMACSHA224Duplicate(const IppsHMACSHA224State* pSrcCtx,
                                IppsHMACSHA224State* pDstCtx);
```

Parameters

<code>pSrcCtx</code>	Pointer to the source <code>IppsHMACSHA224State</code> context.
<code>pDstCtx</code>	Pointer to the source <code>IppsHMACSHA224State</code> context to be cloned.

Description

The function is declared in the `ippcp.h` file. The function copies one `IppsHMACSHA224State` context to another.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

HMACSHA224Update

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippSHMACSHA224Update(const Ipp8u *pSrcMesg, int mesglen,  
                                IppSHMACSHA224State *pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part of the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the <code>IppSHMACSHA224State</code> context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length.

The function first integrates the previous partial block with the input message stream and then partitions them into multiple message blocks (as specified by the applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected hash algorithm to transform the block into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

HMACSHA224Final

Completes computation of the HMAC value.

Syntax

```
IppStatus ippshMACSHA224Final(Ipp8u *pMAC, int macLen,
                              IppsHMACSHA224State *pCtx);
```

Parameters

<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.
<i>pCtx</i>	Pointer to the IppsHMACSHA224State context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stores the result into the specified *pMD* memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>macLen</i> is less than one or greater than the length of the hash value.

HMACSHA224MessageDigest

Computes the HMAC value of the message.

Syntax

```
IppStatus ippshMACSHA224MessageDigest(const Ipp8u *pSrcMesg, int
                                       msgLen, const Ipp8u *pKey, int keyLen, Ipp8u *pMAC, int macLen);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length in bytes.
<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey* of the specified key length *keyLen* and applies the keyed hash-based message authentication code scheme to transform the input message into the respective message authentication code *pMAC* of the specified length *macLen*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero and <i>macLen</i> is less than one or greater than the length of the hash value.

HMACSHA256GetSize

Gets the size of the IppsHMACSHA256State context.

Syntax

```
IppStatus ippHMACSHA256GetSize(int *pSize);
```

Parameters

<i>pSize</i>	Pointer to the IppsHMACSHA256State context size value.
--------------	--

Description

The function is declared in the `ippcp.h` file. The function gets the IppsHMACSHA256State context size in bytes and stores it in *pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

HMACSHA256Init

*Initializes user-supplied memory as
IppsHMACSHA256State context for future use.*

Syntax

```
IppStatus ippsha256Init(const Ipp8u *pKey, int keyLen,
                        IppsHMACSHA256State *pCtx);
```

Parameters

<code>pKey</code>	Pointer to the user-supplied key.
<code>keyLen</code>	Key length in bytes.
<code>pCtx</code>	Pointer to the IppsHMACSHA256State context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by `pCtx` as the `IppsHMACSHA256State` context. The function also sets up the initial chaining digest value according to the Hash algorithm specified by the function base name and computes necessary key material from the supplied key `pKey`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <code>keyLen</code> is less than one.

HMACSHA256Duplicate

Copies one IppsHMACSHA256State context to another.

Syntax

```
IppStatus ippsHMACSHA256Duplicate(const IppsHMACSHA256State* pSrcCtx,  
    IppsHMACSHA256State* pDstCtx);
```

Parameters

<i>pSrcCtx</i>	Pointer to the source IppsHMACSHA256State context.
<i>pDstCtx</i>	Pointer to the source IppsHMACSHA256State context to be cloned.

Description

The function is declared in the `ippcp.h` file. The function copies one IppsHMACSHA256State context to another.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

HMACSHA256Update

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippsHMACSHA256Update(const Ipp8u *pSrcMesg, int mesglen,  
    IppsHMACSHA256State *pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part of the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the <code>IppsHMACSHA256State</code> context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length.

The function first integrates the previous partial block with the input message stream and then partitions them into multiple message blocks (as specified by the applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected hash algorithm to transform the block into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

HMACSHA256Final

Completes computation of the HMAC value.

Syntax

```
IppStatus ippHMACSHA256Final(Ipp8u *pMAC, int macLen,
                             IppsHMACSHA256State *pCtx);
```

Parameters

<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.
<i>pCtx</i>	Pointer to the <code>IppsHMACSHA256State</code> context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stores the result into the specified *pMD* memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>macLen</i> is less than one or greater than the length of the hash value.

HMACSHA256MessageDigest

Computes the HMAC value of the message.

Syntax

```
IppStatus ippSHA256MessageDigest(const Ipp8u *pSrcMsg, int  
    msgLen, const Ipp8u *pKey, int keyLen, Ipp8u *pMAC, int macLen);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length in bytes.
<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey* of the specified key length *keyLen* and applies the keyed hash-based message authentication code scheme to transform the input message into the respective message authentication code *pMAC* of the specified length *macLen*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero and <i>macLen</i> is less than one or greater than the length of the hash value.

Example 4-1 SHA256 HMACs of a Message

```
// Compute two keyed-hash based message authentication code
// using the HMAC scheme
//    1-st will correspond of 1/2 message
//    2-nd will correspond of whole message

void HMACSHA256_sample(void){
    // define key
    Ipp8u key[] = "the key for HMAC scheme";

    // get size of HMACSHA256 context
    int ctxSize;
    ippHMACSHA256GetSize(&ctxSize);

    // allocate HMACSHA256 context
    IppsHMACSHA256State* pCtx = (IppsHMACSHA256State*)( new Ipp8u [ctxSize] );
```

Example 4-1 SHA256 HMACs of a Message (continued)

```
// and ini context
ippsHMACSHA256Init(key, strlen((char*)key), pCtx);
// define message
Ipp8u msg[] = "abcdbcdecdefdefgefghfghighijhijkijkljklmklmnlmnomnopnopq";

int n;
// update MAC using piece of message
for(n=0; n<(sizeof(msg)-1)/2; n++)
    ippsHMACSHA256Update(msg+n, 1, pCtx);

// clone HMACSHA256 context
IppsHMACSHA256State* pCtx2 = (IppsHMACSHA256State*)( new Ipp8u [ctxSize] );
ippsHMACSHA256Init(key, strlen((char*)key), pCtx2);
ippsHMACSHA256Duplicate(pCtx, pCtx2);

// finalize and extract digest of a half message
const int macLen = 16;
Ipp8u mac[macLen];
ippsHMACSHA256Final(mac, macLen, pCtx);

// update MAC using HMACSHA256 clone context
ippsHMACSHA256Update(msg+n, sizeof(msg)-1-n, pCtx2);

// finalize and extract digest of a whole message
Ipp8u mac2[macLen];
ippsHMACSHA256Final(mac2, macLen, pCtx2);

delete [] (Ipp8u*)pCtx;
delete [] (Ipp8u*)pCtx2;
}
```

HMACSHA384GetSize

Gets the size of the IppsHMACSHA384State context.

Syntax

```
IppStatus ippsHMACSHA384GetSize(int *pSize);
```

Parameters

pSize Pointer to the IppsHMACSHA384State context size value.

Description

The function is declared in the `ippcp.h` file. The function gets the IppsHMACSHA384State context size in bytes and stores it in *pSize*.

Return Values

`ippStsNoErr` Indicates no error. Any other value indicates an error or warning.
`ippStsNullPtrErr` Indicates an error condition if any of the specified pointers is NULL.

HMACSHA384Init

Initializes user-supplied memory as IppsHMACSHA384State context for future use.

Syntax

```
IppStatus ippsHMACSHA384Init(const Ipp8u *pKey, int keyLen,  
                                IppsHMACSHA384State *pCtx);
```

Parameters

pKey Pointer to the user-supplied key.
keyLen Key length in bytes.
pCtx Pointer to the IppsHMACSHA384State context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed to by `pCtx` as the `IppsHMACSHA384State` context. The function also sets up the initial chaining digest value according to the Hash algorithm specified by the function base name and computes necessary key material from the supplied key `pKey`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <code>keyLen</code> is less than one.

HMACSHA384Duplicate

Copies one `IppsHMACSHA384State` context to another.

Syntax

```
IppStatus ippHMACSHA384Duplicate(const IppsHMACSHA384State* pSrcCtx,  
                                IppsHMACSHA384State* pDstCtx);
```

Parameters

<code>pSrcCtx</code>	Pointer to the source <code>IppsHMACSHA384State</code> context.
<code>pDstCtx</code>	Pointer to the source <code>IppsHMACSHA384State</code> context to be cloned.

Description

The function is declared in the `ippcp.h` file. The function copies one `IppsHMACSHA384State` context to another.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

HMACSHA384Update

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippsHMACSHA384Update(const Ipp8u *pSrcMesg, int mesglen,
                                IppsHMACSHA384State *pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part of the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the IppsHMACSHA384State context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length.

The function first integrates the previous partial block with the input message stream and then partitions them into multiple message blocks (as specified by the applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected hash algorithm to transform the block into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

HMACSHA384Final

Completes computation of the HMAC value.

Syntax

```
IppStatus ippshMACSHA384Final(Ipp8u *pMAC, int macLen,  
    IppshMACSHA384State *pCtx);
```

Parameters

<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.
<i>pCtx</i>	Pointer to the IppshMACSHA384State context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stores the result into the specified *pMD* memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>macLen</i> is less than one or greater than the length of the hash value.

HMACSHA384MessageDigest

Computes the HMAC value of the message.

Syntax

```
IppStatus ippshMACSHA384MessageDigest(const Ipp8u *pSrcMesg, int  
    mesgLen, const Ipp8u *pKey, int keyLen, Ipp8u *pMAC, int macLen);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length in bytes.
<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey* of the specified key length *keyLen* and applies the keyed hash-based message authentication code scheme to transform the input message into the respective message authentication code *pMAC* of the specified length *macLen*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero and <i>macLen</i> is less than one or greater than the length of the hash value.

HMACSHA512GetSize

Gets the size of the IppsHMACSHA512State context.

Syntax

```
IppStatus ippHMACSHA512GetSize(int *pSize);
```

Parameters

<i>pSize</i>	Pointer to the IppsHMACSHA512State context size value.
--------------	--

Description

The function is declared in the `ippcp.h` file. The function gets the IppsHMACSHA512State context size in bytes and stores it in *pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

HMACSHA512Init

*Initializes user-supplied memory as
IppsHMACSHA512State context for future use.*

Syntax

```
IppStatus ippsha512Init(const Ipp8u *pKey, int keyLen,  
                        IppsHMACSHA512State *pCtx);
```

Parameters

<code>pKey</code>	Pointer to the user-supplied key.
<code>keyLen</code>	Key length in bytes.
<code>pCtx</code>	Pointer to the IppsHMACSHA512State context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by `pCtx` as the `IppsHMACSHA512State` context. The function also sets up the initial chaining digest value according to the Hash algorithm specified by the function base name and computes necessary key material from the supplied key `pKey`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <code>keyLen</code> is less than one.

HMACSHA512Duplicate

Copies one IppsHMACSHA512State context to another.

Syntax

```
IppStatus ippsHMACSHA512Duplicate(const IppsHMACSHA512State* pSrcCtx,
                                   IppsHMACSHA512State* pDstCtx);
```

Parameters

<i>pSrcCtx</i>	Pointer to the source IppsHMACSHA512State context.
<i>pDstCtx</i>	Pointer to the source IppsHMACSHA512State context to be cloned.

Description

The function is declared in the `ippcp.h` file. The function copies one IppsHMACSHA512State context to another.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

HMACSHA512Update

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippsHMACSHA512Update(const Ipp8u *pSrcMesg, int mesglen,
                                IppsHMACSHA512State *pCtx);
```

Parameters

<i>pSrcMsg</i>	Pointer to the buffer containing a part of the whole message.
<i>msglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the <code>IppsHMACSHA512State</code> context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length.

The function first integrates the previous partial block with the input message stream and then partitions them into multiple message blocks (as specified by the applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected hash algorithm to transform the block into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

HMACSHA512Final

Completes computation of the HMAC value.

Syntax

```
IppStatus ippHMACSHA512Final(Ipp8u *pMAC, int macLen,  
                             IppsHMACSHA512State *pCtx);
```

Parameters

<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.
<i>pCtx</i>	Pointer to the <code>IppsHMACSHA512State</code> context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stores the result into the specified *pMD* memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>macLen</i> is less than one or greater than the length of the hash value.

HMACSHA512MessageDigest

Computes the HMAC value of the message.

Syntax

```
IppStatus ippHMACSHA512MessageDigest(const Ipp8u *pSrcMesg, int
    msgLen, const Ipp8u *pKey, int keyLen, Ipp8u *pMAC, int macLen);
```

Parameters

<i>pSrcMesg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length in bytes.
<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey* of the specified key length *keyLen* and applies the keyed hash-based message authentication code scheme to transform the input message into the respective message authentication code *pMAC* of the specified length *macLen*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero and <i>macLen</i> is less than one or greater than the length of the hash value.

HMACMD5GetSize

Gets the size of the IppsHMACMD5State context.

Syntax

```
IppStatus ippSHMACMD5GetSize(int *pSize);
```

Parameters

pSize Pointer to the IppsHMACMD5State context size value.

Description

The function is declared in the `ippcp.h` file. The function gets the IppsHMACMD5State context size in bytes and stores it in *pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

HMACMD5Init

Initializes user-supplied memory as IppsHMACMD5State context for future use.

Syntax

```
IppStatus ippSHMACMD5Init(const Ipp8u *pKey, int keyLen,  
                          IppsHMACMD5State *pCtx);
```


Parameters

<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length in bytes.
<i>pCtx</i>	Pointer to the <code>IppsHMACMD5State</code> context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as the `IppsHMACMD5State` context. The function also sets up the initial chaining digest value according to the Hash algorithm specified by the function base name and computes necessary key material from the supplied key *pKey*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>keyLen</i> is less than one.

HMACMD5Duplicate

Copies one `IppsHMACMD5State` context to another.

Syntax

```
IppStatus ippSHMACMD5Duplicate(const IppsHMACMD5State* pSrcCtx,
                               IppsHMACMD5State* pDstCtx);
```

Parameters

<i>pSrcCtx</i>	Pointer to the source <code>IppsHMACMD5State</code> context.
<i>pDstCtx</i>	Pointer to the source <code>IppsHMACMD5State</code> context to be cloned.

Description

The function is declared in the `ippcp.h` file. The function copies one `IppsHMACMD5State` context to another.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
--------------------------	--

<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

HMAMD5Update

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippSHMAMD5Update(const Ipp8u *pSrcMesg, int mesglen,  
                           IppSHMAMD5State *pCtx);
```

Parameters

<code>pSrcMesg</code>	Pointer to the buffer containing a part of the whole message.
<code>mesglen</code>	Length of the actual part of the message in bytes.
<code>pCtx</code>	Pointer to the <code>IppSHMAMD5State</code> context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length.

The function first integrates the previous partial block with the input message stream and then partitions them into multiple message blocks (as specified by the applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected hash algorithm to transform the block into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.

HMACMD5Final

Completes computation of the HMAC value.

Syntax

```
IppStatus ippsHMACMD5Final(Ipp8u *pMAC, int macLen, IppsHMACMD5State
    *pCtx);
```

Parameters

<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.
<i>pCtx</i>	Pointer to the IppsHMACMD5State context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stores the result into the specified *pMD* memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>macLen</i> is less than one or greater than the length of the hash value.

HMACMD5MessageDigest

Computes the HMAC value of the message.

Syntax

```
IppStatus ippsHMACMD5MessageDigest(const Ipp8u *pSrcMesg, int mesgLen,
    const Ipp8u *pKey, int keyLen, Ipp8u *pMAC, int macLen);
```

Parameters

<i>pSrcMesg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length in bytes.
<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey* of the specified key length *keyLen* and applies the keyed hash-based message authentication code scheme to transform the input message into the respective message authentication code *pMAC* of the specified length *macLen*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero and <i>macLen</i> is less than one or greater than the length of the hash value.

Example 4-2 MD5 HMAC of a Message

```
void HMACMD5_sample(void) {
    // define key
    Ipp8u key[] = "the key for HMAC scheme";

    // define message
    Ipp8u msg[] = "abcdefghijklmnopqrstuvwxyz";

    // as soon as whole message placed into memory
    // one can use integrated primitive
    int macLen = 12;
    Ipp8u mac[16];
    ippsHMACMD5MessageDigest(msg, strlen((char*)msg),
                             key, strlen((char*)key),
                             mac, macLen);
}
```

CMACRijndael128GetSize

Gets the size of the IppsCMACRijndael128State context.

Syntax

```
IppStatus ippsCMACRijndael128GetSize(int *pSize);
```

Parameters

pSize Pointer to the IppsCMACRijndael128State context.

Description

This function is declared in the `ippcp.h` file. It gets the size of the `IppsCMACRijndael128State` context.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

CMACRijndael128Init

Initializes user-supplied memory as
`IppsCMACRijndael128State` *context for future*
use.

Syntax

```
IppStatus ippSCMACRijndael128Init(const Ipp8u* pKey,  
    IppsRijndaelKeyLength keyLen, IppsCMACRijndael128State* pState);
```

Parameters

<code>pKey</code>	Pointer to the Rijndael128 key.
<code>keyLen</code>	Key bytestream length (in bytes) defined by the <code>IppsRijndaelKeyLength</code> enumerator.
<code>pState</code>	Pointer to the <code>IppsCMACRijndael128State</code> being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory at the address of `pState` as the `IppsCMACRijndael128State` context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <code>keyLen</code> is not equal to <code>IppsRijndaelKey128</code> , <code>IppsRijndaelKey192</code> , or <code>IppsRijndaelKey256</code> .

CMACRijndael128Update

Updates the MAC value depending on the current input message stream of the specified length.

Syntax

```
IppStatus ippsCMACRijndael128Update(const Ipp8u *pSrc, int len,  
    IppsCMACRijndael128State* pState);
```

Parameters

<i>pSrc</i>	Pointer to the buffer containing a part or the entire message.
<i>len</i>	Length of the actual part of the message in bytes.
<i>pState</i>	Pointer to the IppsCMACRijndael128State context.

Description

This function is declared in the `ippcp.h` file. The function updates the MAC value depending on the current input message stream of the specified length. The function first integrates the previous partial message block with the input message stream and then partitions the obtained message into multiple message blocks with a possible additional partial block. For each message block, the function uses the Rijndael128 cipher to transform the input block into a new chaining MAC value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

CMACRijndael128Final

Completes computation of the MAC value.

Syntax

```
IppStatus ippsCMACRijndael128Final(Ipp8u *pMD, int mdLen,  
    IppsCMACRijndael128State *pState);
```

Parameters

<i>pMD</i>	Pointer to the MAC value.
<i>mdLen</i>	Specified length of the MAC.
<i>pState</i>	Pointer to the IppsCMACRijndael128State context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stores the result into memory at the address of *pMD*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>mdLen</i> is less than 1 or greater than cipher's data block length.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

CMACRijndael128MessageDigest

Computes the MAC value of the entire message.

Syntax

```
IppStatus ippsCMACRijndael128MessageDigest(const Ipp8u *pMsg, int msgLen,  
    const Ipp8u *pKey, IppsRijndaelKeyLength keyLen, Ipp8u *pMD, int mdLen);
```


Parameters

<i>pMsg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length.
<i>pMD</i>	Pointer to the resulting MAC value.
<i>mdLen</i>	Specified MAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey* of the specified key length *keyLen* and applies keyed cipher-based message authentication code scheme to transform the *entire* input message into the respective message authentication code *pMAC* of the specified length *mdLen*.

The function is actually a wrapper around the [CMACRijndael128Init](#), [CMACRijndael128Update](#), and [CMACRijndael128Final](#) functions. If your application has no access to *all* the necessary data, you can compute the MAC using a typical sequence of invoking the primitives, like the one described at the beginning of the [Message Authentication Functions](#) section. `CMACMessageDigest` is convenient when the application can access the entire message. In this case, using the function, you can compute the result in a single call.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero, <i>mdLen</i> is less than 1 or greater than cipher's data block length, or <i>keyLen</i> value is illegal.

Data Authentication Functions

Intel IPP Data Authentication (DAA) primitive functions are designed for applications to use various DAA schemes based on the set of symmetric cryptography functions described in the [Symmetric Cryptography Primitive Functions](#) chapter.

The implementation of each DAA scheme is presented as a set of primitive functions.

The full list of Intel IPP DAA Functions is given in [Table 4-2](#).

Table 4-2 Intel IPP Data Authentication Functions

Function Base Name	Operation
DAADESGetSize	Gets the size of the IppsDAADESState context.
DAADESInit	Initializes user-supplied memory as IppsDAADESState context for future use.
DAADESUpdate	Digests the current input message stream of the specified length.
DAADESFinal	Completes computation of the DAC value.
DAADESMessageDigest	Computes the DAC value of the message.
DAATDESGetSize	Gets the size of IppsDAATDESState context.
DAATDESInit	Initializes user-supplied memory as IppsDAATDESState context for future use.
DAATDESUpdate	Digests the current input message stream of the specified length.
DAATDESFinal	Completes computation of the DAC value.
DAATDESMessageDigest	Computes the DAC value of the message.
DAARijndael128GetSize	Gets the size of the IppsDAARijndael128State context.
DAARijndael128Init	Initializes user-supplied memory as IppsDAARijndael128State context for future use.
DAARijndael128Update	Digests the current input message stream of the specified length.
DAARijndael128Final	Completes computation of the DAC value.
DAARijndael128MessageDigest	Computes the DAC value of the message.
DAARijndael192GetSize	Gets the size of the IppsDAARijndael192State context.
DAARijndael192Init	Initializes user-supplied memory as IppsDAARijndael192State context for future use.
DAARijndael192Update	Digests the current input message stream of the specified length.
DAARijndael192Final	Completes computation of the DAC value.
DAARijndael192MessageDigest	Computes the DAC value of the message.

Table 4-2 Intel IPP Data Authentication Functions (continued)

Function Base Name	Operation
<u>DAARijndael256GetSize</u>	Gets the size of the <code>IppsDAARijndael256State</code> context.
<u>DAARijndael256Init</u>	Initializes user-supplied memory as <code>IppsDAARijndael256State</code> context for future use.
<u>DAARijndael256Update</u>	Digests the current input message stream of the specified length.
<u>DAARijndael256Final</u>	Completes computation of the DAC value.
<u>DAARijndael256MessageDigest</u>	Computes the DAC value of the message.
<u>DAABlowfishGetSize</u>	Gets the size of the <code>IppsDAABlowfishState</code> context.
<u>DAABlowfishInit</u>	Initializes user-supplied memory as <code>IppsDAABlowfishState</code> context for future use.
<u>DAABlowfishUpdate</u>	Digests the current input message stream of the specified length.
<u>DAABlowfishFinal</u>	Completes computation of the DAC value.
<u>DAABlowfishMessageDigest</u>	Computes the DAC value of the message.
<u>DAATwofishGetSize</u>	Gets the size of the <code>IppsDAATwofishState</code> context.
<u>DAATwofishInit</u>	Initializes user-supplied memory as <code>IppsDAATwofishState</code> context for future use.
<u>DAATwofishUpdate</u>	Digests the current input message stream of the specified length.
<u>DAATwofishFinal</u>	Completes computation of the DAC value.
<u>DAATwofishMessageDigest</u>	Computes the DAC value of the message.

The primitive implementing a DAA scheme uses the context as the operational vehicle to carry all necessary variables to manage computation of the chaining digest value. For example, the primitive implementing the DAA scheme based on the Rijndael128 block cipher uses `ippsDAARijndael128` context.

The function `Init` (`DAARijndael128Init`, `DAADESInit`, and others) initializes the context and sets up the specified initialization vectors. Once initialized, the function `Update` (`DAARijndael128Update`, `DAADESUpdate`, and others) digests the input message stream with the selected hash algorithm till it exhausts all message blocks. The function `Final` (`DAARijndael128Final`, `DAADESFinal`, and others) is designed to pad the partial message block into a final message block with the specified padding scheme, and further uses the hash algorithm to transform the final block into a message digest value.

The following example illustrates how the application code can apply an implemented DAA based on the Rijndael128 block cipher to digest the input message stream:

1. Call the function [DAARijndael128GetSize](#) to get the size required to configure the `ippsDAARijndael128State` context.
2. Ensure that the required memory space is properly allocated. With the allocated memory, call the function [DAARijndael128Init](#) to set up the initial context state.
3. Keep calling the function [DAARijndael128Update](#) to digest incoming message stream in the queue till its completion.
4. Call the function [DAARijndael128Final](#) for padding the final partial block applying Rijndael128 block cipher and transforming ciphertext block into resultant DAA value.
5. Call the operating system memory free service function to release the `IppsDAARijndaelState` context.

DAADESGetSize

Gets the size of the `IppsDAADESState` context.

Syntax

```
IppStatus ippsDAADESGetSize(int *pSize);
```

Parameters

pSize Pointer to the `IppsDAADESState` context.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

DAADESInit

Initializes user-supplied memory as `IppsDAADESState` context for future use.

Syntax

```
IppStatus ippsDAADESInit(const Ipp8u *pKey, IppsDAADESState *pCtx);
```

Parameters

<i>pKey</i>	Pointer to the DES key.
<i>pCtx</i>	Pointer to the <code>IppsDAADESState</code> context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as the `IppsDAADESState` context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operation.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

DAADESUpdate

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippDAADESUpdate(const Ipp8u *pSrcMesg, int mesglen,
                          IppsDAADESState* pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part or the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the <code>IppsDAADESState</code> context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length. The function first integrates the previous partial message block with the input message stream, and then partitions them into multiple message blocks (as specified by applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected block cipher to transform the block of plaintext into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAADESFinal

Completes computation of the DAC value.

Syntax

```
IppStatus ippSDAADESFinal(Ipp8u *pDAC, int dacLen, IppsDAADESState* pCtx);
```

Parameters

<code>pDAC</code>	Pointer to the DAC value.
<code>dacLen</code>	Specified length of the DAC.
<code>pCtx</code>	Pointer to the <code>IppsDAADESState</code> context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stored result into the specified `pDAC` memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <code>dacLen</code> is less than 1 or greater than cipher's data block length.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAADESMessageDigest

Computes the DAC value of the message.

Syntax

```
IppStatus ippsDAADESMessageDigest(const Ipp8u *pSrcMsg, int msgLen,
                                   const Ipp8u *pKey, Ipp8u *pMAC, int macLen);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey</i>	Pointer to the user-supplied key.
<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey* of the specified key length *keyLen* and applied keyed hash-based message authentication code scheme to transform the input message into the respective message authentication code *pMAC* of the specified length *macLen*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero, <i>macLen</i> is less than 1 or greater than cipher's data block length, <i>keyLen</i> value is illegal.

DAATDESGetSize

Gets the size of the IppsDAATDESState context.

Syntax

```
IppStatus ippsDAATDESGetSize(int *pSize);
```

Parameters

pSize Pointer to the `IppsDAATDESState` context.

Return Values

`ippStsNoErr` Indicates no error. Any other value indicates an error or warning.

`ippStsNullPtrErr` Indicates an error condition if any of the specified pointers is NULL.

DAATDESInit

Initializes user-supplied memory as the `IppsDAATDESState` context for future use.

Syntax

```
IppStatus ippDAATDESInit(const Ipp8u* pKey, const Ipp8u* pKey2, const  
                        Ipp8u* pKey3, IppsDAATDESState* pCtx);
```

Parameters

pKey Pointer to the TDES key.

pKey2 Pointer to the TDES key.

pKey3 Pointer to the TDES key.

pCtx Pointer to the `IppsDAATDESState` context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as the `IppsDAATDESState` context. In addition, `DAATDESInit` uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

`ippStsNoErr` Indicates no error. Any other value indicates an error or warning.

`ippStsNullPtrErr` Indicates an error condition if any of the specified pointers is NULL.

DAATDESUpdate

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippsDAATDESUpdate(const Ipp8u *pSrcMesg, int mesglen,  
                             IppsDAATDESState *pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part or the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the <code>IppsDAATDESState</code> context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length. The function first integrates the previous partial message block with the input message stream, and then partitions them into multiple message blocks (as specified by applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected block cipher to transform the block of plaintext into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAATDESFinal

Completes computation of the DAC value.

Syntax

```
IppStatus ippsDAATDESFinal(Ipp8u *pDAC, int dacLen, IppsDAATDESState* pCtx);
```

Parameters

<i>pDAC</i>	Pointer to the DAC value.
<i>dacLen</i>	Specified length of the DAC.
<i>pCtx</i>	Pointer to the IppsDAATDESState context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stored result into the specified *pDAC* memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>dacLen</i> is less than 1 or greater than cipher's data block length.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAATDESMessageDigest

Computes the DAC value of the message.

Syntax

```
IppStatus ippsDAATDESMessageDigest(const Ipp8u *pSrcMsg, int msgLen,  
    const Ipp8u *pKey1, const Ipp8u *pKey2, const Ipp8u *pKey3, Ipp8u  
    *pMAC, int macLen);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey1</i>	Pointer to the user-supplied key.
<i>pKey2</i>	Pointer to the user-supplied key.
<i>pKey3</i>	Pointer to the user-supplied key.
<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey1*, *pKey2*, and *pKey3* of the specified key length *keyLen* and applied keyed hash-based message authentication code scheme to transform the input message into the respective message authentication code *pMAC* of the specified length *macLen*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero, <i>macLen</i> is less than 1 or greater than cipher's data block length, <i>keyLen</i> value is illegal.

DAARijndael128GetSize

Gets the size of the IppsDAARijndael128State context.

Syntax

```
IppStatus ippDAARijndael128GetSize(int *pSize);
```

Parameters

<i>pSize</i>	Pointer to the IppsDAARijndael128State context.
--------------	---

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
--------------------------	--

`ippStsNullPtrErr` Indicates an error condition if any of the specified pointers is NULL.

DAARijndael128Init

*Initializes user-supplied memory as
IppsDAARijndael128State context for future
use.*

Syntax

```
IppStatus ippSDAARijndael128Init(const Ipp8u* pKey,  
                                IppsRijndaelKeyLength keyLen, IppsDAARijndael128State* pCtx);
```

Parameters

<code>pKey</code>	Pointer to the Rijndael128 key.
<code>keyLen</code>	Key byte stream length in bytes defined by the IppsRijndaelKeyLength enumerator.
<code>pCtx</code>	Pointer to the IppsDAARijndael128State being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed to by `pCtx` as the `IppsDAARijndael128State` context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <code>keyLen</code> is not equal to <code>IppsRijndaelKey128</code> , <code>IppsRijndaelKey192</code> , or <code>IppsRijndaelKey256</code> .

DAARijndael128Update

Digest the current input message stream of the specified length.

Syntax

```
IppStatus ippsDAARijndael128Update(const Ipp8u *pSrcMesg, int mesglen,
    IppsDAARijndael128State* pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part or the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the IppsDAARijndael128State context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length. The function first integrates the previous partial message block with the input message stream, and then partitions them into multiple message blocks (as specified by applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected block cipher to transform the block of plaintext into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAARijndael128Final

Completes computation of the DAC value.

Syntax

```
IppStatus ippsDAARijndael128Final(Ipp8u *pDAC, int dacLen,  
    IppsDAARijndael128State *pCtx);
```

Parameters

<i>pDAC</i>	Pointer to the DAC value.
<i>dacLen</i>	Specified length of the DAC.
<i>pCtx</i>	Pointer to the IppsDAARijndael128State context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stored result into the specified *pDAC* memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>dacLen</i> is less than 1 or greater than cipher's data block length.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAARijndael128MessageDigest

Computes the DAC value of the message.

Syntax

```
IppStatus ippsDAARijndael128MessageDigest(const Ipp8u *pSrcMsg, int msgLen,  
    const Ipp8u *pKey, IppsRijndaelKeyLength keyLen, Ipp8u *pMAC, int macLen);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length.
<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey* of the specified key length *keyLen* and applied keyed hash-based message authentication code scheme to transform the input message into the respective message authentication code *pMAC* of the specified length *macLen*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero, <i>macLen</i> is less than 1 or greater than cipher's data block length, <i>keyLen</i> value is illegal.

DAARijndael192GetSize

Gets the size of the IppsDAARijndael192State context.

Syntax

```
IppStatus ippDAARijndael192GetSize(int *pSize);
```

Parameters

<i>pSize</i>	Pointer to the IppsDAARijndael192State context.
--------------	---

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

DAARijndael192Init

Initializes user-supplied memory as

IppsDAARijndael192State context for future use.

Syntax

```
IppStatus ippsRijndael192Init(const Ipp8u* pKey,  
                             IppsDAARijndaelKeyLength keyLen, IppsRijndael192State* pCtx);
```

Parameters

<i>pKey</i>	Pointer to the Rijndael192 key.
<i>keyLen</i>	Key byte stream length in bytes defined by the <code>IppsRijndaelKeyLength</code> enumerator.
<i>pCtx</i>	Pointer to the <code>IppsDAARijndael192State</code> being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as the `IppsDAARijndael192State` context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>keyLen</i> is not equal to <code>IppsRijndaelKey128</code> , <code>IppsRijndaelKey192</code> , or <code>IppsRijndaelKey256</code> .

DAARijndael192Update

Digest the current input message stream of the specified length.

Syntax

```
IppStatus ippsDAARijndael192Update(const Ipp8u *pSrcMesg, int mesglen,
    IppsDAARijndael192State* pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part or the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the IppsDAARijndael192State context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length. The function first integrates the previous partial message block with the input message stream, and then partitions them into multiple message blocks (as specified by applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected block cipher to transform the block of plaintext into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAARijndael192Final

Completes computation of the DAC value.

Syntax

```
IppStatus ippsDAARijndael192Final(Ipp8u *pDAC, int dacLen,  
    IppsDAARijndael192State *pCtx);
```

Parameters

<i>pDAC</i>	Pointer to the DAC value.
<i>dacLen</i>	Specified length of the DAC.
<i>pCtx</i>	Pointer to the IppsDAARijndael192State context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stored result into the specified *pDAC* memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>dacLen</i> is less than 1 or greater than cipher's data block length.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAARijndael192MessageDigest

Computes the DAC value of the message.

Syntax

```
IppStatus ippsDAARijndael192MessageDigest(const Ipp8u *pSrcMsg, int msgLen,  
    const Ipp8u *pKey, IppsRijndaelKeyLength keyLen, Ipp8u *pDAC, int macLen);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length.
<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey* of the specified key length *keyLen* and applied keyed hash-based message authentication code scheme to transform the input message into the respective message authentication code *pMAC* of the specified length *macLen*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero, <i>macLen</i> is less than 1 or greater than cipher's data block length, <i>keyLen</i> value is illegal.

DAARijndael256GetSize

Gets the size of the IppsDAARijndael256State context.

Syntax

```
IppStatus ippDAARijndael256GetSize(int *pSize);
```

Parameters

<i>pSize</i>	Pointer to the IppsDAARijndael256State context.
--------------	---

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

DAARijndael256Init

Initializes user-supplied memory as

IppsDAARijndael256State context for future use.

Syntax

```
IppStatus ippsRijndael256Init(const Ipp8u* pKey,  
                             IppsDAARijndaelKeyLength keyLen, IppsRijndael256State* pCtx);
```

Parameters

<i>pKey</i>	Pointer to the Rijndael256 key.
<i>keyLen</i>	Key byte stream length in bytes defined by the <code>IppsRijndaelKeyLength</code> enumerator.
<i>pCtx</i>	Pointer to the <code>IppsDAARijndael256State</code> being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as the `IppsDAARijndael256State` context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>keyLen</i> is not equal to <code>IppsRijndaelKey128</code> , <code>IppsRijndaelKey192</code> , or <code>IppsRijndaelKey256</code> .

DAARijndael256Update

Digest the current input message stream of the specified length.

Syntax

```
IppStatus ippsDAARijndael256Update(const Ipp8u *pSrcMesg, int mesglen,
    IppsDAARijndael256State* pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part or the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the IppsDAARijndael256State context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length. The function first integrates the previous partial message block with the input message stream, and then partitions them into multiple message blocks (as specified by applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected block cipher to transform the block of plaintext into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAARijndael256Final

Completes computation of the DAC value.

Syntax

```
IppStatus ippsDAARijndael256Final(Ipp8u *pDAC, int dacLen,  
    IppsDAARijndael256State* pCtx);
```

Parameters

<i>pDAC</i>	Pointer to the DAC value.
<i>dacLen</i>	Specified length of the DAC.
<i>pCtx</i>	Pointer to the IppsDAARijndael256State context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stored result into the specified *pDAC* memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>dacLen</i> is less than 1 or greater than cipher's data block length.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAARijndael256MessageDigest

Computes the DAC value of the message.

Syntax

```
IppStatus ippsDAARijndael256MessageDigest(const Ipp8u *pSrcMsg, int msgLen,  
    const Ipp8u *pKey, IppsRijndaelKeyLength keyLen, Ipp8u *pMAC, int macLen);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length.
<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey* of the specified key length *keyLen* and applied keyed hash-based message authentication code scheme to transform the input message into the respective message authentication code *pMAC* of the specified length *macLen*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero, <i>macLen</i> is less than 1 or greater than cipher's data block length, <i>keyLen</i> value is illegal.

DAABlowfishGetSize

Gets the size of the IppsDAABlowfishState context.

Syntax

```
IppStatus ippDAABlowfishGetSize(int *pSize);
```

Parameters

<i>pSize</i>	Pointer to the IppsDAABlowfishState context.
--------------	--

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

DAABlowfishInit

*Initializes user-supplied memory as
IppsDAABlowfishState context for future use.*

Syntax

```
IppStatus ippsDAABlowfishInit(const Ipp8u* pKey, int keylen,  
    IppsDAABlowfishState* pCtx);
```

Parameters

<i>pKey</i>	Pointer to the DAABlowfish key.
<i>keylen</i>	Key byte stream length in bytes.
<i>pCtx</i>	Pointer to the IppsDAABlowfishState context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as the `IppsDAABlowfishState` context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>keylen</i> is less than 1 or greater than 56.

DAABlowfishUpdate

*Digests the current input message stream of the
specified length.*

Syntax

```
IppStatus ippsDAABlowfishUpdate(const Ipp8u *pSrcMesg, int mesglen,  
    IppsDAABlowfishState* pCtx);
```


Parameters

<i>pSrcMsg</i>	Pointer to the buffer containing a part or the whole message.
<i>msglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the <code>IppsDAABlowfishState</code> context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length. The function first integrates the previous partial message block with the input message stream, and then partitions them into multiple message blocks (as specified by applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected block cipher to transform the block of plaintext into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAABlowfishFinal

Completes computation of the DAC value.

Syntax

```
IppStatus ippDAABlowfishFinal(Ipp8u *pDAC, int dacLen,
                              IppsDAABlowfishState* pCtx);
```

Parameters

<i>pDAC</i>	Pointer to the DAC value.
<i>dacLen</i>	Specified length of the DAC.
<i>pCtx</i>	Pointer to the <code>IppsDAABlowfishState</code> context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stored result into the specified *pDAC* memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>dacLen</i> is less than 1 or greater than cipher's data block length.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAABlowfishMessageDigest

Computes the DAC value of the message.

Syntax

```
IppStatus ippDAABlowfishMessageDigest(const Ipp8u *pSrcMsg, int msgLen,  
                                       const Ipp8u *pKey, int keyLen, Ipp8u *pMAC, int macLen);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length.
<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey* of the specified key length *keyLen* and applied keyed hash-based message authentication code scheme to transform the input message into the respective message authentication code *pMAC* of the specified length *macLen*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero, <i>macLen</i> is less than 1 or greater than cipher's data block length, <i>keyLen</i> value is illegal.

DAATwofishGetSize

Gets the size of the `IppsDAATwofishState` context.

Syntax

```
IppStatus ippDAATwofishGetSize(int *pSize);
```

Parameters

pSize Pointer to the `IppsDAATwofishState` context.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

DAATwofishInit

Initializes user-supplied memory as `IppsDAATwofishState` context for future use.

Syntax

```
IppStatus ippDAATwofishInit(const Ipp8u* pKey, int keylen,
                             IppsDAATwofishState* pCtx);
```

Parameters

pKey Pointer to the DAATwofish key.
keylen Key byte stream length in bytes.

pCtx Pointer to the `IppsDAATwofishState` context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as the `IppsDAATwofishState` context. In addition, the function uses the key to provide all necessary key material for both encryption and decryption operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if <i>keylen</i> is less than 1 or greater than 32.

DAATwofishUpdate

Digests the current input message stream of the specified length.

Syntax

```
IppStatus ippDAATwofishUpdate(const Ipp8u *pSrcMesg, int mesglen,  
                              IppsDAATwofishState* pCtx);
```

Parameters

<i>pSrcMesg</i>	Pointer to the buffer containing a part or the whole message.
<i>mesglen</i>	Length of the actual part of the message in bytes.
<i>pCtx</i>	Pointer to the <code>IppsDAATwofishState</code> context.

Description

This function is declared in the `ippcp.h` file. The function digests the current input message stream of the specified length. The function first integrates the previous partial message block with the input message stream, and then partitions them into multiple message blocks (as specified by applied hash algorithm) with a possible additional partial block. For each message block, the function uses the selected block cipher to transform the block of plaintext into a new chaining digest value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the input data stream length is less than zero.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAATwofishFinal

Completes computation of the DAC value.

Syntax

```
IppStatus ippDAATwofishFinal(Ipp8u *pDAC, int dacLen,
                             IppsDAATwofishState* pCtx);
```

Parameters

<code>pDAC</code>	Pointer to the DAC value.
<code>dacLen</code>	Specified length of the DAC.
<code>pCtx</code>	Pointer to the <code>IppsDAATwofishState</code> context.

Description

This function is declared in the `ippcp.h` file. The function completes calculation of the digest value and stored result into the specified `pDAC` memory.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <code>dacLen</code> is less than 1 or greater than cipher's data block length.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

DAATwofishMessageDigest

Computes the DAC value of the message.

Syntax

```
IppStatus ippsDAATwofishMessageDigest(const Ipp8u *pSrcMsg, int msgLen,  
    const Ipp8u *pKey, int keyLen, Ipp8u *pMAC, int macLen);
```

Parameters

<i>pSrcMsg</i>	Pointer to the input message.
<i>msgLen</i>	Message length in bytes.
<i>pKey</i>	Pointer to the user-supplied key.
<i>keyLen</i>	Key length.
<i>pMAC</i>	Pointer to the resultant HMAC value.
<i>macLen</i>	Specified HMAC length.

Description

This function is declared in the `ippcp.h` file. The function takes the input key *pKey* of the specified key length *keyLen* and applied keyed hash-based message authentication code scheme to transform the input message into the respective message authentication code *pMAC* of the specified length *macLen*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>msgLen</i> is less than zero, <i>macLen</i> is less than 1 or greater than cipher's data block length, <i>keyLen</i> value is illegal.

Public Key Cryptography Functions



This chapter introduces Intel® Integrated Performance Primitives (Intel® IPP) [Big Number Arithmetic](#), [Montgomery Reduction Scheme Functions](#), [Pseudorandom Number Generation Functions](#), [Prime Number Generation Functions](#), [RSA Algorithm Functions](#), [Discrete-Logarithm-Based Cryptography Functions](#), and [Elliptic Curve Cryptography Functions](#) (ECC) functions.

Big Number Arithmetic

This section describes primitives for performing arithmetic operations with integer big numbers of variable length.

The full list of functions for big number arithmetic is given in [Table 5-1](#).

Table 5-1 Intel IPP Big Number Arithmetic Functions

Function Base Name	Operation
Unsigned Big Number Arithmetic Functions	
Add_BNU	Adds two unsigned integer big numbers of the same length.
Sub_BNU	Subtracts one integer big number from another integer big number of the same length.
MulOne_BNU	Multiplies unsigned integer big number by 32-bit unsigned integer.
MACOne_BNU_I	Computes multiplication of unsigned integer big number by 32-bit integer and accumulates the result with another integer big number.
Mul_BNU4	Multiplies two unsigned integers of 4*32 bits.
Mul_BNU8	Multiplies two unsigned integers of 8*32 bits.
Div_64u32u	Divides unsigned 64-bit integer by unsigned 32-bit integer.
Sqr_32u64u	Computes the square of 32-bit words in the input array.
Sqr_BNU4	Computes the square of an unsigned integer big number of 4*32 bits.

Table 5-1 Intel IPP Big Number Arithmetic Functions (continued)

Function Base Name	Operation
Sqr_BNU8	Computes the square of an unsigned integer big number of 8*32 bits.
SetOctString_BNU	Converts octet string into unsigned integer big number.
GetOctString_BNU	Converts unsigned integer big number into octet string.
Signed Big Number Arithmetic Functions	
BigNumGetSize	Gets the size of the IppsBigNumState context.
BigNumInit	Initializes context and partitions allocated buffer.
Set_BN	Defines the sign and value of the context.
SetOctString_BN	Converts octet string into a positive Big Number.
GetSize_BN	Returns the maximum length of the integer big number the structure can store.
Get_BN	Extracts the sign and value of the integer big number from the input structure.
GetOctString_BN	Converts a positive Big Number into octet String.
Cmp_BN	Compares two Big Numbers.
CmpZero_BN	Checks the value of the input data field.
Add_BN	Adds two integer big numbers.
Sub_BN	Subtracts one integer big number from another.
Mul_BN	Multiplies two integer big numbers.
MAC_BN_I	Multiplies two integer big numbers and accumulates the result with the third integer big number.
Div_BN	Divides one integer big number by another.
Mod_BN	Computes modular reduction for input integer big number with respect to specified modulus.
Gcd_BN	Computes the greatest common divisor.
ModInv_BN	Computes multiplicative inverse of a positive integer big number with respect to specified modulus.

The magnitude of an integer big number is specified by an array of unsigned integer data type `Ipp32u rp[length]` and corresponds to the mathematical value

$$r = \sum_{0 \leq i < length} rp[i] \times 2^{32i}.$$

This section uses the following definition to define the sign of an integer big number:

```
typedef enum {
    IppsBigNumNEG=0,
```



```
IppsBigNumPOS=1  
} IppsBigNumSGN;
```

The functions described in this section use the context `IppsBigNumState` to serve as an operational vehicle that carries not only the sign and value of the data, but also a sufficient working buffer reserved for various arithmetic operations. The length of the context `IppsBigNumState` is defined as the length of the data carried by the structure and the size of the context `IppsBigNumState` is therefore defined as the maximal length of the data that this operational vehicle can carry.



NOTE. In all unsigned big number arithmetic functions described below, integers pointed to by a , b , and r are all of $(n*32)$ bits.

Add_BNU

Adds two unsigned integer big numbers of the same length.

Syntax

```
IppStatus ippsAdd_BNU(const Ipp32u *a, const Ipp32u *b, Ipp32u *r, int n,  
    Ipp32u *carry);
```

Parameters

a	First unsigned integer big number of $n*32$ bits.
b	Second unsigned integer big number of $n*32$ bits.
n	Size specified for the input parameters a and b and the result parameter r . The size is expressed in the number of 32-bit words.
r	On output, addition result.
$carry$	On output, addition carry. The possible value is 0 or 1.

Description

This function is declared in the `ippcp.h` file. The function adds two unsigned integer big numbers of the same length and returns the result of the operation with a possible carry.

The following pseudocode represents this function:

$$(carry|(*r)) \leftarrow (*a) + (*b).$$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>n</i> is less than or equal to 0.

Sub_BNU

Subtracts one integer big number from another integer big number of the same length.

Syntax

```
IppStatus ippSub_BNU(const Ipp32u *a, const Ipp32u *b, Ipp32u *r, int n,
                    Ipp32u *carry);
```

Parameters

<i>a</i>	First unsigned integer big number of <i>n</i> *32 bits.
<i>b</i>	Second unsigned integer big number of <i>n</i> *32 bits.
<i>n</i>	Size specified for the input parameters <i>a</i> and <i>b</i> and the result parameter <i>r</i> . The size is expressed in the number of 32-bit words.
<i>r</i>	Subtraction result.
<i>carry</i>	Subtraction borrow. Possible value is 0 or 1.

Description

This function is declared in the `ippcp.h` file. The function subtracts one integer big number from another big-number integer of the same length and returns the result of the operation with a possible borrow returned as the *carry* argument.

The following pseudo code represents this function:

$$(*r) \leftarrow (*a) - (*b);$$

$$carry = ((*a) < (*b)).$$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if n is less than or equal to 0.

MulOne_BNU

Multiplies unsigned integer big number by 32-bit unsigned integer.

Syntax

```
IppStatus ippMulOne_BNU(const Ipp32u *a, Ipp32u *r, int n, Ipp32u w,
                        Ipp32u *carry);
```

Parameters

a	Unsigned integer big number of $n \cdot 32$ bits.
w	Unsigned long integer of 32 bits serving as multiplier for the operation.
n	Size specified for the input parameter a and the result parameter r . The size is expressed in the number of 32-bit words.
r	Multiplication result.
$carry$	Multiplication carry.

Description

This function is declared in the `ippcp.h` file. The function computes the multiplication of an unsigned integer big number a by a 32-bit unsigned integer w . The function returns the result to the length array r , and the carry of the result to $carry$.

The following pseudo code represents this function:

$$(carry | (*r)) \leftarrow (*a)w.$$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if n is less than or equal to 0.

MACOne_BNU_I

Computes multiplication of unsigned integer big number by 32-bit integer and accumulates the result with another integer big number.

Syntax

```
IppStatus ippsMACOne_BNU_I(const Ipp32u *a, Ipp32u *r, int n, Ipp32u w,
                          Ipp32u *carry);
```

Parameters

<i>a</i>	Multiplicand, an unsigned integer big number.
<i>w</i>	32-bit unsigned long integer multiplier.
<i>n</i>	Size specified for the input parameters <i>a</i> and the result parameter <i>r</i> . The size is expressed in the number of 32-bit words.
<i>r</i>	Unsigned integer big number accumulator.
<i>carry</i>	Operation carry.

Description

This function is declared in the `ippcp.h` file. The function computes the multiplication of an unsigned integer big number *a* by a 32-bit integer *w* and accumulates the result with another integer big number *r* of same length. The result of the operation is returned to *r*, and the carry of the result is returned as *carry*.

The following pseudo code represents this function:

$$(carry|(r)) \leftarrow (r) + (a)w.$$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>n</i> is less than or equal to 0.

Mul_BNU4

*Multiplies two unsigned integers of 4*32 bits.*

Syntax

```
IppStatus ippsMul_BNU4(const Ipp32u *a, const Ipp32u *b, Ipp32u *r);
```

Parameters

<i>a</i>	Multiplicand, an integer big number of 4*32 bits.
<i>w</i>	Multiplier, an integer big number of 4*32 bits.
<i>r</i>	Multiplication result of 8*32 bits.

Description

This function is declared in the `ippcp.h` file. The function multiplies two unsigned integers of 4*32 bit and returns the result of 8*32 bits.

The following pseudo code represents this function:

$$(*r) \leftarrow (*a)(*b).$$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

Mul_BNU8

*Multiplies two unsigned integers of 8*32 bits.*

Syntax

```
IppStatus ippsMul_BNU8(const Ipp32u *a, const Ipp32u *b, Ipp32u *r);
```

Parameters

<i>a</i>	Multiplicand, an integer big number of 8*32 bits.
<i>b</i>	Multiplier, an integer big number of 8*32 bits.
<i>r</i>	Multiplication result of 16*32 bits.

Description

This function is declared in the `ippcp.h` file. The function multiplies two unsigned integers of 8×32 bit and returns the result of 16×32 bits.

The following pseudo code represents this function:

$$(*r) \leftarrow (*a)(*b).$$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

Div_64u32u

Divides unsigned 64-bit integer by unsigned 32-bit integer.

Syntax

```
IppStatus ippDiv_64u32u(Ipp64u a, Ipp32u b, Ipp32u *q, Ipp32u *r);
```

Parameters

<i>a</i>	Dividend of 64 bits.
<i>b</i>	Divisor of 32 bits.
<i>q</i>	Quotient of 32 bits.
<i>r</i>	Remainder of 32 bits.

Description

This function is declared in the `ippcp.h` file. The function divides a 64-bit unsigned integer dividend by a 32-bit unsigned divisor and returns the quotient and remainder.

The following pseudo code represents this function:

$$a = b * q + r.$$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
--------------------------	--

<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if the quotient is not 32 bits. In this case the result is undefined.
<code>ippStsDivByZeroErr</code>	Indicates an error condition if zero divisor is used.

Sqr_32u64u

Computes the square of 32-bit words in the input array.

Syntax

```
IppStatus ippsSqr_32u64u(const Ipp32u *src, int n, Ipp64u *dst);
```

Parameters

<code>src</code>	Array of 32-bit words.
<code>n</code>	Input array size.
<code>dst</code>	Pointer to the result.

Description

This function is declared in the `ippcp.h` file. The function computes the square of each unsigned 32-bit long word in the input array. The result of the operation is stored in the array of 64-bit unsigned integers.

The following pseudo code represents this function:

$dst[i] \leftarrow src[i] * src[i]$ where $i = 0, 1, 2, \dots, n-1$.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if n is less than or equal to 0.

Sqr_BNU4

*Computes the square of an unsigned integer big number of 4*32 bits.*

Syntax

```
IppStatus ippsSqr_BNU4(const Ipp32u *a, Ipp32u *r);
```

Parameters

<i>a</i>	Multiplicand of 4*32 bits.
<i>r</i>	Square operation result of 8*32 bits.

Description

This function is declared in the `ippcp.h` file. The function computes the square of an unsigned integer big number of 4*32 bits and stores the result in the memory.

The following pseudo code represents this function:

$$(*r) \leftarrow (*a)(*a).$$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

Sqr_BNU8

*Computes the square of an unsigned integer big number of 8*32 bits.*

Syntax

```
IppStatus ippsSqr_BNU8(const Ipp32u *a, Ipp32u *r);
```

Parameters

<i>a</i>	Multiplicand of 8*32 bits.
<i>r</i>	Square operation result of 16*32 bits.

Description

This function is declared in the `ippcp.h` file. The function computes the square of an unsigned integer big number of 8×32 bits and stores the result in the memory.

The following pseudo code represents this function:

```
(*r) <- (*a) (*a)
```

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

SetOctString_BNU

Converts octet string into unsigned integer big number.

Syntax

```
IppStatus ippSetOctString_BNU(const Ipp8u* pOctStr, int strLen, Ipp32u*  
    pBNU, int* pBNUsize);
```

Parameters

<code>pOctStr</code>	Pointer to the input octet string <code>OctStr</code> .
<code>strLen</code>	Length of the octet string.
<code>pBNU</code>	Pointer to the unsigned integer big number BNU.
<code>pBNUsize</code>	Pointer to the size (in 32-bit items) of the unsigned integer big number.

Description

The function is declared in the `ippcp.h` file. This function converts octet string into unsigned integer big number.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if specified <code>strLen</code> is less than 1.

ippStsSizeErr

Indicates an error condition if specified **pBNUsize* is not sufficient for keeping actual *strLen*.

GetOctString_BNU

Converts unsigned integer big number into octet string.

Syntax

```
IppStatus ippGetOctString_BNU(const Ipp32u* pBNU, int bnuSize, Ipp8u*
    pOctStr, int strLen);
```

Parameters

pBNU

Pointer to the source unsigned integer big number BNU.

bnuSize

Unsigned integer big number size (in 32-bit intems)

pOctStr

Pointer to the source octet string OctStr.

strLen

Length of the octet string.

Description

The function is declared in the `ippcp.h` file. This function converts unsigned integer big number BNU into octet string OctStr.

Return Values

ippStsNoErr

Indicates no error. Any other value indicates an error or warning.

ippStsNullPtrErr

Indicates an error condition if any of the specified pointers is NULL.

ippStsLengthErr

Indicates an error condition if specified *bnuSize* is less than 1.

ippStsRangeErr

Indicates an error condition if 256^{strLen} is less than the unsigned integer big number.

BigNumGetSize

Gets the size of the `IppsBigNumState` context in bytes.

Syntax

```
IppStatus ippsBigNumGetSize(int length, int *size);
```

Parameters

<i>length</i>	Integer big number length in <code>Ipp32u</code> .
<i>size</i>	Size of the buffer in bytes required for initialization.

Description

This function is declared in the `ippcp.h` file. The function specifies the buffer size required to define a structuralized working buffer of the context `IppsBigNumState` for the storage and operations on an integer big number in bytes.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if <i>length</i> is less than or equal to 0.

BigNumInit

Initializes context and partitions allocated buffer.

Syntax

```
IppStatus ippsBigNumInit(int length, IppsBigNumState *b);
```

Parameters

<i>length</i>	Size of the big number for the context initialization.
<i>b</i>	Pointer to the supplied buffer used to store the initialized context <code>IppsBigNumState</code> .

Description

This function is declared in the `ippcp.h` file. The function initializes the context `IppsBigNumState` using the specified buffer space and partitions the given buffer to store and execute arithmetic operations on an integer big number of the *length* size.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>length</i> is less than or equal to 0.

Set_BN

Defines the sign and value of the context.

Syntax

```
IppStatus ippSet_BN(IppsBigNumSGN sgn, int length, const Ipp32u *data,
                   IppsBigNumState *x);
```

Parameters

<i>sgn</i>	Sign of <code>IppsBigNumState *x</code> .
<i>length</i>	Array length of the input data.
<i>data</i>	Data array.
<i>x</i>	On output, the context <code>IppsBigNumState</code> updated with the input data.

Description

This function is declared in the `ippcp.h` file. The function defines the sign and value for `IppsBigNumState *x` with the specified inputs `IppsBigNumSGN sgn` and `const Ipp32u *data`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsLengthErr</code>	Indicates an error condition if <i>length</i> is less than or equal to 0.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if <i>length</i> is more than the size of <code>IppsBigNumState *x</code> .
<code>ippStsBadArgErr</code>	Indicates an error condition if the big number is set to zero with the negative sign.

Example 5-1 Create a Big Number

```
IppsBigNumState* New_BN(int size, const Ipp32u* pData=0){
    // get the size of the Big Number context
    int ctxSize;
    ippBigNumGetSize(size, &ctxSize);
    // allocate the Big Number context
    IppsBigNumState* pBN = (IppsBigNumState*)( new Ipp8u [ctxSize] );
    // and initialize one
    ippBigNumInit(size, pBN);

    // if any data was supplied, then set up the Big Number value
    if(pData)
        ippSet_BN(IppsBigNumPOS, size, pData, pBN);

    // return pointer to the Big Number context for future use
    return pBN;
}
```

SetOctString_BN

Converts octet string into a positive Big Number.

Syntax

```
IppStatus ippSetOctString_BN(const Ipp8u* pOctStr, int strLen,
    IppsBigNumState* pBN);
```

Parameters

<i>pOctStr</i>	Pointer to the input octet string.
<i>strLen</i>	Octet string length in bytes.
<i>pBN</i>	Pointer to the context of the output Big Number.

Description

The function is declared in the `ippcp.h` file. This function converts octet string into a positive Big Number.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if specified <i>strLen</i> is less than 1.
<code>ippStsSizeErr</code>	Indicates an error condition if insufficient space has been reserved for Big Number.

Example 5-2 Create a Big Number from a String

```
void Set_BN_sample(void){
    // desired value of Big Number is 0x123456789abcdef0fedcba9876543210
    Ipp8u desiredBNvalue[] = "\x12\x34\x56\x78\x9a\xbc\xde\xfo"
                             "\xfe\xdc\xba\x98\x76\x54\x32\x10";

    // estimate required size of Big Number
    //int size = (sizeof(desiredBNvalue)+3)/4;
    int size = (sizeof(desiredBNvalue)-1+3)/4;

    // and create new (and empty) one
    IppsBigNumState* pBN = New_BN(size);

    // set up the value from the string
    ippsSetOctString_BN(desiredBNvalue, sizeof(desiredBNvalue)-1, pBN);

    Type_BN("Big Number value is:\n", pBN);
}
```

GetSize_BN

Returns the maximum length of the integer big number the structure can store.

Syntax

```
IppStatus ippsGetSize_BN(const IppsBigNumState *b, int *size);
```

Parameters

<i>b</i>	Integer big number of the data type IppsBigNumState.
<i>size</i>	Maximum length of the integer big number.

Description

This function is declared in the `ippcp.h` file. The function evaluates the working buffer assigned to the context `IppsBigNumState` and returns the size of the structure to indicate the maximum length of the integer big number that the structure can store.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

Get_BN

Extracts the sign and value of the integer big number from the input structure.

Syntax

```
IppStatus ippGet_BN(IppsBigNumSGN *sgn, int *length, Ipp32u *data, const
    IppsBigNumState *x);
```

Parameters

<i>sgn</i>	Sign of <code>IppsBigNumState *x</code> .
<i>length</i>	Array length of the input data.
<i>data</i>	Data array.
<i>x</i>	Integer big number of the context <code>IppsBigNumState</code> .

Description

This function is declared in the `ippcp.h` file. The function extracts the sign and value of the integer big number from the input structure.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

GetOctString_BN

Converts a positive Big Number into octet String.

Syntax

```
IppStatus ippsGetOctString_BN(const Ipp8u* pOctStr, int strLen, const  
    IppsBigNumState* pBN);
```

Parameters

<i>pOctStr</i>	Pointer to the input octet string.
<i>strLen</i>	Octet string length in bytes.
<i>pBN</i>	Pointer to the context of the input Big Number.

Description

The function is declared in the `ippcp.h` file. This function converts a positive Big Number into the octet string.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if specified <i>pOctStr</i> is insufficient in length.
<code>ippStsRangeErr</code>	Indicates an error condition if Big Number is negative.

Example 5-3 Type a Big Number

```
void Type_BN(const char* pMsg, const IppsBigNumState* pBN){
    // size of Big Number
    int size;
    ippsGetSize_BN(pBN, &size);

    // extract Big Number value and convert it to the string presentation
    Ipp8u* bnValue = new Ipp8u [size*4];
    ippsGetOctString_BN(bnValue, size*4, pBN);

    // type header
    if(pMsg)
        cout <<pMsg;

    // type value
    for(int n=0; n<size*4; n++)
        cout<<hex <<(int)bnValue[n];
    cout <<endl;

    delete [] bnValue;
}
```

Cmp_BN

Compares two Big Numbers.

Syntax

```
IppStatus ippsCmp_BN(const IppsBigNumState *pA, const IppsBigNumState
    *pB, Ipp32u *pResult);
```

Parameters

<i>pA</i>	Pointer to the context of the Big Number A.
<i>pB</i>	Pointer to the context of the Big Number B.
<i>pResult</i>	Pointer to the result of the comparison.

Description

The function is declared in `ippcp.h` file. This function compares Big Numbers A and B and sets up the result according to the following conditions:

- if $A=B$, then *pResult* = `IS_ZERO`
- if $A > B$, then *pResult* = `GREATER_THAN_ZERO`
- if $A < B$, then *pResult* = `LESS_THAN_ZERO`

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

CmpZero_BN

Checks the value of the input data field.

Syntax

```
IppStatus ippCmpZero_BN(const IppsBigNumState *b, Ipp32u *result);
```

Parameters

<i>b</i>	Integer big number of the data type <code>IppsBigNumState</code> .
<i>result</i>	Indicates whether the input integer big number is positive, negative, or zero.

Description

This function is declared in the `ippcp.h` file. The function scans the data field of the input `const IppsBigNumState *b` and returns

- `IS_ZERO` if the value held by `IppsBigNumState *b` is zero
- `GREATER_THAN_ZERO` if the input is more than zero
- `LESS_THAN_ZERO` if the input is less than zero.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

Add_BN

Adds two integer big numbers.

Syntax

```
IppStatus ippAdd_BN(IppsBigNumState *a, IppsBigNumState *b,
                    IppsBigNumState *r);
```

Parameters

<i>a</i>	First unsigned integer big number of the data type <code>IppsBigNumState</code> .
<i>b</i>	Second unsigned integer big number of the data type <code>IppsBigNumState</code> .
<i>r</i>	Addition result.

Description

This function is declared in the `ippcp.h` file. The function adds two integer big numbers regardless of their signs and sizes and returns the result of the operation.

The following pseudo code represents this function:

$(*r) \leftarrow (*a) + (*b).$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

`ippStsOutOfRangeErr`

Indicates an error condition if the size of r is smaller than the resulting data length.



NOTE. The function executes only under the condition that size of `IppsBigNumState *r` is not less than either the length of `IppsBigNumState *a` or that of `IppsBigNumState *b`.

Example 5-4 Add Big Numbers

```
void Add_BN_sample(void){
    // define and set up Big Number A
    const Ipp32u bnuA[] = {0x01234567,0x9abcdeff,0x11223344};
    IppsBigNumState* bnA = New_BN(sizeof(bnuA)/sizeof(Ipp32u));

    // define and set up Big Number B
    const Ipp32u bnuB[] = {0x76543210,0xfedcabee,0x44332211};
    IppsBigNumState* bnB = New_BN(sizeof(bnuB)/sizeof(Ipp32u), bnuB);

    // define Big Number R
    int sizeR = max(sizeof(bnuA), sizeof(bnuB));
    IppsBigNumState* bnR = New_BN(1+sizeR/sizeof(Ipp32u));

    // R = A+B
    ippAdd_BN(bnA, bnB, bnR);

    // type R
    Type_BN("R=A+B:\n", bnR);

    delete [] (Ipp8u*)bnA;
    delete [] (Ipp8u*)bnB;
    delete [] (Ipp8u*)bnR;
}
```

Sub_BN

Subtracts one integer big number from another.

Syntax

```
IppStatus ippsSub_BN(IppsBigNumState *a, IppsBigNumState *b,
    IppsBigNumState *r);
```

Parameters

<i>a</i>	First unsigned integer big number of the data type IppsBigNumState.
<i>b</i>	Second unsigned integer big number of the data type IppsBigNumState.
<i>r</i>	Subtraction result.

Description

This function is declared in the `ippcp.h` file. The function subtracts one integer big number from another regardless of their signs and sizes and returns the result of the operation.

The following pseudo code represents this function:

$(*r) \leftarrow (*a) - (*b).$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if <code>IppsBigNumState *r</code> is smaller than the result data length.



NOTE. The function executes only under the condition that size of `IppsBigNumState *r` is not less than either the length of `IppsBigNumState *a` or that of `IppsBigNumState *b`.

Mul_BN

Multiplies two integer big numbers.

Syntax

```
IppStatus ippsMul_BN(IppsBigNumState *a, IppsBigNumState *b,  
    IppsBigNumState *r);
```

Parameters

<i>a</i>	Multiplicand of IppsBigNumState.
<i>b</i>	Multiplier of IppsBigNumState.
<i>r</i>	Multiplication result.

Description

This function is declared in the `ippcp.h` file. The function multiplies an integer big number by another integer big number regardless of their signs and sizes and returns the result of the operation.

The following pseudo code represents this function:

$r \leftarrow a * b.$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if IppsBigNumState * <i>r</i> is smaller than the result data length.



NOTE. The function executes only under the condition that the size IppsBigNumState **r* is not less than the sum of the lengths of IppsBigNumState **a* or that of IppsBigNumState **b* minus one.

MAC_BN_I

Multiplies two integer big numbers and accumulates the result with the third integer big number.

Syntax

```
IppStatus ippsMAC_BN_I(IppsBigNumState *a, IppsBigNumState *b,
    IppsBigNumState *r);
```

Parameters

<i>a</i>	Multiplicand of IppsBigNumState.
<i>b</i>	Multiplier of IppsBigNumState.
<i>r</i>	Multiplication result.

Description

This function is declared in the `ippcp.h` file. The function multiplies one integer big number by another and accumulates the result with the third input integer big number regardless of their signs and sizes. The function subsequently returns the result of the operation.

The following pseudocode represents this function:

$r \leftarrow r + a * b.$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if IppsBigNumState * <i>r</i> is smaller than the result data length.



NOTE. The function executes only under the condition that the size IppsBigNumState **r* is not less than the sum of the lengths of IppsBigNumState **a* or that of IppsBigNumState **b* minus one.

Div_BN

Divides one integer big number by another.

Syntax

```
IppStatus ippsDiv_BN(IppsBigNumState *a, IppsBigNumState *b,  
    IppsBigNumState *q, IppsBigNumState *r);
```

Parameters

<i>a</i>	Dividend of IppsBigNumState.
<i>b</i>	Divisor of IppsBigNumState.
<i>q</i>	Quotient of IppsBigNumState.
<i>r</i>	Remainder of IppsBigNumState.

Description

This function is declared in the `ippcp.h` file. The function divides an integer big number dividend by another integer big number regardless of their signs and sizes and returns the quotient of the division and the respective remainder.

The following pseudo code represents this function:

$$q \leftarrow a/b$$
$$r \leftarrow a - b*q$$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if IppsBigNumState * <i>r</i> is smaller than the length of IppsBigNumState * <i>b</i> or when the size of IppsBigNumState * <i>q</i> is smaller than the quotient result data length.
<code>ippStsDivByZeroErr</code>	Indicates an error condition if the zero divisor is attempted.



NOTE. The size of `IppsBigNumState *q` should not be less than $(\text{length of } *a) - (\text{length of } *b) + 1$, and the size of `IppsBigNumState *r` should be no less than the length of `IppsBigNumState *b`.

Mod_BN

Computes modular reduction for input integer big number with respect to specified modulus.

Syntax

```
IppStatus ippsMod_BN(IppsBigNumState *a, IppsBigNumState *m,
    IppsBigNumState *r);
```

Parameters

<i>a</i>	Integer big number of <code>IppsBigNumState</code> .
<i>m</i>	Modulus integer of <code>IppsBigNumState</code> .
<i>r</i>	Modular reduction result.

Description

This function is declared in the `ippcp.h` file. The function computes the modular reduction for an input integer big number with respect to the modulus specified by a positive integer big number and returns the modular reduction result in the range of $[0, (m-1)]$.

The following pseudo code represents this function:

$r \leftarrow a \bmod m.$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsOutOfRangeErr</code>	Indicates an error condition if <code>IppsBigNumState *r</code> is smaller than the length of <code>IppsBigNumState *m</code> .
<code>ippStsBadModulusErr</code>	Indicates an error condition if the modulus <code>IppsBigNumState *m</code> is not a positive integer.



NOTE. The size of `IppsBigNumState *r` should not be less than the length of `IppsBigNumState *m`.

Gcd_BN

Computes greatest common divisor.

Syntax

```
IppStatus ippGcd_BN(IppsBigNumState *a, IppsBigNumState *b,  
    IppsBigNumState *g);
```

Parameters

<code>a</code>	First integer big number of <code>IppsBigNumState</code> .
<code>b</code>	Second integer big number of <code>IppsBigNumState</code> .
<code>g</code>	Greatest common divisor to <code>a</code> and <code>b</code> .

Description

This function is declared in the `ippcp.h` file. The function computes the greatest common divisor (GCD) for two positive integer big numbers.

The following pseudo code represents this function:

$g \leftarrow \text{gcd}(a, b).$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

`ippStsOutOfRangeErr`

Indicates an error condition if `IppsBigNumState *g` is smaller than the length of `IppsBigNumState *a` or `IppsBigNumState *b`.



NOTE. The size of `IppsBigNumState *g` should not be less than either the length of `IppsBigNumState *a` and `IppsBigNumState *b`.

ModInv_BN

Computes multiplicative inverse of a positive integer big number with respect to specified modulus.

Syntax

```
IppStatus ippModInv_BN(IppsBigNumState *e, IppsBigNumState *m,
    IppsBigNumState *d);
```

Parameters

<i>e</i>	Integer big number of <code>IppsBigNumState</code> .
<i>m</i>	Modulus integer of <code>IppsBigNumState</code> .
<i>d</i>	Multiplicative inverse.

Description

This function is declared in the `ippcp.h` file. The function uses the extended Euclidean algorithm to compute the multiplicative inverse of a given positive integer big number *e* with respect to the modulus specified by another positive integer big number *m*, where $\text{gcd}(e, m) = 1$.

The following pseudo code represents this function:

compute *d* such that $d * e = 1 \bmod m$.

Return Values

`ippStsNoErr`

Indicates no error. Any other value indicates an error or warning.

<code>ippStsBadArgErr</code>	Indicates an error condition if e is less than or equal to 0.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsBadModulusErr</code>	Indicates an error condition if the modulus e is more than m , or $\gcd(e, m)$ is more than 1, or m is less than or equal to 0.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if <code>IppsBigNumState *d</code> is smaller than the length of <code>IppsBigNumState *m</code> .



NOTE. The size of `IppsBigNumState *d` should not be less than the length of `IppsBigNumState *m`.

Montgomery Reduction Scheme Functions

This section describes Montgomery reduction scheme functions. The full list of these functions is given in [Table 5-2](#).

Table 5-2 Intel IPP Montgomery Reduction Scheme Functions

Function Base Name	Operation
<code>MontGetSize</code>	Gets the size of the <code>IppsMontState</code> context.
<code>MontInit</code>	Initializes the context and partitions the specified buffer space.
<code>MontSet</code>	Sets the input integer big number to a value and computes the Montgomery reduction index.
<code>MontGet</code>	Extracts the big number modulus.
<code>MontForm</code>	Converts input positive integer big number into Montgomery form.
<code>MontMul</code>	Computes Montgomery modular multiplication for positive integer big numbers of Montgomery form.
<code>MontExp</code>	Computes Montgomery exponentiation.

Montgomery reduction is a technique for efficient implementation of modular multiplication without explicitly carrying out the classical modular reduction step.

This section describes functions for Montgomery modular reduction, Montgomery modular multiplication, and Montgomery modular exponentiation.

Let n be a positive integer, and let R and T be integers such that $R > n$, $\gcd(n, R) = 1$, and $0 < T < nR$. The Montgomery reduction of T modulo n with respect to R is defined as $TR^{-1} \bmod n$.

For better results, functions included in the cryptography package use $R = b^k$ where $b = 2^{32}$ and k is the Montgomery index integer computed by the ceiling function of the bit length of the integer n over 32.

All functions use employ the context `IppsMontState` to serve as an operational vehicle to carry the Montgomery reduction index k , the integer big number modulus n , the least significant word n_0 of the multiplicative inverse of the modulus n with respect to the Montgomery reduction factor R , and a sufficient working buffer reserved for various Montgomery modular operations.

Furthermore, two new terms are introduced in this section:

- length of the context `IppsMontState` is defined as the data length of the modulus n carried by the structure
- size of the context `IppsMontState` is therefore defined as the maximum data length of such an integer modulus n that could be carried by this operational vehicle.

The following example can briefly illustrate the procedure of using the primitives described in this section to compute a classical modular exponentiation $T = x^e \bmod n$. Consider computing $T = x^4 \bmod n$, for some integer x with $0 < x < n$.

First get the buffer size required to configure the context `IppsMontState` by calling [MontGetSize](#) and then allocate the working buffer using OS service function, with allocated buffer to call [MontInit](#) to initialize the context `IppsMontState`.

Set the modulus n by calling [MontSet](#) and then convert x into its respective Montgomery form by calling [MontForm](#), that is, computing $\underline{x} = xR \bmod n$. Then compute the Montgomery reduction of $\underline{x}\underline{x}$ using the function [MontMul](#) to generate $T = \underline{x}\underline{x}R^{-1} \bmod n$. The Montgomery reduction of $T \cdot T \bmod n$ with respect to R is

$$T^2 R^{-1} \bmod n = (\underline{x}^2 R^{-1})^2 R^{-1} \bmod n = x^4 R \bmod n.$$

Further applying [MontMul](#) with this value and the value of 1 yields the desired result $T = x^4 \bmod n$.

The classical modular exponentiation should be computed by performing the following sequence of operations:

1. Get the buffer size required to configure the context `IppsMontState` by calling the function [MontGetSize](#). For limited memory system, choose binary method, and otherwise, choose sliding window method. Using the binary method reduces the buffer size significantly while using sliding window method enhances the performance.
2. Allocate working buffer through an operating system memory allocation function and configure the structure `IppsMontState` by calling the function [MontInit](#) with the allocated buffer and the choice made on the modular exponential method at time invoking [MontGetSize](#).
3. Call the function [MontSet](#) to set the integer big number module for `IppsMontState`.
4. Call the function [MontForm](#) to convert the integer x to be its Montgomery form.
5. Call the function [MontExp](#) to compute the Montgomery modular exponentiation.
6. Call the function [MontMul](#) to compute the Montgomery modular multiplication of the above result with the integer 1 as to convert the above result back to the desired classical modular exponential result.
7. Free the memory using an operating system memory free function, if needed.

MontGetSize

Gets the size of the `IppsMontState` context.

Syntax

```
IppStatus ippsMontGetSize(IppsExpMethod method, int length, int *size);
```

Parameters

<i>method</i>	Selected exponential method.
<i>length</i>	Data field length for the modulus.
<i>size</i>	Size of the buffer required for initialization.

Description

This function is declared in the `ippcp.h` file. The function specifies the buffer size required to define the structuralized working buffer of the context `IppsMontState` to store the modulus and perform operations using various Montgomery modulus schemes.

The function returns the required buffer size based on the selected exponential method. The binary method helps to significantly reduce the buffer size, while the sliding windows method results in enhanced performance.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>length</i> is less than or equal to 0.

MontInit

Initializes the context and partitions the specified buffer space.

Syntax

```
IppStatus ippMontInit(IppsExpMethod method, int length, IppsMontState
    *m) ;
```

Parameters

<i>method</i>	Selected exponential method.
<i>buffer</i>	Buffer for initializing <i>m</i> .
<i>length</i>	Data field length for the modulus.
<i>m</i>	Pointer to the context <code>IppsMontState</code> .

Description

This function is declared in the `ippcp.h` file. The function initializes the context using the specified buffer space. The function then partitions the buffer using the selected modular exponential method in such a way as to carry up to *length***sizeof*(`Ipp32u`)-bit big number modulus and execute various Montgomery modulus operations.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>length</i> is less than or equal to 0.

MontSet

Sets the input integer big number to a value and computes the Montgomery reduction index.

Syntax

```
IppStatus ippsMontSet(const Ipp32u *n, int length, IppsMontState *m);
```

Parameters

<i>n</i>	Input big number modulus.
<i>m</i>	Pointer to the context <code>IppsMontState</code> capturing the modulus and the least significant word of the multiplicative inverse <i>Ni</i> .

Description

This function is declared in the `ippcp.h` file. The function sets the input positive integer big number *n* to be the modulus for the context `IppsMontState *m`, computes the Montgomery reduction index *k* with respect to the input big number modulus *n* and the least significant 32-bit word of the multiplicative inverse *Ni* with respect to the modulus *R*, that satisfies

$$R * R^{-1} - n * Ni = 1.$$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsBadModulusErr</code>	Indicates an error condition if the modulus is not a positive odd integer.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>length</i> is less than or equal to 0.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if <i>length</i> is larger than <code>IppsMontState *m</code> .

MontGet

Extracts the big number modulus.

Syntax

```
IppStatus ippsMontGet(Ipp32u *n, int *length, const IppsMontState *m);
```

Parameters

<i>m</i>	context IppsMontState.
<i>n</i>	Modulus data field.
<i>length</i>	Modulus data length.

Description

This function is declared in the `ippcp.h` file. The function extracts the big number modulus from the input IppsMontState **m*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

MontForm

Converts input positive integer big number into Montgomery form.

Syntax

```
IppStatus ippsMontForm(IppsBigNumState *a, IppsMontState *m,  
    IppsBigNumState *r);
```

Parameters

<i>a</i>	Input integer big number within the range $[0, m - 1]$.
<i>m</i>	Input big number modulus of IppsBigNumState.
<i>r</i>	Resulting Montgomery form $r = a * R \bmod m$.

Description

This function is declared in the `ippcp.h` file. The function converts an input positive integer big number into the Montgomery form with respect to the big number modulus and stores the conversion result.

The following pseudo code represents this function:

```
r <- a*R mod m.
```

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsBadArgErr</code>	Indicates an error condition if <i>a</i> is a negative integer.
<code>ippStsScaleRangeErr</code>	Indicates an error condition if <i>a</i> is more than <i>m</i> .
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if <code>IppsBigNumState *r</code> is larger than <code>IppsMontState *m</code> .



NOTE. The size of `IppsBigNumState *r` should not be less than the data length of the modulus *m*.

MontMul

Computes Montgomery modular multiplication for positive integer big numbers of Montgomery form.

Syntax

```
IppStatus ippMontMul(IppsBigNumState *a, IppsBigNumState *b,  
                    IppsMontState *m, IppsBigNumState *r);
```

Parameters

<i>a</i>	Multiplicand within the range $[0, m - 1]$.
<i>b</i>	Multiplier within the range $[0, m - 1]$.

<i>m</i>	Modulus.
<i>r</i>	Montgomery multiplication result.

Description

This function is declared in the `ippcp.h` file. The function computes the Montgomery modular multiplication for positive integer big numbers of Montgomery form with respect to the modulus `IppsMontState *m`. As a result, `IppsBigNumState *r` holds the product.

The following pseudo code represents this function:

$$r \leftarrow a * b * R^{-1} \bmod m.$$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsBadArgErr</code>	Indicates an error condition if <i>a</i> or <i>b</i> is a negative integer.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsScaleRangeErr</code>	Indicates an error condition if <i>a</i> or <i>b</i> is more than <i>m</i> .
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if <code>IppsBigNumState *r</code> is larger than <code>IppsMontState *m</code> .



NOTE. The size of `IppsBigNumState *r` should not be less than the data length of the modulus *m*.

Example 5-5 Montgomery Multiplication

```
void MontMul_sample(void) {
    int size;
```

Example 5-5 Montgomery Multiplication (continued)

```
// define and initialize Montgomery Engine over Modulus N
Ipp32u bnuN = 19;
ippsMontGetSize(IppsBinaryMethod, 1, &size);
IppsMontState* pMont = (IppsMontState*)( new Ipp8u [size] );
ippsMontInit(IppsBinaryMethod, 1, pMont);
ippsMontSet(&bnuN, 1, pMont);

// define and init Big Number multiplicand A
Ipp32u bnuA = 12;
IppsBigNumState* bnA = New_BN(1, &bnuA);
// encode A into Montfomery form
ippsMontForm(bnA, pMont, bnA);

// define and init Big Number multiplicand A
Ipp32u bnuB = 15;
IppsBigNumState* bnB = New_BN(1, &bnuB);

// compute R = A*B mod N
IppsBigNumState* bnR = New_BN(1);
ippsMontMul(bnA, bnB, pMont, bnR);

Type_BN("R = A*B mod N:\n", bnR);

delete [] (Ipp8u*)pMont;
delete [] (Ipp8u*)bnA;
delete [] (Ipp8u*)bnB;
delete [] (Ipp8u*)bnR;
}
```

MontExp

Computes Montgomery exponentiation.

Syntax

```
IppStatus ippsMontExp(IppsBigNumState *a, IppsBigNumState *e,
    IppsMontState *m, IppsBigNumState *r);
```

Parameters

<i>a</i>	Big number Montgomery integer within the range of $[0, m - 1]$.
<i>e</i>	Big number exponent.
<i>m</i>	Modulus.
<i>r</i>	Montgomery exponentiation result.

Description

This function is declared in the `ippcp.h` file. The function computes Montgomery exponentiation with the exponent specified by the input positive integer big number to the given positive integer big number of the Montgomery form with respect to the modulus *m*.

The following pseudo code represents this function:

$$r \leftarrow a^{eR^{-(e-1)}} \bmod m.$$

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsBadArgErr</code>	Indicates an error condition if <i>a</i> or <i>e</i> is a negative integer.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsScaleRangeErr</code>	Indicates an error condition if <i>a</i> or <i>e</i> is more than <i>m</i> .
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if <code>IppsBigNumState *r</code> is larger than <code>IppsMontState *m</code> .



NOTE. The size of `IppsBigNumState *r` should not be less than the data length of the modulus `m`.

Pseudorandom Number Generation Functions

Many cryptographic systems rely on pseudorandom number generation functions in their design that make the unpredictable nature inherited from a pseudorandom number generator the security foundation to ensure safe communication over open channels and protection against potential adversaries.

The full list of Intel IPP Pseudorandom Number Generation Functions is given in [Table 5-3](#).

Table 5-3 Intel IPP Pseudorandom Number Generation Functions

Function Base Name	Operation
Pseudorandom Number Generation Functions	
<code>PRNGGetSize</code>	Gets the size of the <code>IppsPRNGState</code> context.
<code>PRNGInit</code>	Initializes user-supplied memory as <code>IppsPRNGState</code> context for future use.
<code>PRNGSetSeed</code>	Sets the initial state with the given input seed for pseudorandom number generation.
<code>PRNGSetAugment</code>	Sets the initial state with the given input entropy for the pseudorandom number generation.
<code>PRNGSetModulus</code>	Sets the initial state with the given input modulus for the pseudorandom number generation.
<code>PRNGSetH0</code>	Sets the initial state with the given input IV for the SHA-1 algorithm.
<code>PRNGen</code>	Generates a pseudorandom unsigned Big Number of the specified bitlength.
<code>PRNGen_BN</code>	Generates a pseudorandom positive Big Number of the specified bitlength.

This section describes functions that comprise the pseudorandom bit sequence generator implemented by a US FIPS-approved method and based on a SHA-1 one-way hash function specified by [[FIPS PUB 186-2](#)], appendix 3.

The application code for generating a sequence of pseudorandom bits should perform the following sequence of operations:

1. Call the function `PRNGGetSize` to get the size required to configure the `IppsPRNGState` context.
2. Ensure that the required memory space is properly allocated. With the allocated memory, call the [PRNGInit](#) function to set up the default value of the parameters for pseudorandom generation process.
3. If the default values of the parameters are not satisfied, call the function [PRNGSetSeed](#) and/or `PRNGSetArgument` and/or `PRNGSetModulus` and/or `PRNGSetH0` to reset any of the control pseudorandom generator parameters.
4. Keep calling the function `PRNGGen` or `PRNGGen_BN` to generate pseudo random value of the desired format.
5. Free the memory allocated for the `IppsPRNGState` context by calling the operating system memory free service function.

User’s Implementation of a Pseudorandom Number Generator

Both functions `ippsPRNGGen` and `ippsPRNGGen_BN` as well as their supplementary functions represent the implementation of the pseudorandom number generator in the IPPCP library. This given implementation is based on recommendations made in [\[FIPS PUB 186-2\]](#). If you prefer to use the implementation of the pseudorandom number generator which is different from the given, you can still use IPPCP library. To do this, use the following definition of the generator introduced by the IPPCP library:

Syntax

```
typedef IppStatus(_STDCALL *IppBitSupplier)(Ipp32u* pData, int nBits,
void* pEbsParams);
```

Parameters

<i>pData</i>	Pointer to the output data.
<i>nBits</i>	Number of generated data bits.
<i>pEbsParams</i>	Pointer to the user defined context.

Description

This declaration is included in the `ippcp.h` file. The function generates any data (probably pseudorandom numbers) of the specified *nBits* length.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsErr</code>	Indicates an error condition.

PRNGGetSize

Gets the size of the `IppsPRNGState` context in bytes.

Syntax

```
IppStatus ippsPRNGGetSize(int *pSize);
```

Parameters

pSize Pointer to the `IppsPRNGState` context size in bytes.

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsPRNGState` context size in bytes and stores it in **pSize*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

PRNGInit

Initializes user-supplied memory as `IppsPRNGState` context for future use.

Syntax

```
IppStatus ippsPRNGInit(int seedBits, IppsPRNGState* pCtx);
```

Parameters

<i>seedBits</i>	Size in bits for the seed value.
<i>pCtx</i>	Pointer to the <code>IppsPRNGState</code> context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by `pCtx` as the `IppsPRNGState` context. In addition, the function sets up the default internal random generator parameters (seed, entropy augment, modulus, and initial hash value H0 of the SHA-1 algorithm). PRNG default parameters are as follows:

- seed = 0x0
- entropy augment = 0x0
- modulus = 0xFF
- H0 = 0xC3D2E1F01032547698BADCFEEFCDAB8967452301

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>seedBits</i> is less than 1 or greater than 512.

PRNGSetSeed

Sets up the seed value for the pseudorandom number generator.

Syntax

```
IppStatus ippSPRNGSetSeed(const IppsBigNumState* pSeed, IppsPRNGState*
    pCtx);
```

Parameters

<i>pSeed</i>	Pointer to the seed value being set up.
<i>pCtx</i>	Pointer to the <code>IppsPRNGState</code> context.

Description

This function is declared in the `ippcp.h` file. The function resets the seed value with the supplied value of *seedBits* bit length. The supplied big number should be created prior to the function call using the appropriate Big Number Arithmetic functions (see [Example 5-1](#)).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.



NOTE. This function restarts the pseudorandom number generation process, which results in losing already generated pseudorandom numbers.

PRNGSetAugment

Sets the initial state with the given input entropy for the pseudorandom number generation.

Syntax

```
IppsStatus ippPRNGSetAugment(const IppsBigNumState* pAugment,  
                             IppsPRNGState* pCtx);
```

Parameters

<code>pAugment</code>	Pointer to the entropy augment value being set up.
<code>pCtx</code>	Pointer to the <code>IppsPRNGState</code> context.

Description

This function is declared in the `ippcp.h` file. The function resets entropy augment value with the supplied value of the `seedBits` bit length. The supplied big number should be created prior to the function call using the appropriate Big Number Arithmetic functions (see [Example 5-1](#)).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
--------------------------	--

<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

PRNGSetModulus

Sets the initial state with the given input modulus for the pseudorandom number generation.

Syntax

```
IppStatus ippSPRNGSetModulus(const IppsBigNumState* pModulus,
                             IppsPRNGState* pCtx);
```

Parameters

<i>pModulus</i>	Pointer to the modulus value being set up.
<i>pCtx</i>	Pointer to the IppsPRNGState context.

Description

This function is declared in the `ippcp.h` file. The function resets the modulus value with the supplied value up to 160 bit length. The supplied big number should be created prior to the function call using the appropriate Big Number Arithmetic functions (see [Example 5-1](#)).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

PRNGSetH0

Sets the initial state with the given input IV for the SHA-1 algorithm.

Syntax

```
IppStatus ippSPRNGSetH0(const IppsBigNumState* pH0, IppsPRNGState* pCtx);
```

Parameters

<i>pH0</i>	Pointer to the initial hash value being set up.
<i>pCtx</i>	Pointer to the IppsPRNGState context.

Description

This function is declared in the `ippcp.h` file. The function resets the initial hash value with the supplied value up to 160 bit length. The supplied big number should be created prior to the function call using the appropriate Big Number Arithmetic functions (see [Example 5-1](#)).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

PRNGen

Generates a pseudorandom unsigned Big Number of the specified bitlength.

Syntax

```
IppStatus ippSPRNGen(Ipp32u* pRandBNU, int nBits, void* pCtx);
```

Parameters

<i>pRandBNU</i>	Pointer to the output pseudorandom unsigned integer big number.
-----------------	---

<i>nBits</i>	Number of the generated pseudorandom bit.
<i>pCtx</i>	Pointer to the IppsPRNGState context.

Description

This function is declared in the `ippcp.h` file. The function generates pseudorandom unsigned integer big number of the specified *nBits* length.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>nBits</i> is less than 1.

PRNGen_BN

Generates a pseudorandom positive Big Number of the specified bitlength.

Syntax

```
IppStatus ippPRNGen_BN(IppsBigNumState* pRandBN, int nBits, void* pCtx);
```

Parameters

<i>pRandBN</i>	Pointer to the output pseudorandom Big Number.
<i>nBits</i>	Number of the generated pseudorandom bit.
<i>pCtx</i>	Pointer to the IppsPRNGState context.

Description

This function is declared in the `ippcp.h` file. The function generates pseudorandom positive Big Number of the specified *nBits* length.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
--------------------------	--

<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>nBits</i> is less than 1.

Example 5-6 Find Pseudorandom Co-primes

```
void FindCoPrimes(void){
    int size;

    // define Pseudo Random Generator (default settings)
    ippSPRNGGetSize(&size);
    IppsSPRNGState* pPrng = (IppsSPRNGState*)(new Ipp8u [size] );
    ippSPRNGInit(160, pPrng);

    // define 256-bits Big Numbers X and Y
    const int bnBitSize = 256;
    IppsBigNumState* bnX = New_BN(bnBitSize/32);
    IppsBigNumState* bnY = New_BN(bnBitSize/32);

    // define temporary Big Numbers GCD and 1
    IppsBigNumState* bnGCD = New_BN(bnBitSize/32);
    Ipp32u one = 1;
    IppsBigNumState* bnOne = New_BN(1, &one);
```

Example 5-6 Find Pseudorandom Co-primes (continued)

```
// generate pseudo random X and Y
// while GCD(X,Y) != 1
Ipp32u result;
int counter;
for(counter=0,result=1; result; counter++) {
    ippsPRNGen_BN(bnX, bnBitSize, pPrng);
    ippsPRNGen_BN(bnY, bnBitSize, pPrng);
    ippsGcd_BN(bnX, bnY, bnGCD);
    ippsCmp_BN(bnGCD, bnOne, &result);
}

cout <<"Coprimes:" <<endl;
Type_BN("X: ", bnX); cout <<endl;
Type_BN("Y: ", bnY); cout <<endl;
cout <<"were found on " <<counter <<" attempt" <<endl;

delete [] (Ipp8u*)pPrng;
delete [] (Ipp8u*)bnX;
delete [] (Ipp8u*)bnY;
delete [] (Ipp8u*)bnGCD;
delete [] (Ipp8u*)bnOne;
}
```

Prime Number Generation Functions

This section introduces Intel® Integrated Performance Primitives (Intel® IPP) functions for prime number generation.

The full list of prime number generation functions is given in [Table 5-4](#).

Table 5-4 Intel IPP Prime Number Generation Functions

Function Base Name	Operation
Prime Number Generation Functions	
PrimeGetSize	Gets the size of the IppsPrimeState context.
PrimeInit	Initializes user-supplied memory as the IppsPrimeState context for future use.
PrimeGen	Generates a random probable prime number of the specified bitlength.
PrimeTest	Tests the given integer for being a probable prime.
PrimeSet	Sets the Big Number for primality testing.
PrimeSet_BN	Sets the Big Number for primality testing.
PrimeGet	Extracts the probable prime unsigned integer big number.
PrimeGet_BN	Extracts the probable prime positive Big Number.

This section describes Intel IPP functions for generating probable prime numbers of variable lengths and validating probable prime numbers through a probabilistic primality test scheme for cryptographic use. A probable prime number is thus defined as an integer that passes the Miller-Rabin probabilistic primality-based test.

The scheme adopted for the probable prime number generation is based on a well-known prime number theorem. Study shows that the number of primitives that are no greater than the given large integer x is closely approximated by the expression. Let denote the number of primes that are not greater than x . In this case the statement is true

$$\lim_{x \rightarrow \infty} \frac{\pi(x)}{x / (\ln x)} = 1.$$

Further study indicates that if X represents the event where the tested k -bit integer n is composite and if Y_t denotes the event where the Miller-Rabin test with the security parameter t declares n to be a prime, the test error probability is upper bounded by

$$P_{k,t} \leq k^{2/3} 2^{t-1/2} t^{4^{2-\sqrt{t}k}} \text{ for } t = 2, k \geq 88, \text{ or } 3 \leq t \leq k/9, k \geq 21.$$

Subsequently, a practical strategy for generating a random k -bit probable prime is to repeatedly pick k -bit random odd integers until finding one integer that can pass a recognized probabilistic primality test scheme as a probable prime. The available set of

probable prime number generation functions enables you to specify an appropriate value of the security parameter t used in the Miller-Rabin primality test to meet the cryptographic requirements for your application.

All Intel IPP for prime number generation use the context `IppsPrimeState` as an operational vehicle that carries the bitlength of the target probable prime number, the structure capturing the state of the pseudorandom number generation, the structuralized working buffer used for Montgomery modular computation in the Miller-Rabin primality test, and the buffer to store the generated probable prime number.

The following sequence of operations is required to generate a probable prime number of the specified bitlength:

1. Call the function [PrimeGetSize](#) to get the size required to configure the `IppsPrimeState` context.
2. Allocate memory through the operating system memory allocation function and configure the `IppsPrimeState` context by calling the function [PrimeInit](#).
3. Generate probable prime number of the specified bitlength by calling the function [PrimeGen](#). If the returned `IppStatus` is `ippStsInsufficientEntropy`, then change the parameters of the pseudorandom generator and call the function [PrimeGen](#) again.
4. Extract the generated probable prime number by calling the functions [PrimeGet](#) and `PrimeGet_BN`.
5. Free the memory allocated to the `IppsPrimeState` context by calling the operating system memory-free service function.

PrimeGetSize

Gets the size of the `IppsPrimeState` context in bytes.

Syntax

```
IppStatus ippPrimeGetSize(int nMaxBits, int* pSize);
```

Parameters

<i>nMaxBits</i>	Maximum length of the probable prime number in bits.
<i>pSize</i>	Pointer to the <code>IppsPrimeState</code> context size in bytes.

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsPrimeState` context size in bytes and stores it in `pSize`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <code>nMaxBits</code> is less than 1.

PrimeInit

*Initializes user-supplied memory as
IppsPrimeState context for future use.*

Syntax

```
IppStatus ippPrimeInit(int nMaxBits, IppsPrimeState* pCtx);
```

Parameters

<code>nMaxBits</code>	Maximum length of the probable prime number in bits.
<code>pCtx</code>	Pointer to the <code>IppsPrimeState</code> context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by `pCtx` as the `IppsPrimeState` context.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <code>nMaxBits</code> is less than 1.

PrimeGen

Generates a random probable prime number of the specified bitlength.

Syntax

```
IppStatus ippsPrimeGen(int nBits, int nTrials, IppsPrimeState* pCtx,
    IppBitSupplier rndFunc, void* pRndParam);
```

Parameters

<i>nbits</i>	Target bitlength for the desired probable prime number.
<i>nTrials</i>	Security parameter specified for the Miller-Rabin probable primality.
<i>pCtx</i>	Pointer to the <code>IppsPrimeState</code> context.
<i>rndFunc</i>	Specified Random Generator.
<i>pRndParam</i>	Pointer to the Random Generator context.

Description

This function is declared in the `ippcp.h` file. The function employs the `rndFuncRandom` Generator specified by the user to generate a random probable prime number of the specified *nBits* length. The generated probable prime number is further validated by the Miller-Rabin primality test scheme with the specified security parameter *nTrials*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>nBits</i> is less than 1.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsBadArgErr</code>	Indicates an error condition if <i>nTrials</i> is less than 1.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if <i>nBits</i> > <i>nMaxBits</i> (see PrimeGetSize and PrimeInit)
<code>ippStsInsufficientEntropy</code>	Indicates a warning condition if prime generation fails due to poor choice of entropy.

PrimeTest

Tests the given integer for being a probable prime.

Syntax

```
IppStatus ippsPrimeTest(int nTrials, Ipp32u *pResult, IppsPrimeState*  
    pCtx, IppBitSupplier rndFunc, void* pRndParam);
```

Parameters

<i>nTrials</i>	Security parameter specified for the Miller-Rabin probable primality.
<i>pResult</i>	Pointer to the result of the primality test.
<i>pCtx</i>	Pointer to the <code>IppsPrimeState</code> context.
<i>rndFunc</i>	Specified Random Generator.
<i>pRndParam</i>	Pointer to the Random Generator context.

Description

This function is declared in the `ippcp.h` file. The function uses the Miller-Rabin probabilistic primality test scheme with the given security parameter to test if the given integer is a probable prime. The pseudorandom number used in the Miller-Rabin test is generated by the specified `rndFunc` Random Generator. The function sets up the **pResult* as `IS_PRIME` or `IS_COMPOSITE` to show if the input probable prime passes the Miller-Rabin test.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsBadArgErr</code>	Indicates an error condition if <i>nTrials</i> is less than 1.

PrimeSet

Sets the *Big Number* for primality testing.

Syntax

```
IppStatus ippsPrimeSet(const Ipp32u* pBNU, int nBits, IppsPrimeState* pCtx);
```

Parameters

<i>pBNU</i>	Pointer to the unsigned integer big number.
<i>nBits</i>	Unsigned integer big number length in bits.
<i>pCtx</i>	Pointer to the IppsPrimeState context.

Description

This function is declared in the `ippcp.h` file. The function sets a probable prime number and its length for the probabilistic primality test.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if <i>nBits</i> is less than 1.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if <i>nBits</i> is too large to fit <i>pCtx</i> .

PrimeSet_BN

Sets the *Big Number* for primality testing.

Syntax

```
IppStatus ippsPrimeSet_BN(const IppsBigNumState* pBN, IppsPrimeState* pCtx);
```

Parameters

<i>pBN</i>	Pointer to the Big Number context.
<i>pCtx</i>	Pointer to the IppsPrimeState context.

Description

This function is declared in the `ippcp.h` file. The function sets the Big Number for probabilistic primality test.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if the Big Number is too large to fit <i>pCtx</i> .

PrimeGet

Extracts the probable prime unsigned integer big number.

Syntax

```
IppStatus ippPrimeGet(Ipp32u* pBNU, int *pSize, const IppsPrimeState *pCtx);
```

Parameters

<i>pBNU</i>	Pointer to the unsigned integer big number.
<i>pSize</i>	Pointer to the length of the unsigned integer big number.
<i>p</i>	Pointer to the IppsPrimeState context.

Description

This function is declared in the `ippcp.h` file. The function extracts the probable prime number from **pCtx* context and stores it into the specified unsigned integer big number.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

PrimeGet_BN

Extracts the probable prime positive Big Number.

Syntax

```
IppStatus IppsPrimeGet_BN(IppsBigNumState* pBN, const IppsPrimeState *pCtx);
```

Parameters

<code>pBN</code>	Pointer to the Big Number context.
<code>pCtx</code>	Pointer to the <code>IppsPrimeState</code> context.

Description

This function is declared in the `ippcp.h` file. The function extracts the probable prime positive big number from the `*pCtx` context and stores it into the specified Big Number context.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if the Big Number is too small to store probable prime number.

Example 5-7 Check Primality

```
void CheckPrime(void){
    Ipp32u result;
    int size;

    // define 256-bit Prime Generator
    int maxBitSize = 256;
    ippsPrimeGetSize(maxBitSize, &size);
    IppsPrimeState* pPrimeGen = (IppsPrimeState*)( new Ipp8u [size] );
    ippsPrimeInit(maxBitSize, pPrimeGen);

    // define Pseudo Random Generator (default settings)
    ippsPRNGGetSize(&size);
    IppsPRNGState* pPrng = (IppsPRNGState*)(new Ipp8u [size] );
    ippsPRNGInit(160, pPrng);

    // define known prime value  $(2^{128} - 3)/76439$ 
    Ipp32u bnuPrime1[] = {
        0xBEAD208B, 0x5E668076, 0x2ABF62E3, 0xDB7C};
    IppsBigNumState* bnP1 = New_BN(4, bnuPrime1);
    // make sure P1 is really prime
    ippsPrimeSet_BN(bnP1, pPrimeGen);
    ippsPrimeTest(50, &result, pPrimeGen,
        ippsPRNGGen, pPrng);
    IS_PRIME==result?
        cout <<"Primality of P1 is confirmed\n" :
        cout <<"Primality of P1 isn't confirmed\n";
```

Example 5-7 Check Primality (continued)

```

        // define another known prime value 2^128 - 2^97 - 1
Ipp32u bnuPrime2[] = {
    0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFFFD};
IppsBigNumState* bnP2 = New_BN(4, bnuPrime2);
// make sure P2 is really prime
ippsPrimeSet_BN(bnP2, pPrimeGen);
ippsPrimeTest(50, &result, pPrimeGen,
              ippsPRNGen, pPrng);
IS_PRIME==result?
    cout <<"Primality of P2 is confirmed\n" :
    cout <<"Primality of P2 isn't confirmed\n";

// define composite Big Number C = P1*P2
IppsBigNumState* bnC = New_BN(8);
ippsMul_BN(bnP1, bnP2, bnC);
// make sure C is really composite
ippsPrimeSet_BN(bnC, pPrimeGen);
ippsPrimeTest(50, &result, pPrimeGen,
              ippsPRNGen, pPrng);
IS_PRIME==result?
    cout <<"Strange, but C=P1*P2 is prime\n" :
    cout <<"OK, C=P1*P2 is composite\n";

delete [] (Ipp8u*)pPrimeGen;
delete [] (Ipp8u*)pPrng;
delete [] (Ipp8u*)bnP1;
delete [] (Ipp8u*)bnP2;
delete [] (Ipp8u*)bnC;
}

```

RSA Algorithm Functions

This section introduces Intel® Integrated Performance Primitives (Intel® IPP) functions for RSA algorithm. The section describes a set of primitives to perform operations required for RSA cryptographic systems [PKCS 1.2.1]. This set of primitives offers a flexible user interface that enables the RSA crypto key size scalability with the limit of up to 4096 bits.

RSA algorithm functions include

- [Functions for Building RSA System](#), the system being then used by functions listed below.
- [RSA Primitives](#), which perform RSA encryption and decryption.
- RSA-based schemes, which combine RSA cryptographic primitives with other techniques, such as computing hash message digests or applying mask generation functions (MGFs), to achieve a particular security goal. IPPCP contains two groups of functions implementing RSA-based schemes: [RSA-OAEP Scheme Functions](#) and [RSA-SSA Scheme Functions](#).

Functions for Building RSA System

The list of functions for building RSA cryptographic system is given in [Table 5-5](#).

Table 5-5 Intel IPP RSA Algorithm Functions

Function Base Name	Operation
RSAGetSize	Gets the size of the <code>IppRSASState</code> context.
RSAInit	Initializes user-supplied memory as the <code>IppRSASState</code> context for future use.
RSASetKey	Sets the tag-designated key component into the established RSA context.
RSAGetKey	Extracts the tag-designated key component from the RSA context.
RSAGenerate	Generates key components for the desired RSA cryptographic system.
RSAValidate	Validates key components of the RSA cryptographic system.

You can use the primitives to build an RSA cryptographic system with the supplied randomized seed and stimulus. The function [RSAGenerate](#) generates the RSA system probable primes p and q , the system composite integer n , as well as the key pair, the RSA public key e and its respective private key d .

[RSA Primitives](#) and RSA-based schemes ([RSA-OAEP Scheme Functions](#) and [RSA-SSA Scheme Functions](#)) use `IppRSASState` context, which is initialized using the [RSAInit](#) function, as an operational vehicle carrying the RSA system composite integer, a pair of RSA probable primes, RSA key pair, and working buffers.

RSAGetSize

Gets the size of the IppsRSAState context.

Syntax

```
IppStatus ippsRSAGetSize(int nBitsN, int nBitsP, IppRSAKeyType flag, int*
    pSize);
```

Parameters

<i>nBitsN</i>	Length of the RSA system in bits (that is, the length of the composite RSA modulus n in bits).
<i>nBitsP</i>	Length in bits of the largest of two prime factors of the RSA modulus.
<i>flag</i>	The flag indicating RSA system for encryption or decryption operation.
<i>pSize</i>	Pointer to the IppsRSAState context size in bytes.

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsRSAState` context size in bytes and stores it in `*pSize`. Use the `flag == IppRSAPublic` to establish RSA for encryption or `flag == IppRSAprivate` to establish RSA for the decryption operation. Refer to [RSA-OAEP Scheme Functions](#) and [RSA-SSA Scheme Functions](#) on which `flag` value to select for the schemes.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsNotSupportedModeErr</code>	Indicates an error condition if $nBitsN < 32$ or $nBitsN > 4096$.
<code>ippStsBadArgErr</code>	Indicates an error condition if $(nBitsP \geq nBitsN)$ or $(2 * nBitsP < nBitsN)$ or <code>flag</code> value is illegal.

RSAInit

Initializes user-supplied memory as the IppsRSAState context for future use.

Syntax

```
IppStatus ippsRSAInit(int nBitsN, int nBitsP, IppRSAKeyType flag,
                     IppsRSAState* pCtx);
```

Parameters

<i>nBitsN</i>	Length of the RSA system in bits (that is, the length of the composite RSA modulus <i>n</i> in bits)
<i>nBitsP</i>	Length in bits of the largest of two prime factors of the RSA modulus.
<i>flag</i>	The flag indicating RSA system for encryption or decryption operation.
<i>pCtx</i>	Pointer to the IppsRSAState context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as the IppsRSAState context. Use the *flag* == IppRSAPublic to initialize RSA context for encryption or *flag* == IppRSAprivate to initialize RSA context for the decryption operation. Refer to [RSA-OAEP Scheme Functions](#) and [RSA-SSA Scheme Functions](#) on which *flag* value to select for the schemes.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsNotSupportedModeErr</code>	Indicates an error condition if <i>nBitsN</i> < 32 or <i>nBitsN</i> > 4096.
<code>ippStsBadArgErr</code>	Indicates an error condition if <i>nBitsN</i> > <i>nBitsP</i> or illegal <i>flag</i> value.

RSASetKey

Sets the tag-designated key component into the established RSA context.

Syntax

```
IppStatus ippsRSASetKey(const IppsBigNumState* pBN, ippsRSAKeyTag tag,
                        IppsRSAState* pCtx);
```

Parameters

<i>pBN</i>	Pointer to the Big Number context presented by key component.
<i>tag</i>	The tag of the key component being set up.
<i>pCtx</i>	Pointer to the <code>IppsRSAState</code> context.

Description

This function is declared in the `ippcp.h` file. The function sets the key specified by *pBN* to the tag-designated component of the `IppsRSAState` context:

- *tag* == `IppRSAkeyN`, the function sets up the RSA system composite integer *n*
- *tag* == `IppRSAkeyP`, the function sets up the RSA system prime factor *p*
- *tag* == `IppRSAkeyQ`, the function sets up the RSA system prime factor *q*
- *tag* == `IppRSAkeyE`, the function sets up the RSA system public exponent *e*
- *tag* == `IppRSAkeyD`, the function sets up the RSA system public exponent *d*.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsLengthErr</code>	Indicates an error condition if the length of the Big Number specified by <i>pBN</i> is less than 1.
<code>ippStsBadArgErr</code>	Indicates an error condition if some of the arguments are invalid: Big Number specified by <i>pBN</i> is negative, key component specified by <i>tag</i> does not match the RSA context.

<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if the length of the Big Number specified by <code>pBN</code> is too big to be stored in the <code>IppsRSAState</code> context.

RSAGetKey

Extracts the tag-designated key component from the RSA context.

Syntax

```
IppStatus ippRSAGetKey(IppsBigNumState* pBN, IppsRSAKeyTag tag,
                      IppsRSAState* pCtx);
```

Parameters

<code>pBN</code>	Pointer to the output Big Number context.
<code>tag</code>	The tag of the key component being extracted from.
<code>pCtx</code>	Pointer to the <code>IppsRSAState</code> context.

Description

This function is declared in the `ippcp.h` file. The function extracts the tag-designated key component from the `*pCtx` RSA context and stores it in the specified `*pBN` Big Number context:

- `tag == IppRSAkeyN`, the function extracts the RSA system composite integer n
- `tag == IppRSAkeyP`, the function extracts the RSA system prime factor p
- `tag == IppRSAkeyQ`, the function extracts the RSA system prime factor q
- `tag == IppRSAkeyE`, the function extracts the RSA system public exponent e
- `tag == IppRSAkeyD`, the function extracts the RSA system public exponent d .

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsBadArgErr</code>	Indicates an error condition if the key component specified by tag does not match the RSA context.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if the length of the Big Number specified by <i>pBN</i> is too small to be stored as a key component.

RSAGenerate

Generates key components for the desired RSA cryptographic system.

Syntax

```
IppStatus ippRSAGenerate(IppsBigNumState* pE, int nBitsN, int nBitsP, int
    nTrials, IppsRSAState* pCtx, IppBitSupplier rndFunc, void* pRndParam);
```

Parameters

<i>pE</i>	Pointer to the <code>IppsBigNumState</code> context of the newly generated RSA public exponent key.
<i>nBitsN</i>	Length of the RSA system in bits (that is, the length of the composite RSA modulus <i>n</i> in bits)
<i>nBitsP</i>	Length in bits of one of the two prime factors of the RSA modulus.
<i>nTrials</i>	Security parameter specified for the Miller-Rabin probable primality
<i>pCtx</i>	Pointer to the <code>IppsRSAState</code> context.
<i>rndFunc</i>	Specified Random Generator
<i>pRndParam</i>	Pointer to the random Generator context.

Description

The function is declared in the `ippcp.h` file. This function generates the desired RSA cryptographic system based on:

- the input **pE* specifying the initial value for searching the RSA public exponent
- input parameters *nBitsN* and *nBitsP* specifying the bit lengths of the composite RSA modulus *n* and the largest RSA prime factor *p* respectively.

This function employs the specified *rndFunc* Random Generator to generate a random probable prime numbers *p* and *q*.

The function then computes the RSA composite modulus $n = (p*q)$, and the RSA private exponent d , and it further computes all other CRT-related RSA components.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsNotSupportedModeErr</code>	Indicates an error condition if RSA context was initialized with the <code>IppRSAPublic</code> flag.
<code>ippStsBadArgErr</code>	Indicates an error condition in cases not explicitly mentioned above.
<code>ippStsInsufficientEntropy</code>	Indicates a warning condition if prime generation fails due to poor choice of entropy.

RSASValidate

Validates key components of the RSA cryptographic system.

Syntax

```
IppStatus ippSRASValidate(const IppsBigNumState* pE, int nTrials, Ipp32u*  
    pResult, IppsRSASState* pCtx, IppBitSuppler rndFunc, void* pRndParam);
```

Parameters

<code>pE</code>	Pointer to the <code>IppsBigNumState</code> context of the RSA public exponent.
<code>nTrials</code>	Security parameter specified for the Miller-Rabin probable primality.
<code>pResult</code>	Pointer to the result of validation.
<code>pCtx</code>	Pointer to the <code>IppsRSASState</code> context.
<code>rndFunc</code>	Specified Random Generator.
<code>pRndParam</code>	Pointer to the Random Generator context.

Description

This function is declared in the `ippcp.h` file. The function validates key components of the RSA cryptographic system and stores the result (`IS_VALID_KEY` or `IS_INVALID_KEY`) of the validation procedure in `*pResult`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsBadArgErr</code>	Indicates an error condition if <code>nBitsN > nBitsP</code> or <code>nTrials < 1</code> .
<code>ippStsLengthErr</code>	Indicates an error condition if the length of the Big Number specified by <code>pE</code> is less than 1.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if the length of the Big Number specified by <code>pE</code> is too big to be stored in the <code>IppsRSASState</code> context.

RSA Primitives

The list of RSA cryptographic primitives is given in [Table 5-6](#).

Table 5-6 Intel IPP RSA Primitives

Function Base Name	Operation
RSAEncrypt	Performs the RSA encryption operation.
RSADecrypt	Performs the RSA decryption operation.

- The application code for conducting a typical RSA encryption must perform the following sequence of operations, starting with building of a crypto system:
1. Call the function [RSAGetSize](#) to get the size required to configure `IppsRSASState` context.
 2. Ensure that the required memory space is properly allocated. With the allocated memory, call the [RSAInit](#) function to initialize the context for the RSA encryption.
 3. Keep calling the [RSASetKey](#) to set up RSA public key (n, e).

4. Invoke the [RSAEncrypt](#) function with the established RSA public key to encode the plaintext into the respective ciphertext.
5. Free the memory allocated for the `IppsRSAState` context by calling the operating system memory free service function.

The typical application code for the RSA decryption must perform the following sequence of operations:

1. Call the function [RSAGetSize](#) to get the size required to configure `IppsRSAState` context.
2. Ensure that the required memory space is properly allocated. With the allocated memory, call the [RSAInit](#) function to initialize the context for the RSA decryption.
3. Establish the RSA private key by means of either [RSAGenerate](#) function or by key setup function [RSASetKey](#). The `RSAGenerate` function computes the private key d with respect to the generated public key e , as well as several other components for applying the CRT. When using [RSASetKey](#), you have an option of presenting the private key either as a pair (n, d) or quantiple $(p, q, dP, dQ, qInv)$.
4. Invoke the [RSADecrypt](#) function with the established RSA public key to decode the ciphertext into the respective plaintext.
5. Free the memory allocated for the `IppsRSAState` context by calling the operating system memory free service function.

RSAEncrypt

Performs the RSA encryption operation.

Syntax

```
IppStatus ippsRSAEncrypt(const IppsBigNumState* pX, IppsBigNumState* pY,
    IppsRSAState* pCtx);
```

Parameters

<code>pX</code>	Pointer to the <code>IppsBigNumState</code> context of the plaintext.
<code>pY</code>	Pointer to the <code>IppsBigNumState</code> context of the ciphertext.
<code>pCtx</code>	Pointer to the <code>IppsRSAState</code> context.

Description

This function is declared in the `ippcp.h` file. The function performs the RSA encryption operation.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsInvalidCryptoKeyErr</code>	Indicates an error condition if the RSA context has not been properly set up for the operation.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if: length of the Big Number specified by <i>pX</i> is too big length of the Big Number specified by <i>pY</i> is too small.

RSADecrypt

Performs the RSA decryption operation.

Syntax

```
IppStatus ippRSADecrypt(IppsBigNumState* pX, IppsBigNumState* pY,
                        IppsRSAState* pCtx);
```

Parameters

<i>pX</i>	Ppointer to the <code>IppsBigNumState</code> context of the ciphertext.
<i>pY</i>	Pointer to the <code>IppsBigNumState</code> context of the plaintext.
<i>pCtx</i>	Pointer to the <code>IppsRSAState</code> context.

Description

This function is declared in the `ippcp.h` file. The function performs the RSA encryption operation.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsInvalidCryptoKeyErr</code>	Indicates an error condition if the RSA context has not been properly set up for the operation.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if: length of the Big Number specified by pX is too big or length of the Big Number specified by pY is too small.

The example below illustrates the use of RSA primitives. The example uses the `BigNumber` class and functions creating some cryptographic contexts, whose source code can be found in Appendix B.

Example 5-8 The Use of RSA Primitives

```
static Ipp32u dataN[] = {
    0x091DBDCB, 0x46F8E5FD,
    0xCA2A8F59, 0xE2537298, 0xF6C1687F, 0x527A9A41, 0x7B61A51F, 0xE0AAB12D,
    0x4598394E, 0x8834B245, 0x06095374, 0xEE6A649D, 0xD93A2584, 0x3EE6B4B7,
    0xDFC73772, 0xAFB8E0A3, 0x5B8B807F, 0x19719D8A, 0x60E1EC46, 0x76ED520D,
    0xEB6FCD48, 0x61EA48CE, 0x035C02AB, 0xB8DFBAAF, 0x7454F51F, 0x40D6B6F0,
    0xD41043A4, 0x368D07EE, 0x9DA871F7, 0x2338AC2B, 0x0682CE9C, 0xBBF82F09
};

static Ipp32u dataP[] = {
    0x58FB6599, 0x7541BA2A, 0x459D1F39, 0x5B252176,
    0xAA040A2D, 0x7E28FAE7, 0x6E5D1E3B, 0x124EF023, 0x3D84F632, 0x93B81A9E,
    0xAEF4FDA4, 0x99EB9F44, 0xA1B56001, 0x08810B10, 0xB1B9B3C9, 0xEECF8E81
};

static Ipp32u dataQ[] = {
    0xAF461503, 0xA441E700, 0x4D0416A5, 0xCE335252,
    0x3204B5CF, 0xEA0DA3B4, 0x66B42E92, 0x9840B416, 0x028B9D86, 0x5A0F2035,
    0x8866B1D0, 0x3F6C42D0, 0xAAD1D935, 0x341233EA, 0x27F453F6, 0xC97FB1F0
};

static Ipp32u dataE[] = {0x11};
```

Example 5-8 The Use of RSA Primitives (continued)

```
int RSA_sample(void)
{
    BigNumber P(dataP, sizeof(dataP)/sizeof(dataP[0]));
    BigNumber Q(dataQ, sizeof(dataQ)/sizeof(dataQ[0]));
    BigNumber N = P*Q;
    BigNumber E(dataE, sizeof(dataE)/sizeof(dataE[0]));
    IppsRSAState* pRSAPub = newRSA(N.BitSize(), P.BitSize(), IppRSApublic);
    IppsRSAState* pRSAPrv1 = newRSA(N.BitSize(), P.BitSize(), IppRSAprivate);
    IppsRSAState* pRSAPrv2 = newRSA(N.BitSize(), P.BitSize(), IppRSAprivate);

    // compute private key
    BigNumber phi = (P-BigInteger(1))*(Q-BigInteger(1));
    BigNumber D = phi.InverseMul(E);

    // set up public RSA (N,E)
    ippsRSASetKey(N, IppRSAkeyN, pRSAPub);
    ippsRSASetKey(E, IppRSAkeyE, pRSAPub);

    // set up private (no CRT) RSA (N, D)
    ippsRSASetKey(N, IppRSAkeyN, pRSAPrv1);
    ippsRSASetKey(D, IppRSAkeyD, pRSAPrv1);

    // set up private (CRT) RSA (P,Q,D)
    ippsRSASetKey(P, IppRSAkeyP, pRSAPrv2);
    ippsRSASetKey(Q, IppRSAkeyQ, pRSAPrv2);
    ippsRSASetKey(D, IppRSAkeyD, pRSAPrv2);

    // validate RSA
    IppsPRNGState* pRand = newPRNG();
    Ipp32u result;
    ippsRSAValidate(E, 50, &result, pRSAPrv2, ippsPRNGen, pRand);
}
```

Example 5-8 The Use of RSA Primitives (continued)

```

    if (IS_VALID_KEY != result) {
        cout << "validation fail" << endl;
        return 0;
    }
    // validation pass

    // plaintext
    Ipp32u dataM[] = {
        0x4D353E2D, 0xD2F1B76D,
        0x5281CE32, 0x7BC27519, 0x2F3AC14F, 0x0448DB97, 0xD095AEB4, 0x82FB3E87,
        0x1BE392F9, 0x43581159, 0xD5024121, 0xB48D2869, 0x2BAAD29A, 0xA1B7C136,
        0xF47728B4, 0x4CDCFE4F, 0x839A2DDB, 0xFF8AE10E, 0x25C9C2B3, 0xF93EDCFB,
        0x4626F5AF, 0xD7E0B2C0, 0xB4251F84, 0xC31B2E8B, 0xA8F55267, 0x5C68F1EE,
        0x26DCD87D, 0xCA82310B, 0x504B45E2, 0x6350E329, 0xACE9E300, 0x00EB7A19
    };
    BigNumber M(dataM, sizeof(dataM)/sizeof(dataM[0]));

    // encrypt plaintext
    BigNumber C(0, N.DwordSize());
    ippsRSAEncrypt(M, C, pRSAPub);

    // decrypt ciphertext using pRSAprv1
    BigNumber Z1(0, N.DwordSize());
    ippsRSADecrypt(C, Z1, pRSAprv1);

```

Example 5-8 The Use of RSA Primitives (continued)

```
// decrypt ciphertext using pRSAprv2
BigNumber Z2(0,N.DwordSize());
ippsRSADecrypt(C, Z2, pRSAprv2);

deleteRSA(pRSAPub);
deleteRSA(pRSAprv1);
deleteRSA(pRSAprv2);

return (M==Z1) && (M==Z2);
}
```

RSA-OAEP Scheme Functions

This subsection describes functions implementing RSA-OAEP encryption scheme, featured in [\[PKCS 1.2.1\]](#).

The full list of these functions is given in [Table 5-7](#).

Table 5-7 Intel IPP RSA-based Encryption Scheme Functions

Function Base Name	Operation
RSAOAEPEncrypt	Carries out the RSA-OAEP encryption scheme.
RSAOAEPEncrypt_MD5	MD5-based helper of the RSA-OAEP encryption scheme.
RSAOAEPEncrypt_SHA1	SHA-1-based helper of the RSA-OAEP encryption scheme.
RSAOAEPEncrypt_SHA224	SHA-224-based helper of the RSA-OAEP encryption scheme.
RSAOAEPEncrypt_SHA256	SHA-256-based helper of the RSA-OAEP encryption scheme.
RSAOAEPEncrypt_SHA384	SHA-384-based helper of the RSA-OAEP encryption scheme.
RSAOAEPEncrypt_SHA512	SHA-512-based helper of the RSA-OAEP encryption scheme.
RSAOAEPDecrypt	Carries out the RSA-OAEP decryption scheme.
RSAOAEPDecrypt_MD5	MD5-based helper of the RSA-OAEP decryption scheme.
RSAOAEPDecrypt_SHA1	SHA-1-based helper of the RSA-OAEP decryption scheme.
RSAOAEPDecrypt_SHA224	SHA-224-based helper of the RSA-OAEP decryption scheme.
RSAOAEPDecrypt_SHA256	SHA-256-based helper of the RSA-OAEP decryption scheme.

Table 5-7 Intel IPP RSA-based Encryption Scheme Functions (continued)

Function Base Name	Operation
RSAOAEPDecrypt_SHA384	SHA-384-based helper of the RSA-OAEP decryption scheme.
RSAOAEPDecrypt_SHA512	SHA-512-based helper of the RSA-OAEP decryption scheme.

To invoke a function that carries out RSA-OAEP scheme, the `IppsRSASState` context must be initialized (by the [RSAInit](#) function) with a proper value of the *flag* parameter:

- For the RSA-OAEP encryption scheme, *flag* == `IppRSAPublic`
- For the RSA-OAEP decryption scheme, *flag* == `IppRSAprivate`.

RSAOAEPEncrypt

Carries out the RSA-OAEP encryption scheme.

Syntax

```
IppStatus ippsRSAOAEPEncrypt(const Ipp8u* pSrc, int srcLen, const Ipp8u*
    pLabel, int labLen, const Ipp8u* pSeed, Ipp8u* pDst, IppsRSASState*
    pCtx, IppHash hushFunc, int hashLen, IppMGF mgfFunc);
```

Parameters

<i>pSrc</i>	Pointer to the octet message to be encrypted.
<i>srcLen</i>	Length of the message to be encrypted.
<i>pLabel</i>	Pointer to the optional label to be associated with the message.
<i>labLen</i>	Length of the optional label.
<i>pSeed</i>	Pointer to the random octet string of length <i>hashLen</i> .
<i>pDst</i>	Pointer to the output octet ciphertext string.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.
<i>hashFunc</i>	Hash function, which meets the General Definition of a Hash Function given in chapter 3.
<i>hashLen</i>	Length of the hash function output, in octets.
<i>mgfFunc</i>	Mask generation function (MGF), which meets the definition given in section User's Implementation of a Mask Generation Function in chapter 3.

Description

This function is declared in the `ippcp.h` file. The function carries out the RSA-OAEP encryption scheme, defined in [PKCS 1.2.1].

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsInvalidCryptoKeyErr</code>	Indicates an error condition if the RSA context has not been properly set up for the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input length parameters do not meet any of the following conditions: <i>srcLen</i> > 0, <i>srcLen</i> > $N - 2 \cdot \textit{hashLen} - 2$, where <i>N</i> is the length of the RSA modulus in octets, <i>labLen</i> > 0, <i>hashLen</i> > 0 .

RSAOAEPDecrypt_MD5

Carries out the RSA-OAEP encryption scheme using MD5 hash algorithm.

Syntax

```
IppStatus ippsRSAOAEPDecrypt_MD5(const Ipp8u* pSrc, int srcLen, const
    Ipp8u* pLabel, int labLen, const Ipp8u* pSeed, Ipp8u* pDst,
    IppsRSAState* pCtx);
```

Parameters

<i>pSrc</i>	Pointer to the octet message to be encrypted.
<i>srcLen</i>	Length of the message to be encrypted.
<i>pLabel</i>	Pointer to the optional label to be associated with the message.

<i>labLen</i>	Length of the optional label.
<i>pSeed</i>	Pointer to the random octet string of length <i>hashLen</i> , where <i>hashLen</i> is the length (in octets) of the hash message digest.
<i>pDst</i>	Pointer to the output octet ciphertext string.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses MD5 hash function and MD5-based MGF implemented in Intel IPP to carry out the RSA-OAEP encryption scheme and thus is a specific form of the general `RSASOAEP_Encrypt` function.

Return Values

The function may return any of the values that the general [RSASOAEP_Encrypt](#) function returns.

RSASOAEP_Encrypt_SHA1

Carries out the RSA-OAEP encryption scheme using SHA-1 hash algorithm.

Syntax

```
IppStatus ippRSASOAEP_Encrypt_SHA1(const Ipp8u* pSrc, int srcLen, const  
    Ipp8u* pLabel, int labLen, const Ipp8u* pSeed, Ipp8u* pDst,  
    IppsRSASState* pCtx);
```

Parameters

<i>pSrc</i>	Pointer to the octet message to be encrypted.
<i>srcLen</i>	Length of the message to be encrypted.
<i>pLabel</i>	Pointer to the optional label to be associated with the message.
<i>labLen</i>	Length of the optional label.
<i>pSeed</i>	Pointer to the random octet string of length <i>hashLen</i> , where <i>hashLen</i> is the length (in octets) of the hash message digest.
<i>pDst</i>	Pointer to the output octet ciphertext string.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-1 hash function and SHA-1-based MGF implemented in Intel IPP to carry out the RSA-OAEP encryption scheme and thus is a specific form of the general `RSAOAEP_Encrypt` function.

Return Values

The function may return any of the values that the general [RSAOAEP_Encrypt](#) function returns.

RSAOAEP_Encrypt_SHA224

Carries out the RSA-OAEP encryption scheme using SHA-224 hash algorithm.

Syntax

```
IppStatus ippRSAOAEP_Encrypt_SHA224(const Ipp8u* pSrc, int srcLen, const Ipp8u*
    pLabel, int labLen, const Ipp8u* pSeed, Ipp8u* pDst, IppsRSAStruct* pCtx);
```

Parameters

<i>pSrc</i>	Pointer to the octet message to be encrypted.
<i>srcLen</i>	Length of the message to be encrypted.
<i>pLabel</i>	Pointer to the optional label to be associated with the message.
<i>labLen</i>	Length of the optional label.
<i>pSeed</i>	Pointer to the random octet string of length <i>hashLen</i> , where <i>hashLen</i> is the length (in octets) of the hash message digest.
<i>pDst</i>	Pointer to the output octet ciphertext string.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSAStruct</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-224 hash function and SHA-224-based MGF implemented in Intel IPP to carry out the RSA-OAEP encryption scheme and thus is a specific form of the general `RSAOAEP_Encrypt` function.

Return Values

The function may return any of the values that the general [RSAOAEP_Encrypt](#) function returns.

RSAOAEPDecrypt_SHA256

Carries out the RSA-OAEP encryption scheme using SHA-256 hash algorithm.

Syntax

```
IppStatus ippsRSAOAEPDecrypt_SHA256(const Ipp8u* pSrc, int srcLen, const Ipp8u* pLabel, int labLen, const Ipp8u* pSeed, Ipp8u* pDst, IppsRSASState* pCtx);
```

Parameters

<i>pSrc</i>	Pointer to the octet message to be encrypted.
<i>srcLen</i>	Length of the message to be encrypted.
<i>pLabel</i>	Pointer to the optional label to be associated with the message.
<i>labLen</i>	Length of the optional label.
<i>pSeed</i>	Pointer to the random octet string of length <i>hashLen</i> , where <i>hashLen</i> is the length (in octets) of the hash message digest.
<i>pDst</i>	Pointer to the output octet ciphertext string.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-256 hash function and SHA-256-based MGF implemented in Intel IPP to carry out the RSA-OAEP encryption scheme and thus is a specific form of the general `RSAOAEPDecrypt` function.

Return Values

The function may return any of the values that the general [RSAOAEPDecrypt](#) function returns.

RSAOAEPDecrypt_SHA384

Carries out the RSA-OAEP encryption scheme using SHA-384 hash algorithm.

Syntax

```
IppStatus ippsRSAOAEPDecrypt_SHA384(const Ipp8u* pSrc, int srcLen, const Ipp8u* pLabel, int labLen, const Ipp8u* pSeed, Ipp8u* pDst, IppsRSASState* pCtx);
```

Parameters

<i>pSrc</i>	Pointer to the octet message to be encrypted.
<i>srcLen</i>	Length of the message to be encrypted.
<i>pLabel</i>	Pointer to the optional label to be associated with the message.
<i>labLen</i>	Length of the optional label.
<i>pSeed</i>	Pointer to the random octet string of length <i>hashLen</i> , where <i>hashLen</i> is the length (in octets) of the hash message digest.
<i>pDst</i>	Pointer to the output octet ciphertext string.
<i>pCtx</i>	Pointer to the properly initialized <i>IppsRSASState</i> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-384 hash function and SHA-384-based MGF implemented in Intel IPP to carry out the RSA-OAEP encryption scheme and thus is a specific form of the general [RSAOAEPDecrypt](#) function.

Return Values

The function may return any of the values that the general [RSAOAEPDecrypt](#) function returns.

RSAOAEPDecrypt_SHA512

Carries out the RSA-OAEP encryption scheme using SHA-512 hash algorithm.

Syntax

```
IppStatus ippsRSAOAEPDecrypt_SHA512(const Ipp8u* pSrc, int srcLen, const Ipp8u* pLabel, int labLen, const Ipp8u* pSeed, Ipp8u* pDst, IppsRSASState* pCtx);
```

Parameters

<i>pSrc</i>	Pointer to the octet message to be encrypted.
<i>srcLen</i>	Length of the message to be encrypted.
<i>pLabel</i>	Pointer to the optional label to be associated with the message.
<i>labLen</i>	Length of the optional label.
<i>pSeed</i>	Pointer to the random octet string of length <i>hashLen</i> , where <i>hashLen</i> is the length (in octets) of the hash message digest.
<i>pDst</i>	Pointer to the output octet ciphertext string.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-512 hash function and SHA-512-based MGF implemented in Intel IPP to carry out the RSA-OAEP encryption scheme and thus is a specific form of the general `RSAOAEPDecrypt` function.

Return Values

The function may return any of the values that the general [RSAOAEPDecrypt](#) function returns.

RSAOAEPDecrypt

Carries out the RSA-OAEP decryption scheme.

Syntax

```
IppStatus ippsRSOAEPDecrypt(const Ipp8u* pSrc, const Ipp8u* pLabel, int
    labLen, Ipp8u* pDst, int* pDstLen, IppsRSASState* pCtx, IppHash
    hushFunc, int hashLen, IppMGF mgfFunc);
```

Parameters

<i>pSrc</i>	Pointer to the octet ciphertext to be decrypted.
<i>pLabel</i>	Pointer to the optional label to be associated with the message.
<i>labLen</i>	Length of the optional label.
<i>pDst</i>	Pointer to the output octet plaintext message.
<i>pDstLen</i>	Pointer to the length of the decrypted message.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.
<i>hashFunc</i>	Hash function, which meets the General Definition of a Hash Function given in chapter 3.
<i>hashLen</i>	Length of the hash function output, in octets.
<i>mgfFunc</i>	Mask generation function (MGF), which meets the definition given in section User's Implementation of a Mask Generation Function in chapter 3.

Description

This function is declared in the `ippcp.h` file. The function carries out the RSA-OAEP decryption scheme defined in [\[PKCS 1.2.1\]](#).

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsInvalidCryptoKeyErr</code>	Indicates an error condition if the RSA context has not been properly set up for the operation.

<code>ippStsLengthErr</code>	Indicates an error condition if the input length parameters do not meet any of the following conditions: $labLen > 0$, $hashLen > 0$, $2 \cdot hashLen + 2 \geq N$, where N is the length of the RSA modulus in octets.L
------------------------------	--

RSAOAEPDecrypt_MD5

Carries out the RSA-OAEP decryption scheme using MD5 hash algorithm.

Syntax

```
IppStatus ippRSOAEPDecrypt_MD5(const Ipp8u* pSrc, const Ipp8u* pLabel,  
                                int labLen, Ipp8u* pDst, int* pDstLen, IppsRSASState* pCtx);
```

Parameters

<code>pSrc</code>	Pointer to the octet ciphertext to be decrypted.
<code>pLabel</code>	Pointer to the optional label to be associated with the message.
<code>labLen</code>	Length of the optional label.
<code>pDst</code>	Pointer to the output octet plaintext message.
<code>pDstLen</code>	Pointer to the length of the decrypted message.
<code>pCtx</code>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses MD5 hash function and MD5-based MGF implemented in Intel IPP to carry out the RSA-OAEP decryption scheme and thus is a specific form of the general `RSAOAEPDecrypt` function.

Return Values

The function may return any of the values that the general [RSAOAEPDecrypt](#) function returns.

RSAOAEPDecrypt_SHA1

Carries out the RSA-OAEP decryption scheme using SHA-1 hash algorithm.

Syntax

```
IppStatus ippsRSAOAEPDecrypt_SHA1(const Ipp8u* pSrc, const Ipp8u*
    pLabel, int labLen, Ipp8u* pDst, int* pDstLen, IppsRSASState* pCtx);
```

Parameters

<i>pSrc</i>	Pointer to the octet ciphertext to be decrypted.
<i>pLabel</i>	Pointer to the optional label to be associated with the message.
<i>labLen</i>	Length of the optional label.
<i>pDst</i>	Pointer to the output octet plaintext message.
<i>pDstLen</i>	Pointer to the length of the decrypted message.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-1 hash function and SHA-1based MGF implemented in Intel IPP to carry out the RSA-OAEP decryption scheme and thus is a specific form of the general `RSAOAEPDecrypt` function.

Return Values

The function may return any of the values that the general [RSAOAEPDecrypt](#) function returns.

RSAOAEPDecrypt_SHA224

Carries out the RSA-OAEP decryption scheme using SHA-224 hash algorithm.

Syntax

```
IppStatus ippsRSAOAEPDecrypt_SHA224(const Ipp8u* pSrc, const Ipp8u*
    pLabel, int labLen, Ipp8u* pDst, int* pDstLen, IppsRSASState* pCtx);
```

Parameters

<i>pSrc</i>	Pointer to the octet ciphertext to be decrypted.
<i>pLabel</i>	Pointer to the optional label to be associated with the message.
<i>labLen</i>	Length of the optional label.
<i>pDst</i>	Pointer to the output octet plaintext message.
<i>pDstLen</i>	Pointer to the length of the decrypted message.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-224 hash function and SHA-224-based MGF implemented in Intel IPP to carry out the RSA-OAEP decryption scheme and thus is a specific form of the general `RSASOAEPDecrypt` function.

Return Values

The function may return any of the values that the general [RSASOAEPDecrypt](#) function returns.

RSASOAEPDecrypt_SHA256

Carries out the RSA-OAEP decryption scheme using SHA-256 hash algorithm.

Syntax

```
IppStatus ippRSASOAEPDecrypt_SHA256(const Ipp8u* pSrc, const Ipp8u*  
    pLabel, int labLen, Ipp8u* pDst, int* pDstLen, IppsRSASState* pCtx);
```

Parameters

<i>pSrc</i>	Pointer to the octet ciphertext to be decrypted.
<i>pLabel</i>	Pointer to the optional label to be associated with the message.
<i>labLen</i>	Length of the optional label.
<i>pDst</i>	Pointer to the output octet plaintext message.
<i>pDstLen</i>	Pointer to the length of the decrypted message.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-256 hash function and SHA-256-based MGF implemented in Intel IPP to carry out the RSA-OAEP decryption scheme and thus is a specific form of the general `RSAOAEPDecrypt` function.

Return Values

The function may return any of the values that the general [RSAOAEPDecrypt](#) function returns.

RSAOAEPDecrypt_SHA384

Carries out the RSA-OAEP decryption scheme using SHA-384 hash algorithm.

Syntax

```
IppStatus ippRSAOAEPDecrypt_SHA384(const Ipp8u* pSrc, const Ipp8u*
    pLabel, int labLen, Ipp8u* pDst, int* pDstLen, IppsRSASState* pCtx);
```

Parameters

<i>pSrc</i>	Pointer to the octet ciphertext to be decrypted.
<i>pLabel</i>	Pointer to the optional label to be associated with the message.
<i>labLen</i>	Length of the optional label.
<i>pDst</i>	Pointer to the output octet plaintext message.
<i>pDstLen</i>	Pointer to the length of the decrypted message.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-384 hash function and SHA-384-based MGF implemented in Intel IPP to carry out the RSA-OAEP decryption scheme and thus is a specific form of the general `RSAOAEPDecrypt` function.

Return Values

The function may return any of the values that the general [RSAOAEPDecrypt](#) function returns.

RSAOAEPDecrypt_SHA512

Carries out the RSA-OAEP decryption scheme using SHA-512 hash algorithm.

Syntax

```
IppStatus ippsRSAOAEPDecrypt_SHA512(const Ipp8u* pSrc, const Ipp8u*
    pLabel, int labLen, Ipp8u* pDst, int* pDstLen, IppsRSASState* pCtx);
```

Parameters

<i>pSrc</i>	Pointer to the octet ciphertext to be decrypted.
<i>pLabel</i>	Pointer to the optional label to be associated with the message.
<i>labLen</i>	Length of the optional label.
<i>pDst</i>	Pointer to the output octet plaintext message.
<i>pDstLen</i>	Pointer to the length of the decrypted message.
<i>pCtx</i>	Pointer to the properly initialized IppsRSASState context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-512 hash function and SHA-512-based MGF implemented in Intel IPP to carry out the RSA-OAEP decryption scheme and thus is a specific form of the general `RSAOAEPDecrypt` function.

Return Values

The function may return any of the values that the general [RSAOAEPDecrypt](#) function returns.

RSA-SSA Scheme Functions

This subsection describes functions implementing RSA-SSA signature scheme with appendix, featured in [\[PKCS 1.2.1\]](#).

The full list of these functions is given in [Table 5-8](#).

Table 5-8 Intel IPP RSA-based Signature Scheme Functions

Function Base Name	Operation
RSASSASign	Carries out the RSA-SSA signature generation scheme.

Table 5-8 Intel IPP RSA-based Signature Scheme Functions (continued)

Function Base Name	Operation
RSASSASign_MD5	MD5-based helper of the RSA-SSA signature generation scheme.
RSASSASign_SHA1	SHA-1-based helper of the RSA-SSA signature generation scheme.
RSASSASign_SHA224	SHA-224-based helper of the RSA-SSA signature generation scheme.
RSASSASign_SHA256	SHA-256-based helper of the RSA-SSA signature generation scheme.
RSASSASign_SHA384	SHA-384-based helper of the RSA-SSA signature generation scheme.
RSASSASign_SHA512	SHA-512-based helper of the RSA-SSA signature generation scheme.
RSASSAVerify	Carries out the RSA-SSA signature verification scheme.
RSASSAVerify_MD5	MD5 based helper of the RSA-SSA signature verification scheme.
RSASSAVerify_SHA1	SHA-1-based helper of the RSA-SSA signature verification scheme.
RSASSAVerify_SHA224	SHA-224-based helper of the RSA-SSA signature verification scheme.
RSASSAVerify_SHA256	SHA-256-based helper of the RSA-SSA signature verification scheme.
RSASSAVerify_SHA384	SHA-384-based helper of the RSA-SSA signature verification scheme.
RSASSAVerify_SHA512	SHA-512-based helper of the RSA-SSA signature verification scheme.

To invoke a function that carries out RSA-SSA scheme, the `IppsRSASState` context must be initialized (by the [RSAInit](#) function) with a proper value of the *flag* parameter:

- For the RSA-SSA signing scheme, *flag* == `IppRSAPrivate`
- For the RSA-SSA verification scheme, *flag* == `IppRSAPublic`.

RSASSASign

Carries out the RSA-SSA signature generation scheme.

Syntax

```
IppStatus ippsRSASSASign(const Ipp8u* pHMsg, int hashLen, const Ipp8u*
    pSalt, int saltLen, Ipp8u* pSign, IppsRSASState* pCtx, IppHash
    hushFunc, IppMGF mgfFunc);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash to be signed.
<i>hashLen</i>	Length of the message hash <i>*pHMsg</i> in octets.

<i>pSalt</i>	Pointer to the random octet salt string
<i>saltLen</i>	Length of the salt string in octets.
<i>pSign</i>	Pointer to the output octet signature.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.
<i>hashFunc</i>	Hash function, which meets the General Definition of a Hash Function given in chapter 3.
<i>mgfFunc</i>	MGF, which meets the definition given in section User's Implementation of a Mask Generation Function in chapter 3.

Description

This function is declared in the `ippcp.h` file. The function generates a message signature according to the RSASSA-PSS scheme defined in [PKCS 1.2.1]. Intel IPP implementation of the scheme assumes that its first step, i.e. computing the hash digest of the original message, is executed prior to the function call and the resulting message hash **pHMsg* is passed to the function. The use of a message hash instead of the original message reduces the length of the function input message, limited by the upper bound of the integer data type $(2^{32} - 1) \cdot 8$ bit), and thus allows applying the entire RSA-SSA scheme to input messages of greater lengths. To compute the original message hash to be passed to the function, you should use the same hash function as the one specified by the *hashFunc* parameter and applied at the subsequent steps of the scheme.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsInvalidCryptoKeyErr</code>	Indicates an error condition if the RSA context has not been properly set up for the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input length parameters do not meet any of the following conditions: $hashLen > 0$, $saltLen \geq 0$, $N > hashLen + saltLen + 2$, where N is the length of the RSA modulus in octets.

RSASSASign_MD5

Carries out the RSA-SSA signature generation scheme using MD5 hash algorithm.

Syntax

```
IppStatus ippsRSASSASign_MD5(const Ipp8u* pHMsg, const Ipp8u* pSalt, int
    saltLen, Ipp8u* pSign, IppsRSASState* pCtx);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash to be signed.
<i>pSalt</i>	Pointer to the random octet salt string.
<i>saltLen</i>	Length of the salt string in octets.
<i>pSign</i>	Pointer to the output octet signature.
<i>pCtx</i>	Pointer to the properly initialized IppsRSASState context.

Description

This function is declared in the `ippcp.h` file. The function uses MD5 hash function and MD5-based MGF implemented in Intel IPP to generate a message signature according to the RSASSA-PSS scheme and thus is a specific form of the general [RSASSASign](#) function. Provided the IppsRSASState context is properly initialized, the entire RSA-SSA signing scheme is carried out in two steps:

1. Call the [MD5MessageDigest](#) function to generate the hash message to be signed.
2. Call RSASSASign_MD5 with *pHMsg* pointing to the resulting message hash.

Return Values

The function may return any of the values that the general [RSASSASign](#) function returns.

RSASSASign_SHA1

Carries out the RSA-SSA signature generation scheme using SHA-1 hash algorithm.

Syntax

```
IppStatus ippsRSASSASign_SHA1(const Ipp8u* pHMsg, const Ipp8u* pSalt, int  
    saltLen, Ipp8u* pSign, IppsRSASState* pCtx);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash to be signed.
<i>pSalt</i>	Pointer to the random octet salt string.
<i>saltLen</i>	Length of the salt string in octets.
<i>pSign</i>	Pointer to the output octet signature.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-1 hash function and SHA-1-based MGF implemented in Intel IPP to generate a message signature according to the RSASSA-PSS scheme and thus is a specific form of the general [RSASSASign](#) function. Provided the `IppsRSASState` context is properly initialized, the entire RSA-SSA signing scheme is carried out in two steps:

1. Call the [SHA1MessageDigest](#) function to generate the hash message to be signed.
2. Call `RSASSASign_SHA1` with *pHMsg* pointing to the resulting message hash.

Return Values

The function may return any of the values that the general [RSASSASign](#) function returns.

RSASSASign_SHA224

Carries out the RSA-SSA signature generation scheme using SHA-224 hash algorithm.

Syntax

```
IppStatus ippsRSASSASign_SHA224(const Ipp8u* pHMsg, const Ipp8u* pSalt,  
                                int saltLen, Ipp8u* pSign, IppsRSASState* pCtx);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash to be signed.
<i>pSalt</i>	Pointer to the random octet salt string.
<i>saltLen</i>	Length of the salt string in octets.
<i>pSign</i>	Pointer to the output octet signature.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-224 hash function and SHA-224-based MGF implemented in Intel IPP to generate a message signature according to the RSASSA-PSS scheme and thus is a specific form of the general [RSASSASign](#) function. Provided the `IppsRSASState` context is properly initialized, the entire RSA-SSA signing scheme is carried out in two steps:

1. Call the [SHA224MessageDigest](#) function to generate the hash message to be signed.
2. Call `RSASSASign_SHA224` with *pHMsg* pointing to the resulting message hash.

Return Values

The function may return any of the values that the general [RSASSASign](#) function returns.

RSASSASign_SHA256

Carries out the RSA-SSA signature generation scheme using SHA-256 hash algorithm.

Syntax

```
IppStatus ippsRSASSASign_SHA256(const Ipp8u* pHMsg, const Ipp8u* pSalt,  
    int saltLen, Ipp8u* pSign, IppsRSASState* pCtx);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash to be signed.
<i>pSalt</i>	Pointer to the random octet salt string.
<i>saltLen</i>	Length of the salt string in octets.
<i>pSign</i>	Pointer to the output octet signature.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-256 hash function and SHA-256-based MGF implemented in Intel IPP to generate a message signature according to the RSASSA-PSS scheme and thus is a specific form of the general [RSASSASign](#) function. Provided the `IppsRSASState` context is properly initialized, the entire RSA-SSA signing scheme is carried out in two steps:

1. Call the [SHA256MessageDigest](#) function to generate the hash message to be signed.
2. Call `RSASSASign_SHA256` with *pHMsg* pointing to the resulting message hash.

Return Values

The function may return any of the values that the general [RSASSASign](#) function returns.

RSASSASign_SHA384

Carries out the RSA-SSA signature generation scheme using SHA-384 hash algorithm.

Syntax

```
IppStatus ippsRSASSASign_SHA384(const Ipp8u* pHMsg, const Ipp8u* pSalt,
                                int saltLen, Ipp8u* pSign, IppsRSASState* pCtx);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash to be signed.
<i>pSalt</i>	Pointer to the random octet salt string.
<i>saltLen</i>	Length of the salt string in octets.
<i>pSign</i>	Pointer to the output octet signature.
<i>pCtx</i>	Pointer to the properly initialized IppsRSASState context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-384 hash function and SHA-384-based MGF implemented in Intel IPP to generate a message signature according to the RSASSA-PSS scheme and thus is a specific form of the general [RSASSASign](#) function. Provided the IppsRSASState context is properly initialized, the entire RSA-SSA signing scheme is carried out in two steps:

1. Call the [SHA384MessageDigest](#) function to generate the hash message to be signed.
2. Call RSASSASign_SHA384 with *pHMsg* pointing to the resulting message hash.

Return Values

The function may return any of the values that the general [RSASSASign](#) function returns.

RSASSASign_SHA512

Carries out the RSA-SSA signature generation scheme using SHA-512 hash algorithm.

Syntax

```
IppStatus ippsRSASSASign_SHA512(const Ipp8u* pHMsg, const Ipp8u* pSalt,  
    int saltLen, Ipp8u* pSign, IppsRSASState* pCtx);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash to be signed.
<i>pSalt</i>	Pointer to the random octet salt string.
<i>saltLen</i>	Length of the salt string in octets.
<i>pSign</i>	Pointer to the output octet signature.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-512 hash function and SHA-512-based MGF implemented in Intel IPP to generate a message signature according to the RSASSA-PSS scheme and thus is a specific form of the general [RSASSASign](#) function. Provided the `IppsRSASState` context is properly initialized, the entire RSA-SSA signing scheme is carried out in two steps:

1. Call the [SHA512MessageDigest](#) function to generate the hash message to be signed.
2. Call `RSASSASign_SHA512` with *pHMsg* pointing to the resulting message hash.

Return Values

The function may return any of the values that the general [RSASSASign](#) function returns.

RSASSAVerify

Carries out the RSA-SSA signature verification scheme.

Syntax

```
IppStatus ippRSASSAVerify(const Ipp8u* pHMsg, int hashLen, const Ipp8u* pSign,
    IppBool* pIsValid, IppsRSASState* pCtx, IppHash hushFunc, IppMGF mgfFunc);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash that has been signed.
<i>hashLen</i>	Length in octets of the message hash <i>*pHMsg</i> .
<i>pSign</i>	Pointer to the octet signature string to be verified.
<i>pIsValid</i>	Pointer to the verification result.
<i>pCtx</i>	Pointer to the properly initialized IppsRSASState context.
<i>hashFunc</i>	Hash function, which meets the General Definition of a Hash Function given in chapter 3.
<i>mgfFunc</i>	MGF, which meets the definition given in section User's Implementation of a Mask Generation Function in chapter 3.

Description

This function is declared in the `ippcp.h` file. The function carries out the RSASSA-PSS signature verification scheme defined in [[PKCS 1.2.1](#)].

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsInvalidCryptoKeyErr</code>	Indicates an error condition if the RSA context has not been properly set up for the operation.
<code>ippStsLengthErr</code>	Indicates an error condition if the input length parameter does not meet any of the following conditions:

$hashLen > 0$,
 $hashLen + 2 > N$, where N is the length of the RSA modulus in octets.

RSASSAVerify_MD5

Carries out the RSA-SSA signature verification scheme using MD5 hash algorithm.

Syntax

```
IppStatus ippRSASSAVerify_MD5(const Ipp8u* pHMsg, const Ipp8u* pSign,  
    IppBool* pIsValid, IppsRSASState* pCtx);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash that has been signed.
<i>pSign</i>	Pointer to the octet signature string to be verified.
<i>pIsValid</i>	Pointer to the verification result.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses MD5 hash function and MD5-based MGF implemented in Intel IPP to verify a signature according to the RSASSA-PSS scheme and thus is a specific form of the general [RSASSAVerify](#) function.

Return Values

The function may return any of the values that the general [RSASSAVerify](#) function returns.

RSASSAVerify_SHA1

Carries out the RSA-SSA signature verification scheme using SHA-1 hash algorithm.

Syntax

```
IppStatus ippsRSASSAVerify_SHA1(const Ipp8u* pHMsg, const Ipp8u* pSign,  
                                IppBool* pIsValid, IppsRSASState* pCtx);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash that has been signed.
<i>pSign</i>	Pointer to the octet signature string to be verified.
<i>pIsValid</i>	Pointer to the verification result.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-1 hash function and SHA-1-based MGF implemented in Intel IPP to verify a signature according to the RSASSA-PSS scheme and thus is a specific form of the general [RSASSAVerify](#) function.

Return Values

The function may return any of the values that the general [RSASSAVerify](#) function returns.

RSASSAVerify_SHA224

Carries out the RSA-SSA signature verification scheme using SHA-224 hash algorithm.

Syntax

```
IppStatus ippsRSASSAVerify_SHA224(const Ipp8u* pHMsg, const Ipp8u*  
                                   pSign, IppBool* pIsValid, IppsRSASState* pCtx);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash that has been signed.
--------------	---

<i>pSign</i>	Pointer to the octet signature string to be verified.
<i>pIsValid</i>	Pointer to the verification result.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-224 hash function and SHA-224-based MGF implemented in Intel IPP to verify a signature according to the RSASSA-PSS scheme and thus is a specific form of the general [RSASSAVerify](#) function.

Return Values

The function may return any of the values that the general [RSASSAVerify](#) function returns.

RSASSAVerify_SHA256

Carries out the RSA-SSA signature verification scheme using SHA-256 hash algorithm.

Syntax

```
IppStatus ippRSASSAVerify_SHA256(const Ipp8u* pHMsg, const Ipp8u*  
    pSign, IppBool* pIsValid, IppsRSASState* pCtx);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash that has been signed.
<i>pSign</i>	Pointer to the octet signature string to be verified.
<i>pIsValid</i>	Pointer to the verification result.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-256 hash function and SHA-256-based MGF implemented in Intel IPP to verify a signature according to the RSASSA-PSS scheme and thus is a specific form of the general [RSASSAVerify](#) function.

Return Values

The function may return any of the values that the general [RSASSAVerify](#) function returns.

RSASSAVerify_SHA384

Carries out the RSA-SSA signature verification scheme using SHA-384 hash algorithm.

Syntax

```
IppStatus ippsRSASSAVerify_SHA384(const Ipp8u* pHMsg, const Ipp8u*
    pSign, IppBool* pIsValid, IppsRSASState* pCtx);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash that has been signed.
<i>pSign</i>	Pointer to the octet signature string to be verified.
<i>pIsValid</i>	Pointer to the verification result.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-384 hash function and SHA-384-based MGF implemented in Intel IPP to verify a signature according to the RSASSA-PSS scheme and thus is a specific form of the general [RSASSAVerify](#) function.

Return Values

The function may return any of the values that the general [RSASSAVerify](#) function returns.

RSASSAVerify_SHA512

Carries out the RSA-SSA signature verification scheme using SHA-512 hash algorithm.

Syntax

```
IppStatus ippsRSASSAVerify_SHA512(const Ipp8u* pHMsg, const Ipp8u*
    pSign, IppBool* pIsValid, IppsRSASState* pCtx);
```

Parameters

<i>pHMsg</i>	Pointer to the octet message hash that has been signed.
--------------	---

<i>pSign</i>	Pointer to the octet signature string to be verified.
<i>pIsValid</i>	Pointer to the verification result.
<i>pCtx</i>	Pointer to the properly initialized <code>IppsRSASState</code> context.

Description

This function is declared in the `ippcp.h` file. The function uses SHA-512 hash function and SHA-512-based MGF implemented in Intel IPP to verify a signature according to the RSASSA-PSS scheme and thus is a specific form of the general [RSASSAVerify](#) function.

Return Values

The function may return any of the values that the general [RSASSAVerify](#) function returns.

Discrete-Logarithm-Based Cryptography Functions

This section introduces Intel® Integrated Performance Primitives (Intel® IPP) functions allowing for different operations with Discrete Logarithm (DL) based cryptosystem over a prime finite field $GF(p)$. The functions are mainly based on the [\[IEEE P1363A\]](#) standard. Implementation of the Digital Signature operations is based on [\[FIPS PUB 186-2\]](#). The Diffie-Hellman (DH) Agreement scheme is based on [\[X9.42\]](#).

The full list of Intel IPP DL-based cryptography functions is given in [Table 5-9](#).

Table 5-9 Intel IPP Discrete-Logarithm-Based Cryptography Functions

Function Base Name	Operation
DLPGetSize	Gets the size of the <code>IppsDLPState</code> context.
DLPInit	Initializes user-supplied memory as the <code>IppsDLPState</code> context for future use.
DLPSet	Sets up domain parameters of the DL-based cryptosystem over $GF(p)$.
DLPGet	Retrieves domain parameters of the DL-based cryptosystem over $GF(p)$.
DLPSetDP	Sets up a particular domain parameter of the DL-based cryptosystem over $GF(p)$.
DLPGetDP	Retrieves a particular domain parameter of the DL-based cryptosystem over $GF(p)$.
DLPGenKeyPair	Generates a private key and computes public keys of the DL-based cryptosystem over $GF(p)$.
DLPPublicKey	Computes a public key from the given private key of the DL-based cryptosystem over $GF(p)$.

Table 5-9 Intel IPP Discrete-Logarithm-Based Cryptography Functions (continued)

Function Base Name	Operation
<u>DLPValidateKeyPair</u>	Validates private and public keys of the DL-based cryptosystem over $GF(p)$.
<u>DLPSetKeyPair</u>	Sets private and/or public keys of the DL-based cryptosystem over $GF(p)$.
<u>DLPGenerateDSA</u>	Generates domain parameters of the DL-based cryptosystem over $GF(p)$ to use DSA.
<u>DLPValidateDSA</u>	Validates domain parameters of the DL-based cryptosystem over $GF(p)$ to use DSA.
<u>DLPSignDSA</u>	Performs the DSA digital signature signing operation.
<u>DLPVerifyDSA</u>	Verifies the input DSA digital signature.
<u>DLPGenerateDH</u>	Generates domain parameters of the DL-based cryptosystem over $GF(p)$ to use the DH Agreement scheme.
<u>DLPValidateDH</u>	Validates domain parameters of the DL-based cryptosystem over $GF(p)$ to use the DH Agreement scheme.
<u>DLPSharedSecretDH</u>	Computes a shares secret field element by using the Diffie-Hellman scheme.

All functions described in this section employ the `IppsDLPState` context as operational vehicle that carries domain parameters of the DL cryptosystem, a pair of keys, and working buffers.

The application code intended for executing typical operations should perform the following sequence of operations:

1. Call the function [DLPGetSize](#) to get the size required to configure the `IppsDLPState` context.
2. Ensure that the required memory space is properly allocated. With the allocated memory, call the [DLPInit](#) function to initialize the context of the DL-based cryptosystem.
3. Set domain parameters of the DL-based cryptosystem by calling the [DLPSet](#) function, or generate domain parameters by calling the [DLPGenerateDSA](#) or [DLPGenerateDH](#).
4. Call one of the functions [DLPSignDSA](#), [DLPVerifyDSA](#), and [DLPSharedSecretDH](#) to compute digital signature, to verify authenticity of the digital signature, and to compute the shared element accordingly.
5. Free the memory allocated for the `IppsDLPState` context by calling the operating system memory free service function unless the context is no longer needed.

Free the memory allocated for the `IppsPRNGState` context by calling the operating system memory free service function.

DLPGetSize

Gets the size of the `IppsDLPState` context.

Syntax

```
IppsStatus ippsDLPGetSize(int peBits, int reBits, int *pSize);
```

Parameters

<i>peBits</i>	Bitsize of the $GF(p)$ element (that is, the length of the DL-based cryptosystem in bits)
<i>reBits</i>	Bitsize of the multiplicative subgroup $GF(r)$.
<i>pSize</i>	Pointer to the <code>IppsDLPState</code> context size in bytes.

Description

This function is declared in the `ippcp.h` file. The function gets the `IppsDLPState` context size in bytes and stores in **pSize*. DL-based cryptosystem over $GF(p)$ assumes that $r/p-1$ where both p and r are primes.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsSizeErr</code>	Indicates an error condition if $peBits \leq reBits$.

DLPInit

Initializes user-supplied memory as the `IppsDLPState` context for future use.

Syntax

```
IppsStatus IppsDLPInit(int peBits, int reBits, IppsDLPState* pCtx);
```

Parameters

<i>peBits</i>	Bitsize of the $GF(p)$ element (that is, the length of the DL-based cryptosystem in bits)
---------------	---

<i>reBits</i>	Bitsize of the multiplicative subgroup $GF(r)$.
<i>pCtx</i>	Pointer to the <code>IppsDLPState</code> context being initialized.

Description

This function is declared in the `ippcp.h` file. The function initializes the memory pointed by *pCtx* as the `IppsDLPState` context.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsSizeErr</code>	Indicates an error condition if $peBits \leq reBits$.

DLPSet

Sets up domain parameters of the DL-based cryptosystem over $GF(p)$.

Syntax

```
IppStatus ippDLPSet(const IppsBigNumState* pP, const IppBigNumState*
    pQ, const IppsBigNumState* pG, IppsDLPState* pCtx);
```

Parameters

<i>pP</i>	Pointer to the characteristic p of the prime finite field $GF(p)$.
<i>pQ</i>	Pointer to the characteristic q of the multiplicative subgroup $GF(q)$.
<i>pG</i>	Pointer to the generator G of the multiplicative subgroup $GF(r)$.
<i>pCtx</i>	Pointer to the cryptosystem context.

Description

This function is declared in the `ippcp.h` file. The function sets up DL-based cryptosystem domain parameters into the cryptosystem context.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
--------------------------	--

<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsRangeErr</code>	Indicates an error condition if any of the Big Numbers specified by pP , pR , and pG is too big to be stored in the <code>IppsDLPState</code> context.

DLPGet

Retrieves domain parameters of the DL-based cryptosystem over $GF(p)$.

Syntax

```
IppStatus ippDLPGet(IppsBigNumState* pP, IppsBigNumState* pQ,  
    IppsBigNumState* pG, ippDLPState* pCtx);
```

Parameters

pP	Pointer to the characteristic p of the prime finite field $GF(p)$.
pQ	Pointer to the characteristic q of the multiplicative subgroup $GF(q)$.
pG	Pointer to the generator G of the multiplicative subgroup $GF(r)$.
$pCtx$	Pointer to the cryptosystem context.

Description

This function is declared in the `ippcp.h` file. The function retrieves DL-based cryptosystem domain parameters into the cryptosystem context.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsIncompleteContextErr</code>	Indicates an error condition if the cryptosystem context has not been properly set up.

`ippStsRangeErr`

Indicates an error condition if any of the Big Numbers specified by pP , pR , and pG is too small for the DL parameter.

DLPSetDP

Sets up a particular domain parameter of the DL-based cryptosystem over $GF(p)$.

Syntax

```
IppStatus ippDLPSetDP(const IppsBigNumState* pDP, IppsDLPKeyTag tag,
    IppsDLPState* pCtx);
```

Parameters

<code>pDP</code>	Pointer to the domain parameter value to be set.
<code>tag</code>	Tag specifying the desired domain parameter.
<code>pCtx</code>	Pointer to the cryptosystem context.

Description

This function is declared in the `ippcp.h` file. The function assigns the value specified by `pDP` to a particular domain parameter of the DL-based cryptosystem. The domain parameter to be set up is determined by `tag` as follows:

- If `tag == IppDLPkeyP`, the function assigns value to the characteristic p , the size of the prime finite field $GF(p)$.
- If `tag == IppDLPkeyR`, the function assigns value to the characteristic r , the prime divisor of $(p-1)$ and the order of g .
- If `tag == IppDLPkeyG`, the function assigns value to the characteristic g , the element of $GF(p)$ generating a multiplicative subgroup of order r .

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

<code>ippStsRangeErr</code>	Indicates an error condition if the Big Number specified by <code>pDP</code> is too big to be stored in the <code>IppsDLPState</code> context.
<code>ippStsBadArgErr</code>	Indicates an error condition if some of the function parameters are invalid: • Big Number specified by <code>pDP</code> is negative • Domain parameter specified by <code>tag</code> does not match the <code>IppsDLPState</code> context.

DLPGetDP

Retrieves a particular domain parameter of the DL-based cryptosystem over $GF(p)$.

Syntax

```
IppStatus ippDLGetDP(const IppsBigNumState* pDP, IppsDLPKeyTag tag,
                    IppsDLPState* pCtx);
```

Parameters

<code>pDP</code>	Pointer to the output Big Number context.
<code>tag</code>	Tag specifying the domain parameter to be retrieved.
<code>pCtx</code>	Pointer to the cryptosystem context.

Description

This function is declared in the `ippcp.h` file. The function retrieves value of a particular domain parameter of the DL-based cryptosystem from the `IppsDLPState` context and stores the value in the Big Number context `*pDP`. The domain parameter to be retrieved is determined by `tag` as follows:

- If `tag == IppDLPkeyP`, the function retrieves value of the characteristic p , the size of the prime finite field $GF(p)$.
- If `tag == IppDLPkeyR`, the function retrieves value of the characteristic r , the prime divisor of $(p-1)$ and the order of g .
- If `tag == IppDLPkeyG`, the function retrieves value of the characteristic g , the element of $GF(p)$ generating a multiplicative subgroup of order r .

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsIncompleteContextErr</code>	Indicates an error condition if the cryptosystem context has not been properly set up.
<code>ippStsOutOfRangeErr</code>	Indicates an error condition if the Big Number specified by <i>pDP</i> is too small for the DL parameter.
<code>ippStsBadArgErr</code>	Indicates an error condition if the domain parameter specified by the tag does not match the <code>IppsDLPState</code> context.

DLPGenKeyPair

Generates a private key and computes public keys of the DL-based cryptosystem over $GF(p)$.

Syntax

```
IppStatus ippSDLPGenKeyPair(ippBigNumState* pPrivate, ippBigNumState*
    pPublic, IppsDLPState* pCtx, IppBitSupplier rndFunc, void* pRndParam);
```

Parameters

<i>pPrivate</i>	Pointer to the private key <i>privKey</i> .
<i>pPublic</i>	Pointer to the public key <i>pubKey</i> .
<i>pCtx</i>	Pointer to the cryptosystem context.
<i>rndFunc</i>	Specified Random Generator.
<i>pRndParam</i>	Pointer to the Random Generator context.

Description

This function is declared in the `ippcp.h` file. The function generates a private key *privKey* and computes a public key *pubKey* of the DL-based cryptosystem. The function employs specified *rndFunc* Random Generator to generate a pseudorandom private key. The value of the private key *privKey* is a random number that lies in the range of $[2, R-2]$.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsIncompleteContextErr</code>	Indicates an error condition if the cryptosystem context has not been properly set up.
<code>ippStsRangeErr</code>	Indicates an error condition if any of the Big Numbers specified by <i>pPrivate</i> and <i>pPublic</i> is too small for the DL key.

DLPPublicKey

Computes a public key from the given private key of the DL-based cryptosystem over $GF(p)$.

Syntax

```
IppStatus ippDLPPublicKey(const IppsBigNumState* pPrivate,  
                          IppsBigNumState* pPublic, IppsDLPState* pCtx);
```

Parameters

<i>pPrivate</i>	Pointer to the input private key <i>privKey</i> .
<i>pPublic</i>	Pointer to the output public key <i>pubKey</i> .
<i>pCtx</i>	Pointer to the cryptosystem context.

Description

This function is declared in the `ippcp.h` file. The function computes a public key *pubKey* of the DL-based cryptosystem.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsIncompleteContextErr</code>	Indicates an error condition if the cryptosystem context has not been properly set up.
<code>ippStsInvalidPrivateKey</code>	Indicates an error condition if the <i>privKey</i> has an illegal value.
<code>ippStsRangeErr</code>	Indicates an error condition if Big Number specified by <i>pPublic</i> is too small for the DL public key.

DLPValidateKeyPair

Validates private and public keys of the DL-based cryptosystem over GF(p).

Syntax

```
IppStatus ippDLValidateKeyPair(const IppsBigNumState* pPrivate, const
    IppsBigNumState* pPublic, IppDLResult* pResult, IppsDLPState* pCtx);
```

Parameters

<i>pPrivate</i>	Pointer to the input private key <i>privKey</i> .
<i>pPublic</i>	Pointer to the output public key <i>pubKey</i> .
<i>pResult</i>	Pointer to the validation result.
<i>pCtx</i>	Pointer to the cryptosystem context.

Description

This function is declared in the `ippcp.h` file. The function validates the private key *privKey* and the public key *pubKey* of the DL-based cryptosystem. The result of the validation is stored in the **pResult* and may be assigned to one of the enumerators listed below:

<code>ippDLValid</code>	Validation has passed successfully.
<code>ippDLInvalidPrivateKey</code>	$1 < private < (R-1)$ is false.
<code>ippDLInvalidPublicKey</code>	$(1 < public \leq (P-1))$ is false.
<code>ippDLInvalidKeyPair</code>	$public \neq G^{private} \pmod{P}$.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsIncompleteContextErr</code>	Indicates an error condition if the cryptosystem context has not been properly set up.

DLPSetKeyPair

Sets private and/or public keys of the DL-based cryptosystem over $GF(p)$.

Syntax

```
IppStatus ippDLPSetKeyPair(const IppsBigNumState* pPrivate, const
    IppsBigNumState* pPublic, IppsDLPState* pCtx);
```

Parameters

<i>pPrivate</i>	Pointer to the input private key <i>privKey</i> .
<i>pPublic</i>	Pointer to the output public key <i>pubKey</i> .
<i>pCtx</i>	Pointer to the cryptosystem context.

Description

This function is declared in the `ippcp.h` file. The function stores the private key *privKey* and public key *pubKey* in the cryptosystem context.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

<code>ippStsIncompleteContextErr</code>	Indicates an error condition if the cryptosystem context has not been properly set up.
---	--

DLPGenerateDSA

Generates domain parameters of the DL-based cryptosystem over $GF(p)$ to use DSA.

Syntax

```
ippStatus ippDLPGenerateDSA(const IppsBigNumState* pSeedIn, int nTrials,
    IppsDLPState* pCtx, IppsBigNumState* pSeedOut, int* pCounter, IppBitSupplier
    rndFunc, void* pRndParam);
```

Parameters

<code>pSeedIn</code>	Pointer to the input <i>Seed</i> .
<code>nTrials</code>	Security parameter specified for the Miller-Rabin probable primality.
<code>pCtx</code>	Pointer to the cryptosystem context.
<code>pSeedOut</code>	Pointer to the output <i>Seed</i> value (if requested).
<code>pCounter</code>	Pointer to the <i>counter</i> value (if requested).
<code>rndFunc</code>	Specified Random Generator.
<code>pRndParam</code>	Pointer to the Random Generator context.

Description

This function is declared in the `ippcp.h` file. The function generates domain parameters of the DL-based cryptosystem over $GF(p)$ to use DSA. The function uses a procedure specified in [\[FIPS PUB 186-2\]](#) for generating both a 160-bit randomized prime r and a *LpeBits* prime p based on the input `*pSeedIn`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.

<code>ippStsSizeErr</code>	Indicates an error condition if: <i>peBits</i> < 512, <i>peBits</i> is not divided by 64, <i>reBits</i> != 160.
<code>ippStsRangeErr</code>	Indicates an error condition if: bitsize of the input <i>Seed</i> value is less than 160, bitsize of the input <i>Seed</i> value is greater than <i>peBits</i> , not enough space to store the output <i>Seed</i> value (if requested).
<code>ippStsBadArgErr</code>	Indicates an error condition if <i>nTrials</i> < 1.
<code>ippStsInsuffucientEntropy</code>	Indicates a warning condition if prime generation fails due to a poor choice of the entropy.

DLPValidateDSA

Validates domain parameters of the DL-based cryptosystem over GF(p) to use DSA.

Syntax

```
IppStatus ippDLValidateDSA(int nTrials, IppDLResult* pResult,  
    IppsDLPState* pCtx, IppBitSupplier rndFunc, void* pRndParam);
```

Parameters

<i>nTrials</i>	Security parameter specified for the Miller-Rabin probable primality.
<i>pResult</i>	Pointer to the validation result.
<i>pCtx</i>	Pointer to the cryptosystem context.
<i>rndFunc</i>	Specified Random Generator.
<i>pRndParam</i>	Pointer to the Random Generator context.

Description

This function is declared in the `ippcp.h` file. The function validates domain parameters of the DL-based cryptosystem over GF(*p*) to use DSA. The result of validation is stored in the **pResult* and may be assigned to one of the enumerators listed below:

<code>ippDLValid</code>	Validation has passed successfully.
<code>ippDLBaseIsEven</code>	<i>P</i> is even.

<code>ippDLOrderIsEven</code>	R is even.
<code>ippDLInvalidBaseRange</code>	$P \leq 2^{peBits-1}$ or $P \geq 2^{peBits}$.
<code>ippDLInvalidOrderRange</code>	$R \leq 2^{reBits-1}$ or $R \geq 2^{reBits}$.
<code>ippDLCompositeBase</code>	P is not a prime.
<code>ippDLCompositeOrder</code>	R is not a prime.
<code>ippDLInvalidCofactor</code>	R is not divided by $(P-1)$.
<code>ippDLInvalidGenerator</code>	$1 < G < (P-1)$ is false or $G^R \neq 1 \pmod{P}$.

To ensure that both p and r are primes, the function applies $nTrial$ -round Miller-Rabin primality test. Test data for primality test is provided by the specified `rndFunc` Random Generator.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsIncompleteContextErr</code>	Indicates an error condition if the cryptosystem context has not been properly set up.
<code>ippStsBadArgErr</code>	Indicates an error condition if $nTrials < 1$.

DLPSignDSA

Performs the DSA digital signature signing operation.

Syntax

```
IppsStatus ippDLPSignDSA(const IppsBigNumState* pMsg, const
    IppsBigNumState* pPrivate, IppsBigNumState* pSignR, IppsBigNumState*
    pSignS, IppsDLPState* pCtx);
```

Parameters

<code>pMsg</code>	Pointer to the message representation <i>msgRep</i> to be signed.
<code>pPrivate</code>	Pointer to the signer's private key <i>privKey</i> .

<i>pSignR</i>	Pointer to the <i>r</i> -component of the signature.
<i>pSignS</i>	Pointer to the <i>s</i> -component of the signature.
<i>pCtx</i>	Pointer to the cryptosystem context.

Description

This function is declared in the `ippcp.h` file. The function performs the DSA digital signature signing operation provided that the ephemeral signer's key pair (both private and public) was previously computed (generated by [DLPGenKeyPair](#) or computed by [DLPPublicKey](#)) and then set up into the DLP context by the [DLPSetKeyPair](#) function.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsIncompleteContextErr</code>	Indicates an error condition if the cryptosystem context has not been properly set up.
<code>ippStsMessageErr</code>	Indicates an error condition if the value of <i>msgRep</i> is greater than the multiplicative subgroup characteristic (<i>q</i>).
<code>ippStsInvalidPrivateKey</code>	Indicates an error condition if an illegal value has been assigned to <i>privKey</i> .
<code>ippStsRangeErr</code>	Indicates an error condition if any of the signature components has not enough space.

DLPVerifyDSA

Verifies the input DSA digital signature.

Syntax

```
IppsStatus ippDLVerifyDSA(const IppsBigNumState* pMsg, const
    IppsBigNumState* pSignR, const IppsBigNumState* pSignS, IppDLResult*
    pResult, IppsDLPState* pCtx);
```

Parameters

<i>pMsg</i>	Pointer to the message representation <i>msgRep</i> .
<i>pSignR</i>	Pointer to the signature <i>r</i> -component to be verified.
<i>pSignS</i>	Pointer to the signature <i>s</i> -component to be verified.
<i>pResult</i>	Pointer to the result of the verification.
<i>pCtx</i>	Pointer to the cryptosystem context.

Description

This function is declared in the `ippcp.h` file. The function verifies the input DSA digital signature's components **pSignR* and **pSignS* with the supplied message representation *msgRep*. Signer's public key must be stored by the [DLPSetsKeyPair](#) function before the `DLPVerifyDSA` operation.

The function sets the **pResult* to `ippDLValid` if it validates the input DSA digital signature, or to `ippDLInvalidSignature` if the DSA digital signature verification fails.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsIncompleteContextErr</code>	Indicates an error condition if the cryptosystem context has not been properly set up.
<code>ippStsMessageErr</code>	Indicates an error condition if the value of <i>msgRep</i> is greater than the multiplicative subgroup characteristic (<i>q</i>).

The example below illustrates the use of functions `DLPSignDSA` and `DLPVerifyDSA`. The example uses the `BigNumber` class and functions creating some cryptographic contexts, whose source code can be found in Appendix B.

Example 5-9 The Use of DLPSignDSA and DLPVerifyDSA

```
//
// known domain parameters
//
static const int M = 512; // DSA system bitsize
static const int L = 160; // DSA order  bitsize

static
BigInteger P("0x8DF2A494492276AA3D25759BB06869CBEAC0D83AFB8D0CF7" \
             "CBB8324F0D7882E5D0762FC5B7210EAF2E9ADAC32AB7AAC" \
             "49693DFBF83724C2EC0736EE31C80291");

static
BigInteger Q("0xC773218C737EC8EE993B4F2DED30F48EDACE915F");

static
BigInteger G("0x626D027839EA0A13413163A55B4CB500299D5522956CEFCB" \
             "3BFF10F399CE2C2E71CB9DE5FA24BABF58E5B79521925C9C" \
             "C42E9F6F464B088CC572AF53E6D78802");

//
// known DSA regular key pair
//
static
BigInteger X("0x2070B3223DBA372FDE1C0FFC7B2E3B498B260614");
```

Example 5-9 The Use of DLPSignDSA and DLPVerifyDSA (continued)

```
static
BigNumber Y("0x19131871D75B1612A819F29D78D1B0D7346F7AA77BB62A85" \
            "9BFD6C5675DA9D212D3A36EF1672EF660B8C7C255CC0EC74" \
            "858FBA33F44C06699630A76B030EE333");

int DSAsign_verify_sample(void)
{
    // DLP context
    IppsDLPState *DLPState = newDLP(M, L);

    // set up DLP crypto system
    ippsDLPSet(P, Q, G, DLPState);

    // message
    Ipp8u message[] = "abc";

    // compute message digest to be signed
    Ipp8u md[SHA1_DIGEST_LENGTH/8];
    ippsSHA1MessageDigest(message, sizeof(message)-1, md);
    BigNumber digest(0, BITS_2_WORDS(SHA1_DIGEST_LENGTH));
    ippsSetOctString_BN(md, SHA1_DIGEST_LENGTH/8, digest);

    // generate ephemeral key pair (ephX,ephY)
    BigNumber ephX(0, BITS_2_WORDS(L));
    BigNumber ephY(0, BITS_2_WORDS(M));
```

Example 5-9 The Use of DLPSignDSA and DLPVerifyDSA (continued)

```
IppsPRNGState* pRand = newPRNG();
ippsDLPGenKeyPair(ephX, ephY, DLPState, ippsPRNGen, pRand);
deletePRNG(pRand);
//
// generate signature
//
BigNumber signR(0, BITS_2_WORDS(L));      // R and S signature's component
BigNumber signS(0, BITS_2_WORDS(L));
ippsDLPSetKeyPair(ephX, ephY, DLPState);    // set up ephemeral keys
ippsDLPSignDSA(digest, X,                  // sign digest
               signR, signS,
               DLPState);

//
// verify signature
//
ippsDLPSetKeyPair(0, Y, DLPState);          // set up regular public key
IppDLResult result;
ippsDLPVerifyDSA(digest, signR, signS,      // verify
                 &result, DLPState);

deleteDLP(DLPState);
return result==ippDLValid;
}
```

DLPGenerateDH

Generates domain parameters of the DL-based cryptosystem over $GF(p)$ to use the DH Agreement scheme.

Syntax

```
IppStatus IppsDLPGenerateDH(const IppsBigNumState* pSeedIn, int nTrials,
    IppsDLPState* pCtx, IppsBigNumState* pSeedOut, int* pCounter,
    IppBitSupplier rndFunc, void* pRndParam);
```

Parameters

<i>pSeedIn</i>	Pointer to the input <i>Seed</i> .
<i>nTrials</i>	Security parameter specified for the Miller-Rabin probable primality.
<i>pCtx</i>	Pointer to the cryptosystem context.
<i>pSeedOut</i>	Pointer to the output <i>Seed</i> value (if requested).
<i>pCounter</i>	Pointer to the <i>counter</i> value (if requested).
<i>rndFunc</i>	Specified Random Generator.
<i>pRndParam</i>	Pointer to the Random Generator context.

Description

This function is declared in the `ippcp.h` file. The function generates domain parameters of the DL-based cryptosystem over $GF(p)$ to use Diffie-Hellman Agreement scheme. The function uses a procedure specified in [\[X9.42\]](#) for generating both randomized prime p and r based on the input **pSeedIn*.

Generated primes r and p are further validated through a *nTrial*-round Miller-Rabin primality test. Both generation and primality test procedures employ specified *rndFunc* Random Generator.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsSizeErr</code>	Indicates an error condition if: <i>peBits</i> < 512 or <i>reBits</i> < 160, <i>peBits</i> is not divided by 256.
<code>ippStsRangeErr</code>	Indicates an error condition if: bitsize of the input <i>Seed</i> value is less than <i>reBits</i> , not enough space to store the output <i>Seed</i> value (if requested).
<code>ippStsBadArgErr</code>	Indicates an error condition if <i>nTrials</i> < 1.
<code>ippStsInsuffucientEntropy</code>	Indicates a warning condition if prime generation fails due to a poor choice of the entropy.

DLPValidateDH

Validates domain parameters of the DL-based cryptosystem over GF(p) to use the DH Agreement scheme.

Syntax

```
IppStatus ippSDLPValidateDH(int nTrials, IppDLResult* pResult,  
    IppDLPState* pCtx, IppBitSupplier rndFunc, void* pRndParam);
```

Parameters

<i>nTrials</i>	Security parameter specified for the Miller-Rabin probable primality.
<i>pResult</i>	Pointer to the validation result.
<i>pCtx</i>	Pointer to the cryptosystem context.
<i>rndFunc</i>	Specified Random Generator.
<i>pRndParam</i>	Pointer to the Random Generator context.

Description

This function is declared in the `ippcp.h` file. The function validates domain parameters of the DL-based cryptosystem over GF(*p*) to use Diffie-Hellman Agreement scheme. The result of validation is stored in the **pResult* and may be assigned to one of the enumerators listed below:

<code>ippDLValid</code>	Validation has passed successfully.
<code>ippDLBaseIsEven</code>	P is even.
<code>ippDLOrderIsEven</code>	R is even.
<code>ippDLInvalidBaseRange</code>	$P \leq 2^{peBits-1}$ or $P \geq 2^{peBits}$.
<code>ippDLInvalidOrderRange</code>	$R \leq 2^{reBits-1}$ or $R \geq 2^{reBits}$.
<code>ippDLCompositeBase</code>	P is not a prime.
<code>ippDLCompositeOrder</code>	R is not a prime.
<code>ippDLInvalidCofactor</code>	R is not divided by $(P-1)$.
<code>ippDLInvalidGenerator</code>	$1 < G < (P-1)$ is false or $G^R \neq 1 \pmod{P}$.

To ensure that both p and r are primes, the function applies $nTrial$ -round Miller-Rabin primality test. Test data for primality test is provided by the specified `rndFunc` Random Generator.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsIncompleteContextErr</code>	Indicates an error condition if the cryptosystem context has not been properly set up.
<code>ippStsBadArgErr</code>	Indicates an error condition if $nTrials < 1$.

DLPSecretDH

Computes a shared field element by using the Diffie-Hellman scheme.

Syntax

```
IssStatus ippDLPSecretDH(const IppsBigNumState* pPrivateA, const
    IppsBigNumState* pPublicB, IppsBigNumState* pShare, IppsDLPState* pCtx);
```

Parameters

`pPrivateA` Pointer to your own private key `privateKeyA`.

<i>pPublicB</i>	Pointer to the public key <i>pubKeyB</i> belonging to the other party.
<i>pShare</i>	Pointer to the shared secret element <i>Share</i> .
<i>pCtx</i>	Pointer to the cryptosystem context.

Description

This function is declared in the `ippcp.h` file. The function computes a shared secret element $\text{FG}(p) \text{ } pubKeyB^{privateKeyA} \pmod{p}$.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the context parameter does not match the operation.
<code>ippStsIncompleteContextErr</code>	Indicates an error condition if the cryptosystem context has not been properly set up.
<code>ippStsRangeErr</code>	Indicates an error condition if <i>Share</i> does not have enough space.

Elliptic Curve Cryptography Functions

Intel® Integrated Performance Primitives (Intel® IPP) for cryptography offer functions allowing for different operations with an elliptic curve defined over a prime finite field $\text{GF}(p)$ and binary finite field $\text{GF}(2^m)$. The functions are based on standards ([\[IEEE P1363A\]](#), [\[SEC1\]](#), and [\[ANSI\]](#). For more information on parameters recommended for the functions, see [\[SEC2\]](#).

The full list of Elliptic Curve Cryptography functions is given in [Table 5-10](#).

Table 5-10 Intel IPP Elliptic Curve Cryptography Functions

Function Base Name	Operation
Functions Operating over $\text{GF}(p)$	
ECCPGetSize	Gets the size of the <code>IppsECCPState</code> context.
ECCPInit	Initializes context for the elliptic curve cryptosystem over $\text{GF}(p)$.
ECCPSet	Sets up elliptic curve domain parameters over $\text{GF}(p)$.
ECCPSetStd	Sets up a recommended set of elliptic curve domain parameters over $\text{GF}(p)$.

Table 5-10 Intel IPP Elliptic Curve Cryptography Functions (continued)

Function Base Name	Operation
ECCPGet	Retrieves elliptic curve domain parameters over $GF(p)$.
ECCPGetOrderBitSize	Retrieves order size of the elliptic curve base point over $GF(p)$ in bits
ECCPValidate	Checks validity of the elliptic curve domain parameters over $GF(p)$.
ECCPPointGetSize	Gets the size of the <code>IppsECCPPoint</code> context in bytes for a point on the elliptic curve point defined over $GF(p)$.
ECCPPointInit	Initializes context for a point on the elliptic curve defined over $GF(p)$.
ECCPSetPoint	Sets coordinates of a point on the elliptic curve defined over $GF(p)$.
ECCPSetPointAtInfinity	Sets the point at infinity.
ECCPGetPoint	Retrieves coordinates of the point on the elliptic curve defined over $GF(p)$.
ECCPCheckPoint	Checks correctness of the point on the elliptic curve defined over $GF(p)$.
ECCPComparePoint	Compares two points on the elliptic curve defined over $GF(p)$.
ECCPNegativePoint	Finds an elliptic curve point which is an additive inverse for the given point over $GF(p)$.
ECCPAddPoint	Computes the addition of two elliptic curve points over $GF(p)$.
ECCPMulPointScalar	Performs scalar multiplication of a point on the elliptic curve defined over $GF(p)$.
ECCPGenKeyPair	Generates a private key and computes public keys of the elliptic cryptosystem over $GF(p)$.
ECCPPublicKey	Computes a public key from the given private key of the elliptic cryptosystem over $GF(p)$.
ECCPValidateKeyPair	Validates private and public keys of the elliptic cryptosystem over $GF(p)$.
ECCPSetKeyPair	Sets private and/or public keys of the elliptic cryptosystem over $GF(p)$.
ECCPSharedSecretDH	Computes a shared secret field element by using the Diffie-Hellman scheme.
ECCPSharedSecretDHC	Computes a shared secret field element by using the Diffie-Hellman scheme and the elliptic curve cofactor.
ECCPSignDSA	Computes a digital signature over a message digest.
ECCPVerifyDSA	Verifies authenticity of the digital signature over a message digest (ECDSA).
ECCPSignNR	Computes the digital signature over a message digest (the Nyberg-Rueppel scheme).
ECCPVerifyNR	Verifies authenticity of the digital signature over a message digest (the Nyberg-Rueppel scheme).
Functions Operating over $GF(2^m)$	
ECCBGetSize	Gets the size of the <code>IppsECCBState</code> context.

Table 5-10 Intel IPP Elliptic Curve Cryptography Functions (continued)

Function Base Name	Operation
<u>ECCBInit</u>	Initializes context for the elliptic curve cryptosystem over $GF(2^m)$.
<u>ECCBSet</u>	Sets up elliptic curve domain parameters over $GF(2^m)$.
<u>ECCBSetStd</u>	Sets up a recommended set of elliptic curve domain parameters over $GF(2^m)$.
<u>ECCBGet</u>	Retrieves elliptic curve domain parameters over $GF(2^m)$.
<u>ECCBGetOrderBitSize</u>	Retrieves order size of the elliptic curve base point over $GF(2^m)$ in bits.
<u>ECCBValidate</u>	Checks validity of the elliptic curve domain parameters over $GF(2^m)$.
<u>ECCBPointGetSize</u>	Gets the size of the IppsECCBPoint context in bytes for a point on the elliptic curve point defined over $GF(2^m)$.
<u>ECCBPointInit</u>	Initializes context for a point on the elliptic curve defined over $GF(2^m)$.
<u>ECCBSetPoint</u>	Sets coordinates of a point on the elliptic curve defined over $GF(2^m)$.
<u>ECCBSetPointAtInfinity</u>	Sets the point at infinity.
<u>ECCBGetPoint</u>	Retrieves coordinates of the point on the elliptic curve defined over $GF(2^m)$.
<u>ECCBCheckPoint</u>	Checks correctness of the point on the elliptic curve defined over $GF(2^m)$.
<u>ECCBComparePoint</u>	Compares two points on the elliptic curve defined over $GF(2^m)$.
<u>ECCBNegativePoint</u>	Finds an elliptic curve point which is an additive inverse for the given point over $GF(2^m)$.
<u>ECCBAddPoint</u>	Computes the addition of two elliptic curve points over $GF(2^m)$.
<u>ECCBMulPointScalar</u>	Performs scalar multiplication of a point on the elliptic curve defined over $GF(2^m)$.
<u>ECCBGenKeyPair</u>	Generates a private key and computes public keys of the elliptic cryptosystem over $GF(2^m)$.
<u>ECCBPublicKey</u>	Computes a public key from the given private key of the elliptic cryptosystem over $GF(2^m)$.
<u>ECCBValidateKeyPair</u>	Validates private and secret keys of the elliptic cryptosystem over $GF(2^m)$.
<u>ECCBSetKeyPair</u>	Sets private and/or public keys in the elliptic cryptosystem over $GF(2^m)$.
<u>ECCBSharedSecretDH</u>	Computes a shared secret field element by using the Diffie-Hellman scheme.
<u>ECCBSharedSecretDHC</u>	Computes a shared secret field element by using the Diffie-Hellman scheme and the elliptic curve cofactor.
<u>ECCBSignDSA</u>	Computes a digital signature over a message digest.
<u>ECCBVerifyDSA</u>	Verifies authenticity of the digital signature over a message digest (ECDSA).
<u>ECCBSignNR</u>	Computes the digital signature over a message digest (the Nyberg-Rueppel scheme).

Table 5-10 Intel IPP Elliptic Curve Cryptography Functions (continued)

Function Base Name	Operation
ECCBVerifyNR	Computes authenticity of the digital signature over a message digest (the Nyberg-Rueppel scheme).

Public key cryptography successfully allows to solve problems of information security by enabling secure communication over insecure channels. Although elliptic curves are well studied as a branch of mathematics, an interest to the cryptographic schemes based on elliptic curves is constantly rising due to the advantages that the elliptic curve algorithms provide in the wireless communications: shorter processing time and key length.

Elliptic curve cryptosystems (ECCs) implement a different way of creating public keys. Because elliptic curve calculation is based on the addition of the rational points in the (x,y) plane and it is difficult to solve a discrete logarithm from these points, a higher level of security is achieved through the cryptographic schemes that use the elliptic curves. The cryptographic systems that encrypt messages by using the properties of elliptic curves are hard to attack due to the extreme complexity of deciphering the private key.

Use of elliptic curves allows for shorter public key length and encourage cryptographers to create cryptosystems with the same or higher encryption strength as the RSA or DSA cryptosystems. Because of the relatively short key length, ECCs do encryption and decryption faster on the hardware that requires less computation processing volumes. For example, with a key length of 150-350 bits, ECCs provide the same encryption strength as the cryptosystems who have to use 600 -1400 bits. Alternatively, it requires considerably less time to create a digital signature with the ECDSA (Elliptic Curve Digital Signature Algorithm) algorithm rather than with the RSA algorithm even though you are using the same processor machine.

Functions Based on GF(p)

- This section describes functions designed to specify the elliptic curve cryptosystem and perform various operations on the elliptic curve defined over a prime finite field. The examples of the operations are shown below:
- Setting up operations
[ECCPSet](#) sets up elliptic curve domain parameters. [ECCPSetKeyPair](#) sets a pair of public and private keys for the given cryptosystem.
 - Computation operations
[ECCPAddPoint](#) adds two points on the elliptic curve. [ECCPMulPointScalar](#) performs the scalar multiplication of a point on the elliptic curve. [ECCPSignDSA](#) computes the digital signature of a message.
 - Validation operations
[ECCPValidate](#) checks validity of the elliptic curve domain parameters. [ECCPValidateKeyPair](#) validates correctness of the public and private keys.

- Generation operations
[ECCPGenKeyPair](#) generates a private key and computes a public key for the given elliptic cryptosystem.
- Retrieval operations
[ECCPGet](#) retrieves elliptic curve domain parameters. [ECCPGetOrderBitSize](#) retrieves the size of a base point in bytes.

All functions described in this section employ a context `IppsECCPState` that catches several auxiliary components specifying operations performed on the elliptic curve or entire elliptic cryptosystem. ECCP stands for Elliptic Curve Cryptography Prime and means that all functions whose name include this abbreviation perform operations over a [prime finite field \$GF\(p\)\$](#) .

ECCPGetSize

Gets the size of the `IppsECCPState` context.

Syntax

```
IppStatus ippsECCPGetSize(int feBitSize, int *pSize);
```

Parameters

<i>feBitSize</i>	Size (in bits) of the field element.
<i>pSize</i>	Pointer to the size (in bytes) of the context.

Description

The function computes the size of the context in bytes for the elliptic cryptosystem over a prime finite field $GF(p)$.

Context is a structure `IppsECCPState` designed to store information about the cryptosystem status.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsSizeErr</code>	Indicates an error condition if the value of the parameter <i>feBitSize</i> is less than 2.

ECCPInit

Initializes context for the elliptic curve cryptosystem over $GF(p)$.

Syntax

```
IppStatus ippseCCPInit(int feBitSize, IppsECCPState* pECC);
```

Parameters

<i>feBitSize</i>	Size (in bits) of a field element.
<i>pECC</i>	Pointer to the cryptosystem context.

Description

The function initializes the context of the elliptic curve cryptosystem over the prime finite field $GF(p)$.

Context is a structure `IppsECCPState` designed to store information about the cryptosystem status.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NIULL</code> .
<code>ippStsSizeErr</code>	Indicates an error condition if the value of the parameter <i>feBitSize</i> is less than 2.

ECCPSet

Sets up elliptic curve domain parameters over $GF(p)$.

Syntax

```
IppStatus ippseCCPSet(const IppsBigNumState* pPrime, const
    IppsBigNumState* pA, const IppsBigNumState* pB, const
    IppsBigNumState* pGX, const IppsBigNumState* pGY, const
    IppsBigNumState* pOrder, int cofactor, IppsECCPState* pECC);
```

Parameters

<i>pPrime</i>	Pointer to the characteristic p of the prime finite field $\text{GF}(p)$.
<i>pA</i>	Pointer to the coefficient A of the equation defining the elliptic curve.
<i>pB</i>	Pointer to the coefficient B of the equation defining the elliptic curve.
<i>pGX</i>	Pointer to the x -coordinate of the elliptic curve base point.
<i>pGY</i>	Pointer to the y -coordinate of the elliptic curve base point.
<i>pOrder</i>	Pointer to the order of the elliptic curve base point.
<i>cofactor</i>	Cofactor.
<i>pECC</i>	Pointer to the context of the cryptosystem.

Description

The function sets up the elliptic curve domain parameters over a prime finite field $\text{GF}(p)$. These are as follows:

- *pPrime* sets up the characteristic p of a finite field $\text{GF}(p)$ where p is a prime number.
- *pA*, *pB* set up the coefficients A and B of the equation defining the elliptic curve:

$$y^2 = x^3 + A \cdot x + B \pmod{p} .$$
- *pGX*, *pGY* are pointers to the affine coordinates of the elliptic curve base point G .
- *pOrder* is a pointer to the order n of the elliptic curve base point G such that $n \cdot G = O$ where O is the point at infinity and n is a prime number.
- *cofactor* sets up the ratio h of a general number of points $\#E$ on the elliptic curve (including the point at infinity) to the order n of the base point :

$$h = \frac{\#E}{n} .$$

The domain parameters are set in the cryptosystem context which must be already created by the `ippsECCPGetSize` and `ippsECCPInit` functions.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPrime</i> , <i>pA</i> , <i>pB</i> , <i>pGX</i> , <i>pGY</i> , <i>pOrder</i> , and <i>pECC</i> is not valid.
<code>ippStsRangeErr</code>	Indicates an error condition if of one of the parameters pointed by <i>pPrime</i> , <i>pA</i> , <i>pB</i> , <i>pGX</i> , <i>pGY</i> , and <i>pOrder</i> cannot embed the <i>feBitSize</i> bits length or the value of <i>cofactor</i> is less than 1.

ECCPSetStd

Sets up a recommended set of elliptic curve domain parameters over GF(p).

Syntax

```
IppStatus ippseCCPSetStd(IppECCType flag, IppsECCPState* pECC);
```

Parameters

<i>flag</i>	Set specifier.
<i>pECC</i>	Pointer to the cryptosystem context.

Description

The function sets a recommended set of elliptic curve domain parameters over a prime finite field GF(*p*).

The set is defined by the value of the parameter *flag*. Possible values of the parameter are as follows:

<code>IppECCPStd112r1</code>	For the cryptosystem context where <i>feBitSize</i> ==112
<code>IppECCPStd112r2</code>	For the cryptosystem context where <i>feBitSize</i> ==112
<code>IppECCPStd128r1</code>	For the cryptosystem context where <i>feBitSize</i> ==128
<code>IppECCPStd128r2</code>	For the cryptosystem context where <i>feBitSize</i> ==128
<code>IppECCPStd160r1</code>	For the cryptosystem context where <i>feBitSize</i> ==160
<code>IppECCPStd160r2</code>	For the cryptosystem context where <i>feBitSize</i> ==160
<code>IppECCPStd192r1</code>	For the cryptosystem context where <i>feBitSize</i> ==192
<code>IppECCPStd224r1</code>	For the cryptosystem context where <i>feBitSize</i> ==224
<code>IppECCPStd256r1</code>	For the cryptosystem context where <i>feBitSize</i> ==256
<code>IppECCPStd384r1</code>	For the cryptosystem context where <i>feBitSize</i> ==384

IppsECCPStd521r1 For the cryptosystem context where *feBitSize*==521.

For more information on parameter values for the recommended elliptic curves, see [\[SEC2\]](#).

The cryptosystem context must be already created by the `ippsECCPGetSize` and `ippsECCPInit` functions. The value of *feBitSize* is applied when these functions are called and predetermines the possible choice of the *flag* value.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if the cryptosystem context is not valid.
<code>ippStsECCInvalidFlagErr</code>	Indicates an error condition if the value of the parameter <i>flag</i> is not valid.

ECCPGet

Retrieves elliptic curve domain parameters over GF(p).

Syntax

```
IppsStatus ippsECCPGet(IppsBigNumState* pPrime, IppsBigNumState* pA,  
    IppsBigNumState* pB, IppsBigNumState* pGX, IppsBigNumState* pGY,  
    IppsBigNumState* pOrder, int* cofactor, IppsECCPState* pECC);
```

Parameters

<i>pPrime</i>	Pointer to the characteristic <i>p</i> of the prime finite field GF(<i>p</i>).
<i>pA</i>	Pointer to the coefficient <i>A</i> of the equation defining the elliptic curve.
<i>pB</i>	Pointer to the coefficient <i>B</i> of the equation defining the elliptic curve.
<i>pGX</i>	Pointer to the <i>x</i> -coordinate of the elliptic curve base point.
<i>pGY</i>	Pointer to the <i>y</i> -coordinate of the elliptic curve base point.
<i>pOrder</i>	Pointer to the order <i>n</i> of the elliptic curve base point.

<i>cofactor</i>	Pointer to the cofactor <i>h</i> .
<i>pECC</i>	Pointer to the context of the cryptosystem.

Description

The function retrieves elliptic curve domain parameters from the context of the elliptic cryptosystem over a finite field GF(*p*) and allocates them in accordance with the pointers *pPrime*, *pA*, *pB*, *pGX*, *pGY*, *pOrder*, and *cofactor*. The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCPSet` or `ippsECCPSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPrime</i> , <i>pA</i> , <i>pB</i> , <i>pGX</i> , <i>pGY</i> , <i>pOrder</i> , or <i>pECC</i> is not valid.
<code>ippStsRangeErr</code>	Indicates an error condition if the memory size of one of the parameters pointed by <i>pPrime</i> , <i>pA</i> , <i>pB</i> , <i>pGX</i> , <i>pGY</i> , <i>pOrder</i> , and <i>pECC</i> is less than the value of <i>feBitSize</i> in the <code>ippsECCPInit</code> function.

ECCPGetOrderBitSize

*Retrieves order size of the elliptic curve base point over GF(*p*) in bits.*

Syntax

```
IppStatus ippsECCPGetOrderBitSize(int* pBitSize, IppsECCPState* pECC);
```

Parameters

<i>pBitSize</i>	Pointer to the size of the base point (in bits).
<i>pECC</i>	Pointer to the cryptosystem context.

Description

The function retrieves the order size (in bits) of the elliptic curve base point G from the context of elliptic cryptosystem over a prime finite field $GF(p)$ and allocates it in accordance with the pointer *pBitsSize*. The elliptic curve domain parameters must be hitherto defined by one of the functions: *ippsECCPSet* or *ippsECCPSetStd*.

Return Values

<i>ippStsNoErr</i>	Indicates no error. Any other value indicates an error or warning.
<i>ippStsNullPtrErr</i>	Indicates an error condition if any of the specified pointers is NULL.
<i>ippStsContextMatchErr</i>	Indicates an error condition if the cryptosystem context is not valid.

ECCPValidate

Checks validity of the elliptic curve domain parameters over $GF(p)$.

Syntax

```
IppStatus ippsECCPValidate(int nTrials, IppECResult* pResult,  
    IppsECCPState* pECC, IppBitSupplier rndFunc, void* pRndParam);
```

Parameters

<i>nTrials</i>	A number of attempts made to check the number for primality.
<i>pResult</i>	Pointer to the result received upon the check of the elliptic curve domain parameters.
<i>pECC</i>	Pointer to the cryptosystem context.
<i>rndFunc</i>	Specified Random Generator.
<i>pRndParam</i>	Pointer to Random Generator context.

Description

The function checks validity of the elliptic curve domain parameters over a prime finite field $GF(p)$ and stores the result of the check in accordance with the pointer *pResult*.

Elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCPSet` or `ippsECCPSetStd`. The purpose of the parameters `rndFunc`, `pRndParam`, and `nTrials` is analogous to that of the parameters `rndFunc`, `pRndParam`, and `nTrials` in the `ippsPrimeTest` function.

The result of the elliptic curve domain parameters check can take one of the following values:

<code>ippecValid</code>	The parameters are valid.
<code>ippecCompositeBase</code>	The prime finite field characteristic p is a composite number.
<code>ippecIsNotAG</code>	The solutions of the elliptic curve equation do not form the abelian group because the only requirement that $4 \cdot a^3 + 27 \cdot b^2 \neq 0 \pmod{p}$ is not met.
<code>ippecPointIsNotValid</code>	The base point G is not on the elliptic curve.
<code>ippecCompositeOrder</code>	The order n of the base point G is a composite number.
<code>ippecInvalidOrder</code>	The order n of the base point G is not valid because the requirement that $n \cdot G = O$ where O is the point at infinity is not met.
<code>ippecIsWeakSSSA</code>	The order n of the base point G is equal to the finite field characteristic p .
<code>ippecIsWeakMOV</code>	The curve is excluded because it is subject to the MOV reduction attack.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <code>c</code> or <code>pECC</code> is not valid.
<code>ippStsBadArgErr</code>	Indicates an error condition if the memory size of the parameter <code>seed</code> is less than five words (32 bytes in each) or the value of the parameter <code>nTrials</code> is less than 1.

ECCPPointGetSize

Gets the size of the `IppsECCPPoint` context in bytes for a point on the elliptic curve point defined over $GF(p)$.

Syntax

```
IppStatus ippsECCPPointGetSize(int feBitSize, int* pSize);
```

Parameters

<i>feBitSize</i>	Size (in bits) of the field element.
<i>pSize</i>	Pointer to the context size.

Description

The function computes the context size in bytes for a point on the elliptic curve defined over a prime finite field $GF(p)$.

Context is a structure `IppsECCPPoint` intended for storing the information about a point on the elliptic curve defined over $GF(p)$.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsSizeErr</code>	Indicates an error condition if the value of the parameter <i>feBitSize</i> is less than 2.

ECCPPointInit

Initializes the context for a point on the elliptic curve defined over $GF(p)$.

Syntax

```
IppStatus ippsECCPPointInit(int feBitSize, IppsECCPPoint* pPoint);
```

Parameters

<i>feBitSize</i>	Size (in bits) of the field element.
<i>pECC</i>	Pointer to the context of the elliptic curve point.

Description

The function initializes the context for a point on the elliptic curve defined over a finite field $GF(p)$.

Context is a structure `IppsECCPPoint` intended for storing the information about a point on the elliptic curve defined over $GF(p)$.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsSizeErr</code>	Indicates an error condition if the value of the parameter <i>feBitSize</i> is less than 2.

ECCPSetPoint

Sets coordinates of a point on the elliptic curve defined over $GF(p)$.

Syntax

```
IppStatus ippseccpSetPoint(const IppsBigNumState* pX, const
    IppsBigNumState* pY, IppsECCPPoint* pPoint, IppsECCPState* pECC);
```

Parameters

<i>pX</i>	Pointer to the x-coordinate of the point on the elliptic curve.
<i>pY</i>	Pointer to the y-coordinate of the point on the elliptic curve.
<i>pPoint</i>	Pointer to the context of the elliptic curve point.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function sets the coordinates of a point on the elliptic curve defined over a prime finite field $GF(p)$.

The context of the point on the elliptic curve must be already created by functions: `ippsECCPPointGetSize` and `ippsECCPPointInit`. The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCPSet` or `ippsECCPSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pX</i> , <i>pY</i> , <i>pPoint</i> , or <i>pECC</i> is not valid.

ECCPSetPointAtInfinity

Sets the point at infinity.

Syntax

```
IppsStatus ippsECCPSetPointAtInfinity(IppsECCPPoint* pPoint,  
                                       IppsECCPState* pECC);
```

Parameters

<i>pPoint</i>	Pointer to the context of the elliptic curve point.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function sets the point at infinity. The context of the elliptic curve point must be already created by functions: `ippsECCPPointGetSize` and `ippsECCPPointInit`. The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCPSet` or `ippsECCPSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPoint</i> or <i>pECC</i> is not valid.

ECCPGetPoint

Retrieves coordinates of the point on the elliptic curve defined over $GF(p)$.

Syntax

```
IppStatus ippseCCPGetPoint(IppsBigNumState* pX, IppsBigNumState* pY,
    const IppsECCPPoint* pPoint, IppsECCPState* pECC);
```

Parameters

<i>pX</i>	Pointer to the x-coordinate of the point on the elliptic curve.
<i>pY</i>	Pointer to the y-coordinate of the point on the elliptic curve.
<i>pPoint</i>	Pointer to the context of the elliptic curve point.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function retrieves the coordinates of the point on the elliptic curve defined over a prime finite field $GF(p)$ from the point context and allocates them in accordance with the set pointers *pX* and *pY*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: *ippseCCPSet* or *ippseCCPSetStd*.

Return Values

<i>ippStsNoErr</i>	Indicates no error. Any other value indicates an error or warning.
<i>ippStsNullPtrErr</i>	Indicates an error condition if any of the specified pointers is NULL.
<i>ippStsContextMatchErr</i>	Indicates an error condition if one of the contexts pointed by <i>pX</i> , <i>pY</i> , <i>pPoint</i> , or <i>pECC</i> is not valid.

ECCPCheckPoint

Checks correctness of the point on the elliptic curve defined over $GF(p)$.

Syntax

```
IppStatus ippseccpCheckPoint(const IppseccpPoint* pP, IppECResult*
    pResult, IppseccpState* pECC);
```

Parameters

<i>pP</i>	Pointer to the elliptic curve point.
<i>pResult</i>	Pointer to the result of the check.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function checks the correctness of the point on the elliptic curve defined over a prime finite field $GF(p)$ and allocates the result of the check in accordance with the pointer *pResult*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippseccpSet` or `ippseccpSetStd`.

The result of the check for the correctness of the point can take one of the following values:

<code>ippECValid</code>	Point is on the elliptic curve.
<code>ippECPointIsValid</code>	Point is not on the elliptic curve and is not the point at infinity.
<code>ippECPointIsAtInfinite</code>	Point is the point at infinity.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pP</i> or <i>pECC</i> is not valid.

ECCPComparePoint

Compares two points on the elliptic curve defined over $GF(p)$.

Syntax

```
IppStatus ippseCCPComparePoint(const IppsECCPPoint* pP, const
    IppsECCPPoint* pQ, IppECResult* pResult, IppsECCPState* pECC);
```

Parameters

<i>pP</i>	Pointer to the elliptic curve point <i>P</i> .
<i>pQ</i>	Pointer to the elliptic curve point <i>Q</i> .
<i>pResult</i>	Pointer to the comparison result of two points: <i>P</i> and <i>Q</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function compares two points *P* and *Q* on the elliptic curve defined over a prime finite field $GF(p)$ and allocates the comparison result in accordance with the pointer *pResult*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: *ippseCCPSet* or *ippseCCPSetStd*.

The comparison result of two points *P* and *Q* can take one of the following values:

<i>ippECPointIsEqual</i>	Points <i>P</i> and <i>Q</i> are equal.
<i>ippECPointIsNotEqual</i>	Points <i>P</i> and <i>Q</i> are different.

Return Values

<i>ippStsNoErr</i>	Indicates no error. Any other value indicates an error or warning.
<i>ippStsNullPtrErr</i>	Indicates an error condition if any of the specified pointers is NULL.
<i>ippStsContextMatchErr</i>	Indicates an error condition if one of the contexts pointed by <i>pP</i> or <i>pECC</i> is not valid.

ECCPNegativePoint

Finds an elliptic curve point which is an additive inverse for the given point over $GF(p)$.

Syntax

```
IppStatus ippseCCPNegativePoint(const IppsECCPPoint* pP, IppsECCPPoint*  
    pR, IppsECCPState* pECC);
```

Parameters

<i>pP</i>	Pointer to the elliptic curve point P .
<i>pR</i>	Pointer to the elliptic curve point R .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function finds an elliptic curve point R over a prime finite field $GF(p)$, which is an additive inverse of the given point P , that is, $P = -P$.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippseCCPSet` or `ippseCCPSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pP</i> , <i>pR</i> , or <i>pECC</i> is not valid.

ECCPAddPoint

Computes the addition of two elliptic curve points over $GF(p)$.

Syntax

```
IppStatus ippseCCPAddPoint(const IppseCCPPoint* pP, const IppseCCPPoint*
    pQ, IppseCCPPoint* pR, IppseCCPState* pECC);
```

Parameters

<i>pP</i>	Pointer to the elliptic curve point <i>P</i> .
<i>pQ</i>	Pointer to the elliptic curve point <i>Q</i> .
<i>pR</i>	Pointer to the elliptic curve point <i>R</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function calculates the addition of two elliptic curve points *P* and *Q* over a finite field $GF(p)$ with the result in a point *R* such that $R = P + Q$.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippseCCPSet` or `ippseCCPSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pP</i> , <i>pQ</i> , <i>pR</i> , or <i>pECC</i> is not valid.

ECCPMulPointScalar

Performs scalar multiplication of a point on the elliptic curve defined over $GF(p)$.

Syntax

```
IppStatus ippseCCPMulPointScalar(const IppsECCPPoint* pP, const  
    IppsBigNumState* pK, IppsECCPPoint* pR, IppsECCPState* pECC);
```

Parameters

<i>pP</i>	Pointer to the elliptic curve point <i>P</i> .
<i>pK</i>	Pointer to the scalar <i>K</i> .
<i>pR</i>	Pointer to the elliptic curve point <i>R</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function performs the *K* scalar multiplication of an elliptic curve point *P* over $GF(p)$ with the result in a point *R* such that $R = K \cdot P$.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippseCCPSet` or `ippseCCPSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pP</i> , <i>pK</i> , <i>pR</i> , or <i>pECC</i> is not valid.

ECCPGenKeyPair

Generates a private key and computes public keys of the elliptic cryptosystem over $GF(p)$.

Syntax

```
IppStatus ippsECCPGenKeyPair(IppsBigNumState* pPrivate,
    IppsECCPointState* pPublic, IppsECCPState* pECC, IppBitSupplier
    rndFunc, void* pRndParam);
```

Parameters

<i>pPrivate</i>	Pointer to the private key <i>privKey</i> .
<i>pPublic</i>	Pointer to the public key <i>pubKey</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.
<i>rndFunc</i>	Specified Random Generator.
<i>pRndParam</i>	Pointer to the Random Generator context.

Description

The function generates a private key *privKey* and computes a public key *pubKey* of the elliptic cryptosystem over a finite field $GF(p)$. The generation process employs the user specified *rndFunc* Random Generator.

The private key *privKey* is a number that lies in the range of $[1, n-1]$ where n is the order of the elliptic curve base point.

The public key *pubKey* is an elliptic curve point such that $pubKey = privKey \cdot G$, where G is the base point of the elliptic curve.

The memory size of the parameter *privKey* pointed by *pPrivate* must be less than that of the base point which can also be defined by the function `ippsECCPGetOrderBitSize`.

The context of the point *pubKey* as an elliptic curve point must be created by using the functions `ippsECCPPointGetSize` and `ippsECCPPointInit`.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCPSet` or `ippsECCPSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
--------------------------	--

<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPrivate</i> , <i>pPublic</i> , or <i>pECC</i> is not valid.
<code>ippStsSizeErr</code>	Indicates an error condition if the memory size of the parameter <i>privKey</i> pointed by <i>pPrivate</i> is less than that of the order of the elliptic curve base point.

ECCPPublicKey

Computes a public key from the given private key of the elliptic cryptosystem over GF(p).

Syntax

```
IppStatus ippseccppublickey(const IppsBigNumState* pPrivate,  
    IppsECCPoint* pPublic, IppsECCPState* pECC);
```

Parameters

<i>pPrivate</i>	Pointer to the private key <i>privKey</i> .
<i>pPublic</i>	Pointer to the public key <i>pubKey</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function computes the public key *pubKey* from the given private key *privKey* of the elliptic cryptosystem over a finite field GF(p).

The private key *privKey* is a number that lies in the range of $[1, n-1]$ where n is the order of the elliptic curve base point. The public key *pubKey* is an elliptic curve point such that $pubKey = privKey \cdot G$, where G is the base point of the elliptic curve.

The context of the point *pubKey* as an elliptic curve point must be created by using the functions `ippseccppointgetsize` and `ippseccppointinit`.

The elliptic curve domain parameters must be defined by one of the functions: `ippseccpset` or `ippseccpsetstd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
--------------------------	--

<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPrivate</i> , <i>pPublic</i> , or <i>pECC</i> is not valid.
<code>ippStsInvalidPrivateKey</code>	Indicates an error condition if the value of the private key falls outside the range of $[1, n-1]$.

ECCPValidateKeyPair

Validates private and public keys of the elliptic cryptosystem over $GF(p)$.

Syntax

```
IppStatus ippseCCPValidateKeyPair(const IppsBigNumState* pPrivate, const
    IppsECCPPoint* pPublic, IppeCResult* pResult, IppsECCPState* pECC);
```

Parameters

<i>pPrivate</i>	Pointer to the private key <i>privKey</i> .
<i>pPublic</i>	Pointer to the public key <i>pubKey</i> .
<i>pResult</i>	Pointer to the validation result.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function validates the private key *privKey* and public key *pubKey* of the elliptic cryptosystem over a finite field $GF(p)$ and allocates the result of the validation in accordance with the pointer *pResult*.

The private key *privKey* is a number that lies in the range of $[1, n-1]$, where n is the order of the elliptic curve base point. The public key *pubKey* is an elliptic curve point such that $pubKey = privKey \cdot G$ where G is the base point of the elliptic curve.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippseCCPSet` or `ippseCCPSetStd`.

The result of the cryptosystem keys validation for correctness can take one of the following values:

<code>ippeCValid</code>	Keys are valid.
<code>ippeCInvalidKeyPair</code>	Keys are not valid because $privKey \cdot G \neq pubKey$

<code>ippECInvalidPrivateKey</code>	Key <i>privKey</i> falls outside the range of $[1, n-1]$.
<code>ippECPPointIsAtInfinite</code>	Key <i>pubKey</i> is the point at infinity.
<code>ippECInvalidPublicKey</code>	Key <i>pubKey</i> is not valid because $n \cdot \text{pubKey} \neq O$, where O is the point at infinity.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPrivate</i> , <i>pPublic</i> , or <i>pECC</i> is not valid.

ECCPSetKeyPair

Sets private and/or public keys of the elliptic cryptosystem over $GF(p)$.

Syntax

```
IppStatus ippseccpSetKeyPair(const IppsBigNumState* pPrivate, const  
    IppsECCPPoint* pPublic, IppBool regular, IppsECCPState* pECC);
```

Parameters

<i>pPrivate</i>	Pointer to the private key <i>privKey</i> .
<i>pPublic</i>	Pointer to the public key <i>pubKey</i> .
<i>regular</i>	Key status flag.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function sets a private key *privKey* and/or public key *pubKey* in the elliptic cryptosystem defined over a prime finite field $GF(p)$.

The private key *privKey* is a number that lies in the range of $[1, n-1]$, where n is the order of the elliptic curve base point. The public key *pubKey* is an elliptic curve point such that $\text{pubKey} = \text{privKey} \cdot G$, where G is the base point of the elliptic curve.

The two possible values of the parameter *regular* define the key timeliness status:

ippTrue

Keys are regular.

ippFalse

Keys are ephemeral.

The elliptic curve domain parameters must be hitherto defined by one of the functions: ippSECCPSet or ippSECCPSetStd.

Return Values

ippStsNoErr

Indicates no error. Any other value indicates an error or warning.

ippStsNullPtrErr

Indicates an error condition if any of the specified pointers is NULL.

ippStsContextMatchErr

Indicates an error condition if one of the contexts pointed by *pPrivate*, *pPublic*, or *pECC* is not valid.

ECCPSharedSecretDH

Computes a shared secret field element by using the Diffie-Hellman scheme.

Syntax

```
IppStatus ippSECCPSharedSecretDH(const IppsBigNumState* pPrivate, const
    IppsECCPPointState* pPublic, IppsBigNumState* pShare,
    IppsECCPState* pECC);
```

Parameters

pPrivate

Pointer to your own public key *pubKey*.

pPublic

Pointer to the public key *pubKey*.

pShare

Pointer to the secret number *bnShare*.

pECC

Pointer to the context of the elliptic cryptosystem.

Description

The function computes a secret number *bnShare*, which is a secret key shared between two participants of the cryptosystem.

In cryptography, metasyntactic names such as Alice as Bob are normally used as examples and in discussions and stand for participant A and participant B.

Both participants (Alice and Bob) use the cryptosystem for receiving a common secret point on the elliptic curve called a secret key. To receive a secret key, participants apply the Diffie-Hellman key-agreement scheme involving public key exchange. The value of the secret key entirely depends on participants.

According to the scheme, Alice and Bob perform the following operations:

1. Alice calculates her own public key *pubKeyA* by using her private key *privKeyA*:

$$pubKeyA = privKeyA \cdot G$$
where *G* is a base point of the elliptic curve. Alice passes the public key to Bob.
2. Bob calculates his own public key *pubKeyB* by using his private key *privKeyB*:

$$pubKeyB = privKeyB \cdot G$$
where *G* is a base point of the elliptic curve. Bob passes the public key to Alice.
3. Alice gets Bob's public key and calculates the secret point *shareA*. When calculating, she uses her own private key and Bob's public key and applies the following formula:

$$shareA = privKeyA \cdot pubKeyB = privKeyA \cdot privKeyB \cdot G$$
4. Bob gets Alice's public key and calculates the secret point *shareB*. When calculating, he uses his own private key and Alice's public key and applies the following formula:

$$shareB = privKeyB \cdot pubKeyA = privKeyB \cdot privKeyA \cdot G$$

Because the following equation is true

$privKeyA \cdot privKeyB \cdot G = privKeyB \cdot privKeyA \cdot G$, the result of both calculations is the same, that is, the equation $shareA = shareB$ is true. The secret point serves as a secret key.

Shared secret *bnShare* is a x-coordinate of the secret point on the elliptic curve.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCPSet` or `ippsECCPSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPublic</i> , <i>pShare</i> , or <i>pECC</i> is not valid.
<code>ippStsRangeErr</code>	Indicates an error condition if the memory size of <i>bnShare</i> pointed by <i>pShare</i> is less than the value of <i>feBitSize</i> in the function <code>ippsECCPInit</code> .
<code>ippStsShareKeyErr</code>	Indicates an error condition if the shared secret key is not valid. (For example, the shared secret key is invalid if the result of the secret point calculation is the point at infinity.

ECCPSharedSecretDHC

Computes a shared secret field element by using the Diffie-Hellman scheme and the elliptic curve cofactor.

Syntax

```
IppStatus ippsECCPSharedSecretDHC(const IppsBigNumState* pPrivate, const
    IppsECCPPointState* pPublic, IppsBigNumState* pShare, IppsECCPState* pECC);
```

Parameters

<i>pPrivate</i>	Pointer to your own public key <i>pubKey</i> .
<i>pPublic</i>	Pointer to the public key <i>pubKey</i> .
<i>pShare</i>	Pointer to the secret number <i>bnShare</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function computes a secret number *bnShare* which is a secret key shared between two participants of the cryptosystem. Both participants (Alice and Bob) use the cryptosystem for getting a common secret point on the elliptic curve by using the Diffie-Hellman scheme and elliptic curve cofactor *h*.

Alice and Bob perform the following operations:

1. Alice calculates her own public key *pubKeyA* by using her private key *privKeyA*:

$$pubKeyA = privKeyA \cdot G$$
 where *G* is a base point of the elliptic curve. Alice passes the public key to Bob.
2. Bob calculates his own public key *pubKeyB* by using his private key *privKeyB*:

$$pubKeyB = privKeyB \cdot G$$
 where *G* is a base point of the elliptic curve. Bob passes the public key to Alice.
3. Alice gets Bob's public key and calculates the secret point *shareA*. When calculating, she uses her own private key and Bob's public key and applies the following formula:

$$shareA = h \cdot privKeyA \cdot pubKeyB = h \cdot privKeyA \cdot privKeyB \cdot G$$
 where *h* is the elliptic curve cofactor.
4. Bob gets Alice's public key and calculates the secret point *shareB*. When calculating, he uses his own private key and Alice's public key and applies the following formula:

$$shareB = h \cdot privKeyB \cdot pubKeyA = h \cdot privKeyB \cdot privKeyA \cdot G$$
 where *h* is the elliptic curve cofactor.

Shared secret *bnShare* is a x-coordinate of the secret point on the elliptic curve.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCPSet` or `ippsECCPSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPublic</i> , <i>pPShare</i> , or <i>pECC</i> is not valid.
<code>ippStsRangeErr</code>	Indicates an error condition if the memory size of <i>bnShare</i> pointed by <i>pShare</i> is less than the value of <i>feBitSize</i> in the function <code>ippsECCPInit</code> .
<code>ippStsShareKeyErr</code>	Indicates an error condition if the shared secret key is not valid. (For example, the shared secret key is invalid if the result of the secret point calculation is the point at infinity.

ECCPSignDSA

Computes a digital signature over a message digest.

Syntax

```
IppStatus ippsECCPSignDSA(const IppsBigNumState* pMsgDigest, const  
    IppsBigNumState* pPrivate, IppsBigNumState* pSignX, IppsBigNumState*  
    pSignY, IppsECCPState* pECC);
```

Parameters

<i>pMsgDigest</i>	Pointer to the message digest <i>msg</i> to be digitally signed, that is, to be encrypted with a private key.
<i>pPrivate</i>	Pointer to the signer's regular private key.
<i>pSignX</i>	Pointer to the integer <i>r</i> of the digital signature.
<i>pSignY</i>	Pointer to the integer <i>s</i> of the digital signature.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

A message digest is a fixed size number derived from the original message with an applied hash function over the binary code of the message. The signer's private key and the message digest are used to create a signature.

A digital signature over a message consists of a pair of large numbers r and s which the given function computes.

The scheme used for computing a digital signature is analogue of the ECDSA scheme, an elliptic curve analogue of the DSA scheme. ECDSA assumes that the following keys are hitherto set by a message signer:

ephPrivKey Ephemeral private key.

ephPubKey Ephemeral public key.

The keys can be generated and set up by the unctons `ippsECCPGenKeyPair` and `ippsECCPSetKeyPair` with only requirement that the key *regPrivKey* be different from the key *ephPrivKey*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCPSet` or `ippsECCPSetStd`.

For more information on digital signatures, please refer to the [\[ANSI\]](#) standard.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pMsgDigest</i> , <i>pSignX</i> , <i>pSignY</i> , or <i>ECC</i> is not valid.
<code>ippStsMessageErr</code>	Indicates an error condition if the value of <i>msg</i> pointed by <i>pMsgDigest</i> falls outside the range of $[1, 1-n]$ where n is the order of the elliptic curve base point G .
<code>ippStsRangeErr</code>	Indicates an error condition if one of the parameters pointed by <i>pSignX</i> or <i>pSignY</i> has a less memory size than the order n of the elliptic curve base point G .
<code>ippStsEphemeralKeyErr</code>	Indicates an error condition if the values of the ephemeral keys <i>ephPrivKey</i> and <i>ephPubKey</i> are not valid. (Either $r = 0$ or $s = 0$ is received as a result of the digital signature calculation).

ECCPVerifyDSA

Verifies authenticity of the digital signature over a message digest (ECDSA).

Syntax

```
IppStatus ippseCCPVerifyDSA(const IppsBigNumState* pMsgDigest, const
    IppsBigNumState* pSignX, const IppsBigNumState* pSignY,
    IppeCResult* pResult, IppsECCPState* pECC);
```

Parameters

<i>pMsgDigest</i>	Pointer to the message digest <i>msg</i> .
<i>pSignX</i>	Pointer to the integer <i>r</i> of the digital signature.
<i>pSignY</i>	Pointer to the integer <i>s</i> of the digital signature.
<i>pResult</i>	Pointer to the digital signature verification result.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function verifies authenticity of the digital signature over a message digest *msg*. The signature consists of two large integers: *r* and *s*.

The scheme used to verify the signature is an elliptic curve analogue of the DSA scheme and assumes that the following cryptosystem key be hitherto set:

regPubKey Message sender's regular public key.

The *regPubKey* is set by the function `ippseCCPSetKeyPair`.

The result of the digital signature verification can take one of two possible values:

<code>ippeCValid</code>	Digital signature is valid.
<code>ippeCInvalidSignature</code>	Digital signature is not valid.

The call to the `ippseCCPVerifyDSA` function must be preceded by the call to the `ippseCCPSignDSA` function which computes the digital signature over the message digest *msg* and represents the signature with two numbers: *r* and *s*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippseCCPSet` or `ippseCCPSetStd`.

For more information on digital signatures, please refer to the [\[ANSI\]](#) standard.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pMsgDigest</i> , <i>pSignX</i> , <i>pSignY</i> , or <i>ECC</i> is not valid.
<code>ippStsMessageErr</code>	Indicates an error condition if the value of <i>msg</i> pointed by <i>pMsgDigest</i> falls outside the range of $[1, 1-n]$ where n is the order of the elliptic curve base point G .

ECCPSignNR

Computes the digital signature over a message digest (the Nyberg-Rueppel scheme).

Syntax

```
ippStatus ippseccpsignnr(const IppsBigNumState* pMsgDigest,
    IppsBigNumState* pSignX, IppsBigNumState* pSignY, IppsECCPState*
    pECC);
```

Parameters

<i>pMsgDigest</i>	Pointer to the message digest <i>msg</i> .
<i>pSignX</i>	Pointer to the integer r of the digital signature.
<i>pSignY</i>	Pointer to the integer s of the digital signature.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function computes two large numbers r and s which form the digital signature over a message digest *msg*.

The scheme used to compute the digital signature is an elliptic curve analogue of the El-Gamal Digital Signature scheme with the message recovery (the Nyberg-Rueppel signature scheme). The scheme that the given function uses assumes that the following cryptosystem keys are hitherto set up by the message sender:

<i>regPrivKey</i>	Regular private key.
<i>ephPrivKey</i>	Ephemeral private key.
<i>ephPubKey</i>	Ephemeral public key.

The keys can be generated and set up by the functions `ippsECCPGenKeyPair` and `ippsECCPSetKeyPair` with only requirement that the key *regPrivKey* be different from the key *ephPrivKey*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCPSet` or `ippsECCPSetStd`.

For more information on digital signatures, please refer to the [\[ANSI\]](#) standard.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pMsgDigest</i> , <i>pSignX</i> , <i>pSignY</i> , or <i>ECC</i> is not valid.
<code>ippStsMessageErr</code>	Indicates an error condition if the value of <i>msg</i> pointed by <i>pMsgDigest</i> falls outside the range of $[1, 1-n]$ where <i>n</i> is the order of the elliptic curve base point <i>G</i> .
<code>ippStsRangeErr</code>	Indicates an error condition if one of the parameters pointed by <i>pSignX</i> or <i>pSignY</i> has a less memory size than the order <i>n</i> of the elliptic curve base point <i>G</i> .
<code>ippStsEphemeralKeyErr</code>	Indicates an error condition if the values of the ephemeral keys <i>ephPrivKey</i> and <i>ephPubKey</i> are not valid. (Either $r = 0$ or $s = 0$ is received as a result of the digital signature calculation).

ECCPVerifyNR

Verifies authenticity of the digital signature over a message digest (the Nyberg-Rueppel scheme).

Syntax

```
IppStatus ippsECCPVerifyNR(const IppsBigNumState* pMsgDigest, const
    IppsBigNumState* pSignX, const IppsBigNumState* pSignY,
    IppECResult* pResult, IppsECCPState* pECC);
```

Parameters

<i>pMsgDigest</i>	Pointer to the message digest <i>msg</i> .
<i>pSignX</i>	Pointer to the integer <i>r</i> of the digital signature.
<i>pSignY</i>	Pointer to the integer <i>s</i> of the digital signature.
<i>pResult</i>	Pointer to the digital signature verification result.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function verifies authenticity of the digital signature over a message digest *msg*. The signature is presented with two large integers *r* and *s*.

The scheme used to compute the digital signature is an elliptic curve analogue of the El-Gamal Digital Signature scheme with the message recovery (the Nyberg-Rueppel signature scheme). The scheme that the given function uses assumes that the following cryptosystem keys be hitherto set up by the message sender:

regPubKey Message sender's regular private key.

The key can be generated and set up by the function `ippsECCPGenKeyPair`.

The result of the digital signature verification can take one of two possible values:

<code>ippECValid</code>	The digital signature is valid.
<code>ippECInvalidSignature</code>	The digital signature is not valid.

The call to the `ippsECCPVerifyNR` function must be preceded by the call to the `ippsECCPSignNR` function which computes the digital signature over the message digest *msg* and represents the signature with two numbers: *r* and *s*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCPSet` or `ippsECCPSetStd`.

For more information on digital signatures, please refer to the [\[ANSI\]](#) standard.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pMsgDigest</i> , <i>pSignX</i> , <i>pSignY</i> , or <i>ECC</i> is not valid.
<code>ippStsMessageErr</code>	Indicates an error condition if the value of <i>msg</i> pointed by <i>pMsgDigest</i> falls outside the range of $[1, 1-n]$ where n is the order of the elliptic curve base point G .

Functions Based on $GF(2^m)$

This section describes functions designed to specify the elliptic curve cryptosystem and perform various operations on the elliptic curve defined over a binary finite field. The examples of the operations are illustrated below:

- Setting up operations
[ECCBSet](#) sets up elliptic curve domain parameters. [ECCBSetKeyPair](#) sets a pair of public and private keys for the given cryptosystem.
- Computation operations
[ECCBAddPoint](#) adds two points on the elliptic curve. [ECCBMulPointScalar](#) performs the scalar multiplication of a point on the elliptic curve. [ECCBSignDSA](#) computes the digital signature of a message.
- Validation operations
[ECCBValidate](#) checks validity of the elliptic curve domain parameters. [ECCBValidateKeyPair](#) validates correctness of the public and private keys.
- Generation operations
[ECCBGenKeyPair](#) generates a private key and computes a public key for the given elliptic cryptosystem.
- Retrieval operations
[ECCBGet](#) retrieves elliptic curve domain parameters. [ECCBGetOrderBitSize](#) retrieves the size of a base point in bytes.

All functions described in this section employ a context `IppsECCBState` that catches several auxiliary components specifying operations performed on the elliptic curve or entire elliptic cryptosystem. ECCB stands for Elliptic Curve Cryptography Binary and means that all functions whose name include this abbreviation perform operations over a [binary finite field \$GF\(2^m\)\$](#) .

ECCBGetSize

Gets the size of the IppsECCBState context.

Syntax

```
IppStatus ippsECCBGetSize(int feBitSize, int *pSize);
```

Parameters

<i>feBitSize</i>	Size (in bits) of the field element.
<i>pSize</i>	Pointer to the size of the context (in bytes).

Description

The function computes the size of the context in bytes for the elliptic cryptosystem over a binary finite field $GF(2^m)$.

Context is a structure `IppsECCBState` designed to store information about the cryptosystem status.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsSizeErr</code>	Indicates an error condition if the value of the parameter <i>feBitSize</i> is less than 1.

ECCBInit

Initializes context for the elliptic curve cryptosystem over $GF(2^m)$.

Syntax

```
IppStatus ippsECCBInit(int feBitSize, IppsECCBState* pECC)
```

Parameters

<i>feBitSize</i>	Size (in bits) of a field element.
<i>pECC</i>	Pointer to the cryptosystem context.

Description

The function initializes the context of the elliptic curve cryptosystem over a binary finite field $GF(2^m)$.

Context is a structure `IppsECCBState` designed to store information about the cryptosystem status.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsSizeErr</code>	Indicates an error condition if the value of the parameter <i>feBitSize</i> is less than 1.

ECCBSet

Sets up elliptic curve domain parameters over $GF(2^m)$.

Syntax

```
IppStatus ippseCCBSet(const IppsBigNumState* pPrime, const
    IppsBigNumState* pA, const IppsBigNumState* pB, const
    IppsBigNumState* pGX, const IppsBigNumState* pGY, const
    IppsBigNumState* pOrder, int cofactor, IppsECCBState* pECC);
```

Parameters

<i>pPrime</i>	Pointer to the irreducible binary polynomial $f(x)$ of degree m which specifies the presentation of the field $GF(2^m)$.
<i>pA</i>	Pointer to the coefficient A of the equation defining the elliptic curve.
<i>pB</i>	Pointer to the coefficient B of the equation defining the elliptic curve.
<i>pGX</i>	Pointer to the x -coordinate of the elliptic curve base point.
<i>pGY</i>	Pointer to the y -coordinate of the elliptic curve base point.
<i>pOrder</i>	Pointer to the order of the elliptic curve base point.
<i>cofactor</i>	Cofactor.
<i>pECC</i>	Pointer to the context of the cryptosystem.

Description

The function sets up the elliptic curve domain parameters over a binary finite field $\text{GF}(2^m)$. These are as follows:

- *pPrime* sets up the irreducible binary polynomial $f(x)$ of degree m which specifies the presentation of the field $\text{GF}(2^m)$.
- *pA*, *pB* set up the coefficients A and B of the equation defining the elliptic curve:

$$y^2 + x \cdot y = x^3 + A \cdot x^2 + B \quad \text{in } \text{GF}(2^m).$$
- *pGX*, *pGY* set up coordinates of the elliptic curve base point G .
- *pOrder* sets up the order n of the elliptic curve base point G such that $n \cdot G = O$ where O is the point at infinity and n is a prime number.
- *cofactor* sets up the ratio h of a general number of points $\#E$ on the elliptic curve (including the point at infinity) to the order n of the base point :

$$h = \frac{\#E}{n}.$$

The domain parameters are set in the cryptosystem context which must be already created by the `ippsECCBGetSize` and `ippsECCBInit` functions.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPrime</i> , <i>pA</i> , <i>pB</i> , <i>pGX</i> , <i>pGY</i> , <i>pOrder</i> , and <i>pECC</i> is not valid.
<code>ippStsRangeErr</code>	Indicates an error condition if the memory size of one of the parameters pointed by <i>pPrime</i> , <i>pA</i> , <i>pB</i> , <i>pGX</i> , <i>pGY</i> , <i>pOrder</i> , and <i>pECC</i> is more than the value of <i>feBitSize</i> in the <code>ippsECCBInit</code> function or the value of <i>cofactor</i> is less than or equal to zero.

ECCBSetStd

Sets up a recommended set of elliptic curve domain parameters over $GF(2^m)$.

Syntax

```
IppStatus ippseCCBSetStd(IppECCType flag, IppsECCBState* pECC);
```

Parameters

flag Set specifier.

pECC Pointer to the cryptosystem context.

Description

The function sets a recommended set of elliptic curve domain parameters over a binary finite field $GF(2^m)$.

The set is defined by the value of the parameter *flag*. Possible values of the parameter are as follows:

IppECCBStd113r1	For the cryptosystem context where <i>feBitSize</i> ==113
IppECCBStd113r2	For the cryptosystem context where <i>feBitSize</i> ==113
IppECCBStd131r1	For the cryptosystem context where <i>feBitSize</i> ==131
IppECCBStd131r2	For the cryptosystem context where <i>feBitSize</i> ==131
IppECCBStd163k1	For the cryptosystem context where <i>feBitSize</i> ==163
IppECCBStd163r1	For the cryptosystem context where <i>feBitSize</i> ==163
IppECCBStd163r2	For the cryptosystem context where <i>feBitSize</i> ==163
IppECCBStd193r1	For the cryptosystem context where <i>feBitSize</i> ==193
IppECCBStd193r2	For the cryptosystem context where <i>feBitSize</i> ==193
IppECCBStd233k1	For the cryptosystem context where <i>feBitSize</i> ==233
IppECCBStd233r1	For the cryptosystem context where <i>feBitSize</i> ==233
IppECCBStd239k1	For the cryptosystem context where <i>feBitSize</i> ==239
IppECCBStd283k1	For the cryptosystem context where <i>feBitSize</i> ==283
IppECCBStd283r1	For the cryptosystem context where <i>feBitSize</i> ==283
IppECCBStd409k1	For the cryptosystem context where <i>feBitSize</i> ==409.
IppECCBStd409r1	For the cryptosystem context where <i>feBitSize</i> ==409.

IppECCBStd571k1

For the cryptosystem context where *feBitSize*==571.

IppECCBStd571r1

For the cryptosystem context where *feBitSize*==571.

For more information on parameter values for the recommended elliptic curves, see [\[SEC2\]](#).

The cryptosystem context must be already created by the `ippsECCBGetSize` and `ippsECCBInit` functions. The value of *feBitSize* is applied when these function are called and predetermines the possible choice of the *flag* value.

Return Values

ippStsNoErr

Indicates no error. Any other value indicates an error or warning.

ippStsNullPtrErr

Indicates an error condition if any of the specified pointers is NULL.

ippStsContextMatchErr

Indicates an error condition if the cryptosystem context is not valid.

ippStsECCInvalidFlagErr

Indicates an error condition if the value of the parameter *flag* is not valid.

ECCBGet

Retrieves elliptic curve domain parameters over $GF(2^m)$.

Syntax

```
IppStatus ippsECCBGet(IppsBigNumState* pPrime, IppsBigNumState* pA,
    IppsBigNumState* pB, IppsBigNumState* pGX, IppsBigNumState* pGY,
    IppsBigNumState* pOrder, int* cofactor, IppsECCBState* pECC);
```

Parameters

pPrime

Pointer to the irreducible binary polynomial $f(x)$ of degree m which specifies the presentation of the field $GF(2^m)$.

pA

Pointer to the coefficient A of the equation defining the elliptic curve.

pB

Pointer to the coefficient B of the equation defining the elliptic curve.

pGX

Pointer to the x -coordinate of the elliptic curve base point.

<i>pGY</i>	Pointer to the y -coordinate of the elliptic curve base point.
<i>pOrder</i>	Pointer to the order n of the elliptic curve base point.
<i>cofactor</i>	Pointer to the cofactor h .
<i>pECC</i>	Pointer to the context of the cryptosystem.

Description

The function retrieves elliptic curve domain parameters from the context of the elliptic cryptosystem over a binary finite field $GF(2^m)$ and allocates them in accordance with the pointers *pPrime*, *pA*, *pB*, *pGX*, *pGY*, *pOrder*, and *cofactor*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: *ippsECCBSet* or *ippsECCBSetStd*.

Return Values

<i>ippStsNoErr</i>	Indicates no error. Any other value indicates an error or warning.
<i>ippStsNullPtrErr</i>	Indicates an error condition if any of the specified pointers is NULL.
<i>ippStsContextMatchErr</i>	Indicates an error condition if one of the contexts pointed by <i>pPrime</i> , <i>pA</i> , <i>pB</i> , <i>pGX</i> , <i>pGY</i> , <i>pOrder</i> , or <i>pECC</i> is not valid.
<i>ippStsRangeErr</i>	Indicates an error condition if the memory size of one of the parameters pointed by <i>pPrime</i> , <i>pA</i> , <i>pB</i> , <i>pGX</i> , <i>pGY</i> , <i>pOrder</i> , and <i>pECC</i> is less than the value of <i>feBitSize</i> in the <i>ippsECCBInit</i> function.

ECCBGetOrderBitSize

Retrieves order size of the elliptic curve base point over $GF(2^m)$ in bits.

Syntax

```
IppStatus ippsECCBGetOrderBitSize(int* pBitSize, IppsECCBState* pECC);
```

Parameters

<i>pBitSize</i>	Pointer to the size of the base point (in bits).
<i>pECC</i>	Pointer to the cryptosystem context.

Description

The function retrieves the order size (in bits) of the elliptic curve base point G from the context of elliptic cryptosystem over a binary finite field $GF(2^m)$ and allocates it in accordance with the pointer *pBitsSize*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: *ippSECCBSet* or *ippSECCBSetStd*.

Return Values

<i>ippStsNoErr</i>	Indicates no error. Any other value indicates an error or warning.
<i>ippStsNullPtrErr</i>	Indicates an error condition if any of the specified pointers is NULL.
<i>ippStsContextMatchErr</i>	Indicates an error condition if the cryptosystem context is not valid.

ECCBValidate

Checks validity of the elliptic curve domain parameters over $GF(2^m)$.

Syntax

```
IppStatus ippSECCBValidate(int nTrials, IppECResult* pResult,
                          IppsECCBState* pECC, IppBitSupplier rndFunc, void* pRndParam);
```

Parameters

<i>nTrials</i>	A number of attempts made to check the number for primality.
<i>pResult</i>	Pointer to the result received upon the check of the elliptic curve domain parameters.
<i>pECC</i>	Pointer to the cryptosystem context.
<i>rndFunc</i>	Specified Random Generator.
<i>pRndParam</i>	Pointer to the Random Generator context.

Description

The function checks validity of the elliptic curve domain parameters over a binary finite field $GF(2^m)$ and stores the result of the check in accordance with the pointer *pResult*.

Elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCBSet` or `ippsECCBSetStd`. The purpose of the parameters `rndFunc`, `pRndParam`, and `nTrials` is analogous to that of the parameters `rndFunc`, `pRndParam`, and `nTrials` in the `ippsPrimeTest` function.

The result of the elliptic curve domain parameters check can take one of the following values:

<code>ippECValid</code>	The parameters are valid.
<code>ippECCompllicatedBase</code>	The irreducible binary polynomial $f(x)$ of degree m which specifies the presentation of the field $GF(2^m)$ is not valid because the set of polynomials consists of more than five elements.
<code>ippECCompositeBase</code>	The binary polynomial $f(x)$ is not irreducible.
<code>ippECIsSupersingular</code>	The coefficient in the elliptic curve equation is <code>NULL</code> .
<code>ippECPointAtInfinite</code>	The elliptic curve base point G is the point at infinity.
<code>ippECPointIsNotValid</code>	Base point G is not on the elliptic curve.
<code>ippECCompositeOrder</code>	The order n of the base point G is a composite number.
<code>ippECInvalidOrder</code>	The order n of the base point G is not valid because the requirement that $n \cdot G = O$ where O is the point at infinity is not met.
<code>ippECIsWeakSSSA</code>	$h \cdot n = 2^m$ where h is a cofactor and n is the order n of the base point.
<code>ippECIsWeakMOV</code>	The curve is excluded because it is subject to the MOV reduction attack.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is <code>NULL</code> .
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <code>c</code> or <code>pECC</code> is not valid.
<code>ippStsBadArgErr</code>	Indicates an error condition if the memory size of the parameter <code>seed</code> is less than five words (32 bytes in each) or the value of the parameter <code>nTrials</code> is less than 1.

ECCBPointGetSize

Gets the size of the `IppsECCBPoint` context in bytes for a point on the elliptic curve point defined over $GF(2^m)$.

Syntax

```
IppStatus ippsECCBPointGetSize(int feBitSize, int* pSize);
```

Parameters

<i>feBitSize</i>	Size (in bits) of the field element.
<i>pSize</i>	Pointer to the context size.

Description

The function computes the context size in bytes for a point on the elliptic curve defined over a binary finite field $GF(2^m)$.

Context is a structure `IppsECCBPoint` intended for storing the information about a point on the elliptic curve defined over $GF(2^m)$.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsSizeErr</code>	Indicates an error condition if the value of the parameter <i>feBitSize</i> is less than 1.

ECCBPointInit

Initializes the context for a point on the elliptic curve defined over $GF(2^m)$.

Syntax

```
IppStatus ippsECCBPointInit(int feBitSize, IppsECCBPoint* pPoint);
```

Parameters

<i>feBitSize</i>	Size (in bits) of the field element.
<i>pECC</i>	Pointer to the context of the elliptic curve point.

Description

The function initializes the context for a point on the elliptic curve defined over a binary finite field $\text{GF}(2^m)$.

Context is a structure `IppsECCBPoint` intended for storing the information about a point on the elliptic curve defined over $\text{GF}(2^m)$.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsSizeErr</code>	Indicates an error condition if the value of the parameter <i>feBitSize</i> is less than 1.

ECCBSetPoint

Sets coordinates of a point on the elliptic curve defined over $\text{GF}(2^m)$.

Syntax

```
IppStatus ippseccbSetPoint(const IppsBigNumState* pX, const  
    IppsBigNumState* pY, IppsECCBPoint* pPoint, IppsECCBState* pECC);
```

Parameters

<i>pX</i>	Pointer to the x-coordinate of the point on the elliptic curve.
<i>pY</i>	Pointer to the y-coordinate of the point on the elliptic curve.
<i>pPoint</i>	Pointer to the context of the elliptic curve point.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function sets the coordinates of a point on the elliptic curve defined over a binary finite field $\text{GF}(2^m)$.

The context of the point on the elliptic curve must be already created by functions: `ippsECCBPointGetSize` and `ippsECCBPointInit`. The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCBSet` or `ippsECCBSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pX</i> , <i>pY</i> , <i>pPoint</i> , or <i>pECC</i> is not valid.

ECCBSetPointAtInfinity

Sets the point at infinity.

Syntax

```
IppStatus ippsECCBSetPointAtInfinity(IppsECCBPoint* pPoint,
                                     IppsECCBState* pECC);
```

Parameters

<i>pPoint</i>	Pointer to the context of the elliptic curve point.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function sets the point at infinity. The context of the elliptic curve point must be already created by functions: `ippsECCBPointGetSize` and `ippsECCBPointInit`. The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCBSet` or `ippsECCBSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPoint</i> or <i>pECC</i> is not valid.

ECCBGetPoint

Retrieves coordinates of the point on the elliptic curve defined over $GF(2^m)$.

Syntax

```
IppStatus ippsECCBGetPoint(IppsBigNumState* pX, IppsBigNumState* pY,  
    const IppsECCBPoint* pPoint, IppsECCBState* pECC);
```

Parameters

<i>pX</i>	Pointer to the x-coordinate of the point on the elliptic curve.
<i>pY</i>	Pointer to the y-coordinate of the point on the elliptic curve.
<i>pPoint</i>	Pointer to the context of the elliptic curve point.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function retrieves the coordinates of the point on the elliptic curve defined over a binary finite field $GF(2^m)$ from the point context and allocates them in accordance with the set pointers *pX* and *pY*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: *ippsECCBSet* or *ippsECCBSetStd*.

Return Values

<i>ippStsNoErr</i>	Indicates no error. Any other value indicates an error or warning.
<i>ippStsNullPtrErr</i>	Indicates an error condition if any of the specified pointers is NULL.
<i>ippStsContextMatchErr</i>	Indicates an error condition if one of the contexts pointed by <i>pX</i> , <i>pY</i> , <i>pPoint</i> , or <i>pECC</i> is not valid.

ECCBCheckPoint

Checks correctness of the point on the elliptic curve defined over $GF(2^m)$.

Syntax

```
IppStatus ippsECCBCheckPoint(const IppsECCBPoint* pP, IppeCResult*
    pResult, IppsECCBState* pECC);
```

Parameters

<i>pP</i>	Pointer to the elliptic curve point.
<i>pResult</i>	Pointer to the result of the check.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function checks the correctness of the point on the elliptic curve defined over a binary finite field $GF(2^m)$ and allocates the result of the check in accordance with the pointer *pResult*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCBSet` or `ippsECCBSetStd`.

The result of the check for the correctness of the point can take one of the following values:

<code>ippeCValid</code>	Point is on the elliptic curve.
<code>ippeCPointIsValid</code>	Point is not on the elliptic curve and is not the point at infinity.
<code>ippeCPointIsAtInfinite</code>	Point is the point at infinity.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pP</i> or <i>pECC</i> is not valid.

ECCBComparePoint

Compares two points on the elliptic curve defined over $GF(2^m)$.

Syntax

```
IppStatus ippseCCBComparePoint(const IppsECCBPoint* pP, const  
    IppsECCBPoint* pQ, IppeCResult* pResult, IppsECCBState* pECC);
```

Parameters

<i>pP</i>	Pointer to the elliptic curve point <i>P</i> .
<i>pQ</i>	Pointer to the elliptic curve point <i>Q</i> .
<i>pResult</i>	Pointer to the comparison result of two points: <i>P</i> and <i>Q</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function compares two points *P* and *Q* on the elliptic curve defined over a binary finite field $GF(2^m)$ and allocates the comparison result in accordance with the pointer *pResult*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: *ippseCCBSet* or *ippseCCBSetStd*.

The comparison result of two points *P* and *Q* can take one of the following values:

<i>ippeCPointIsEqual</i>	Points <i>P</i> and <i>Q</i> are equal.
<i>ippeCPointIsNotEqual</i>	Points <i>P</i> and <i>Q</i> are different.

Return Values

<i>ippStsNoErr</i>	Indicates no error. Any other value indicates an error or warning.
<i>ippStsNullPtrErr</i>	Indicates an error condition if any of the specified pointers is NULL.
<i>ippStsContextMatchErr</i>	Indicates an error condition if one of the contexts pointed by <i>pP</i> or <i>pECC</i> is not valid.

ECCBNegativePoint

Finds the elliptic curve point which is an additive inverse for the given point over $GF(2^m)$.

Syntax

```
IppStatus ippseCCBNegativePoint(const IppsECCBPoint* pP, IppsECCBPoint*
                                pR, IppsECCBState* pECC);
```

Parameters

<i>pP</i>	Pointer to the elliptic curve point <i>P</i> .
<i>pR</i>	Pointer to the elliptic curve point <i>R</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function finds an elliptic curve point *R* over a binary finite field $GF(2^m)$ which is an additive inverse of the given point *P*, i.e., $P = -P$.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippseCCBSet` or `ippseCCBSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pP</i> , <i>pR</i> , or <i>pECC</i> is not valid.

ECCBAddPoint

Computes the addition of two elliptic curve points over $GF(2^m)$.

Syntax

```
IppStatus ippseCCBAddPoint(const IppsECCBPoint* pP, const IppsECCBPoint*  
    pQ, IppsECCBPoint* pR, IppsECCBState* pECC);
```

Parameters

<i>pP</i>	Pointer to the elliptic curve point <i>P</i> .
<i>pQ</i>	Pointer to the elliptic curve point <i>Q</i> .
<i>pR</i>	Pointer to the elliptic curve point <i>R</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function calculates the addition of two elliptic curve points *P* and *Q* over a binary finite field $GF(2^m)$ with the result in a point *R* such that $R = P + Q$.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippseCCBSet` or `ippseCCBSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pP</i> , <i>pQ</i> , <i>pR</i> , or <i>pECC</i> is not valid.

ECCBMulPointScalar

Performs scalar multiplication of a point on the elliptic curve defined over $GF(2^m)$.

Syntax

```
IppStatus ippseCCBMulPointScalar(const IppsECCBPoint* pP, const
    IppsBigNumState* pK, IppsECCBPoint* pR, IppsECCBState* pECC);
```

Parameters

<i>pP</i>	Pointer to the elliptic curve point <i>P</i> .
<i>pK</i>	Pointer to the scalar <i>K</i> .
<i>pR</i>	Pointer to the elliptic curve point <i>R</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function performs the *K* scalar multiplication of an elliptic curve point *P* over $GF(2^m)$ with the result in a point *R* such that $R = K \cdot P$.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippseCCBSet` or `ippseCCBSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pP</i> , <i>pK</i> , <i>pR</i> , or <i>pECC</i> is not valid.

ECCBGenKeyPair

Generates a private key and computes public keys of the elliptic cryptosystem over $GF(2^m)$.

Syntax

```
IppStatus ippsECCBGenKeyPair(IppsBigNumState* pPrivate,  
    IppsECCBPointState* pPublic, IppsECCBState* pECC, IppBitSupplier  
    rndFunc, void* pRndParam);
```

Parameters

<i>pPrivate</i>	Pointer to the private key <i>privKey</i> .
<i>pPublic</i>	Pointer to the public key <i>pubKey</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.
<i>rndFunc</i>	Specified Random Generator.
<i>pRndParam</i>	Pointer to the Random Generator context.

Description

The function generates a private key *privKey* and computes a public key *pubKey* of the elliptic cryptosystem over a binary finite field $GF(2^m)$. The generation process employs user specified *rndFunc* Random Generator.

The private key *privKey* is a number that lies in the range of $[1, n-1]$ where n is the order of the elliptic curve base point.

The public key *pubKey* is an elliptic curve point such that $pubKey = privKey \cdot G$ where G is the base point of the elliptic curve.

The memory size of the parameter *privKey* pointed by *pPrivate* must be less than that of the base point which can also be defined by the function `ippsECCBGetOrderBitSize`.

The context of the point *pubKey* as an elliptic curve point must be created by using the functions `ippsECCBPointGetSize` and `ippsECCBPointInit`.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCBSet` or `ippsECCBSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPrivate</i> , <i>pPublic</i> , or <i>pECC</i> is not valid.
<code>ippStsSizeErr</code>	Indicates an error condition if the memory size of the parameter <i>privKey</i> pointed by <i>pPrivate</i> is less than that of the order of the elliptic curve base point.

ECCBPublicKey

Computes a public key from the given private key of the elliptic cryptosystem over $GF(2^m)$.

Syntax

```
IppStatus ippseCCBPublicKey(const IppsBigNumState* pPrivate,
                             IppsECCBPoint* pPublic, IppsECCBState* pECC);
```

Parameters

<i>pPrivate</i>	Pointer to the private key <i>privKey</i> .
<i>pPublic</i>	Pointer to the public key <i>pubKey</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function computes the public key *pubKey* from the given private key *privKey* of the elliptic cryptosystem over a binary finite field $GF(2^m)$.

The private key *privKey* is a number that lies in the range of $[1, n-1]$ where n is the order of the elliptic curve base point. The public key *pubKey* is an elliptic curve point such that $pubKey = privKey \cdot G$ where G is the base point of the elliptic curve.

The context of the point *pubKey* as an elliptic curve point must be created by using the functions `ippseCCBPointGetSize` and `ippseCCBPointInit`.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippseCCBSet` or `ippseCCBSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.

<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <code>pPrivate</code> , <code>pPublic</code> , or <code>pECC</code> is not valid.
<code>ippStsInvalidPrivateKey</code>	Indicates an error condition if the value of the private key falls outside the range of $[1, n-1]$.

ECCBValidateKeyPair

Validates private and secret keys of the elliptic cryptosystem over $GF(2^m)$.

Syntax

```
IppStatus ippseCCBValidateKeyPair(const IppsBigNumState* pPrivate, const
    IppsECCBPoint* pPublic, IppECResult* pResult, IppsECCBState* pECC);
```

Parameters

<code>pPrivate</code>	Pointer to the private key <code>privKey</code> .
<code>pPublic</code>	Pointer to the public key <code>pubKey</code> .
<code>pResult</code>	Pointer to the validation result.
<code>pECC</code>	Pointer to the context of the elliptic cryptosystem.

Description

The function validates the private key `privKey` and public key `pubKey` of the elliptic cryptosystem over a binary finite field $GF(2^m)$ and allocates the result of the validation in accordance with the pointer `pResult`.

The private key `privKey` is a number that lies in the range of $[1, n-1]$ where n is the order of the elliptic curve base point. The public key `pubKey` is an elliptic curve point such that $pubKey = privKey \cdot G$ where G is the base point of the elliptic curve.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippseCCBSet` or `ippseCCBSetStd`.

The result of the cryptosystem keys validation for correctness can take one of the following values:

<code>ippECValid</code>	Keys are valid.
<code>ippECInvalidKeyPair</code>	Keys are not valid because $privKey \cdot G \neq pubKey$
<code>ippECInvalidPrivateKey</code>	Key <code>privKey</code> falls outside the range of $[1, n-1]$.
<code>ippECPPointIsAtInfinite</code>	Key <code>pubKey</code> is the point at infinity.

ippECInvalidPublicKey

Key *pubKey* is not valid because $n \cdot \textit{pubKey} \neq O$ where O is the point at infinity.

Return Values

ippStsNoErr

Indicates no error. Any other value indicates an error or warning.

ippStsNullPtrErr

Indicates an error condition if any of the specified pointers is NULL.

ippStsContextMatchErr

Indicates an error condition if one of the contexts pointed by *pPrivate*, *pPublic*, or *pECC* is not valid.

ECCBSetKeyPair

Sets private and/or public keys in the elliptic cryptosystem over $GF(2^m)$.

Syntax

```
IppStatus ippsECCBSetKeyPair(const IppsBigNumState* pPrivate, const
    IppsECCBPoint* pPublic, IppBool regular, IppsECCBState* pECC);
```

Parameters

pPrivate

Pointer to the private key *privKey*.

pPublic

Pointer to the public key *pubKey*.

regular

Key status flag.

pECC

Pointer to the context of the elliptic cryptosystem.

Description

The function sets the private key *privKey* and/or public key *pubKey* in the elliptic cryptosystem defined over a binary finite field $GF(2^m)$.

The private key *privKey* is a number that lies in the range of $[1, n-1]$ where n is the order of the elliptic curve base point. The public key *pubKey* is an elliptic curve point such that $\textit{pubKey} = \textit{privKey} \cdot G$ where G is the base point of the elliptic curve.

The two possible values of the parameter *regular* define the key timeliness status:

ippTrue

Keys are regular.

ippFalse

Keys are ephemeral.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCBSet` or `ippsECCBSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPrivate</i> , <i>pPublic</i> , or <i>pECC</i> is not valid.

ECCBSharedSecretDH

Computes a shared secret field element by using the Diffie-Hellman scheme.

Syntax

```
IppStatus ippsECCBSharedSecretDH(const IppsBigNumState* pPrivate, const  
    IppsECCBPointState* pPublic, IppsBigNumState* pShare, IppsECCBState* pECC);
```

Parameters

<i>pPrivate</i>	Pointer to your own public key <i>pubKey</i> .
<i>pPublic</i>	Pointer to the public key <i>pubKey</i> .
<i>pShare</i>	Pointer to the secret number <i>bnShare</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function computes a secret number *bnShare*, which is a secret key shared between two participants of the cryptosystem.

In cryptography, metasyntactic names such as Alice as Bob are normally used as examples and in discussions and stand for participant A and participant B.

Both participants (Alice and Bob) use the cryptosystem for receiving a common secret point on the elliptic curve called a secret key. To receive a secret key, participants apply the Diffie-Hellman key-agreement scheme involving public key exchange. The value of the secret key entirely depends on participants.

According to the scheme, Alice and Bob perform the following operations:

1. Alice calculates her own public key *pubKeyA* by using her private key *privKeyA*:

$$pubKeyA = privKeyA \cdot G$$
where *G* is a base point of the elliptic curve. Alice passes the public key to Bob.
2. Bob calculates his own public key *pubKeyB* by using his private key *privKeyB*:

$$pubKeyB = privKeyB \cdot G$$
where *G* is a base point of the elliptic curve. Bob passes the public key to Alice.
3. Alice gets Bob's public key and calculates the secret point *shareA*. When calculating, she uses her own private key and Bob's public key and applies the following formula:

$$shareA = privKeyA \cdot pubKeyB = privKeyA \cdot privKeyB \cdot G$$
4. Bob gets Alice's public key and calculates the secret point *shareB*. When calculating, he uses his own private key and Alice's public key and applies the following formula:

$$shareB = privKeyB \cdot pubKeyA = privKeyB \cdot privKeyA \cdot G$$

Because the following equation is true

$privKeyA \cdot privKeyB \cdot G = privKeyB \cdot privKeyA \cdot G$, the result of both calculations is the same, that is, the equation $shareA = shareB$ is true. The secret point serves as a secret key.

Shared secret *bnShare* is a x-coordinate of the secret point on the elliptic curve.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCBSet` or `ippsECCBSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPublic</i> , <i>pShare</i> , or <i>pECC</i> is not valid.
<code>ippStsRangeErr</code>	Indicates an error condition if the memory size of <i>bnShare</i> pointed by <i>pShare</i> is less than the value of <i>feBitSize</i> in the function <code>ippsECCBInit</code> .
<code>ippStsShareKeyErr</code>	Indicates an error condition if the shared secret key is not valid. (For example, the shared secret key is invalid if the result of the secret point calculation is the point at infinity.

ECCBSharedSecretDHC

Computes a shared secret field element by using the Diffie-Hellman scheme and the elliptic curve cofactor.

Syntax

```
IppStatus ippseccbsharedsecretDHC(const IppsBigNumState* pPrivate, const
    IppsECCBPointState* pPublic, IppsBigNumState* pShare, IppsECCBState* pECC);
```

Parameters

<i>pPrivate</i>	Pointer to your own public key <i>pubKey</i> .
<i>pPublic</i>	Pointer to the public key <i>pubKey</i> .
<i>pShare</i>	Pointer to the secret number <i>bnShare</i> .
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function computes a secret number *bnShare* which is a secret key shared between two participants of the cryptosystem. Both participants (Alice and Bob) use the cryptosystem for getting a common secret point on the elliptic curve by using the Diffie-Hellman scheme and elliptic curve cofactor *h*.

Alice and Bob perform the following operations:

1. Alice calculates her own public key *pubKeyA* by using her private key *privKeyA*:

$$pubKeyA = privKeyA \cdot G$$
 where *G* is a base point of the elliptic curve. Alice passes the public key to Bob.
2. Bob calculates his own public key *pubKeyB* by using his private key *privKeyB*:

$$pubKeyB = privKeyB \cdot G$$
 where *G* is a base point of the elliptic curve. Bob passes the public key to Alice.
3. Alice gets Bob's public key and calculates the secret point *shareA*. When calculating, she uses her own private key and Bob's public key and applies the following formula:

$$shareA = h \cdot privKeyA \cdot pubKeyB = h \cdot privKeyA \cdot privKeyB \cdot G$$
 where *h* is the elliptic curve cofactor.
4. Bob gets Alice's public key and calculates the secret point *shareB*. When calculating, he uses his own private key and Alice's public key and applies the following formula:

$$shareB = h \cdot privKeyB \cdot pubKeyA = h \cdot privKeyB \cdot privKeyA \cdot G$$
 where *h* is the elliptic curve cofactor.

Shared secret *bnShare* is a x-coordinate of the secret point on the elliptic curve.

To define a secret key, the call to the `ippsECCBSharedSecretDH` function must be preceded by the call to the `ippsECCBSetKeyPair` function.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCBSet` or `ippsECCBSetStd`.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pPublic</i> , <i>pShare</i> , or <i>pECC</i> is not valid.
<code>ippStsRangeErr</code>	Indicates an error condition if the memory size of <i>bnShare</i> pointed by <i>pShare</i> is less than the value of <i>feBitSize</i> in the function <code>ippsECCBInit</code> .
<code>ippStsShareKeyErr</code>	Indicates an error condition if the shared secret key is not valid. (For example, the shared secret key is invalid if the result of the secret point calculation is the point at infinity.

ECCBSignDSA

Computes a digital signature over a message digest.

Syntax

```
IppStatus ippsECCBSignDSA(const IppsBigNumState* pMsgDigest, const
    IppsBigNumState* pPrivate, IppsBigNumState* pSignX, IppsBigNumState*
    pSignY, IppsECCBState* pECC);
```

Parameters

<i>pMsgDigest</i>	Pointer to the message digest <i>msg</i> to be digitally signed, that is, to be encrypted with a private key.
<i>pPrivate</i>	Pointer to the signer's regular private key.
<i>pSignX</i>	Pointer to the integer <i>r</i> of the digital signature.
<i>pSignY</i>	Pointer to the integer <i>s</i> of the digital signature.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

A message digest is a fixed size number derived from the original message with an applied hash function over the binary code of the message. The signer's private key and the message digest are used to create a signature.

A digital signature over a message consists of a pair of large numbers r and s which the given function computes.

The scheme used for computing a digital signature is analogue of the ECDSA scheme, an elliptic curve analogue of the DSA scheme. ECDSA assumes that the following keys are hitherto set by a message signer:

ephPrivKey Ephemeral private key.

ephPubKey Ephemeral public key.

The keys can be generated and set up by the functions `ippsECCBGenKeyPair` and `ippsECCBSetKeyPair` with only requirement that the key *regPrivKey* be different from the key *ephPrivKey*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCBSet` or `ippsECCBSetStd`.

For more information on digital signatures, please refer to the [\[ANSI\]](#) standard.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pMsgDigest</i> , <i>pSignX</i> , <i>pSignY</i> , or <i>ECC</i> is not valid.
<code>ippStsMessageErr</code>	Indicates an error condition if the value of <i>msg</i> pointed by <i>pMsgDigest</i> falls outside the range of $[1, 1-n]$ where n is the order of the elliptic curve base point G .
<code>ippStsRangeErr</code>	Indicates an error condition if one of the parameters pointed by <i>pSignX</i> or <i>pSignY</i> has a less memory size than the order n of the elliptic curve base point G .
<code>ippStsEphemeralKeyErr</code>	Indicates an error condition if the values of the ephemeral keys <i>ephPrivKey</i> and <i>ephPubKey</i> are not valid. (Either $r = 0$ or $s = 0$ is received as a result of the digital signature calculation).

ECCBVerifyDSA

Verifies authenticity of the digital signature over a message digest (ECDSA).

Syntax

```
IppStatus ippsECCBVerifyDSA(const IppsBigNumState* pMsgDigest, const
    IppsBigNumState* pSignX, const IppsBigNumState* pSignY,
    IppECResult* pResult, IppsECCBState* pECC);
```

Parameters

<i>pMsgDigest</i>	Pointer to the message digest <i>msg</i> .
<i>pSignX</i>	Pointer to the integer <i>r</i> of the digital signature.
<i>pSignY</i>	Pointer to the integer <i>s</i> of the digital signature.
<i>pResult</i>	Pointer to the digital signature verification result.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function verifies authenticity of the digital signature over a message digest *msg*. The signature consists of two large integers: *r* and *s*.

The scheme used to verify the signature is an elliptic curve analogue of the DSA scheme and assumes that the following cryptosystem key be hitherto set:

regPubKey Message sender's regular public key.

The *regPubKey* is set by the function `ippsECCBSetKeyPair`.

The result of the digital signature verification can take one of two possible values:

<code>ippecValid</code>	Digital signature is valid.
<code>ippecInvalidSignature</code>	Digital signature is not valid.

The call to the `ippsECCBVerifyDSA` function must be preceded by the call to the `ippsECCBSignDSA` function which computes the digital signature over the message digest *msg* and represents the signature with two numbers: *r* and *s*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCBSet` or `ippsECCBSetStd`.

For more information on digital signatures, please refer to the [\[ANSI\]](#) standard.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <code>pMsgDigest</code> , <code>pSignX</code> , <code>pSignY</code> , or <code>ECC</code> is not valid.
<code>ippStsMessageErr</code>	Indicates an error condition if the value of <code>msg</code> pointed by <code>pMsgDigest</code> falls outside the range of $[1, 1-n]$ where n is the order of the elliptic curve base point G .

ECCBSignNR

Computes the digital signature over a message digest (the Nyberg-Rueppel scheme).

Syntax

```
IppStatus ippseCCBSignNR(const IppsBigNumState* pMsgDigest, IppsBigNumState*  
    pSignX, IppsBigNumState* pSignY, IppsECCBState* pECC);
```

Parameters

<code>pMsgDigest</code>	Pointer to the message digest <code>msg</code> .
<code>pSignX</code>	Pointer to the integer r of the digital signature.
<code>pSignY</code>	Pointer to the integer s of the digital signature.
<code>pECC</code>	Pointer to the context of the elliptic cryptosystem.

Description

The function computes two large numbers r and s which form the digital signature over a message digest `msg`.

The scheme used to compute the digital signature is an elliptic curve analogue of the El-Gamal Digital Signature scheme with the message recovery (the Nyberg-Rueppel signature scheme). The scheme that the given function uses assumes that the following cryptosystem keys are hitherto set up by the message sender:

`regPrivKey` Regular private key.

ephPrivKey Ephemeral private key.

ephPubKey Ephemeral public key.

The keys can be generated and set up by the functions `ippsECCBGenKeyPair` and `ippsECCBSetKeyPair` with only requirement that the key *regPrivKey* be different from the key *ephPrivKey*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCBSet` or `ippsECCBSetStd`.

For more information on digital signatures, please refer to the [\[ANSI\]](#) standard.

Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pMsgDigest</i> , <i>pSignX</i> , <i>pSignY</i> , or <i>ECC</i> is not valid.
<code>ippStsMessageErr</code>	Indicates an error condition if the value of <i>msg</i> pointed by <i>pMsgDigest</i> falls outside the range of $[1, 1-n]$ where n is the order of the elliptic curve base point G .
<code>ippStsRangeErr</code>	Indicates an error condition if one of the parameters pointed by <i>pSignX</i> or <i>pSignY</i> has a less memory size than the order n of the elliptic curve base point G .
<code>ippStsEphemeralKeyErr</code>	Indicates an error condition if the values of the ephemeral keys <i>ephPrivKey</i> and <i>ephPubKey</i> are not valid. (Either $r = 0$ or $s = 0$ is received as a result of the digital signature calculation).

ECCBVerifyNR

Computes authenticity of the digital signature over a message digest (the Nyberg-Rueppel scheme).

Syntax

```
IppsStatus ippsECCBVerifyNR(const IppsBigNumState* pMsgDigest, const
    IppsBigNumState* pSignX, const IppsBigNumState* pSignY,
    IppECResult* pResult, IppsECCBState* pECC);
```


Parameters

<i>pMsgDigest</i>	Pointer to the message digest <i>msg</i> .
<i>pSignX</i>	Pointer to the integer <i>r</i> of the digital signature.
<i>pSignY</i>	Pointer to the integer <i>s</i> of the digital signature.
<i>pResult</i>	Pointer to the digital signature verification result.
<i>pECC</i>	Pointer to the context of the elliptic cryptosystem.

Description

The function verifies authenticity of the digital signature over a message digest *msg*. The signature is presented with two large integers *r* and *s*.

The scheme used to compute the digital signature is an elliptic curve analogue of the El-Gamal Digital Signature scheme with the message recovery (the Nyberg-Rueppel signature scheme). The scheme that the given function uses assumes that the following cryptosystem keys be hitherto set up by the message sender:

regPubKey Message sender's regular private key.

The key can be generated and set up by the function `ippsECCBGenKeyPair`.

The result of the digital signature verification can take one of two possible values:

<code>ippECValid</code>	The digital signature is valid.
<code>ippECInvalidSignature</code>	The digital signature is not valid.

The call to the `ippsECCBVerifyNR` function must be preceded by the call to the `ippsECCBSignNR` function which computes the digital signature over the message digest *msg* and represents the signature with two numbers: *r* and *s*.

The elliptic curve domain parameters must be hitherto defined by one of the functions: `ippsECCBSet` or `ippsECCBSetStd`.

For more information on digital signatures, please refer to the [\[ANSI\]](#) standard.

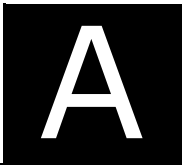
Return Values

<code>ippStsNoErr</code>	Indicates no error. Any other value indicates an error or warning.
<code>ippStsNullPtrErr</code>	Indicates an error condition if any of the specified pointers is NULL.
<code>ippStsContextMatchErr</code>	Indicates an error condition if one of the contexts pointed by <i>pMsgDigest</i> , <i>pSignX</i> , <i>pSignY</i> , or <i>ECC</i> is not valid.

`ippStsMessageErr`

Indicates an error condition if the value of *msg* pointed by *pMsgDigest* falls outside the range of $[1, 1-n]$ where n is the order of the elliptic curve base point G .

Functions Removed from Intel[®] Integrated Performance Primitives 5.0



The table below lists functions removed from Intel[®] Integrated Performance Primitives 5.0 (Intel[®] IPP 5.0) for cryptography and suggests Intel IPP 5.0 functions to be called instead.

Table A-1 Functions Removed from Intel IPP 5.0

Removed Function	Function for Substitution or Workaround
ippsBigNumBufferSize	ippsBigNumGetSize
ippsBlowfishBufferSize	ippsBlowfishGetSize
ippsDAABlowfishBufferSize	ippsDAABlowfishGetSize
ippsDAADESBufferSize	ippsDAADESGetSize
ippsDAARijndael128BufferSize	ippsDAARijndael128GetSize
ippsDAARijndael192BufferSize	ippsDAARijndael192GetSize
ippsDAARijndael256BufferSize	ippsDAARijndael256GetSize
ippsDAATDESBufferSize	ippsDAATDESGetSize
ippsDAATtwofishBufferSize	ippsDAATtwofishGetSize
ippsDESBufferSize	ippsDESGetSize
ippsDESDecrypt	ippsDESDecryptECB(...length==8...)
ippsDESDecrypt_I	ippsDESDecryptECB(...length==8...)
ippsDESEncrypt	ippsDESEncryptECB(...length==8...)
ippsDESEncrypt_I	ippsDESEncryptECB(...length==8...)
ippsDSABufferSizes	ippsDLPGetSize
ippsDSAInit	ippsDLPInit
ippsDSAKeyCheck	ippsDLPValidateDSA
ippsDSAKeyGen	ippsDLPGenerateDSA
ippsDSAKeyGet	ippsDLPGet

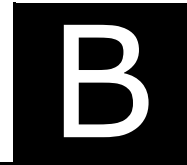
Table A-1 Functions Removed from Intel IPP 5.0 (continued)

Removed Function	Function for Substitution or Workaround
ippsDSAKeySet	ippsDLPSet
ippsDSASign	ippsDLPSignDSA
ippsDSAVerify	ippsDLPVerifyDSA
ippsHMACMD5BufferSize	ippsHMACMD5GetSize
ippsHMACSHA1BufferSize	ippsHMACSHA1GetSize
ippsHMACSHA256BufferSize	ippsHMACSHA256GetSize
ippsHMACSHA384BufferSize	ippsHMACSHA384GetSize
ippsHMACSHA512BufferSize	ippsHMACSHA512GetSize
ippsMD5BufferSize	ippsMD5GetSize
ippsMontBufferSize	ippsMontGetSize
ippsPrimeBufferSize	ippsPrimeGetSize
ippsPRNGAdd	Functionality is not obvious, not needed, removed.
ippsPRNGBufferSize	ippsPRNGGetSize
ippsPRNGGen	ippsPRNGen , ippsPRNGen_BN
ippsPRNGGetRand	ippsPRNGen , ippsPRNGen_BN
ippsPRNGSetPrimeQ	ippsPRNGSetModulus
ippsRijndael128BufferSize	ippsRijndael128GetSize
ippsRijndael192BufferSize	ippsRijndael192GetSize
ippsRijndael256BufferSize	ippsRijndael256GetSize
ippsRSABufferSizes	ippsRSAGetGetSize
ippsRSAKeyCheck	ippsRSAValidate
ippsRSAKeyGen	ippsRSAGenerate
ippsRSAKeyGet	ippsRSAGetKey
ippsRSAKeySet	ippsRSASetKey
ippsSHA1BufferSize	ippsSHA1GetSize
ippsSHA256BufferSize	ippsSHA256GetSize
ippsSHA384BufferSize	ippsSHA384GetSize
ippsSHA512BufferSize	ippsSHA512GetSize
ippsTDESDecrypt	ippsTDESDecryptECB (...length==8...)
ippsTDESDecrypt_I	ippsTDESDecryptECB (...length==8...)
ippsTDESEncrypt	ippsTDESEncryptECB (...length==8...)

Table A-1 Functions Removed from Intel IPP 5.0 (continued)

Removed Function	Function for Substitution or Workaround
<code>ippsTDESEncrypt_I</code>	<code>ippsTDESEncryptECB</code> (...length==8...)
<code>ippsTwofishBufferSize</code>	<code>ippsTwofishGetSize</code>

Support Functions and Classes



This appendix contains miscellaneous information on support functions and classes that may be helpful to users of Intel® Integrated Performance Primitives (Intel® IPP) for cryptography.

The [Version Information Function](#) section describes an Intel IPP function that provides version information for cryptography software.

The [Classes and Functions Used in Examples](#) section presents source code of classes and functions needed for examples given in the manual chapters.

Version Information Function

GetLibVersion

Returns information about the active version of the Intel IPP software for cryptography.

Syntax

```
const IppLibraryVersion* ippcpGetLibVersion(void);
```

Description

The function `ippcpGetLibVersion` is declared in the `ippcp.h` file. This function returns a pointer to a static data structure `IppLibraryVersion` that contains information about the current version of the Intel IPP software for cryptography. There is no need for you to release memory referenced by the returned pointer, as it points to a static variable. The following fields of the `IppLibraryVersion` structure are available:

<i>major</i>	is the major number of the current library version.
<i>minor</i>	is the minor number of the current library version.
<i>majorBuild</i>	is the number of builds for the (<i>major.minor</i>) version.
<i>build</i>	is the total number of IPP builds.
<i>Name</i>	is the name of the current library version.
<i>Version</i>	is the library version string.
<i>BuildDate</i>	is the library version actual build date.

For example, if the library version is "5.2 gold", library name is "ippcppxl.lib", and build date is "Nov 14 06", then the fields in this structure are set as follows:

```
major = 5, minor = 2, Name = "ippcppxl.lib",  
Version = "5.2 gold", BuildDate = "Nov 14 2006".
```

[Example B-1](#) shows how to use the function `ippcpGetLibVersion`.

Example B-1 Using the `ippcpGetLibVersion` Function

```
void libinfo(void) {  
    const IppLibraryVersion* lib = ippcpGetLibVersion();  
    printf("%s %s %d.%d.%d.%d\n", lib->Name, lib->Version,  
        lib->major, lib->minor, lib->majorBuild, lib->build);  
}
```

Output:

```
ippcppxl.lib 5.2 gold 5.2.35.285
```

Classes and Functions Used in Examples

This section presents source code of functions and classes used in [Example 5-8](#) and [Example 5-9](#) given in the [Public Key Cryptography Functions](#) chapter.

BigNumber Class

The section presents source code of the `BigNumber` class.

Declarations

Contents of the header file (`bignum.h`) declaring the `BigNumber` class are presented below:

```
#if !defined _BIGNUMBER_H_
#define _BIGNUMBER_H_

#include "ippcp.h"

#include <iostream>
#include <vector>
#include <iterator>
using namespace std;

class BigNumber
{
public:
    BigNumber(Ipp32u value=0);
    BigNumber(Ipp32s value);
    BigNumber(const IppsBigNumState* pBN);
    BigNumber(const Ipp32u* pData, int length=1, IppsBigNumSGN sgn=IppsBigNumPOS);
    BigNumber(const BigNumber& bn);
    BigNumber(const char *s);
    virtual ~BigNumber();

    // set value
    void Set(const Ipp32u* pData, int length=1, IppsBigNumSGN sgn=IppsBigNumPOS);

    // conversion to IppsBigNumState
    friend IppsBigNumState* BN(const BigNumber& bn) {return bn.m_pBN;}
    operator IppsBigNumState* () const { return m_pBN; }
```



```
// some useful constatns
static const BigNumber& Zero();
static const BigNumber& One();
static const BigNumber& Two();

// arithmetic operators probably need
BigNumber& operator = (const BigNumber& bn);
BigNumber& operator += (const BigNumber& bn);
BigNumber& operator -= (const BigNumber& bn);
BigNumber& operator *= (Ipp32u n);
BigNumber& operator *= (const BigNumber& bn);
BigNumber& operator /= (const BigNumber& bn);
BigNumber& operator %= (const BigNumber& bn);
friend BigNumber operator + (const BigNumber& a, const BigNumber& b);
friend BigNumber operator - (const BigNumber& a, const BigNumber& b);
friend BigNumber operator * (const BigNumber& a, const BigNumber& b);
friend BigNumber operator * (const BigNumber& a, Ipp32u);
friend BigNumber operator % (const BigNumber& a, const BigNumber& b);
friend BigNumber operator / (const BigNumber& a, const BigNumber& b);

// modulo arithmetic
BigNumber Modulo(const BigNumber& a) const;
BigNumber ModAdd(const BigNumber& a, const BigNumber& b) const;
BigNumber ModSub(const BigNumber& a, const BigNumber& b) const;
BigNumber ModMul(const BigNumber& a, const BigNumber& b) const;
BigNumber InverseAdd(const BigNumber& a) const;
BigNumber InverseMul(const BigNumber& a) const;

// comparisons
friend bool operator < (const BigNumber& a, const BigNumber& b);
friend bool operator > (const BigNumber& a, const BigNumber& b);
friend bool operator == (const BigNumber& a, const BigNumber& b);
friend bool operator != (const BigNumber& a, const BigNumber& b);
friend bool operator <= (const BigNumber& a, const BigNumber& b)
{return !(a>b);}
    friend bool operator >= (const BigNumber& a, const BigNumber& b)
{return !(a<b);}

// easy tests
bool IsOdd() const;
bool IsEven() const { return !IsOdd(); }

// size of BigNumber
int MSB() const;
```

```

int LSB() const;
int BitSize() const { return MSB()+1; }
int DwordSize() const { return (BitSize()+31)>>5;}
friend int Bit(const vector<Ipp32u>& v, int n);

// conversion and output
void num2hex( string& s )    const; // convert to hex string
void num2vec( vector<Ipp32u>& v ) const; // convert to 32-bit word vector

friend ostream& operator << (ostream& os,    const BigNumber& a);

protected:
    bool create(const Ipp32u* pData, int length, IppsBigNumSGN
sgn=IppsBigNumPOS);
    int compare(const BigNumber& ) const;

    IppsBigNumState* m_pBN;
};

// convert bit size into 32-bit words
#define BITSIZE_WORD(n) (((n)+31)>>5))

#endif // _BIGNUMBER_H_

```

Definitions

C++ definitions for the `BigNumber` class methods are given below. For the declarations to be included, see the preceding [Declarations](#) section.

```

#include "bignum.h"

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
//
//  BigNumber
//
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

BigNumber::~BigNumber()
{
    delete [] (Ipp8u*)m_pBN;
}

```

```
bool BigNumber::create(const Ipp32u* pData, int length, IppsBigNumSGN
sgn)
{
    int size;
    ippsBigNumGetSize(length, &size);
    m_pBN = (IppsBigNumState*)( new Ipp8u[size] );
    if(!m_pBN)
        return false;
    ippsBigNumInit(length, m_pBN);
    if(pData)
        ippsSet_BN(sgn, length, pData, m_pBN);
    return true;
}

// constructors
//
BigNumber::BigNumber(Ipp32u value)
{
    create(&value, 1, IppsBigNumPOS);
}

BigNumber::BigNumber(Ipp32s value)
{
    Ipp32s avalue = abs(value);
    create((Ipp32u*)&avalue, 1, (value<0)? IppsBigNumNEG : IppsBigNumPOS);
}

BigNumber::BigNumber(const IppsBigNumState* pBN)
{
    int bnLen;
    ippsGetSize_BN(pBN, &bnLen);
    Ipp32u* bnData = new Ipp32u[bnLen];
    IppsBigNumSGN sgn;
    ippsGet_BN(&sgn, &bnLen, bnData, pBN);
    //
    create(bnData, bnLen, sgn);
    //
    delete bnData;
}

BigNumber::BigNumber(const Ipp32u* pData, int length, IppsBigNumSGN sgn)
{
    create(pData, length, sgn);
}
```

```

static
char HexDigitList[] = "0123456789ABCDEF";

BigNumber::BigNumber(const char* s)
{
    bool neg = '-' == s[0];
    if(neg) s++;
    bool hex = ('0'==s[0]) && (('x'==s[1]) || ('X'==s[1]));

    int dataLen;
    Ipp32u base;
    if(hex) {
        s += 2;
        base = 0x10;
        dataLen = (strlen(s) + 7)/8;
    }
    else {
        base = 10;
        dataLen = (strlen(s) + 9)/10;
    }

    create(0, dataLen);
    *(this) = Zero();
    while(*s) {
        char tmp[2] = {s[0],0};
        Ipp32u digit = strchrn(HexDigitList, tmp);
        *this = (*this) * base + BigNumber( digit );
        s++;
    }

    if(neg)
        (*this) = Zero() - (*this);
}

BigNumber::BigNumber(const BigNumber& bn)
{
    IppsBigNumSGN sgn;
    int length;
    ippGetSize_BN(bn.m_pBN, &length);
    Ipp32u* pData = new Ipp32u[length];
    ippGet_BN(&sgn, &length, pData, bn.m_pBN);
    //
    create(pData, length, sgn);
}

```

```
//
delete pData;
}

// set value
//
void BigNumber::Set(const Ipp32u* pData, int length, IppsBigNumSGN sgn)
{
    ippsSet_BN(sgn, length, pData, BN(*this));
}

// constants
//
const BigNumber& BigNumber::Zero()
{
    static const BigNumber zero(0);
    return zero;
}

const BigNumber& BigNumber::One()
{
    static const BigNumber one(1);
    return one;
}

const BigNumber& BigNumber::Two()
{
    static const BigNumber two(2);
    return two;
}

// arithmetic operators
//
BigNumber& BigNumber::operator =(const BigNumber& bn)
{
    if(this != &bn) {        // prevent self copy
        int length;
        ippsGetSize_BN(bn.m_pBN, &length);
        Ipp32u* pData = new Ipp32u[length];
        IppsBigNumSGN sgn;
        ippsGet_BN(&sgn, &length, pData, bn.m_pBN);
    }
}
```

```

        //
        delete (Ipp8u*)m_pBN;
        create(pData, length, sgn);
        //
        delete pData;
    }
    return *this;
}

BigNumber& BigNumber::operator += (const BigNumber& bn)
{
    int aLength;
    ippsGetSize_BN(BN(*this), &aLength);
    int bLength;
    ippsGetSize_BN(BN(bn), &bLength);
    int rLength = IPP_MAX(aLength,bLength) + 1;

    BigNumber result(0, rLength);
    ippsAdd_BN(BN(*this), BN(bn), BN(result));
    *this = result;
    return *this;
}

BigNumber& BigNumber::operator -= (const BigNumber& bn)
{
    int aLength;
    ippsGetSize_BN(BN(*this), &aLength);
    int bLength;
    ippsGetSize_BN(BN(bn), &bLength);
    int rLength = IPP_MAX(aLength,bLength);

    BigNumber result(0, rLength);
    ippsSub_BN(BN(*this), BN(bn), BN(result));
    *this = result;
    return *this;
}

BigNumber& BigNumber::operator *= (const BigNumber& bn)
{
    int aLength;
    ippsGetSize_BN(BN(*this), &aLength);
    int bLength;
    ippsGetSize_BN(BN(bn), &bLength);
    int rLength = aLength+bLength;

```

```
        BigNumber result(0, rLength);
        ippsMul_BN(BN(*this), BN(bn), BN(result));
        *this = result;
        return *this;
    }

BigNumber& BigNumber::operator *= (Ipp32u n)
{
    int aLength;
    ippsGetSize_BN(BN(*this), &aLength);
    BigNumber bn(n);

    BigNumber result(0, aLength+1);
    ippsMul_BN(BN(*this), BN(bn), BN(result));
    *this = result;
    return *this;
}

BigNumber& BigNumber::operator %= (const BigNumber& bn)
{
    BigNumber remainder(bn);
    ippsMod_BN(BN(*this), BN(bn), BN(remainder));
    *this = remainder;
    return *this;
}

BigNumber& BigNumber::operator /= (const BigNumber& bn)
{
    BigNumber quotient(*this);
    BigNumber remainder(bn);
    ippsDiv_BN(BN(*this), BN(bn), BN(quotient), BN(remainder));
    *this = quotient;
    return *this;
}

BigNumber operator + (const BigNumber& a, const BigNumber& b )
{
    BigNumber r(a);
    return r += b;
}

BigNumber operator - (const BigNumber& a, const BigNumber& b )
{

```

```
    BigNumber r(a);
    return r -= b;
}

BigNumber operator * (const BigNumber& a, const BigNumber& b )
{
    BigNumber r(a);
    return r *= b;
}

BigNumber operator * (const BigNumber& a, Ipp32u n)
{
    BigNumber r(a);
    return r *= n;
}

BigNumber operator / (const BigNumber& a, const BigNumber& b )
{
    BigNumber q(a);
    return q /= b;
}

BigNumber operator % (const BigNumber& a, const BigNumber& b )
{
    BigNumber r(b);
    ippsMod_BN(BN(a), BN(b), BN(r));
    return r;
}

// modulo arithmetic
//
BigNumber BigNumber::Modulo(const BigNumber& a) const
{
    return a % *this;
}

BigNumber BigNumber::InverseAdd(const BigNumber& a) const
{
    BigNumber t = Modulo(a);
    if(t==BigNumber::Zero())
        return t;
    else
        return *this - t;
}
```



```
    }

    BigNumber BigNumber::InverseMul(const BigNumber& a) const
    {
        BigNumber r(*this);
        ippsModInv_BN(BN(a), BN(*this), BN(r));
        return r;
    }

    BigNumber BigNumber::ModAdd(const BigNumber& a, const BigNumber& b) const
    {
        BigNumber r = this->Modulo(a+b);
        return r;
    }

    BigNumber BigNumber::ModSub(const BigNumber& a, const BigNumber& b) const
    {
        BigNumber r = this->Modulo(a + this->InverseAdd(b));
        return r;
    }

    BigNumber BigNumber::ModMul(const BigNumber& a, const BigNumber& b) const
    {
        BigNumber r = this->Modulo(a*b);
        return r;
    }

    // comparison
    //
    int BigNumber::compare(const BigNumber &bn) const
    {
        Ipp32u result;
        BigNumber tmp = *this - bn;
        ippsCmpZero_BN(BN(tmp), &result);
        return (result==IS_ZERO)? 0 : (result==GREATER_THAN_ZERO)? 1 : -1;
    }

    bool operator < (const BigNumber &a, const BigNumber &b) { return
    a.compare(b) < 0; }
    bool operator > (const BigNumber &a, const BigNumber &b) { return
    a.compare(b) > 0; }
    bool operator == (const BigNumber &a, const BigNumber &b) { return 0 ==
    a.compare(b); }
```

```
bool operator != (const BigNumber &a, const BigNumber &b) { return 0 !=
a.compare(b); }

// easy tests
//
bool BigNumber::IsOdd() const
{
    vector<Ipp32u> v;
    num2vec(v);
    return v[0]&1;
}

// size of BigNumber
//
int BigNumber::LSB() const
{
    if( *this == BigNumber::Zero() )
        return 0;

    vector<Ipp32u> v;
    num2vec(v);

    int lsb = 0;
    vector<Ipp32u>::iterator i;
    for(i=v.begin(); i!=v.end(); i++) {
        Ipp32u x = *i;
        if(0==x)
            lsb += 32;
        else {
            while(0==(x&1)) {
                lsb++;
                x >>= 1;
            }
            break;
        }
    }
    return lsb;
}

int BigNumber::MSB() const
{
    if( *this == BigNumber::Zero() )
        return 0;
```

```
vector<Ipp32u> v;
num2vec(v);

int msb = v.size()*32 -1;
vector<Ipp32u>::reverse_iterator i;
for(i=v.rbegin(); i!=v.rend(); i++) {
    Ipp32u x = *i;
    if(0==x)
        msb -=32;
    else {
        while(!(x&0x80000000)) {
            msb--;
            x <<= 1;
        }
        break;
    }
}
return msb;
}

int Bit(const vector<Ipp32u>& v, int n)
{
    return 0 != ( v[n>>5] & (1<<(n&0x1F)) );
}

// conversions and output
//
void BigNumber::num2vec( vector<Ipp32u>& v ) const
{
    int length;
    ippsGetSize_BN(BN(*this), &length);
    Ipp32u* pData = new Ipp32u[length];
    IppsBigNumSGN sgn;
    ippsGet_BN(&sgn, &length, pData, BN(*this));
    //
    for(int n=0; n<length; n++)
        v.push_back( pData[n] );
    //
    delete pData;
}
```

```

void BigNumber::num2hex( string& s ) const
{
    int length;
    ippGetSize_BN(BN(*this), &length);
    Ipp32u* pData = new Ipp32u[length];
    IppsBigNumSGN sgn;
    ippGet_BN(&sgn, &length, pData, BN(*this));

    s.append(1, (sgn==IppsBigNumNEG)? '-' : ' ');
    s.append(1, '0');
    s.append(1, 'x');
    for(int n=length; n>0; n--) {
        Ipp32u x = pData[n-1];
        for(int nd=8; nd>0; nd--) {
            char c = HexDigitList[(x>>(nd-1)*4)&0xF];
            s.append(1, c);
        }
    }
    delete pData;
}

ostream& operator << ( ostream &os, const BigNumber& a) {
    string s;
    a.num2hex(s);
    os << s.c_str();
    return os;
}

```

Functions for Creation of Cryptographic Contexts

The section presents source code for creation of some cryptographic contexts.

Declarations

Contents of the header file (cpobjs.h) declaring functions for creation of some cryptographic contexts are presented below:

```

#if !defined _CPOBJS_H_
#define _CPOBJS_H_

//
// create new of some ippCP 'objects'
//

```

```
#include "ippcp.h"
#include <stdlib.h>

#define BITS_2_WORDS(n) (((n)+31)>>5)
int Bitsize2Wordsize(int nBits);

Ipp32u* rand32(Ipp32u* pX, int size);

IppsBigNumState* newBN(int len, const Ipp32u* pData=0);
IppsBigNumState* newRandBN(int len);
void deleteBN(IppsBigNumState* pBN);

IppsPRNGState* newPRNG(int seedBitsize=160);
void deletePRNG(IppsPRNGState* pPRNG);

IppsRSAState* newRSA(int lenN, int lenP, IppRSAKeyType type);
void deleteRSA(IppsRSAState* pRSA);

IppsDLPState* newDLP(int lenM, int lenL);
void deleteDLP(IppsDLPState* pDLP);

#endif // _CPOBJS_H_
```

Definitions

C++ definitions of functions creating cryptographic contexts are given below. For the declarations to be included, see the preceding [Declarations](#) section.

```
#include "cpobjjs.h"

// convert bitsize into 32-bit wordsize
int Bitsize2Wordsize(int nBits)
{ return (nBits+31)>>5; }

// new BN number
IppsBigNumState* newBN(int len, const Ipp32u* pData)
{
    int size;
    ippBigNumGetSize(len, &size);
    IppsBigNumState* pBN = (IppsBigNumState*)( new Ipp8u [size] );
    ippBigNumInit(len, pBN);
    if(pData)
```

```

        ippsSet_BN(IppsBigNumPOS, len, pData, pBN);
    return pBN;
}

// desrtoy BN
void deleteBN(IppsBigNumState* pBN)
{ delete [] (Ipp8u*)pBN; }

// set up array of 32-bit items random
Ipp32u* rand32(Ipp32u* pX, int size)
{
    for(int n=0; n<size; n++)
        pX[n] = (rand()<<16) + rand();
    return pX;
}

IppsBigNumState* newRandBN(int len)
{
    Ipp32u* pBuffer = new Ipp32u [len];
    IppsBigNumState* pBN = newBN(len, rand32(pBuffer, len));
    delete [] pBuffer;
    return pBN;
}

//
// 'external' PRNG
//
IppsPRNGState* newPRNG(int seedBitsize)
{
    int seedSize = Bitsize2Wordsize(seedBitsize);
    Ipp32u* seed = new Ipp32u [seedSize];
    Ipp32u* augm = new Ipp32u [seedSize];

    int size;
    IppsBigNumState* pTmp;
    ippsPRNGGetSize(&size);
    IppsPRNGState* pCtx = (IppsPRNGState*)( new Ipp8u [size] );
    ippsPRNGInit(seedBitsize, pCtx);

    ippsPRNGSetSeed(pTmp=newBN(seedSize, rand32(seed, seedSize)), pCtx);
    delete [] (Ipp8u*)pTmp;
    ippsPRNGSetAugment(pTmp=newBN(seedSize, rand32(augm, seedSize)), pCtx);
}

```

```
        delete [] (Ipp8u*)pTmp;

        delete [] seed;
        delete [] augm;
        return pCtx;
    }
void deletePRNG(IppsPRNGState* pPRNG)
{ delete [] (Ipp8u*)pPRNG; }

//
// RSA context
//
IppsRSAState* newRSA(int lenN, int lenP, IppRSAKeyType type)
{
    int size;
    ippsRSAGetSize(lenN, lenP, type, &size);
    IppsRSAState* pCtx = (IppsRSAState*)( new Ipp8u [size] );
    ippsRSAInit(lenN, lenP, type, pCtx);

    return pCtx;
}
void deleteRSA(IppsRSAState* pRSA)
{ delete [] (Ipp8u*)pRSA; }

//
// DLP context
//
IppsDLPState* newDLP(int lenM, int lenL)
{
    int size;
    ippsDLPGetSize(lenM, lenL, &size);
    IppsDLPState *pCtx= (IppsDLPState *)new Ipp8u[ size ];
    ippsDLPInit(lenM, lenL, pCtx);

    return pCtx;
}
void deleteDLP(IppsDLPState* pDLP)
{ delete [] (Ipp8u*)pDLP; }
```

Bibliography

This bibliography provides a list of publications that might be helpful to you in using cryptography functions of Intel IPP.

- [AC] B. Schneier. *Applied Cryptography. Protocols, Algorithms, and Source Code in C. Second Edition*. John Wiley & Sons, Inc., 1996.
- [AES] J. Daemon and V. Rijmen. *The Rijndael Block Cipher. AES Proposal*. Available from <http://www.nist.gov/aes>.
- [ANSI] *ANSI X9.62-1998 Public Key Cryptography for the Financial Services Industry: the Elliptic Curve Digital Signature Algorithm (ECDSA)*. American Bankers Association, 1999.
- [BF] B. Schneier. *Description of a New Variable-Length Key, 64-bit Block Cipher (Blowfish)*. Available from <http://www.schneier.com/blowfish.html>.
- [FIPS PUB 46-3] *Federal Information Processing Standards Publications, FIPS PUB 46-3*. Data Encryption Standard (DES), October 1999. Available from <http://csrc.nist.gov/publications/fips>.
- [FIPS PUB 113] *Federal Information Processing Standards Publications, FIPS PUB 113*. Computer Data Authentication, May 1985. Available from <http://csrc.nist.gov/publications/fips>.
- [FIPS PUB 180-2] *Federal Information Processing Standards Publications, FIPS PUB 180-2*. Secure Hash Standard, August 2002. Available from <http://csrc.nist.gov/publications/fips>.
- [FIPS PUB 186-2] *Federal Information Processing Standards Publications, FIPS PUB 186-2*. Digital Signature Standard (DSS), January 2000. Available from <http://csrc.nist.gov/publications/fips>.
- [FIPS PUB 198] *Federal Information Processing Standards Publications, FIPS PUB 198*. The Key-Hash Message Authentication Code (HMAC), March 2002. Available from <http://csrc.nist.gov/publications/fips>.

- [IEEE P1363A] *Standard Specifications for Public-Key Cryptography: Additional Techniques*. May, 2000. Working Draft.
- [NIST SP 800-38A] *Recommendation for Block Cipher Modes of Operation - Methods and Techniques*. NIST Special Publication 800-38A, December 2001. Available from <http://csrc.nist.gov/publications/nistpubs/800-38a/sp800-38a.pdf>.
- [NIST SP 800-38B] *Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication*. NIST Special Publication 800-38B, May 2005. Available from http://csrc.nist.gov/publications/nistpubs/800-38B/SP_800-38B.pdf
- [NIST SP 800-38C] *Draft Recommendation for Block Cipher Modes of Operation: The CCM Mode for Authentication and Confidentiality*. NIST Special Publication 800-38C, September 2003. Available from <http://csrc.nist.gov/publications/nistpubs/800-38C/SP800-38C.pdf>
- [PKCS 1.2.1] *RSA Laboratories. PKCS #1 v2.1: RSA Cryptography Standard*, June 2002. Available from <http://www.rsasecurity.com/rsalabs/pkcs>.
- [PKCS 7] *RSA Laboratories. PKCS #7: Cryptographic Message Syntax Standard*, An RSA Laboratories Technical Note Version 1.5 Revised, November 1, 1993.
- [RC5] R. Rivest, "The RC5 Encryption Algorithm", Proceedings of the 1994 Leuven Workshop on Algorithms (Springer), 1994. Revised version, dated March 1997, is available from <http://theory.lcs.mit.edu/~cis/pubs/rivest/rc5rev.ps>.
- [RFC 1321] R. Rivest, "The MD5 Message-Digest Algorithm", RFC 1321, MIT and RSA Data Security, Inc, April 1992. Available from <http://www.faqs.org/rfc1321.html>.
- [RFC2401] H.Krawczyk, M. Bellare and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication", RFC 2401, February 1997. Available from <http://www.faqs.org/rfcs/rfc2401.html>.
- [SEC1] *SEC1: Elliptic Curve Cryptography*. Standards for Efficient Cryptography Group, September 2000. Available from http://www.secg.org/secg_docs.htm.

-
- [SEC2] *SEC2: Recommended Elliptic Curve Domain Parameters*. Standards for Efficient Cryptography Group, September 2000. Available from http://www.secg.org/secg_docs.htm/.
- [TF] B. Schneier, J. Kelsey, D. Whiting, D. Wagner, C. Hall and N.Ferguson. *Twofish: A 128-Bit Block Cipher*. Available from <http://www.counterpane.com/twofish.html>.
- [X9.42] *X9.42-2003: Public Key Cryptography for the Financial Services Industry: Agreement of Symmetric Keys Using Discrete Logarithm Cryptography*. American National Standards Institute, 2003.

Index

A

- about this manual, 1-5
- ARCFour Functions, 2-127
 - ARCFourCheckKey, 2-128
 - ARCFourDecrypt, 2-131
 - ARCFourEncrypt, 2-130
 - ARCFourGetSize, 2-128
 - ARCFourInit, 2-129
 - ARCFourReset, 2-132
- ARCFour stream cipher, 2-127
- audience for this manual, 1-6

B

- Big Number Arithmetic, 5-1
- Big Number Arithmetic Functions
 - Add_BN, 5-22
 - Add_BNU, 5-3
 - BigNumGetSize, 5-13
 - BigNumInit, 5-13
 - Cmp_BN, 5-20
 - CmpZero_BN, 5-21
 - Div_64u32u, 5-8
 - Div_BN, 5-27
 - Gcd_BN, 5-29
 - Get_BN, 5-18
 - GetOctString_BN, 5-19
 - GetOctString_BNU, 5-12
 - GetSize_BN, 5-17, 5-18
 - MAC_BN_I, 5-26
 - MACOne_BNU_I, 5-6
 - Mod_BN, 5-28
 - ModInv_BN, 5-30
 - Mul_BN, 5-25
 - Mul_BNU4, 5-7

- Mul_BNU8, 5-7
- MulOne_BNU, 5-5
- Set_BN, 5-14
- SetOctString_BN, 5-15
- SetOctString_BNU, 5-11
- Sqr_32u64u, 5-9
- Sqr_BNU4, 5-10
- Sqr_BNU8, 5-10
- Sub_BN, 5-24
- Sub_BNU, 5-4

- block cipher modes, 2-1
 - see also* symmetric algorithm modes

- Blowfish Functions, 2-73
 - BlowfishDecryptCBC, 2-78
 - BlowfishDecryptCFB, 2-80
 - BlowfishDecryptCTR, 2-84
 - BlowfishDecryptECB, 2-76
 - BlowfishDecryptOFB, 2-82
 - BlowfishEncryptCBC, 2-77
 - BlowfishEncryptCFB, 2-79
 - BlowfishEncryptCTR, 2-83
 - BlowfishEncryptECB, 2-75
 - BlowfishEncryptOFB, 2-81
 - BlowfishGetSize, 2-74
 - BlowfishInit, 2-75

C

- CMAC, 4-1
- CMAC Functions, 4-3
 - CMACRijndael128Final, 4-37
 - CMACRijndael128GetSize, 4-34
 - CMACRijndael128Init, 4-35
 - CMACRijndael128MessageDigest, 4-37
 - CMACRijndael128Update, 4-36
- concepts of IPP, 1-1

cross-architecture alignment, 1-3
cross-platform applications, 1-2

D

DAARijndael192MessageDigest, 4-55
Data Authentication Functions, 4-39
 DAA Rijndael Functions, 4-48
 DAABlowfishFinal, 4-62
 DAABlowfishGetSize, 4-60
 DAABlowfishInit, 4-61
 DAABlowfishMessageDigest, 4-63
 DAABlowfishUpdate, 4-61
 DAADESFinal, 4-43
 DAADESGetSize, 4-41
 DAADESInit, 4-41
 DAADESMessageDigest, 4-44
 DAADESUpdate, 4-42
 DAARijndael128Final, 4-51
 DAARijndael128GetSize, 4-48
 DAARijndael128Init, 4-49
 DAARijndael128MessageDigest, 4-51
 DAARijndael128Update, 4-50
 DAARijndael192Final, 4-55
 DAARijndael192GetSize, 4-52
 DAARijndael192Init, 4-53
 DAARijndael192MessageDigest, 4-55
 DAARijndael192Update, 4-54
 DAARijndael256Final, 4-59
 DAARijndael256GetSize, 4-56
 DAARijndael256Init, 4-57
 DAARijndael256MessageDigest, 4-59
 DAARijndael256Update, 4-58
 DAATDESFinal, 4-47
 DAATDESGetSize, 4-44
 DAATDESInit, 4-45
 DAATDESMessageDigest, 4-47
 DAATDESUpdate, 4-46
 DAATwofishFinal, 4-66
 DAATwofishGetSize, 4-64
 DAATwofishInit, 4-64
 DAATwofishMessageDigest, 4-67
 DAATwofishUpdate, 4-65
Data Encryption Standard (DES), 2-3
DES/TDES Functions, 2-3
 DESDecryptCBC, 2-9
 DESDecryptCFB, 2-11
 DESDecryptCTR, 2-15
 DESDecryptECB, 2-7
 DESDecryptOFB, 2-13

DESEncryptCBC, 2-8
DESEncryptCFB, 2-10
DESEncryptCTR, 2-14
DESEncryptECB, 2-6
DESEncryptOFB, 2-12
DESGetSize, 2-5
DESInit, 2-6
TDESDecryptCBC, 2-19
TDESDecryptCFB, 2-21
TDESDecryptCTR, 2-26
TDESDecryptECB, 2-17
TDESDecryptOFB, 2-24
TDESEncryptCBC, 2-18
TDESEncryptCFB, 2-20
TDESEncryptCTR, 2-25
TDESEncryptECB, 2-16
TDESEncryptOFB, 2-23

Discrete Logarithm Based Functions

DLPGenerateDH, 5-120
DLPGenerateDSA, 5-112
DLPGenKeyPair, 5-108
DLPGet, 5-105
DLPGetDP, 5-107
DLPGetSize, 5-103
DLPInit, 5-103
DLPPublicKey, 5-109
DLPSet, 5-104
DLPSetDP, 5-106
DLPSetKeyPair, 5-111
DLPSharedSecretDH, 5-122
DLPSignDSA, 5-114
DLPValidateDH, 5-121
DLPValidateDSA, 5-113
DLPValidateKeyPair, 5-110
DLPVerifyDSA, 5-115

E

Elliptic Curve Cryptographic Functions

ECCBAddPoint, 5-173
ECCBCheckPoint, 5-170
ECCBComparePoint, 5-171
ECCBGenKeyPair, 5-175
ECCBGet, 5-162
ECCBGetOrderBitSize, 5-163
ECCBGetPoint, 5-169
ECCBGetSize, 5-158
ECCBInit, 5-158
ECCBMulPointScalar, 5-174
ECCBNegativePoint, 5-172

ECCBPointGetSize, 5-166
 ECCBPointInit, 5-166
 ECCBPublicKey, 5-176
 ECCBSet, 5-159
 ECCBSetKeyPair, 5-178
 ECCBSetPoint, 5-167
 ECCBSetPointAtInfinity, 5-168
 ECCBSetStd, 5-161
 ECCBSharedSecretDH, 5-179
 ECCBSharedSecretDHC, 5-181
 ECCBSignDSA, 5-182
 ECCBSignNR, 5-185
 ECCBValidate, 5-164
 ECCBValidateKeyPair, 5-177
 ECCBVerifyDSA, 5-184
 ECCBVerifyNR, 5-186
 ECCPAddPoint, 5-142
 ECCPCheckPoint, 5-139
 ECCPComparePoint, 5-140
 ECCPGenKeyPair, 5-144
 ECCPGet, 5-131
 ECCPGetOrderBitSize, 5-132
 ECCPGetPoint, 5-138
 ECCPGetSize, 5-127
 ECCPInit, 5-128
 ECCPMulPointScalar, 5-143
 ECCPNegativePoint, 5-141
 ECCPPointGetSize, 5-135
 ECCPPointInit, 5-135
 ECCPPublicKey, 5-145
 ECCPSet, 5-128
 ECCPSetKeyPair, 5-147
 ECCPSetPoint, 5-136
 ECCPSetPointAtInfinity, 5-137
 ECCPSetStd, 5-130
 ECCPSharedSecretDH, 5-148
 ECCPSharedSecretDHC, 5-150
 ECCPSignDSA, 5-151
 ECCPSignNR, 5-154
 ECCPValidate, 5-133
 ECCPValidateKeyPair, 5-146
 ECCPVerifyDSA, 5-153
 ECCPVerifyNR, 5-156
 encryption, decryption, and encryption (E-D-E)
 sequence, 2-3

F

font conventions, 1-7
 function descriptions, in manual sections, 1-6

G

GetLibVersion, B-1

H

hardware and software requirements, 1-2
 hash function, 3-1
 Hash Functions for Non-Streaming Messages, 3-25
 general definition, 3-26
 MD5MessageDigest, 3-26
 SHA1MessageDigest, 3-27
 SHA224MessageDigest, 3-29
 SHA256MessageDigest, 3-30
 SHA384MessageDigest, 3-31
 SHA512MessageDigest, 3-32
 user-implemented, 3-25
 HMAC, 4-1

I

initialization vector iv, 2-4
 Intel Performance Library Suite, 1-1
 IPP software, 1-2
 ippcpGetLibVersion, B-1
 ippsAdd_BN, 5-22
 ippsAdd_BNU, 5-3
 ippsARCFive128DecryptCBC, 2-120
 ippsARCFive128DecryptCFB, 2-122
 ippsARCFive128DecryptCTR, 2-126
 ippsARCFive128DecryptECB, 2-118
 ippsARCFive128DecryptOFB, 2-124
 ippsARCFive128EncryptCBC, 2-119
 ippsARCFive128EncryptCFB, 2-121
 ippsARCFive128EncryptCTR, 2-125
 ippsARCFive128EncryptECB, 2-117
 ippsARCFive128EncryptOFB, 2-123
 ippsARCFive128GetSize, 2-116
 ippsARCFive128Init, 2-116
 ippsARCFive64DecryptCBC, 2-109
 ippsARCFive64DecryptCFB, 2-111
 ippsARCFive64DecryptCTR, 2-115
 ippsARCFive64DecryptECB, 2-107
 ippsARCFive64DecryptOFB, 2-113
 ippsARCFive64EncryptCBC, 2-108
 ippsARCFive64EncryptCFB, 2-110
 ippsARCFive64EncryptCTR, 2-114
 ippsARCFive64EncryptECB, 2-106

ippsARCFive64EncryptOFB, 2-112	ippsDAARijndael192GetSize, 4-52
ippsARCFive64GetSize, 2-104	ippsDAARijndael192MessageDigest, 4-55
ippsARCFive64Init, 2-105	ippsDAARijndael192Update, 4-54
ippsARCFourCheckKey, 2-128	ippsDAARijndael256Final, 4-59
ippsARCFourDecrypt, 2-131	ippsDAARijndael256GetSize, 4-56
ippsARCFourEncrypt, 2-130	ippsDAARijndael256MessageDigest, 4-59
ippsARCFourGetSize, 2-128	ippsDAARijndael256Update, 4-58
ippsARCFourInit, 2-129	ippsDAATDESFinal, 4-47
ippsARCFourReset, 2-132	ippsDAATDESGetSize, 4-44
ippsBigNumGetSize, 5-13	ippsDAATDESInit, 4-45
ippsBigNumInit, 5-13	ippsDAATDESMessageDigest, 4-47
ippsBlowfishDecryptCBC, 2-78	ippsDAATDESUpdate, 4-46
ippsBlowfishDecryptCFB, 2-80	ippsDAATwofishFinal, 4-66
ippsBlowfishDecryptCTR, 2-84	ippsDAATwofishGetSize, 4-64
ippsBlowfishDecryptECB, 2-76	ippsDAATwofishInit, 4-64
ippsBlowfishDecryptOFB, 2-82	ippsDAATwofishMessageDigest, 4-67
ippsBlowfishEncryptCBC, 2-77	ippsDAATwofishUpdate, 4-65
ippsBlowfishEncryptCFB, 2-79	ippsDESDecryptCBC, 2-9
ippsBlowfishEncryptCTR, 2-83	ippsDESDecryptCFB, 2-11
ippsBlowfishEncryptECB, 2-75	ippsDESDecryptCTR, 2-15
ippsBlowfishEncryptOFB, 2-81	ippsDESDecryptECB, 2-7
ippsBlowfishGetSize, 2-74	ippsDESDecryptOFB, 2-13
ippsBlowfishInit, 2-75	ippsDESEncryptCBC, 2-8
ippsCMACRijndael128Final, 4-37	ippsDESEncryptCFB, 2-10
ippsCMACRijndael128GetSize, 4-34	ippsDESEncryptCTR, 2-14
ippsCMACRijndael128Init, 4-35	ippsDESEncryptECB, 2-6
ippsCMACRijndael128MessageDigest, 4-37	ippsDESEncryptOFB, 2-12
ippsCMACRijndael128Update, 4-36	ippsDESGetSize, 2-5
ippsCmp_BN, 5-20	ippsDESInit, 2-6
ippsCmpZero_BN, 5-21	ippsDiv_64u32u, 5-8
ippsDAABlowfishFinal, 4-62	ippsDiv_BN, 5-27
ippsDAABlowfishGetSize, 4-60	IppsDLPGenerateDH, 5-120
ippsDAABlowfishInit, 4-61	ippsDLPGenerateDSA, 5-112
ippsDAABlowfishMessageDigest, 4-63	ippsDLPGenKeyPair, 5-108
ippsDAABlowfishUpdate, 4-61	ippsDLPGet, 5-105
ippsDAADESFinal, 4-43	ippsDLPGetDP, 5-107
ippsDAADESGetSize, 4-41	ippsDLPGetSize, 5-103
ippsDAADESInit, 4-41	IppsDLPInit, 5-103
ippsDAADESMessageDigest, 4-44	ippsDLPPublicKey, 5-109
ippsDAADESUpdate, 4-42	ippsDLPSet, 5-104
ippsDAARijndael128Final, 4-51	ippsDLPSetDP, 5-106
ippsDAARijndael128GetSize, 4-48	ippsDLPSetKeyPair, 5-111
ippsDAARijndael128Init, 4-49	ippsDLPSharedSecretDH, 5-122
ippsDAARijndael128MessageDigest, 4-51	ippsDLPSignDSA, 5-114
ippsDAARijndael128Update, 4-50	ippsDLPValidateDH, 5-121
ippsDAARijndael192Final, 4-55	ippsDLPValidateDSA, 5-113

ippsDLPValidateKeyPair, 5-110
ippsDLPVerifyDSA, 5-115
ippsECCBAddPoint, 5-173
ippsECCBCheckPoint, 5-170
ippsECCBComparePoint, 5-171
ippsECCBGenKeyPair, 5-175
ippsECCBGet, 5-162
ippsECCBGetOrderBitSize, 5-163
ippsECCBGetPoint, 5-169
ippsECCBGetSize, 5-158
ippsECCBInit, 5-158
ippsECCBMulPointScalar, 5-174
ippsECCBNegativePoint, 5-172
ippsECCBPointGetSize, 5-166
ippsECCBPointInit, 5-166
ippsECCBPublicKey, 5-176
ippsECCBSet, 5-159
ippsECCBSetKeyPair, 5-178
ippsECCBSetPoint, 5-167
ippsECCBSetPointAtInfinity, 5-168
ippsECCBSetStd, 5-161
ippsECCBSharedSecretDH, 5-179
ippsECCBSharedSecretDHC, 5-181
ippsECCBSignDSA, 5-182
ippsECCBSignNR, 5-185
ippsECCBValidate, 5-164
ippsECCBValidateKeyPair, 5-177
ippsECCBVerifyDSA, 5-184
ippsECCBVerifyNR, 5-186
ippsECCPAddPoint, 5-142
ippsECCPCheckPoint, 5-139
ippsECCPComparePoint, 5-140
ippsECCPGenKeyPair, 5-144
ippsECCPGet, 5-131
ippsECCPGetOrderBitSize, 5-132
ippsECCPGetPoint, 5-138
ippsECCPGetSize, 5-127
ippsECCPInit, 5-128
ippsECCPMulPointScalar, 5-143
ippsECCPNegativePoint, 5-141
ippsECCPPointGetSize, 5-135
ippsECCPPointInit, 5-135
ippsECCPPublicKey, 5-145
ippsECCPSet, 5-128
ippsECCPSetKeyPair, 5-147
ippsECCPSetPoint, 5-136
ippsECCPSetPointAtInfinity, 5-137
ippsECCPSetStd, 5-130
ippsECCPSharedSecretDH, 5-148
ippsECCPSharedSecretDHC, 5-150
ippsECCPSignDSA, 5-151
ippsECCPSignNR, 5-154
ippsECCPValidate, 5-133
ippsECCPValidateKeyPair, 5-146
ippsECCPVerifyDSA, 5-153
ippsECCPVerifyNR, 5-156
ippsGcd_BN, 5-29
ippsGet_BN, 5-18
ippsGetOctString_BN, 5-19
ippsGetOctString_BNU, 5-12
ippsGetSize_BN, 5-17
ippsHMACMD5Duplicate, 4-30
ippsHMACMD5Final, 4-32
ippsHMACMD5GetSize, 4-29
ippsHMACMD5Init, 4-29
ippsHMACMD5MessageDigest, 4-32
ippsHMACMD5Update, 4-31
ippsHMACSHA1Duplicate, 4-5
ippsHMACSHA1Final, 4-7
ippsHMACSHA1GetSize, 4-4
ippsHMACSHA1Init, 4-5
ippsHMACSHA1MessageDigest, 4-8
ippsHMACSHA1Update, 4-6
ippsHMACSHA224Final, 4-12
ippsHMACSHA224GetSize, 4-9
ippsHMACSHA224Init, 4-9
ippsHMACSHA224MessageDigest, 4-12
ippsHMACSHA224Update, 4-11
ippsHMACSHA256Duplicate, 4-15
ippsHMACSHA256Final, 4-16
ippsHMACSHA256Init, 4-14
ippsHMACSHA256MessageDigest, 4-17
ippsHMACSHA256Update, 4-15
ippsHMACSHA384Duplicate, 4-21
ippsHMACSHA384Final, 4-23
ippsHMACSHA384GetSize, 4-20
ippsHMACSHA384Init, 4-20
ippsHMACSHA384MessageDigest, 4-23
ippsHMACSHA384Update, 4-22
ippsHMACSHA512Duplicate, 4-26
ippsHMACSHA512Final, 4-27
ippsHMACSHA512GetSize, 4-24

ippsHMACSHA512Init, 4-25	ippsRijndael128DecryptCBC, 2-36
ippsHMACSHA512MessageDigest, 4-28	ippsRijndael128DecryptCCM, 2-44
ippsHMACSHA512Update, 4-26	ippsRijndael128DecryptCCM_u8, 2-47
ippsMAC_BN_I, 5-26	ippsRijndael128DecryptCFB, 2-38
ippsMACOne_BNU_I, 5-6	ippsRijndael128DecryptCTR, 2-42
ippsMD5Duplicate, 3-6	ippsRijndael128DecryptECB, 2-34
ippsMD5Final, 3-7	ippsRijndael128DecryptOFB, 2-40
ippsMD5GetSize, 3-4	ippsRijndael128EncryptCBC, 2-35
ippsMD5Init, 3-5	ippsRijndael128EncryptCCM, 2-43
ippsMD5MessageDigest, 3-26	ippsRijndael128EncryptCCM_u8, 2-45
ippsMD5Update, 3-6	ippsRijndael128EncryptCFB, 2-37
ippsMGF_MD5, 3-34	ippsRijndael128EncryptCTR, 2-41
ippsMGF_SHA1, 3-34	ippsRijndael128EncryptECB, 2-33
ippsMGF_SHA224, 3-35	ippsRijndael128EncryptOFB, 2-39
ippsMGF_SHA256, 3-36	ippsRijndael128GetSize, 2-32
ippsMGF_SHA384, 3-37	ippsRijndael128Init, 2-33
ippsMGF_SHA512, 3-37	ippsRijndael192DecryptCBC, 2-52
ippsMod_BN, 5-28	ippsRijndael192DecryptCFB, 2-54
ippsModInv_BN, 5-30	ippsRijndael192DecryptCTR, 2-58
ippsMontExp, 5-40	ippsRijndael192DecryptECB, 2-50
ippsMontForm, 5-36	ippsRijndael192DecryptOFB, 2-56
ippsMontGet, 5-36	ippsRijndael192EncryptCBC, 2-51
ippsMontGetSize, 5-33	ippsRijndael192EncryptCFB, 2-53
ippsMontInit, 5-34	ippsRijndael192EncryptCTR, 2-57
ippsMontMul, 5-37	ippsRijndael192EncryptECB, 2-49
ippsMontSet, 5-35	ippsRijndael192EncryptOFB, 2-55
ippsMul_BN, 5-25	ippsRijndael192GetSize, 2-48
ippsMul_BNU4, 5-7	ippsRijndael192Init, 2-49, 4-53
ippsMul_BNU8, 5-7	ippsRijndael256DecryptCBC, 2-63
ippsMulOne_BNU, 5-5	ippsRijndael256DecryptCFB, 2-65
ippsPrimeGen, 5-54	ippsRijndael256DecryptCTR, 2-69
ippsPrimeGet, 5-57	ippsRijndael256DecryptECB, 2-61
IpssPrimeGet_BN, 5-58	ippsRijndael256DecryptOFB, 2-67
ippsPrimeGetSize, 5-52	ippsRijndael256EncryptCBC, 2-62
ippsPrimeInit, 5-53	ippsRijndael256EncryptCFB, 2-64
ippsPrimeSet, 5-56	ippsRijndael256EncryptCTR, 2-68
ippsPrimeSet_BN, 5-56	ippsRijndael256EncryptECB, 2-60
ippsPrimeTest, 5-55	ippsRijndael256EncryptOFB, 2-66
ippsPRNGen, 5-47	ippsRijndael256GetSize, 2-59
ippsPRNGen_BN, 5-48	ippsRijndael256Init, 2-60, 4-57
ippsPRNGGetSize, 5-43	ippsRSADecrypt, 5-70
ippsPRNGInit, 5-43	ippsRSAEncrypt, 5-69
ippsPRNGSetAugment, 5-45	ippsRSAGenerate, 5-66
ippsPRNGSetH0, 5-47	ippsRSAGetKey, 5-65
ippsPRNGSetModulus, 5-46	ippsRSAGetSize, 5-62
ippsPRNGSetSeed, 5-44	ippsRSAInit, 5-63

ippsRSAOAEPDecrypt, 5-82
ippsRSAOAEPDecrypt_MD5, 5-83
ippsRSAOAEPDecrypt_SHA1, 5-84
ippsRSAOAEPDecrypt_SHA224, 5-84
ippsRSAOAEPDecrypt_SHA256, 5-85
ippsRSAOAEPDecrypt_SHA384, 5-86
ippsRSAOAEPDecrypt_SHA512, 5-87
ippsRSAOAEPEncrypt, 5-75
ippsRSAOAEPEncrypt_MD5, 5-76
ippsRSAOAEPEncrypt_SHA1, 5-77
ippsRSAOAEPEncrypt_SHA224, 5-78
ippsRSAOAEPEncrypt_SHA256, 5-79
ippsRSAOAEPEncrypt_SHA384, 5-80
ippsRSAOAEPEncrypt_SHA512, 5-81
ippsRSASetKey, 5-64
ippsRSASSASign, 5-88
ippsRSASSASign_MD5, 5-90
ippsRSASSASign_SHA1, 5-91
ippsRSASSASign_SHA224, 5-92
ippsRSASSASign_SHA256, 5-93
ippsRSASSASign_SHA384, 5-94
ippsRSASSASign_SHA512, 5-95
ippsRSASSAVerify, 5-96
ippsRSASSAVerify_MD5, 5-97
ippsRSASSAVerify_SHA1, 5-98
ippsRSASSAVerify_SHA224, 5-98
ippsRSASSAVerify_SHA256, 5-99
ippsRSASSAVerify_SHA384, 5-100
ippsRSASSAVerify_SHA512, 5-100
ippsRSASValidate, 5-67
ippsSet_BN, 5-14
ippsSetOctString_BN, 5-15
ippsSetOctString_BNU, 5-11
ippsSHA1Duplicate, 3-9
ippsSHA1Final, 3-11
ippsSHA1GetSize, 3-8
ippsSHA1Init, 3-8
ippsSHA1MessageDigest, 3-27
ippsSHA1Update, 3-10
ippsSHA224Duplicate, 3-12
ippsSHA224Final, 3-14
ippsSHA224GetSize, 3-11
ippsSHA224Init, 3-12
ippsSHA224MessageDigest, 3-29
ippsSHA224Update, 3-13
ippsSHA256Duplicate, 3-16
ippsSHA256Final, 3-18
ippsSHA256GetSize, 3-15
ippsSHA256Init, 3-15
ippsSHA256MessageDigest, 3-30
ippsSHA256Update, 3-17
ippsSHA384Duplicate, 3-19
ippsSHA384Final, 3-21
ippsSHA384GetSize, 3-18
ippsSHA384Init, 3-19
ippsSHA384MessageDigest, 3-31
ippsSHA384Update, 3-20
ippsSHA512Duplicate, 3-23
ippsSHA512Final, 3-25
ippsSHA512GetSize, 3-22
ippsSHA512Init, 3-22
ippsSHA512MessageDigest, 3-32
ippsSHA512Updat, 3-24
ippsSqr_32u64u, 5-9
ippsSqr_BNU4, 5-10
ippsSqr_BNU8, 5-10
ippsSub_BN, 5-24
ippsSub_BNU, 5-4
ippsTDESDecryptCBC, 2-19
ippsTDESDecryptCFB, 2-21
ippsTDESDecryptCTR, 2-26
ippsTDESDecryptECB, 2-17
ippsTDESDecryptOFB, 2-24
ippsTDESEncryptCBC, 2-18
ippsTDESEncryptCFB, 2-20
ippsTDESEncryptCTR, 2-25
ippsTDESEncryptECB, 2-16
ippsTDESEncryptOFB, 2-23
ippsTwofishDecryptCBC, 2-92
ippsTwofishDecryptCFB, 2-94
ippsTwofishDecryptCTR, 2-98
ippsTwofishDecryptECB, 2-90
ippsTwofishDecryptOFB, 2-96
ippsTwofishEncryptCBC, 2-91
ippsTwofishEncryptCFB, 2-93
ippsTwofishEncryptCTR, 2-97
ippsTwofishEncryptECB, 2-89
ippsTwofishEncryptOFB, 2-95
ippsTwofishGetSize, 2-88
ippsTwofishInit, 2-89

K

Keyed Hash Functions, 4-1

- HMACMD5Duplicate, 4-30
- HMACMD5Final, 4-32
- HMACMD5GetSize, 4-29
- HMACMD5Init, 4-29
- HMACMD5MessageDigest, 4-32
- HMACMD5Update, 4-31
- HMACSHA1Duplicate, 4-5
- HMACSHA1Final, 4-7
- HMACSHA1GetSize, 4-4
- HMACSHA1Init, 4-5
- HMACSHA1MessageDigest, 4-8
- HMACSHA1Update, 4-6
- HMACSHA224Duplicate, 4-10
- HMACSHA224Final, 4-12
- HMACSHA224GetSize, 4-9
- HMACSHA224Init, 4-9
- HMACSHA224MessageDigest, 4-12
- HMACSHA224Update, 4-11
- HMACSHA256BufferSize, 4-14
- HMACSHA256Duplicate, 4-15
- HMACSHA256Final, 4-16
- HMACSHA256Init, 4-14
- HMACSHA256MessageDigest, 4-17
- HMACSHA256Update, 4-15
- HMACSHA384Duplicate, 4-21
- HMACSHA384Final, 4-23
- HMACSHA384GetSize, 4-20
- HMACSHA384Init, 4-20
- HMACSHA384MessageDigest, 4-23
- HMACSHA384Update, 4-22
- HMACSHA512Duplicate, 4-26
- HMACSHA512Final, 4-27
- HMACSHA512GetSize, 4-24
- HMACSHA512Init, 4-25
- HMACSHA512MessageDigest, 4-28
- HMACSHA512Update, 4-26

M

manual organization, 1-5

mask generation function, 3-32

Mask Generation Functions, 3-32

- MGF_MD5, 3-34
- MGF_SHA1, 3-34
- MGF_SHA224, 3-35
- MGF_SHA256, 3-36
- MGF_SHA384, 3-37
- MGF_SHA512, 3-37

user-implemented, 3-33

MD5 and SHA Algorithms, 3-4

- MD5Duplicate, 3-6
- MD5Final, 3-7
- MD5GetSize, 3-4
- MD5Init, 3-5
- MD5MessageDigest, 3-8
- MD5Update, 3-6
- SHA1Duplicate, 3-9
- SHA1Final, 3-11
- SHA1GetSize, 3-8
- SHA1Init, 3-8
- SHA1MessageDigest, 3-11
- SHA1Update, 3-10
- SHA224Duplicate, 3-12
- SHA224Final, 3-14
- SHA224GetSize, 3-11
- SHA224Init, 3-12
- SHA224Update, 3-13
- SHA256Duplicate, 3-16
- SHA256Final, 3-18
- SHA256GetSize, 3-15
- SHA256Init, 3-15
- SHA256MessageDigest, 3-18
- SHA256Update, 3-17
- SHA384Duplicate, 3-19
- SHA384Final, 3-21
- SHA384GetSize, 3-18
- SHA384Init, 3-19
- SHA384MessageDigest, 3-22
- SHA384Update, 3-20
- SHA512Duplicate, 3-23
- SHA512Final, 3-25
- SHA512GetSize, 3-22
- SHA512Init, 3-22
- SHA512Update, 3-24

Message Authentication Functions, 4-1

CMAC Functions, 4-3

Keyed Hash Functions, 4-1

MGF, 3-32

modes of operation for block ciphers

- CBC, 2-2
- CCM, 2-44
- CFB, 2-2
- CTR, 2-2
- ECB, 2-2
- OFB, 2-2

Montgomery Reduction Scheme Functions, 5-31

- MontExp, 5-40
- MontForm, 5-36

MontGet, 5-36
MontGetSize, 5-33
MontInit, 5-34
MontMul, 5-37
MontSet, 5-35

N

naming conventions, 1-7
notational conventions, 1-7

O

online version, 1-7

P

platforms supported, 1-3
Prime Number Generation Functions, 5-51
 PrimeGen, 5-54
 PrimeGet, 5-57
 PrimeGet_BN, 5-58
 PrimeGetSize, 5-52
 PrimeInit, 5-53
 PrimeSet, 5-56
 PrimeSet_BN, 5-56
 PrimeTest, 5-55
Pseudorandom Number Generation Functions, 5-41
 PRNGen, 5-47
 PRNGen_BN, 5-48
 PRNGGetSize, 5-43
 PRNGInit, 5-43
 PRNGSetAugment, 5-45
 PRNGSetH0, 5-47
 PRNGSetModulus, 5-46
 PRNGSetSeed, 5-44
 user-implemented, 5-42

R

RC4 stream cipher, 2-127
RC5 Functions, 2-102
 ARCFive128DecryptCBC, 2-120
 ARCFive128DecryptCFB, 2-122
 ARCFive128DecryptCTR, 2-126
 ARCFive128DecryptECB, 2-118
 ARCFive128DecryptOFB, 2-124
 ARCFive128EncryptCBC, 2-119
 ARCFive128EncryptCFB, 2-121
 ARCFive128EncryptCTR, 2-125

 ARCFive128EncryptECB, 2-117
 ARCFive128EncryptOFB, 2-123
 ARCFive128GetSize, 2-116
 ARCFive128Init, 2-116
 ARCFive64DecryptCBC, 2-109
 ARCFive64DecryptCFB, 2-111
 ARCFive64DecryptCTR, 2-115
 ARCFive64DecryptECB, 2-107
 ARCFive64DecryptOFB, 2-113
 ARCFive64EncryptCBC, 2-108
 ARCFive64EncryptCFB, 2-110
 ARCFive64EncryptCTR, 2-114
 ARCFive64EncryptECB, 2-106
 ARCFive64EncryptOFB, 2-112
 ARCFive64GetSize, 2-104
 ARCFive64Init, 2-105

reference code, 1-2

Rijndael Functions, 2-29
 Rijndael128DecryptCBC, 2-36
 Rijndael128DecryptCCM, 2-44
 Rijndael128DecryptCCM_u8, 2-47
 Rijndael128DecryptCFB, 2-38
 Rijndael128DecryptCTR, 2-42
 Rijndael128DecryptECB, 2-34
 Rijndael128DecryptOFB, 2-40
 Rijndael128EncryptCBC, 2-35
 Rijndael128EncryptCCM, 2-43
 Rijndael128EncryptCCM_u8, 2-45
 Rijndael128EncryptCFB, 2-37
 Rijndael128EncryptCTR, 2-41
 Rijndael128EncryptECB, 2-33
 Rijndael128EncryptOFB, 2-39
 Rijndael128GetSize, 2-32
 Rijndael128Init, 2-33
 Rijndael192DecryptCBC, 2-52
 Rijndael192DecryptCFB, 2-54
 Rijndael192DecryptCTR, 2-58
 Rijndael192DecryptECB, 2-50
 Rijndael192DecryptOFB, 2-56
 Rijndael192EncryptCBC, 2-51
 Rijndael192EncryptCFB, 2-53
 Rijndael192EncryptCTR, 2-57
 Rijndael192EncryptECB, 2-49
 Rijndael192EncryptOFB, 2-55
 Rijndael192GetSize, 2-48
 Rijndael192Init, 2-49
 Rijndael256DecryptCBC, 2-63
 Rijndael256DecryptCFB, 2-65
 Rijndael256DecryptCTR, 2-69
 Rijndael256DecryptECB, 2-61

version information function, B-1